

Hybrid Quantum Machine Learning Platform for Early Disease Detection

Table of Contents

S. No.	Content
1	Problem Statement
2	Background
3	Proposed Solution
4	Objectives
5	Target Disease Applications
6	Need for Hybrid Quantum Machine Learning
7	System Architecture
8	Data Ingestion Module
9	Data Preprocessing
10	Feature Engineering
11	Feature Selection
12	Dimensionality Reduction
13	Classical Machine Learning Baselines

14	Quantum Machine Learning Models
15	Quantum Feature Encoding
16	Hybrid Quantum-Classical Training Workflow
17	Disease Prediction
18	Explainable AI
19	Performance Evaluation
20	Classical vs Hybrid QML Benchmarking
21	Quantum Simulator Support
22	Near-Term Quantum Hardware Compatibility
23	Model Optimization
24	Platform Dashboard
25	End-to-End System Workflow
26	Technology Stack
27	Software Architecture
28	AI/ML Pipeline
29	Dataset Strategy
30	Training and Testing Strategy

31	Data Leakage Prevention
32	Model Reliability
33	Explainability and Transparency
34	Security and Privacy
35	Prototype Implementation
36	Example Prototype Workflow
37	Innovation
38	Key USP
39	Feasibility
40	Limitations
41	Safety and Medical Responsibility
42	Testing and Validation
43	Expected Results
44	Expected Impact
45	Future Scope
46	Expected Deliverables
47	SIH Demo Flow

Problem Statement

Early and accurate disease detection is critical for improving treatment outcomes and reducing healthcare costs. However, biomedical data such as genomics, medical imaging, and electronic health records is often high-dimensional, noisy, and complex, making accurate pattern detection challenging.

Classical machine learning has shown strong potential in medical diagnosis, but traditional models may face limitations when handling complex biomedical data. Quantum Machine Learning (QML) offers a potential way to capture intricate patterns using quantum computing concepts such as superposition and entanglement.

At the same time, current quantum hardware has limitations such as limited qubits, noise, and computational constraints. Therefore, a purely quantum solution is not yet practical for many real-world applications.

The problem is to develop a hybrid quantum-classical machine learning platform that combines classical data preprocessing and feature engineering with quantum-enhanced learning models to support early disease detection, while comparing its performance against classical machine learning approaches.

Core Challenge:

How can hybrid quantum-classical machine learning be effectively used with complex biomedical data to improve early disease detection while remaining practical, interpretable, scalable, and compatible with quantum simulators and near-term quantum hardware?

Background

Early and accurate detection of diseases is one of the most important factors in improving healthcare outcomes. When a disease is identified at an early stage, appropriate treatment can be started sooner, which can increase the chances of successful recovery, prevent the disease from becoming more serious, and reduce overall healthcare costs. Artificial Intelligence (AI) and Machine Learning (ML) are increasingly being used to support medical diagnosis by analyzing large amounts of patient data and identifying patterns associated with different diseases.

Classical machine learning techniques have achieved significant success in healthcare applications. They can be used to analyze different types of biomedical data, including **medical images, genomic data, laboratory results, and Electronic Health Records (EHRs)**. However, modern healthcare systems generate extremely large and complex datasets. These datasets often contain thousands or even millions of features, making them **high-dimensional**. They may also contain missing values, measurement errors, irrelevant information, and variations in data quality, making them **noisy**. In addition, relationships between different medical factors can be highly complex and difficult to identify using conventional computational methods.

For example, genomic data can contain a very large number of genetic features, while medical imaging data contains detailed visual information about tissues and organs. Electronic Health Records may contain a combination of a patient's medical history, symptoms, test results, medications, and other clinical information. Analyzing these different sources together can provide valuable information for disease detection, but it also creates significant computational and modeling challenges.

Quantum Machine Learning (QML) is an emerging field that combines the principles of quantum computing with machine learning. Quantum computing uses quantum-mechanical

concepts such as **superposition and entanglement** to represent and process information in ways that are different from traditional classical computing. These properties provide the potential to represent complex relationships within data and may offer advantages for certain machine learning tasks.

However, current quantum computers still have important limitations. They have a limited number of usable qubits, are affected by noise and errors, and cannot yet handle the scale and complexity of most real-world medical datasets entirely on their own. Because of these limitations, a completely quantum-based disease detection system is not currently practical for most healthcare applications.

A **Hybrid Quantum-Classical Machine Learning (HQML)** approach provides a practical solution to this challenge. In a hybrid system, classical computers perform tasks such as data collection, preprocessing, noise reduction, feature extraction, feature selection, and other computational operations, while selected parts of the machine learning process are performed using quantum circuits or quantum machine learning algorithms. This allows the system to take advantage of existing classical computing infrastructure while exploring the potential benefits of quantum computing.

The hybrid approach can also be developed and tested using **quantum simulators and near-term quantum devices**, making it suitable for current technological conditions. Instead of requiring a large and fully fault-tolerant quantum computer, the proposed approach can use available quantum resources for specific computational tasks.

Therefore, the development of a **Hybrid Quantum Machine Learning Platform for Early Disease Detection** aims to explore how classical machine learning and quantum computing can work together to analyze complex biomedical data. The platform can provide a foundation for investigating quantum-enhanced approaches to disease prediction while remaining practical with today's quantum simulators and emerging quantum hardware. Ultimately, the goal is to support **faster, more accurate, and data-driven early disease detection**, while contributing to research into the future application of quantum computing in healthcare.

3. Proposed Solution

We propose a **Hybrid Quantum Machine Learning (HQML) Platform for Early Disease Detection** that combines the strengths of classical machine learning with quantum-enhanced learning techniques.

The platform is designed to process complex biomedical datasets such as **genomics, medical imaging-derived data, and electronic health records (EHRs)** and use them for early disease identification.

Instead of attempting to perform the entire process on a quantum computer, the proposed system follows a hybrid quantum-classical architecture. Classical computing handles data preparation, feature engineering, feature selection,

dimensionality reduction, and optimization, while quantum computing is used for selected machine-learning operations.

3.1 Basic Concept

The complete concept can be represented as:

Biomedical Data



Data Ingestion & Validation



Data Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Classical Baseline + Quantum Feature Encoding



Hybrid Quantum-Classical Model



Disease Prediction



Explainability



Performance Evaluation



Classical vs Hybrid QML Comparison

The platform therefore provides a complete workflow from raw biomedical data to disease prediction and model evaluation.

3.2 Data Ingestion

The platform first accepts a biomedical dataset provided by the researcher or system administrator.

The dataset may contain:

- **Patient-related clinical features**
- **Genomic features**
- **Medical measurements**
- **Imaging-derived features**
- **Electronic health record features**
- **Disease labels**

The system validates the dataset structure and identifies:

- **Number of samples**
- **Number of features**
- **Feature types**
- **Target variable**
- **Missing values**
- **Duplicate records**
- **Invalid or inconsistent values**

This ensures that unsuitable data does not directly enter the machine-learning pipeline.

3.3 Classical Data Preprocessing

Biomedical datasets can contain missing values, different numerical scales, categorical variables, noise, and irrelevant information.

The platform therefore performs preprocessing before model training.

This may include:

- **Missing-value handling**
- **Duplicate removal**
- **Data normalization**
- **Numerical feature scaling**
- **Categorical feature encoding**
- **Outlier analysis**
- **Data consistency checking**

The objective is to produce a clean and standardized dataset suitable for machine-learning models.

3.4 Feature Engineering

Feature engineering transforms the processed biomedical data into useful representations for machine learning.

Depending on the selected disease dataset, the platform can generate or transform features using:

- **Statistical transformations**
- **Normalized measurements**
- **Clinical feature combinations**
- **Genomic feature transformations**
- **Image-derived numerical features**

Feature engineering helps the models focus on information that may be useful for disease classification.

3.5 Feature Selection

Biomedical datasets can contain hundreds or thousands of features.

Sending all these features directly to a quantum circuit is difficult because current quantum systems have limited qubits.

Therefore, the platform identifies the most informative features using techniques such as:

- **Correlation analysis**
- **Mutual information**
- **Statistical feature selection**
- **Recursive feature elimination**
- **Tree-based feature importance**

The selected features are then passed to the next stage.

3.6 Dimensionality Reduction

After feature selection, dimensionality reduction can further reduce the number of features.

For example, Principal Component Analysis (PCA) can transform a large feature space into a smaller representation while retaining important information.

This step is particularly important for the quantum pipeline because the number of available qubits limits the size of the input representation.

The process becomes:

High-Dimensional Biomedical Data

→ Feature Selection

→ Dimensionality Reduction

→ Low-Dimensional Representation

→ Quantum Encoding

3.7 Classical Machine Learning Pipeline

The platform maintains a purely classical machine-learning pipeline as a baseline.

Possible models include:

- Logistic Regression
- Support Vector Machine
- Random Forest
- XGBoost
- Neural Networks

These models are trained using the processed biomedical data.

Their results provide a reference point for evaluating the hybrid quantum-classical models.

This is important because the project should determine experimentally whether the hybrid approach provides benefits rather than assuming that QML will always perform better.

3.8 Quantum Feature Encoding

The selected biomedical features are converted into a representation that can be processed by a quantum circuit.

One practical approach is angle encoding, where numerical features are mapped to rotation angles of quantum gates.

Conceptually:

Classical Feature

→ Scaling

→ Quantum Rotation

→ Qubit Representation

Multiple features can therefore be represented across multiple qubits, depending on the selected encoding strategy and available quantum resources.

3.9 Quantum-Enhanced Learning

After encoding the biomedical features, the quantum circuit performs the quantum component of the learning process.

The platform can support models such as:

- **Quantum Support Vector Machine / quantum kernel methods**
- **Variational Quantum Classifier (VQC)**
- **Quantum Neural Network (QNN)**

For example, in a Variational Quantum Classifier:

Input Features

→ Quantum Feature Map

→ Parameterized Quantum Circuit

→ Measurement

→ **Classical Loss Calculation**

→ **Classical Optimizer**

→ **Parameter Update**

The process repeats during training until the model reaches the desired optimization objective.

3.10 Hybrid Quantum-Classical Training

The key part of the proposed solution is the interaction between classical and quantum computing.

The classical computer performs tasks that are currently more practical on classical hardware, while the quantum processor or simulator performs selected quantum computations.

The workflow is:

1. Biomedical data is processed classically.
2. Important features are selected.
3. Features are reduced to a quantum-compatible size.
4. Features are encoded into quantum states.
5. Quantum circuits process the encoded data.
6. Quantum measurements produce outputs.
7. A classical optimizer evaluates the output.
8. Model parameters are updated.
9. The process is repeated during training.
10. The final hybrid model is evaluated on unseen data.

This approach allows the project to use current quantum technology without requiring a large-scale fault-tolerant quantum computer.

3.11 Disease Prediction

After the hybrid model has been trained, new biomedical samples can be provided to the system.

The prediction process is:

New Biomedical Sample

→ **Preprocessing**

→ **Feature Transformation**

→ **Feature Selection**

→ **Quantum Encoding**

→ **Trained Hybrid Model**

→ **Prediction**

The output can provide the predicted disease class or classification result according to the selected dataset.

For example, in a binary classification dataset, the system may classify an input sample into:

Class 0 → Negative

Class 1 → Positive

The exact meaning of each class depends on the selected biomedical dataset.

3.12 Explainability

The platform includes an explainability component so that predictions are not presented as unexplained outputs.

The system can provide information such as:

- **Important input features**
- **Feature contribution**
- **Prediction probability/confidence**
- **Model performance**
- **Comparison with other models**

For classical components, techniques such as feature importance, permutation importance, or SHAP can be used where appropriate.

For hybrid QML models, the platform can expose interpretable information from the classical preprocessing and feature-selection stages along with the model's output and evaluation results.

3.13 Performance Evaluation

The proposed platform evaluates the trained models using multiple metrics.

Important metrics include:

- **Accuracy**
- **Precision**
- **Recall**
- **Sensitivity**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Training time**
- **Inference time**
- **Generalization performance**

This is particularly important in disease detection because accuracy alone may not represent the actual usefulness of a model.

For example, sensitivity is important for understanding how effectively the model detects positive disease cases.

3.14 Classical vs Hybrid Comparison

One of the most important parts of the platform is direct benchmarking.

The same dataset and evaluation methodology can be used to compare:

Classical ML Model

versus

Hybrid Quantum-Classical Model

The platform can present comparisons such as:

Metric	Classical ML	Hybrid QML
Accuracy	Measured	Measured
Sensitivity	Measured	Measured
Specificity	Measured	Measured
Precision	Measured	Measured
F1-score	Measured	Measured

ROC-AUC	Measured	Measured
Training Time	Measured	Measured
Inference Time	Measured	Measured
Generalization	Measured	Measured

The objective is to identify whether the hybrid approach provides meaningful advantages for the selected biomedical problem.

3.15 Quantum Simulator Support

The initial platform can operate using quantum simulators.

This allows the team to develop and test quantum algorithms without requiring physical quantum hardware.

The architecture can therefore follow:

Classical Computer

→ **Quantum Simulator**

→ **Hybrid QML Model**

→ **Evaluation**

Once the model is validated in simulation, it can potentially be adapted to compatible near-term quantum hardware.

3.16 Near-Term Quantum Hardware Compatibility

The system is designed with current quantum hardware limitations in mind.

The platform should consider:

- **Limited qubit availability**
- **Circuit depth**
- **Quantum noise**
- **Measurement limitations**
- **Hardware execution time**

Therefore, the quantum component should use relatively compact circuits suitable for near-term devices.

The platform should remain functional even when quantum hardware is unavailable by supporting simulator-based execution.

3.17 Platform Dashboard

A user-facing dashboard can provide a single interface for the complete workflow.

The dashboard can contain:

Dataset Panel

- **Dataset upload**
- **Number of samples**
- **Number of features**
- **Dataset summary**

Preprocessing Panel

- **Missing values**
- **Selected features**
- **Reduced dimensions**

Model Panel

- **Classical model selection**
- **QML model selection**
- **Training status**

Prediction Panel

- **Predicted class**
- **Prediction probability/confidence**

Explainability Panel

- **Important features**
- **Feature contribution**
- **Model information**

Benchmark Panel

- **Classical vs QML performance**
- **Accuracy**
- **Sensitivity**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Computational efficiency**

3.18 Complete Proposed Solution

The complete platform can therefore be summarized as:

Biomedical Dataset



Data Ingestion



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Classical ML Baseline



Quantum Feature Encoding



Quantum-Enhanced Model



Hybrid Classical Optimization



Disease Prediction



Explainability



Performance Evaluation



Classical vs Hybrid Benchmark



Results Dashboard

3.19 Core Advantage of the Proposed Solution

The main advantage of the proposed platform is that it does not treat quantum machine learning as a standalone model.

Instead, it provides a complete experimental environment where researchers can:

- **Prepare complex biomedical data.**
- **Reduce it to a quantum-compatible representation.**
- **Train quantum-enhanced models.**
- **Compare them with classical models.**
- **Analyze prediction performance.**
- **Understand model outputs.**
- **Evaluate computational efficiency.**
- **Test models on quantum simulators.**
- **Prepare the architecture for compatible near-term quantum hardware.**

Therefore, the platform provides a practical way to investigate the potential of hybrid quantum-classical machine learning for early disease detection without depending on large-scale quantum computers.

4. Objectives

The primary objective of the project is to develop a hybrid quantum-classical machine learning platform for early disease detection using biomedical datasets.

The platform will combine classical data processing and machine learning with quantum-enhanced learning models to investigate their effectiveness for complex biomedical classification tasks.

4.1 Design a Hybrid Quantum-Classical Architecture

Develop an architecture in which classical and quantum computing work together.

The classical part will handle:

- **Data ingestion**
- **Data cleaning**
- **Preprocessing**
- **Feature engineering**
- **Feature selection**
- **Dimensionality reduction**
- **Model optimization**
- **Performance analysis**

The quantum component will handle selected machine-learning operations such as:

- **Quantum feature encoding**
- **Quantum kernels**
- **Variational quantum circuits**
- **Quantum-enhanced classification**

This architecture should remain practical for quantum simulators and near-term quantum devices.

4.2 Process Complex Biomedical Data

Develop a reliable data pipeline capable of handling biomedical datasets such as:

- **Genomics data**
- **Medical imaging-derived data**
- **Electronic health record data**

- **Clinical datasets**
- **Other benchmark biomedical datasets**

The system should be able to identify and handle common data issues such as missing values, inconsistent formats, different feature scales, and irrelevant features.

4.3 Perform Feature Engineering and Selection

Develop mechanisms to transform raw biomedical data into useful machine-learning representations.

The platform should identify the features that contain the most useful information for the selected disease detection task.

Possible techniques include:

- **Statistical feature selection**
- **Correlation analysis**
- **Mutual information**
- **Recursive feature elimination**
- **Tree-based feature importance**

This reduces unnecessary information and prepares the data for efficient model training.

4.4 Reduce High-Dimensional Data

Biomedical datasets can contain a very large number of features, while current quantum systems have limited qubits.

Therefore, the platform should provide dimensionality-reduction techniques such as:

- **PCA**
- **Feature selection**
- **Other suitable dimensionality-reduction methods**

The objective is to produce a compact representation that can be processed by quantum models without unnecessarily losing important information.

4.5 Develop Quantum-Enhanced Machine Learning Models

Implement quantum machine-learning approaches suitable for disease detection.

The platform can support models such as:

- **Quantum Support Vector Machine / quantum kernel methods**
- **Variational Quantum Classifiers**
- **Quantum Neural Networks**

These models will be tested on biomedical datasets to investigate their ability to identify disease-related patterns.

4.6 Develop a Hybrid Training Process

Create a training workflow where quantum and classical components interact.

The process will involve:

Biomedical Data

→ **Classical preprocessing**

→ **Feature selection**

→ **Dimensionality reduction**

→ **Quantum feature encoding**

→ **Quantum computation**

→ **Measurement**

→ **Classical optimization**

→ **Parameter update**

→ **Repeated training**

This allows the project to take advantage of both classical computing and available quantum resources.

4.7 Enable Early Disease Detection

The platform should use trained models to classify previously unseen biomedical samples.

The objective is to identify disease-related classes as early and accurately as possible according to the selected dataset.

The system should focus on important medical classification measures such as:

- **Sensitivity**
- **Specificity**
- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**

The platform is intended as a research and decision-support system, not a replacement for qualified medical diagnosis.

4.8 Improve Detection Performance

Evaluate whether quantum-enhanced models can improve disease detection performance compared with classical approaches.

The project will not assume that QML is automatically superior.

Instead, it will experimentally compare the approaches using the same:

- **Dataset**
- **Data preprocessing strategy**
- **Train/test methodology**
- **Evaluation metrics**

This provides an objective assessment of the usefulness of the hybrid approach.

4.9 Compare with Classical Machine Learning

Develop strong classical baseline models and compare them with hybrid QML models.

Possible classical baselines include:

- **Logistic Regression**
- **Support Vector Machine**
- **Random Forest**
- **XGBoost**
- **Neural Networks**

The comparison will consider:

- **Accuracy**

- Sensitivity
- Specificity
- Precision
- F1-score
- ROC-AUC
- Training time
- Inference time
- Generalization performance

4.10 Develop Explainability Features

Provide interpretable information about model predictions.

The platform should help users understand:

- Which features were considered important.
- How features contributed to predictions where applicable.
- The prediction probability/confidence.
- How the model performed on evaluation data.

For classical components, explainability techniques such as feature importance and SHAP can be used where appropriate.

The objective is to make the system more transparent and easier to evaluate.

4.11 Support Quantum Simulators

Ensure that the platform can operate without requiring physical quantum hardware.

Quantum simulators will allow the team to:

- Develop QML algorithms.
- Test quantum circuits.
- Train models.
- Evaluate different configurations.
- Reproduce experiments.

This makes the project practical with currently available computing resources.

4.12 Ensure Near-Term Quantum Hardware Compatibility

Design the quantum component so that it can potentially run on compatible near-term quantum devices.

The design should consider:

- **Limited qubits**
- **Quantum noise**
- **Circuit depth**
- **Measurement limitations**
- **Hardware execution time**

The system should therefore avoid depending on large-scale fault-tolerant quantum computers.

4.13 Benchmark Computational Efficiency

The project should evaluate not only prediction quality but also computational requirements.

The platform can measure:

- **Training time**
- **Inference time**
- **Quantum circuit execution requirements**
- **Classical preprocessing time**
- **Resource requirements**

This helps determine whether a hybrid approach is practically useful for the selected biomedical problem.

4.14 Evaluate Generalization Performance

The trained models should be evaluated on previously unseen data.

The objective is to determine whether the model has learned meaningful patterns rather than simply memorizing the training dataset.

The evaluation should use appropriate:

- **Train/validation/test splits**
- **Cross-validation where applicable**
- **Unseen test data**

This is especially important when working with biomedical datasets.

4.15 Build a Scalable Platform

The platform should be modular so that researchers can change:

- **Biomedical datasets**
- **Disease classification tasks**
- **Feature-selection methods**
- **Classical ML algorithms**
- **Quantum models**
- **Quantum feature maps**
- **Evaluation metrics**

This allows the same platform to be used for different disease-detection experiments.

4.16 Create a Complete Research Workflow

The final platform should provide an end-to-end workflow:

Data Ingestion

- **Preprocessing**
- **Feature Engineering**
- **Feature Selection**
- **Dimensionality Reduction**
- **Classical Baseline**
- **Quantum Encoding**
- **Hybrid QML Training**
- **Prediction**
- **Explainability**
- **Performance Evaluation**
- **Classical vs QML Benchmarking**

This makes the platform more than an isolated quantum model; it becomes a complete environment for evaluating hybrid QML approaches in early disease detection.

4.17 Primary Objective

The overall objective can be summarized as:

To design and develop a practical, scalable, interpretable, and benchmarkable hybrid quantum-classical machine learning platform that uses biomedical data for early disease detection and evaluates the performance of quantum-enhanced models against classical machine-learning approaches.

5. Target Disease Applications

The proposed Hybrid Quantum Machine Learning Platform is designed as a general-purpose biomedical disease detection platform. It is not restricted to a single disease. The same architecture can be adapted to different biomedical datasets and classification problems.

The problem statement specifically mentions diseases such as cancer, cardiovascular disorders, and neurological conditions. These provide suitable application areas for demonstrating the platform.

5.1 Cancer Detection

Cancer detection is one of the potential application areas of the proposed platform.

Biomedical cancer datasets may contain:

- **Patient clinical measurements**
- **Laboratory results**
- **Genomic features**
- **Tumor-related measurements**
- **Medical imaging-derived features**

These features can be processed through the platform's classical preprocessing and feature-selection pipeline.

The selected features can then be used by both classical machine-learning models and hybrid quantum models.

The system can perform a classification task such as:

Biomedical Features → Model → Cancer-related Classification

The exact classification labels will depend on the selected benchmark dataset.

5.2 Cardiovascular Disease Detection

Cardiovascular disorders are another potential application.

A cardiovascular dataset may contain features such as:

- **Age**
- **Clinical measurements**
- **Blood pressure-related measurements**
- **Laboratory measurements**
- **Other relevant patient attributes**

The platform can process these features and train models to identify patterns associated with the target classification.

The workflow remains:

Patient/Biomedical Data

→ Preprocessing

→ Feature Selection

→ Classical ML / Hybrid QML

→ Prediction

→ Evaluation

The performance of the models can then be compared using sensitivity, specificity, accuracy, F1-score, and other relevant metrics.

5.3 Neurological Disease Detection

The platform can also be applied to neurological conditions.

Depending on the selected dataset, the input may include:

- **Clinical measurements**
- **Patient assessment features**
- **Imaging-derived features**
- **Biomedical measurements**
- **Other disease-specific features**

High-dimensional data can first be processed using classical techniques before selected features are passed to the quantum-enhanced model.

This provides another environment for evaluating whether hybrid QML can effectively learn complex biomedical patterns.

5.4 Genomics-Based Disease Detection

Genomic datasets can contain a very large number of features.

This makes them particularly relevant to the problem statement's discussion of high-dimensional biomedical data.

A possible workflow is:

Genomic Data



Preprocessing



Feature Selection



Dimensionality Reduction



Quantum Feature Encoding



Hybrid QML Model



Disease Classification

The dimensionality-reduction stage is particularly important because current quantum systems have limited qubit resources.

5.5 Medical Imaging-Derived Data

Medical imaging can contain extremely large amounts of information.

For the initial platform, images do not necessarily need to be directly passed into a quantum circuit.

Instead, classical processing can first extract meaningful numerical representations or features from the images.

The workflow can be:

Medical Image



Classical Image Processing / Feature Extraction



Feature Selection



Dimensionality Reduction



Hybrid QML



Disease Classification

This approach reduces the amount of information that must be processed by the quantum component.

5.6 Electronic Health Records

Electronic Health Records (EHRs) can contain multiple types of patient information.

Examples include:

- **Demographic information**
- **Clinical measurements**
- **Laboratory results**
- **Medical history**

- Diagnoses
- Other structured patient information

The platform can preprocess and transform these records into machine-learning features.

After feature selection and dimensionality reduction, the resulting representation can be used by classical and hybrid QML models.

5.7 Disease-Independent Architecture

An important characteristic of the proposed platform is that its architecture is not tied to one specific disease.

For example:

Cancer Dataset

→ Same Platform

Cardiovascular Dataset

→ Same Platform

Neurological Dataset

→ Same Platform

Genomic Dataset

→ Same Platform

The data-processing and evaluation pipeline remains largely the same, while the dataset, features, labels, and model configuration can be changed.

This makes the platform reusable for multiple biomedical research problems.

5.8 Choosing the Disease Dataset for the Prototype

For the SIH prototype, the team should select a well-established benchmark biomedical dataset that:

- Has clear target labels.
- Is publicly available or appropriately accessible.
- Contains sufficient samples.

- Has relevant biomedical features.
- Can be processed within available computational resources.
- Can be reduced to a manageable number of features for the quantum model.

A relatively small structured dataset can be preferable for the first QML demonstration because current quantum simulators and near-term quantum devices have resource limitations.

5.9 Disease Detection as a Classification Problem

The initial implementation can focus on binary classification.

For example:

Input: Biomedical features

Output:

- Class 0 — Negative
- Class 1 — Positive

The exact meaning of Class 0 and Class 1 depends on the selected dataset.

The platform can later be extended to multiclass disease classification if sufficient data and computational resources are available.

5.10 Evaluation Across Different Diseases

If multiple datasets are used, the platform can compare model behavior across different biomedical domains.

For each dataset, the system can measure:

- Accuracy
- Sensitivity
- Specificity
- Precision
- Recall
- F1-score
- ROC-AUC
- Training time
- Inference time
- Generalization performance

This helps determine whether the hybrid quantum-classical approach performs consistently across different types of biomedical data.

5.11 Important Project Positioning

The project should not claim that quantum machine learning will automatically detect diseases better than classical machine learning.

Instead, the objective is to experimentally investigate:

Whether hybrid quantum-classical models can provide useful advantages for early disease detection on suitable biomedical datasets.

The final results should be based on measured experimental performance.

5.12 Medical Use Limitation

The proposed platform is intended as a research and machine-learning decision-support platform.

A prediction generated by the prototype should not be presented as a final medical diagnosis.

Real-world clinical deployment would require:

- **Clinical validation**
- **Representative patient data**
- **Extensive testing**
- **Medical expert evaluation**
- **Appropriate regulatory approval**
- **Privacy and security compliance**

Therefore, the SIH prototype demonstrates the technical feasibility of hybrid QML for biomedical disease detection, rather than claiming clinical deployment.

5.13 Application Summary

Application Area	Possible Input	Platform Objective
Cancer	Clinical/genomic/imaging-derived features	Disease classification
Cardiovascular Disorders	Clinical and physiological features	Early disease classification

Neurological Conditions	Clinical/imaging-derived features	Disease classification
Genomics	High-dimensional genomic features	Pattern-based disease classification
Medical Imaging	Image-derived features	Disease classification
Electronic Health Records	Structured clinical data	Disease prediction

Core Message

The platform is disease-independent: it provides a common hybrid quantum-classical workflow that can be applied to different biomedical datasets while objectively comparing quantum-enhanced models with classical machine-learning baselines.

6. Need for Hybrid Quantum Machine Learning

The proposed platform uses a hybrid quantum-classical machine learning approach because both classical and quantum computing have different strengths and limitations.

Biomedical datasets can contain a large number of features and complex relationships. Classical machine learning is effective for data preparation and many learning tasks, while quantum computing provides new approaches for representing and processing selected features.

At the same time, current quantum hardware is still limited. Therefore, using only quantum computing for the complete disease-detection pipeline is not practical.

The hybrid approach combines the two technologies in a single workflow.

6.1 Limitations of Using Only Classical Machine Learning

Classical machine learning has achieved significant success in biomedical applications. However, biomedical data can be:

- **High-dimensional**
- **Noisy**
- **Complex**

- **Multivariate**
- **Difficult to represent efficiently**

For example, genomic datasets can contain a very large number of features. Medical imaging and electronic health records can also contain complex patterns and relationships.

Classical algorithms can process this data, but the project investigates whether quantum-enhanced learning can provide additional benefits for suitable datasets.

Therefore, classical ML is retained as a strong baseline rather than being completely replaced.

6.2 Potential of Quantum Machine Learning

Quantum Machine Learning combines machine learning with quantum computing.

Quantum computing uses concepts such as:

- **Superposition**
- **Entanglement**
- **Quantum interference**

These properties provide different ways of representing and processing information.

QML techniques such as quantum kernels, variational quantum classifiers, and quantum neural networks can therefore be investigated for biomedical classification tasks.

However, potential quantum benefits depend on the dataset, algorithm, hardware, and experimental conditions.

The project therefore focuses on testing and benchmarking these approaches rather than assuming quantum advantage.

6.3 Why Not Use Only Quantum Computing?

Current quantum hardware has practical limitations, including:

- **Limited number of qubits**
- **Quantum noise**
- **Limited circuit depth**

- **Measurement constraints**
- **Hardware availability**
- **Computational limitations**

High-dimensional biomedical data cannot simply be transferred directly into a small quantum circuit.

Therefore, classical processing is required before quantum processing.

The proposed approach is:

High-Dimensional Biomedical Data



Classical Preprocessing



Feature Selection



Dimensionality Reduction



Quantum-Compatible Features



Quantum Machine Learning

This makes the system more practical for current quantum technology.

6.4 Role of Classical Computing

The classical part of the platform performs tasks that are currently efficient and practical on conventional computers.

These include:

- **Data ingestion**
- **Data validation**
- **Missing-value handling**
- **Data cleaning**

- Normalization
- Feature engineering
- Feature selection
- Dimensionality reduction
- Classical baseline training
- Optimization
- Performance evaluation
- Visualization

Classical computing therefore forms the foundation of the complete platform.

6.5 Role of Quantum Computing

The quantum component is responsible for selected learning operations.

It can be used for:

- Quantum feature encoding
- Quantum kernel computation
- Variational quantum circuits
- Quantum-enhanced classification
- Quantum neural-network components

The quantum component does not need to process the entire biomedical dataset.

Instead, the most suitable representation is selected and transformed into a form that can be processed using available quantum resources.

6.6 Interaction Between Classical and Quantum Components

The main characteristic of the proposed system is the continuous interaction between classical and quantum components.

A typical training cycle is:

Biomedical Data

→ Classical preprocessing

→ Feature selection

→ Dimensionality reduction

→ Quantum feature encoding

- **Quantum circuit**
- **Measurement**
- **Classical loss calculation**
- **Classical optimizer**
- **Parameter update**
- **Quantum circuit execution again**

This process continues until the model is trained according to the selected optimization criteria.

6.7 Hybrid Approach for Current Quantum Hardware

The hybrid architecture is particularly suitable for the Near-Term Intermediate-Scale Quantum (NISQ) era, where quantum devices are available but still have significant limitations.

The platform should therefore use relatively compact quantum circuits and avoid unnecessary circuit complexity.

The system should also remain functional through quantum simulation when physical quantum hardware is unavailable.

6.8 Quantum Simulator as the Initial Platform

Quantum simulators provide a practical development environment.

The development process can be:

Classical Computer

- **Quantum Simulator**
- **Hybrid QML Model**
- **Performance Evaluation**

This allows the team to develop and validate the platform without depending on access to physical quantum hardware.

Once the implementation is validated, compatible quantum hardware can be used for additional experiments.

6.9 Fair Comparison with Classical Models

A major reason for using the hybrid architecture is to enable an objective comparison.

The platform will train both:

Classical Machine Learning Models

and

Hybrid Quantum-Classical Models

using comparable experimental conditions.

The results can then be compared using:

- **Accuracy**
- **Sensitivity**
- **Specificity**
- **Precision**
- **Recall**
- **F1-score**
- **ROC-AUC**
- **Training time**
- **Inference time**
- **Generalization performance**

This allows the project to determine whether QML provides a meaningful benefit for the selected biomedical task.

6.10 No Assumption of Quantum Advantage

An important principle of the project is:

Quantum machine learning should be evaluated experimentally, not assumed to be superior simply because it uses quantum computing.

For some datasets, classical machine learning may perform better.

For other datasets or specific representations, hybrid QML may show useful characteristics.

The purpose of the platform is to provide the infrastructure required to make this comparison scientifically.

6.11 Practical Benefits of the Hybrid Architecture

The hybrid approach provides several practical benefits:

1. Works with Current Technology

The platform does not require a large-scale fault-tolerant quantum computer.

2. Uses Classical Strengths

Complex data preparation can be performed efficiently using classical computing.

3. Enables QML Experiments

Quantum-enhanced models can be tested on suitable feature representations.

4. Supports Benchmarking

Classical and hybrid approaches can be directly compared.

5. Supports Quantum Simulators

The platform can be developed even without physical quantum hardware.

6. Supports Future Hardware

The architecture can later be adapted to compatible near-term quantum devices.

7. Reduces Quantum Resource Requirements

Feature selection and dimensionality reduction reduce the amount of information that must be encoded into quantum circuits.

6.12 Hybrid Architecture Summary

Component	Classical Computing	Quantum Computing
Data ingestion	✓	—
Data cleaning	✓	—
Feature engineering	✓	—
Feature selection	✓	—
Dimensionality reduction	✓	—
Quantum feature encoding	—	✓

Quantum kernel	—	✓
Variational quantum circuit	—	✓
Classical optimization	✓	—
Disease prediction	✓	✓
Performance evaluation	✓	—
Explainability	✓	Supporting role
Visualization	✓	—

6.13 Core Concept

The proposed platform follows the principle:

“Use classical computing where it is currently strongest and quantum computing where it can provide a meaningful learning component.”

Therefore, the system does not attempt to replace classical machine learning.

Instead, it creates a practical experimental bridge between classical AI and quantum machine learning for early disease detection.

6.14 Final Objective of the Hybrid Approach

The ultimate purpose of using a hybrid architecture is to determine whether quantum-enhanced learning can provide measurable benefits for suitable biomedical datasets in terms of:

- Detection performance
- Sensitivity and specificity
- Generalization
- Computational efficiency
- Scalability

The platform will provide the tools necessary to perform this evaluation systematically.

7. System Architecture

The proposed Hybrid Quantum Machine Learning Platform for Early Disease Detection follows a modular architecture that combines classical data processing, classical machine learning, quantum machine learning, explainability, and performance evaluation.

The architecture is designed to support biomedical datasets, quantum simulators, and compatible near-term quantum hardware.

7.1 High-Level Architecture

The complete system can be represented as:

Biomedical Data Sources



Data Ingestion & Validation



Data Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Parallel Model Pipeline



Classical ML Quantum / Hybrid ML



Classical Prediction Quantum Prediction



Model Evaluation & Comparison



Explainability



Results & Visualization Dashboard

7.2 Main Architectural Layers

The platform consists of the following major layers:

- 1. Data Layer**
- 2. Data Preprocessing Layer**
- 3. Feature Engineering Layer**
- 4. Feature Selection Layer**
- 5. Dimensionality Reduction Layer**
- 6. Classical Machine Learning Layer**
- 7. Quantum Machine Learning Layer**
- 8. Hybrid Optimization Layer**
- 9. Prediction Layer**
- 10. Explainability Layer**
- 11. Evaluation and Benchmarking Layer**
- 12. Application and Dashboard Layer**

7.3 Data Layer

The data layer is responsible for receiving and storing biomedical datasets.

Possible data sources include:

- Genomics datasets**
- Medical imaging-derived datasets**
- Electronic health records**
- Clinical datasets**
- Public biomedical benchmark datasets**

The system should support structured data formats such as CSV and other appropriate formats depending on the implementation.

The data layer maintains:

- **Input datasets**
- **Processed datasets**
- **Feature information**
- **Target labels**
- **Experiment configurations**
- **Model results**

For sensitive biomedical information, appropriate privacy and access controls should be applied.

7.4 Data Ingestion and Validation Layer

Before processing begins, the platform validates the incoming dataset.

The validation process checks:

- **Dataset format**
- **Number of samples**
- **Number of features**
- **Data types**
- **Missing values**
- **Duplicate records**
- **Invalid values**
- **Target variable**
- **Class distribution**

For example:

Dataset

→ Check Structure

→ Check Missing Values

→ Check Data Types

→ Check Target Labels

→ Validate

→ Send to Preprocessing

This prevents incorrectly formatted data from entering the machine-learning pipeline.

7.5 Data Preprocessing Layer

The preprocessing layer converts raw biomedical data into a clean representation.

Operations may include:

Missing-value handling

Missing values can be removed or appropriately imputed depending on the dataset.

Normalization

Numerical features can be scaled to suitable ranges.

Categorical encoding

Categorical information can be converted into numerical representations.

Outlier analysis

Potentially abnormal values can be identified for further examination.

Data splitting

The dataset is divided into training, validation, and test sets.

The preprocessing parameters must be learned from the training data to avoid data leakage.

7.6 Feature Engineering Layer

The feature engineering layer transforms processed data into useful machine-learning representations.

Depending on the biomedical application, this may include:

- Statistical transformations
- Derived clinical features
- Genomic transformations
- Image-derived numerical features
- Normalized measurements

The objective is to produce informative features for the subsequent models.

7.7 Feature Selection Layer

The feature-selection layer identifies the most relevant features.

This is important because biomedical datasets can contain a very large number of variables.

Possible methods include:

- Correlation-based selection
- Mutual information
- Recursive feature elimination
- Statistical tests
- Tree-based feature importance

The output is:

Raw Features → Selected Informative Features

Feature selection can reduce noise and model complexity.

It is also important for the quantum pipeline because quantum systems currently have limited qubit resources.

7.8 Dimensionality Reduction Layer

After feature selection, the platform can further reduce the feature space.

Methods such as Principal Component Analysis (PCA) can be used.

Example:

1000 Biomedical Features



Feature Selection



100 Important Features



PCA



10 Quantum-Compatible Features

The exact number depends on the dataset and available quantum resources.

The objective is to preserve useful information while creating a representation that can be processed by the quantum model.

7.9 Classical Machine Learning Layer

The classical pipeline provides the baseline against which the quantum approach is evaluated.

Possible models include:

- **Logistic Regression**
- **Support Vector Machine**
- **Random Forest**
- **XGBoost**
- **Neural Networks**

The classical model receives the processed feature representation and performs disease classification.

The output includes:

- **Predicted class**
- **Prediction probability where supported**
- **Evaluation metrics**

7.10 Quantum Machine Learning Layer

The quantum layer receives the reduced feature representation.

The general process is:

Reduced Biomedical Features



Quantum Feature Encoding



Quantum Circuit



Measurement



Model Output

Possible QML approaches include:

- Quantum kernel methods
- Quantum Support Vector Machine approaches
- Variational Quantum Classifier
- Quantum Neural Network

The quantum model can initially run on a simulator.

7.11 Quantum Feature Encoding

Classical numerical features need to be mapped into quantum states.

For example, using angle encoding:

Classical Feature Values



Feature Scaling



Rotation Angles



Quantum Gates



Encoded Quantum State

The encoding method depends on the selected QML algorithm and quantum resource constraints.

7.12 Quantum Circuit Layer

After encoding, a quantum circuit processes the data.

A simplified circuit consists of:

Input Encoding



Parameterized Quantum Gates



Entanglement



Measurement

The parameterized gates contain trainable parameters.

During training, these parameters are updated to minimize the selected loss function.

7.13 Hybrid Optimization Layer

The hybrid optimization layer connects the quantum circuit with classical optimization.

The process is:

- 1. Classical computers prepare input data.**
- 2. Features are encoded into the quantum circuit.**
- 3. Quantum circuit executes.**
- 4. Quantum measurements are returned.**
- 5. Classical computer calculates the loss.**
- 6. Classical optimizer updates circuit parameters.**
- 7. Updated parameters are sent back to the quantum circuit.**
- 8. The process repeats.**

This creates the hybrid training loop.

Classical Computer ↔ Quantum Model

7.14 Prediction Layer

After training, both classical and hybrid models can be used for prediction.

For a new biomedical sample:

New Sample

↓

Same Preprocessing

↓

Feature Selection

↓

Dimensionality Reduction

↓

Classical Model / Quantum Encoding

↓

Trained Model



Prediction

The prediction output depends on the selected disease dataset.

For a binary classification problem, it may produce:

- **Negative**
- **Positive**

or equivalent labels defined by the dataset.

7.15 Explainability Layer

The explainability layer provides information that helps users understand model behavior.

For classical models, techniques may include:

- **Feature importance**
- **SHAP**
- **Permutation importance**

For the hybrid pipeline, the platform can expose:

- **Selected features**
- **Feature transformations**
- **Model output**
- **Prediction probability/confidence where available**
- **Performance information**

The objective is to make the experimental results more transparent.

7.16 Evaluation Layer

The evaluation layer measures model performance.

It calculates:

- Accuracy
- Precision
- Recall
- Sensitivity
- Specificity
- F1-score
- ROC-AUC

It can also measure:

- Training time
- Inference time
- Resource requirements
- Generalization performance

This layer is responsible for determining whether the trained models are performing effectively.

7.17 Benchmarking Layer

The benchmarking layer performs direct comparison between:

Classical Machine Learning

and

Hybrid Quantum Machine Learning

The system can generate comparative results such as:

Metric	Classical ML	Hybrid QML
Accuracy	Measured	Measured
Sensitivity	Measured	Measured
Specificity	Measured	Measured
Precision	Measured	Measured
F1-score	Measured	Measured

ROC-AUC	Measured	Measured
Training Time	Measured	Measured
Inference Time	Measured	Measured
Generalization	Measured	Measured

The platform should report actual experimental results rather than assuming that the quantum model will always perform better.

7.18 Explainability and Benchmark Integration

The platform combines prediction results with evaluation and explainability.

For example:

Prediction

- Predicted Class**
- Prediction Confidence**
- Important Features**
- Model Metrics**
- Classical vs QML Comparison**

This gives the user a more complete understanding of the experiment.

7.19 Dashboard Layer

The dashboard acts as the main user interface.

It can contain the following sections:

Dataset Dashboard

- **Dataset name**
- **Number of samples**
- **Number of features**
- **Class distribution**
- **Data quality**

Preprocessing Dashboard

- **Missing values**
- **Scaling**
- **Selected features**
- **Reduced dimensions**

Model Dashboard

- **Selected classical model**
- **Selected QML model**
- **Training status**
- **Model configuration**

Prediction Dashboard

- **Input sample**
- **Predicted class**
- **Prediction probability/confidence**

Benchmark Dashboard

- **Classical performance**
- **Hybrid QML performance**
- **Metric comparison**
- **Training/inference time**

Explainability Dashboard

- **Feature importance**
- **Selected features**
- **Prediction explanation**

7.20 Quantum Simulator / Hardware Layer

The quantum layer should be separated from the rest of the platform so that the backend can use either:

Quantum Simulator

or

Compatible Near-Term Quantum Hardware

The architecture therefore becomes:

Hybrid QML Application

↓

Quantum Backend Interface

↙

↘

Simulator

Quantum Hardware

This makes the platform flexible and future-ready.

7.21 Backend Architecture

A backend API can manage communication between the dashboard and machine-learning modules.

A possible implementation is:

React / Next.js Frontend

↓

FastAPI Backend

↓

ML Service

↓

QML Service

↓

Quantum Simulator / Hardware

The backend can manage:

- **Dataset upload**
 - **Experiment configuration**
 - **Model training**
 - **Prediction requests**
 - **Evaluation results**
 - **Experiment history**
-

7.22 Storage Architecture

The platform can maintain different types of information.

Dataset Storage

Stores uploaded or permitted biomedical datasets.

Experiment Storage

Stores:

- **Model configuration**
- **Feature-selection settings**
- **Training parameters**
- **Dataset version**

Results Storage

Stores:

- **Predictions**
- **Evaluation metrics**
- **Benchmark results**
- **Explainability outputs**

A structured database such as PostgreSQL can be used for metadata and experiment records.

7.23 Security Layer

Because biomedical data may contain sensitive information, the platform should include appropriate security mechanisms.

Possible mechanisms include:

- **User authentication**
- **Role-based access control**
- **Secure API access**
- **Encrypted communication**
- **Secure data storage**
- **Access logging**
- **Dataset access restrictions**

For the SIH prototype, public or appropriately de-identified datasets can be used.

7.24 Complete Data Flow

The complete data flow is:

Biomedical Dataset



Data Ingestion



Validation



Preprocessing



Feature Engineering



Feature Selection



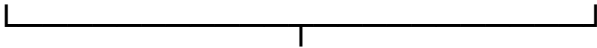
Dimensionality Reduction



Classical ML Quantum Encoding



Classical Output Quantum Output



7.25 Training Data Flow

During training:

Training Dataset



Preprocessing



Feature Selection



Dimensionality Reduction



Quantum Encoding



Quantum Circuit



Measurement



Classical Loss



Optimizer



Updated Parameters



Quantum Circuit



Repeat



Trained Hybrid Model

7.26 Inference Data Flow

During prediction:

New Biomedical Sample



Preprocessing



Feature Transformation



Feature Selection



Dimensionality Reduction



Quantum Encoding



Trained Hybrid Model



Prediction



Explainability



Dashboard

7.27 System Architecture Summary

The proposed architecture combines the strengths of both computing paradigms.

Classical Computing handles:

- **Data preparation**
- **Feature engineering**
- **Feature selection**
- **Dimensionality reduction**
- **Baseline models**
- **Optimization**
- **Evaluation**
- **Visualization**

Quantum Computing handles selected:

- **Quantum feature encoding**
- **Quantum kernels**
- **Variational circuits**
- **Quantum-enhanced learning operations**

The Hybrid Layer connects these components to create an end-to-end disease-detection workflow.

Core Architecture

Biomedical Data → Classical Processing → Quantum-Enhanced Learning → Disease Prediction → Explainability → Classical vs QML Benchmarking

This architecture directly addresses the problem statement by providing a fully functional hybrid quantum machine learning software platform for early disease detection while remaining compatible with quantum simulators and near-term quantum hardware.

8. Data Ingestion Module

The Data Ingestion Module is the first functional component of the proposed Hybrid Quantum Machine Learning Platform. Its purpose is to collect biomedical datasets and convert them into a standardized format that can be processed by the downstream machine-learning and quantum machine-learning pipelines.

Since the proposed platform can be applied to different diseases and biomedical data types, the ingestion module should be designed to support different datasets while maintaining a consistent internal data structure.

8.1 Purpose

The main objectives of the Data Ingestion Module are to:

- Import biomedical datasets.
- Validate the uploaded data.
- Identify features and target labels.
- Detect missing and invalid values.
- Understand numerical and categorical variables.
- Check dataset quality.
- Generate a basic dataset summary.
- Prepare validated data for preprocessing.

The module ensures that only properly structured data proceeds to the machine-learning pipeline.

8.2 Supported Biomedical Data

The platform can work with different types of biomedical datasets, including:

- Genomics datasets
- Clinical datasets
- Electronic Health Record (EHR) datasets
- Medical imaging-derived feature datasets
- Disease-specific benchmark datasets

For the SIH prototype, structured benchmark datasets such as CSV-based datasets can be used because they are easier to process and reproduce.

8.3 Dataset Upload

The user can upload a dataset through the platform dashboard.

A typical workflow is:

User Uploads Dataset

↓

File Validation



Dataset Parsing



Schema Detection



Data Quality Analysis



Dataset Summary



Send to Preprocessing

The platform can display information such as:

- **Dataset name**
- **Number of records**
- **Number of features**
- **Feature names**
- **Data types**
- **Target column**
- **Missing values**
- **Duplicate records**
- **Class distribution**

8.4 Data Format Validation

Before loading the dataset into the ML pipeline, the system verifies that the input format is supported.

For example, for a CSV dataset, the system checks:

- **Whether the file can be read.**
- **Whether columns are properly separated.**
- **Whether column names are valid.**
- **Whether rows contain the expected number of values.**
- **Whether the target column exists.**

If the dataset fails validation, the system should provide a clear error message rather than continuing with unreliable data.

8.5 Schema Detection

The platform identifies the structure of the dataset automatically.

For each column, it can determine:

- Column name
- Data type
- Numerical/categorical status
- Missing-value count
- Unique-value count
- Potential feature/target role

For example:

Feature	Type	Missing Values	Role
Age	Numerical	0	Feature
Blood Pressure	Numerical	2	Feature
Gender	Categorical	0	Feature
Disease	Categorical	0	Target

This information is used to configure the preprocessing pipeline.

8.6 Target Variable Identification

The target variable represents the outcome that the model needs to predict.

For a disease-classification dataset, the target may represent:

- Disease positive
- Disease negative

or multiple disease classes.

The platform should allow the user to select the target column when it cannot be automatically identified.

This prevents the system from incorrectly treating a feature as the prediction target.

8.7 Numerical Feature Detection

Numerical biomedical measurements can include:

- **Age**
- **Blood pressure**
- **Laboratory measurements**
- **Genomic numerical values**
- **Other quantitative measurements**

The ingestion module identifies numerical columns so that the preprocessing system can later apply appropriate operations such as:

- **Scaling**
- **Normalization**
- **Outlier analysis**
- **Statistical transformation**

8.8 Categorical Feature Detection

Biomedical datasets may also contain categorical information.

Examples include:

- **Sex/gender categories**
- **Clinical categories**
- **Disease classes**
- **Other non-numerical attributes**

The system identifies these columns so that they can later be transformed into machine-learning-compatible numerical representations.

8.9 Missing-Value Analysis

Missing information is common in biomedical datasets.

The ingestion module should identify:

- Number of missing values
- Percentage of missing values
- Columns containing missing values
- Rows containing missing values

For example:

Feature	Missing Values	Missing %
Age	0	0%
Blood Pressure	5	1.2%
Biomarker A	12	2.8%

The ingestion module should detect missing values; the actual treatment of missing values belongs to the preprocessing stage.

8.10 Duplicate Detection

Duplicate records can affect model training and evaluation.

The platform can identify:

- Completely duplicated rows
- Potential duplicate records based on selected identifiers/features

The system should report the number of duplicates before preprocessing.

Duplicate removal can then be performed according to the selected preprocessing strategy.

8.11 Invalid Data Detection

The ingestion module can identify potentially invalid values.

Examples include:

- Incorrect data types
- Unexpected text in numerical fields
- Empty columns
- Invalid target labels

- Impossible formatting
- Inconsistent values

The system should flag these records for correction or appropriate preprocessing.

8.12 Class Distribution Analysis

For disease classification, the platform should examine the distribution of target classes.

For example:

Class	Sample s	Percentag e
Negative	650	65%
Positive	350	35%

This is important because highly imbalanced datasets can affect model performance.

The platform can display the class distribution before training so that appropriate evaluation and preprocessing strategies can be considered.

8.13 Dataset Quality Report

After ingestion, the platform can generate an automatic dataset-quality summary.

The report may contain:

Dataset Information

- Total samples
- Total features
- Dataset format

Data Quality

- Missing values
- Duplicate records
- Invalid values

Feature Information

- Numerical features
- Categorical features
- Unique values

Target Information

- Target variable
- Number of classes
- Class distribution

This provides the user with an overview before model training begins.

8.14 Data Validation Pipeline

The ingestion pipeline can follow:

Dataset Upload

↓

File Validation

↓

Schema Detection

↓

Feature/Target Identification

↓

Data-Type Validation

↓

Missing-Value Analysis

↓

Duplicate Detection

↓

Class Distribution



Dataset Quality Report



Validated Dataset



Preprocessing Module

8.15 Data Splitting Preparation

The ingestion module prepares the validated dataset for later splitting into:

- **Training data**
- **Validation data**
- **Test data**

The actual splitting should occur as part of the model-training/preprocessing pipeline.

A key principle is that the test data must remain unseen during model development to ensure reliable evaluation.

8.16 Preventing Data Leakage

The ingestion and preprocessing workflow must be designed to avoid data leakage.

For example, information from the test dataset should not be used to calculate:

- **Feature-selection criteria**
- **Scaling parameters**
- **Dimensionality-reduction parameters**
- **Model parameters**

The correct process is:

Training Data

→ Learn preprocessing parameters

→ Apply parameters to validation/test data

This ensures that evaluation represents performance on unseen data.

8.17 Privacy Considerations

Biomedical datasets may contain sensitive information.

The platform should therefore support appropriate handling practices such as:

- Use of public benchmark datasets for development.
- Use of de-identified data where appropriate.
- Controlled access to uploaded datasets.
- Secure storage.
- Authentication and authorization.
- Protection of sensitive information.

The SIH prototype can primarily demonstrate the platform using publicly available or appropriately de-identified datasets.

8.18 Dataset Versioning

For reproducible experiments, the platform can maintain information about the dataset version used for each experiment.

The system can record:

- Dataset name
- Dataset version
- Number of samples
- Number of features
- Preprocessing configuration
- Selected features
- Experiment date
- Model configuration

This allows different experiments to be reproduced and compared.

8.19 Integration with the ML Pipeline

After validation, the dataset is passed to the preprocessing module.

The complete transition is:

Raw Biomedical Dataset



Data Ingestion



Validation



Quality Analysis



Validated Dataset



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Classical ML / Hybrid QML

The ingestion module therefore acts as the entry point of the entire platform.

8.20 Example

Suppose the user uploads a cardiovascular disease dataset containing 5,000 patient records and 30 features.

The ingestion module may report:

- **Samples: 5,000**
- **Features: 30**
- **Numerical features: 24**
- **Categorical features: 6**
- **Missing values: detected**
- **Duplicate records: detected**
- **Target variable: Disease**
- **Classes: Positive / Negative**
- **Class distribution: displayed**

The validated dataset is then sent to the preprocessing module.

8.21 Expected Output

The Data Ingestion Module should produce:

1. Validated biomedical dataset.
2. Dataset schema.
3. Feature-type information.
4. Target-variable information.
5. Missing-value report.
6. Duplicate-data report.
7. Class-distribution report.
8. Dataset-quality summary.
9. Dataset metadata for experiment tracking.

8.22 Core Data Ingestion Workflow

Upload → Validate → Analyze → Summarize → Standardize → Preprocessing

The Data Ingestion Module ensures that the platform begins with cleanly understood and properly validated biomedical data, creating a reliable foundation for both classical machine learning and hybrid quantum machine learning.

9. Data Preprocessing

The Data Preprocessing Module converts the validated biomedical dataset into a clean, consistent, and machine-learning-ready format.

Biomedical datasets can contain missing values, different numerical scales, categorical variables, outliers, duplicate records, and noisy measurements. These

issues can negatively affect model training and can become especially important for quantum models because quantum circuits generally work with a limited number of input features.

Therefore, preprocessing is a critical stage of the proposed Hybrid Quantum Machine Learning Platform.

9.1 Objectives

The preprocessing module aims to:

- Clean raw biomedical data.
- Handle missing values.
- Remove or manage duplicate records.
- Convert categorical data into numerical form.
- Normalize or standardize numerical features.
- Identify and handle problematic outliers.
- Prepare the target variable.
- Separate input features from target labels.
- Split data into training, validation, and testing sets.
- Prevent data leakage.
- Produce a consistent dataset for downstream ML and QML models.

9.2 Preprocessing Workflow

The proposed workflow is:

Validated Dataset



Data Cleaning



Missing-Value Handling



Duplicate Handling



Categorical Encoding



Numerical Feature Scaling



Outlier Analysis



Target Preparation



Train/Validation/Test Split



Preprocessed Dataset



Feature Engineering



Feature Selection



Dimensionality Reduction



Classical ML / Hybrid QML

9.3 Data Cleaning

The first step is to identify and correct problems in the dataset.

The system can check for:

- **Empty rows**
- **Empty columns**

- Duplicate records
- Incorrect data types
- Invalid values
- Inconsistent representations
- Unexpected symbols or text
- Missing target labels

For example, the same category might appear as:

Male, male, M

These representations may need to be standardized before model training.

The platform should maintain a record of preprocessing operations so that experiments can be reproduced.

9.4 Missing-Value Handling

Biomedical datasets frequently contain missing observations.

For example:

Feature	Missing Values
Age	0
Blood Pressure	12
Cholesterol	8
Biomarker A	25

The platform should first determine the amount and pattern of missing data.

Possible strategies include:

Numerical Features

Depending on the dataset, missing numerical values may be handled using:

- Mean imputation
- Median imputation
- Model-based imputation

Median imputation can be useful when a feature contains significant outliers.

Categorical Features

Possible approaches include:

- Most-frequent category
- Explicit "Unknown" category

The selected method should be configurable rather than permanently fixed.

Important Principle

Missing values must be handled using information from the training data when learning preprocessing parameters. This prevents information from the validation or test set from leaking into the model-development process.

9.5 Duplicate Handling

Duplicate observations can cause the model to see effectively the same patient/sample multiple times.

The system should identify duplicate records and report them.

Depending on the dataset, duplicates may be:

- Removed
- Investigated
- Retained when they represent legitimate repeated observations

For patient-level biomedical data, special care is required because multiple records from the same patient may represent genuine longitudinal observations rather than duplicates.

9.6 Categorical Data Encoding

Most ML and QML models require numerical input.

Therefore, categorical variables must be converted into numerical representations.

For example:

Gende r	Encoded Value
Male	0
Femal e	1

For a feature with multiple categories, suitable encoding methods can be applied depending on the model and dataset.

Possible approaches include:

- **Label encoding**
- **One-hot encoding**
- **Other suitable numerical representations**

The encoding strategy should be fitted using training data and then consistently applied to validation and test data.

9.7 Numerical Feature Scaling

Biomedical features may have very different numerical ranges.

For example:

- **Age → 20–90**
- **Blood pressure → 80–200**
- **Biomarker concentration → 0.001–10**
- **Genomic measurements → potentially very different scales**

Without appropriate scaling, some features may dominate the learning process.

The platform can support methods such as:

Standardization

Transforming features based on their training-set mean and standard deviation.

Normalization

Mapping values into a defined numerical range where appropriate.

Scaling is particularly important before quantum feature encoding because quantum rotation gates typically map numerical feature values into circuit parameters.

9.8 Outlier Analysis

Biomedical measurements can contain extreme values.

For example, an unusually high measurement may represent:

- **A genuine medical observation**
- **Measurement error**
- **Data-entry error**
- **An unusual but important patient case**

Therefore, the system should not automatically remove every outlier.

Possible techniques for detecting outliers include:

- **Interquartile range (IQR)**
- **Z-score analysis**
- **Statistical distribution analysis**

The platform should identify potential outliers and allow an appropriate strategy to be selected based on the dataset.

9.9 Target Variable Preparation

The target variable represents the disease outcome.

For example:

0 → No Disease

1 → Disease

For multiclass problems, multiple labels may be present.

The preprocessing module ensures that:

- Target values are valid.
- Target labels are consistently encoded.
- Missing target values are handled appropriately.
- The target is separated from input features.

The target variable must not accidentally be included as an input feature.

9.10 Feature and Target Separation

After cleaning, the dataset is separated into:

Input Features (**X**)

and

Target Variable (**y**)

For example:

**X = [Age, Blood Pressure, Cholesterol, Biomarker 1,
Biomarker 2, ...]**

y = Disease Status

The model learns the relationship between **X** and **y**.

9.11 Training, Validation, and Test Split

The dataset should be separated into different subsets.

Training Set

Used to train the ML/QML model.

Validation Set

Used for:

- **Model selection**
- **Hyperparameter tuning**
- **Comparing configurations**

Test Set

Used only for final evaluation on unseen data.

A typical example could be:

- **70% Training**
- **15% Validation**
- **15% Testing**

The exact split can be configurable depending on dataset size and experimental requirements.

For smaller biomedical datasets, cross-validation may also be considered.

9.12 Preventing Data Leakage

Data leakage is one of the most important issues in biomedical machine learning.

The platform should ensure that information from the test set is not used during model development.

For example, scaling should follow:

Training Data

- **Calculate scaling parameters**
- **Apply transformation**
- **Validation/Test Data**

Similarly, feature selection and dimensionality-reduction parameters should be learned from the training data within the experimental pipeline.

This ensures that the final evaluation better represents performance on unseen data.

9.13 Handling Class Imbalance

Disease datasets may contain significantly more negative cases than positive cases.

For example:

Class	Samples
No Disease	900
Disease	100

A model could achieve high accuracy simply by predicting the majority class.

Therefore, the preprocessing and evaluation pipeline should identify class imbalance.

Possible strategies include:

- Class weighting
- Appropriate sampling methods
- Stratified data splitting
- Imbalance-aware evaluation metrics

The system should report class distribution rather than assuming that accuracy alone is sufficient.

9.14 Consistent Preprocessing for Classical and Quantum Models

One important feature of the proposed platform is that both model families should receive comparable input data.

The pipeline can be:

Raw Biomedical Data



Common Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



↙ **Classical ML**

↘ **Quantum/Hybrid ML**

This makes the comparison between classical and hybrid QML approaches more meaningful.

9.15 Preparing Data for Quantum Models

Quantum models typically cannot directly accept thousands of raw biomedical features on current small-scale quantum systems.

Therefore, preprocessing prepares the data for later quantum processing.

The workflow can be:

High-Dimensional Biomedical Data



Feature Selection



Dimensionality Reduction



Small Numerical Feature Vector



Quantum Feature Encoding



Quantum Circuit

This allows the platform to work within the practical limitations of current quantum simulators and near-term quantum hardware.

The reduction of features should not be considered a way to guarantee quantum advantage. It is primarily a practical requirement for implementing QML experiments.

9.16 Preprocessing Configuration

The platform should allow preprocessing configurations to be recorded for each experiment.

For example:

Parameter	Example
Missing-value strategy	Median
Categorical encoding	One-hot
Scaling	Standardization
Outlier strategy	IQR analysis

Feature selection	Mutual Information
Dimensionality reduction	PCA
Train/Validation/Test	70/15/15

This configuration can be stored along with the experiment results.

9.17 Preprocessing Pipeline Example

Consider a cardiovascular disease dataset.

Step 1: Raw Data

The dataset contains:

- **5,000 samples**
- **30 features**
- **Disease target**

Step 2: Missing Values

The system identifies missing values in several clinical features.

Step 3: Cleaning

Invalid and unnecessary records are handled.

Step 4: Encoding

Categorical clinical variables are converted into numerical representations.

Step 5: Scaling

Numerical features are standardized.

Step 6: Splitting

The dataset is divided into training, validation, and testing sets.

Step 7: Feature Selection

The most informative features are selected.

Step 8: Dimensionality Reduction

The feature space is reduced to a quantum-compatible size.

Step 9: Model Training

The processed data is sent to:

- **Classical ML models**
- **Quantum-enhanced models**

Step 10: Evaluation

Both approaches are evaluated using the same test data.

9.18 Output of the Preprocessing Module

The module should produce:

- 1. Cleaned dataset.**
 - 2. Encoded categorical features.**
 - 3. Scaled numerical features.**
 - 4. Processed target labels.**
 - 5. Training dataset.**
 - 6. Validation dataset.**
 - 7. Test dataset.**
 - 8. Preprocessing configuration.**
 - 9. Data-quality summary.**
 - 10. Quantum-ready feature representation after subsequent feature-reduction stages.**
-

9.19 Role in the Overall Platform

The Data Preprocessing Module forms the bridge between data ingestion and machine-learning/QML modeling.

The overall process becomes:

Biomedical Dataset

- **Data Ingestion**
- **Validation**
- **Preprocessing**
- **Feature Engineering**
- **Feature Selection**
- **Dimensionality Reduction**
- **Classical ML + Hybrid QML**
- **Disease Prediction**
- **Explainability**
- **Performance Evaluation**

Thus, preprocessing ensures that the models are trained using data that is clean, consistent, appropriately transformed, and suitable for fair classical-vs-quantum comparison.

10. Feature Engineering

The Feature Engineering Module transforms preprocessed biomedical data into meaningful and useful input features for disease detection.

Biomedical datasets often contain a large number of raw variables. Not every raw variable directly provides useful information to the machine-learning model. Feature engineering helps represent the underlying biomedical information in a form that is easier for classical ML and hybrid QML models to learn from.

For the proposed platform, feature engineering is performed after preprocessing and before feature selection and dimensionality reduction.

10.1 Objective

The main objectives of feature engineering are to:

- Convert raw biomedical measurements into meaningful features.
- Improve the representation of disease-related patterns.
- Create informative numerical variables.
- Reduce unnecessary complexity in the input representation.
- Prepare features for classical ML and QML models.
- Maintain consistency between classical and quantum experiments.
- Create reproducible feature-engineering pipelines.

The objective is not to artificially increase the number of features, but to create useful representations that may improve model learning.

10.2 Position in the Overall Pipeline

The feature-engineering stage fits into the following workflow:

Biomedical Dataset



Data Ingestion



Data Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Classical ML / Hybrid QML



Prediction



Explainability



Evaluation

Feature engineering therefore acts as an intermediate transformation layer between cleaned data and model-ready representations.

10.3 Types of Biomedical Features

The platform can work with different types of biomedical information.

Clinical Features

Examples include:

- **Age**
- **Blood pressure**
- **Heart rate**
- **Body measurements**
- **Laboratory values**
- **Clinical history variables**

Genomic Features

Examples include:

- **Gene expression values**
- **Genetic markers**
- **Mutation indicators**
- **Other numerical genomic measurements**

Medical Imaging Features

Raw medical images can be converted into numerical image-derived features using an appropriate preprocessing or feature-extraction pipeline.

Examples include:

- **Texture measurements**
- **Intensity statistics**
- **Shape-related measurements**
- **Region-based numerical descriptors**

For the SIH prototype, structured image-derived features can be used to keep the QML pipeline computationally practical.

EHR Features

Electronic health record data may contain:

- **Demographic information**
- **Previous diagnoses**
- **Laboratory results**
- **Medication-related information**
- **Clinical measurements**

The exact features depend on the selected dataset.

10.4 Statistical Feature Creation

The platform can generate additional statistical features from available biomedical measurements.

For example, if repeated measurements are available, the system may derive:

- **Mean**
- **Median**
- **Minimum**
- **Maximum**
- **Standard deviation**
- **Range**

For example, multiple blood-pressure measurements could be represented using summary statistics.

This can provide the model with information about both the patient's typical measurement and its variability.

However, the platform should only create such features when the dataset structure supports them.

10.5 Ratio and Difference Features

Some biomedical relationships can be represented through derived numerical features.

For example:

Difference Feature

Feature A - Feature B

Ratio Feature

Feature A / Feature B

These transformations can represent relationships that may not be obvious from the individual variables.

For example, when clinically meaningful variables are available, their relative relationship may provide additional predictive information.

The platform should validate such transformations to avoid invalid mathematical operations such as division by zero.

10.6 Interaction Features

Disease-related patterns may depend on combinations of multiple variables.

The platform can therefore support interaction features where appropriate.

For example:

Feature A × Feature B

This allows the model to represent relationships between variables rather than treating every feature independently.

However, generating a large number of interaction features can significantly increase dimensionality.

Therefore, interaction generation should be configurable and followed by feature selection.

10.7 Transformation of Skewed Features

Some biomedical variables may have highly skewed distributions.

Where appropriate, transformations such as logarithmic transformation can be considered.

For example:

$$X' = \log(1 + X)$$

Such transformations can reduce the influence of extreme numerical ranges and make the feature distribution more suitable for certain models.

The transformation should only be applied where it is mathematically valid and appropriate for the selected dataset.

10.8 Domain-Aware Feature Engineering

Feature engineering should be based on the characteristics of the biomedical dataset.

For example:

Cardiovascular Dataset

Potential features may include:

- Age-related measurements
- Blood-pressure relationships
- Heart-rate statistics
- Laboratory-value relationships

Cancer Dataset

Potential features may include:

- **Biomarker measurements**
- **Genomic indicators**
- **Tumor-related numerical characteristics**
- **Statistical summaries**

Neurological Dataset

Potential features may include:

- **Clinical measurements**
- **Imaging-derived numerical features**
- **Neurological assessment scores**

The exact features should depend on the dataset rather than being hard-coded for every disease.

10.9 Feature Engineering for Genomic Data

Genomic datasets can contain thousands of variables.

Directly passing all genomic variables to a quantum circuit is generally impractical on current small-scale quantum systems.

Therefore, the platform can use a staged approach:

Genomic Data

↓

Preprocessing

↓

Feature Engineering

↓

Feature Selection

↓

Dimensionality Reduction



Quantum-Compatible Feature Vector

This allows the system to preserve useful information while reducing the input dimensionality before quantum encoding.

10.10 Feature Engineering for Medical Imaging

Medical images usually contain much more information than a small quantum circuit can directly process.

Therefore, the platform can use an image-to-feature approach.

The conceptual workflow is:

Medical Image



Image Preprocessing



Numerical Feature Extraction



Feature Selection



Dimensionality Reduction



Classical / Quantum Model

Examples of image-derived information may include:

- **Intensity statistics**
- **Texture characteristics**

- Shape descriptors
- Region-level measurements

This allows imaging information to participate in the hybrid QML pipeline without requiring the complete raw image to be directly encoded into a small quantum circuit.

10.11 Feature Engineering for EHR Data

EHR datasets often contain a mixture of numerical and categorical information.

The platform can create suitable representations from:

- Clinical measurements
- Historical records
- Laboratory results
- Demographic variables
- Other structured medical information

When temporal information is available, features such as trends or summary statistics may be created, provided that the dataset structure allows them.

For example:

Current Measurement – Previous Measurement

can represent a change over time.

10.12 Feature Engineering and Data Leakage

Feature engineering must be performed carefully to avoid data leakage.

Suppose a feature is calculated using information from the complete dataset before train/test splitting. This can unintentionally expose information from the test set during training.

The safer workflow is:

Split Dataset

↓

Fit Feature Transformation on Training Data

↓

Apply Transformation to Validation/Test Data

This ensures that the test set remains unseen during model development.

For patient-level datasets, splitting should also respect patient identity where necessary so that records from the same patient do not unintentionally appear across training and testing sets.

10.13 Feature Scaling After Engineering

Newly generated features can have different numerical ranges.

For example:

- **Age → tens**
- **Biomarker → decimals**
- **Ratio → different scale**
- **Interaction feature → potentially much larger values**

Therefore, engineered features may need to be scaled during the preprocessing/model pipeline before they are passed to the learning algorithms.

For quantum models, appropriate numerical scaling is especially important before feature encoding.

10.14 Feature Engineering and Quantum Encoding

The final engineered feature vector must eventually be converted into parameters that can be encoded into a quantum circuit.

For example:

Biomedical Features

$[x_1, x_2, x_3, x_4]$

↓

Selected Features

$[x_1, x_3, x_4]$

↓

Scaled Features

$[x_1', x_3', x_4']$

↓

Quantum Feature Encoding

↓

Quantum Circuit

This creates the connection between classical biomedical feature processing and quantum machine learning.

10.15 Feature Selection After Engineering

Feature engineering may create additional variables.

Therefore, the platform should not send every engineered feature directly to the model.

The next stage performs feature selection using techniques such as:

- Correlation analysis
- Mutual information
- Recursive Feature Elimination
- Statistical methods
- Tree-based feature importance

The selected features are then passed to dimensionality reduction if required.

Thus:

Feature Engineering

→ **Creates potentially useful representations**

Feature Selection

→ **Removes less useful representations**

Dimensionality Reduction

→ **Produces a compact representation suitable for modeling**

10.16 Configurable Feature Engineering

Different biomedical datasets require different feature transformations.

Therefore, the platform should allow the user/researcher to configure feature-engineering operations.

Example configuration:

Operation	Status
Statistical features	Enabled
Ratio features	Enabled
Interaction features	Disabled
Log transformation	Enabled
Domain-specific features	Enabled

This makes the platform reusable across multiple disease datasets.

10.17 Feature Engineering Metadata

For reproducibility, the system should store:

- Original feature names
- Generated feature names
- Transformation type
- Transformation parameters
- Selected features
- Dataset version
- Experiment configuration

For example:

blood_pressure_ratio = systolic / diastolic

The transformation can be recorded as part of the experiment metadata.

10.18 Example Workflow

Consider a cardiovascular dataset containing:

- Age
- Systolic blood pressure
- Diastolic blood pressure
- Heart rate
- Cholesterol
- Glucose
- Disease status

The platform could perform:

Step 1 — Original Features

Age, SBP, DBP, Heart Rate, Cholesterol, Glucose

Step 2 — Derived Features

For example:

Pulse Pressure = SBP - DBP

Additional suitable transformations may be created depending on the dataset.

Step 3 — Scaling

The numerical features are transformed to appropriate ranges.

Step 4 — Feature Selection

The platform identifies the most informative features.

Step 5 — Dimensionality Reduction

If necessary, the selected features are reduced to a compact representation.

Step 6 — Model Input

The resulting representation is sent to:

- **Classical ML models**
- **Quantum-enhanced models**

Step 7 — Comparison

Both model families are evaluated using the same evaluation framework.

10.19 Expected Output

The Feature Engineering Module should produce:

- 1. Engineered feature dataset.**
 - 2. Feature transformation metadata.**
 - 3. New feature descriptions.**
 - 4. Feature statistics.**
 - 5. Configured feature-engineering pipeline.**
 - 6. Reproducible transformation parameters.**
 - 7. Model-ready feature representation.**
-

10.20 Role in the Proposed Hybrid QML Platform

Feature engineering is particularly important because the platform must bridge two different computing environments.

Classical computing can efficiently perform:

- **Data transformation**
- **Statistical calculations**
- **Feature generation**
- **Feature analysis**

Quantum computing can then be used for selected learning operations through:

- **Quantum kernels**
- **Variational quantum classifiers**
- **Quantum neural networks**

The overall concept is:

Raw Biomedical Information



Classical Feature Engineering



Feature Selection



Dimensionality Reduction



Quantum Feature Encoding



Quantum-Enhanced Learning

This approach keeps the system practical for current quantum simulators and near-term quantum devices.

10.21 Key Design Principle

The platform should use domain-relevant and experimentally justified feature engineering, rather than creating features only to increase model complexity.

Every transformation should be evaluated based on its effect on:

- Predictive performance
- Sensitivity
- Specificity
- Generalization
- Computational cost
- Quantum resource requirements

The ultimate goal is to determine whether the engineered representation helps the hybrid QML model perform effectively compared with classical baselines.

10.22 Summary

The Feature Engineering Module converts cleaned biomedical variables into informative representations that can be effectively used by both classical and quantum-enhanced models.

It provides the essential bridge between raw biomedical information and quantum-compatible machine-learning representations, while maintaining reproducibility, avoiding data leakage, and keeping the overall system practical for current quantum computing constraints.

11. Feature Selection

The Feature Selection Module identifies the most relevant and informative features from the engineered biomedical dataset while removing features that are redundant, irrelevant, or less useful for disease prediction.

Feature selection is particularly important in the proposed Hybrid Quantum Machine Learning Platform because biomedical datasets can be extremely high-dimensional, whereas current quantum simulators and near-term quantum hardware have limited resources.

Instead of sending every available feature to the quantum model, the platform selects a smaller and more informative subset.

11.1 Objective

The main objectives of feature selection are to:

- Identify disease-relevant features.

- Remove irrelevant variables.
- Reduce redundant information.
- Reduce model complexity.
- Improve training efficiency.
- Reduce the number of features that need quantum encoding.
- Reduce quantum circuit requirements.
- Improve model interpretability.
- Support fair comparison between classical and hybrid QML models.

The selected features should be determined using measurable criteria rather than arbitrarily choosing a fixed number of variables.

11.2 Position in the Overall Pipeline

Feature selection is performed after feature engineering.

The overall pipeline is:

Biomedical Data

↓

Preprocessing

↓

Feature Engineering

↓

Feature Selection

↓

Dimensionality Reduction

↓

Classical ML / Hybrid QML

↓

Disease Prediction



Explainability



Performance Evaluation

The selected features are then used by both the classical and quantum pipelines to create a fair experimental comparison.

11.3 Why Feature Selection Is Necessary

Biomedical datasets may contain:

- **Hundreds of clinical variables.**
- **Thousands of genomic variables.**
- **Large numbers of imaging-derived features.**
- **Correlated measurements.**
- **Redundant variables.**
- **Irrelevant variables.**
- **Noisy variables.**

For example, a genomic dataset may contain thousands of gene-related measurements while the quantum model may only be able to practically process a small number of input dimensions.

Therefore:

Thousands of Features



Feature Selection



Important Features



Dimensionality Reduction



Quantum-Compatible Representation

This reduces the burden on the quantum component.

11.4 Feature Selection vs Feature Engineering

These two stages have different purposes.

Feature Engineering

Creates or transforms features.

Example:

Feature A + Feature B → New Feature C

Feature Selection

Chooses which existing/generated features should be retained.

Example:

[A, B, C, D, E] → [A, C, E]

Therefore:

Feature Engineering = Create useful representations

Feature Selection = Keep the useful representations

11.5 Selection Methods

The platform can support multiple feature-selection approaches.

A. Correlation-Based Selection

Correlation analysis can identify features that have strong relationships with the target variable or strong relationships with each other.

Highly redundant features can be identified and potentially removed.

For example:

Feature A ↔ Feature B

If two features contain very similar information, retaining both may not provide significant additional value.

However, correlation alone should not be treated as proof that a feature is useless because disease relationships may be nonlinear.

B. Mutual Information

Mutual information can be used to estimate how much information a feature provides about the target.

A feature with higher mutual information may contain a stronger statistical relationship with the disease outcome.

Conceptually:

Feature → Information about Disease Outcome

Features can then be ranked according to their estimated relevance.

C. Statistical Feature Selection

Statistical tests can be used to determine whether differences or associations between features and target classes are meaningful.

Depending on the data type, suitable statistical methods can be considered.

The exact test should depend on:

- **Numerical/categorical feature type.**
 - **Target type.**
 - **Distributional assumptions.**
 - **Dataset characteristics.**
-

D. Recursive Feature Elimination

Recursive Feature Elimination (RFE) starts with a larger feature set and repeatedly removes less important features according to a selected estimator.

Conceptually:

All Features



Train Model



Rank Features



Remove Less Important Features



Retrain



Repeat



Selected Feature Subset

This can help identify a compact feature set.

E. Tree-Based Feature Importance

Tree-based models such as Random Forest can provide feature-importance estimates.

These scores can be used to rank candidate features.

For example:

Feature	Importance
Biomarker A	0.31
Age	0.22
Biomarker B	0.17
Feature D	0.04

The highest-ranked features can then be considered for the final feature set.

Feature importance should be interpreted carefully because importance scores can be affected by feature correlation and model characteristics.

11.6 Hybrid Feature Selection Strategy

Instead of depending on a single method, the platform can support a configurable strategy.

For example:

Step 1

Remove obviously unusable features.

↓

Step 2

Remove highly redundant features.

↓

Step 3

Rank remaining features using mutual information or another relevance method.

↓

Step 4

Optionally apply model-based selection such as RFE.



Step 5

Select the required number of features for the next stage.

This provides flexibility for different biomedical datasets.

11.7 Number of Selected Features

The number of selected features should depend on:

- **Dataset size.**
- **Model requirements.**
- **Feature importance.**
- **Desired quantum circuit size.**
- **Available qubits.**
- **Computational resources.**
- **Experimental configuration.**

The platform should therefore allow the user to test different feature counts.

For example:

- **Top 4 features**
- **Top 6 features**
- **Top 8 features**
- **Top 12 features**

The resulting models can then be compared experimentally.

The system should not claim that a particular number of features is universally optimal.

11.8 Feature Selection for Quantum Models

Feature selection becomes especially important for QML.

If a quantum model uses one qubit per encoded feature, then:

Number of Features $\approx$ Number of Required Qubits

For example, a simplified feature map might use:

$[x_1, x_2, x_3, x_4]$

with four input features corresponding to four qubits.

A dataset containing 1,000 features therefore cannot simply be mapped directly into a small four- or eight-qubit circuit.

Feature selection reduces this problem.

High-Dimensional Dataset

1,000 Features

↓

Feature Selection

50 Important Features

↓

Dimensionality Reduction

8 Components

↓

Quantum Encoding

8 Quantum Inputs

The exact relationship between features, encoding strategy, and qubits depends on the chosen quantum circuit architecture.

11.9 Feature Selection and Dimensionality Reduction

Feature selection and dimensionality reduction are related but different.

Feature Selection

Keeps original features.

Example:

[Age, BP, Cholesterol, Glucose, Biomarker A]

→

[Age, Cholesterol, Biomarker A]

Dimensionality Reduction

Creates a new lower-dimensional representation.

Example:

30 Features

→

5 Principal Components

Therefore, the proposed pipeline can use:

Feature Selection

↓

Dimensionality Reduction

when necessary.

This combination can be useful for high-dimensional biomedical data before quantum encoding.

11.10 Avoiding Data Leakage

Feature selection must be performed correctly to avoid leakage.

For example, if feature ranking is calculated using the entire dataset before splitting into training and test sets, information from the test set can influence the selected features.

The correct approach is:

Training Data

↓

Fit Feature Selection Method

↓

Select Features

↓

Apply Selected Features to Validation/Test Data

This ensures that the test set remains unseen during model development.

This is particularly important when reporting benchmark results.

11.11 Feature Selection and Class Imbalance

Feature-selection methods can also be affected by class imbalance.

For example:

90% No Disease

10% Disease

A feature may appear useful for predicting the majority class while being poor at identifying disease-positive cases.

Therefore, feature selection should be considered together with appropriate evaluation metrics such as:

- Recall
- Sensitivity
- Specificity

- Precision
- F1-score
- ROC-AUC

This is especially important for early disease detection, where missing a positive case can be more significant than simply maximizing overall accuracy.

11.12 Feature Selection for Different Disease Domains

The selected features will depend on the disease and dataset.

Cancer

Potentially informative features may include:

- Biomarkers
- Genomic measurements
- Imaging-derived characteristics
- Clinical variables

Cardiovascular Disease

Potential features may include:

- Age
- Blood pressure
- Heart rate
- Cholesterol
- Other clinical measurements

Neurological Conditions

Potential features may include:

- Clinical assessment variables
- Imaging-derived measurements
- Relevant biomarkers

The platform should therefore use a dataset-driven feature-selection process rather than assuming the same features are important for every disease.

11.13 Feature Ranking Dashboard

The platform can provide a visualization of feature importance or relevance.

For example:

Rank	Feature	Selection Score
1	Biomarker A	0.91
2	Age	0.84
3	Feature B	0.79
4	Cholesterol	0.72
5	Feature C	0.68

This allows the researcher to understand which variables are being used by the model pipeline.

The ranking can also support explainability.

11.14 Feature Selection Configuration

The platform can allow the researcher to configure:

- Selection method.
- Number of features.
- Relevance threshold.
- Redundancy threshold.
- Estimator for model-based selection.
- Random seed.
- Cross-validation strategy where applicable.

Example:

Configuration	Example
---------------	---------

Method	Mutual Information
Features selected	8
Redundancy filtering	Enabled
Random seed	42
Validation strategy	Stratified

The configuration should be stored with the experiment.

11.15 Reproducibility

Every feature-selection experiment should record:

- **Dataset version.**
- **Original number of features.**
- **Selection method.**
- **Selected feature names.**
- **Number of selected features.**
- **Selection parameters.**
- **Random seed where applicable.**
- **Model used for model-based selection.**

This makes experiments reproducible and allows different feature-selection strategies to be compared.

11.16 Example

Suppose a cancer dataset contains:

2,000 genomic features + 20 clinical features

Total:

2,020 features

Directly using all features in a quantum model is impractical for a small-scale quantum system.

The platform may perform:

2,020 Features



Remove Redundant/Low-Value Features



200 Candidate Features



Feature Ranking



Top 30 Features



Dimensionality Reduction



8 Compact Components



Quantum Feature Encoding



Hybrid QML Model

The exact numbers are only an example; the platform should determine suitable values experimentally for each dataset.

11.17 Expected Output

The Feature Selection Module should produce:

1. Selected feature dataset.
 2. Feature ranking.
 3. Feature relevance scores where applicable.
 4. Removed-feature information.
 5. Selected feature names.
 6. Number of selected features.
 7. Selection configuration.
 8. Feature-selection metadata.
 9. Quantum-ready candidate feature set.
-

11.18 Role in Classical vs Quantum Benchmarking

To make the comparison scientifically meaningful, the platform should maintain a consistent data-processing strategy.

For example:

Same Dataset

↓

Same Train/Test Split

↓

Same Feature Engineering

↓

Same Feature Selection

↓

↙ Classical ML

↘ Hybrid QML

This reduces the possibility that differences in performance are caused simply by using different input data.

At the same time, experiments can also test whether different feature-selection strategies affect the performance of classical and quantum models.

11.19 Key Design Principle

Feature selection should be treated as an experimental and data-driven optimization stage, not as a mechanism for guaranteeing better performance.

The selected features should be evaluated based on their effect on:

- Accuracy
- Sensitivity
- Specificity
- Precision
- F1-score
- ROC-AUC
- Generalization
- Training time
- Inference time
- Quantum resource requirements

The best configuration should be selected based on measured experimental results.

11.20 Summary

The Feature Selection Module reduces the complexity of biomedical data by identifying the most relevant features while removing unnecessary or redundant information.

It is especially important for the proposed hybrid QML platform because current quantum systems have limited computational resources.

The complete process is:

Biomedical Data

→ Preprocessing

- **Feature Engineering**
- **Feature Selection**
- **Dimensionality Reduction**
- **Quantum Feature Encoding**
- **Hybrid Quantum Model**
- **Disease Prediction**

The module therefore provides an important connection between high-dimensional biomedical datasets and practical quantum-enhanced machine learning, while maintaining fair benchmarking against classical ML models.

12. Dimensionality Reduction

The Dimensionality Reduction Module converts a high-dimensional biomedical feature set into a smaller and more manageable representation while attempting to preserve the information that is useful for disease prediction.

This stage is particularly important in the proposed Hybrid Quantum Machine Learning Platform because current quantum simulators and near-term quantum hardware have limited numbers of practical qubits and are sensitive to circuit complexity.

Instead of directly sending hundreds or thousands of biomedical features into a quantum circuit, the platform first creates a compact representation.

12.1 Objective

The main objectives of dimensionality reduction are to:

- **Reduce the number of input dimensions.**
- **Preserve important information from the original dataset.**
- **Reduce computational complexity.**
- **Make biomedical data more suitable for quantum processing.**
- **Reduce the number of quantum features that need to be encoded.**
- **Reduce quantum circuit requirements where applicable.**
- **Improve training efficiency.**

- Support experimentation with different feature dimensions.
- Reduce the risk of unnecessary model complexity.

Dimensionality reduction should be evaluated experimentally to determine how much information is retained and how it affects model performance.

12.2 Why Dimensionality Reduction Is Required

Biomedical datasets can contain a very large number of variables.

For example:

- Clinical datasets may contain dozens of variables.
- Imaging-derived datasets may contain hundreds or thousands of features.
- Genomic datasets can contain thousands of measurements.

Current quantum systems generally cannot efficiently encode arbitrary high-dimensional data directly into a small quantum circuit.

Therefore, the platform uses a staged approach:

High-Dimensional Biomedical Data

↓

Feature Engineering

↓

Feature Selection

↓

Dimensionality Reduction

↓

Compact Feature Representation

↓

Quantum Feature Encoding

↓

Quantum Model

This makes the quantum component more practical.

12.3 Feature Selection vs Dimensionality Reduction

These two processes should not be confused.

Feature Selection

Selects existing features.

Example:

1,000 Features

↓

100 Selected Features

The resulting features retain their original meaning.

Dimensionality Reduction

Transforms features into a smaller set of new representations.

Example:

100 Features

↓

8 Components

The resulting components may combine information from multiple original features.

Therefore:

Feature Selection = Select useful original variables

Dimensionality Reduction = Create a compact representation

Both can be used together in the proposed platform.

12.4 Position in the Architecture

The dimensionality-reduction stage is:

Data Ingestion



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



↙ **Classical ML**

↘ **Quantum/Hybrid ML**



Prediction



Explainability



Evaluation

This allows both classical and quantum models to work with a controlled representation of the biomedical data.

12.5 Principal Component Analysis (PCA)

A primary dimensionality-reduction technique that can be supported by the platform is Principal Component Analysis (PCA).

PCA transforms the original features into a smaller number of new variables called principal components.

Conceptually:

Original Features

$X_1, X_2, X_3, \dots, X_n$

↓

PCA

↓

Principal Components

$PC_1, PC_2, PC_3, \dots, PC_k$

where:

$k < n$

The principal components are constructed to capture important patterns of variation in the original feature space.

12.6 Example of PCA

Suppose the feature-selection stage produces:

50 biomedical features

The platform may apply PCA and produce:

8 principal components

The model then receives:

$[PC_1, PC_2, PC_3, \dots, PC_8]$

instead of all 50 features.

These eight components can then be passed to a quantum feature-encoding circuit.

The exact number of components should be determined experimentally.

12.7 Choosing the Number of Components

The platform should allow the number of retained components to be configured.

For example:

- 2 components
- 4 components
- 6 components
- 8 components
- 10 components

Another approach is to select the number of components based on a desired proportion of retained variance.

For example, the researcher may test a configuration that retains approximately a chosen percentage of the training-data variance.

However, retaining more variance does not automatically mean better disease-prediction performance.

The final choice should be evaluated using the model's validation performance.

12.8 Quantum Resource Considerations

Dimensionality reduction can directly influence quantum resource requirements.

For a simple encoding scheme:

Number of Input Features

can correspond approximately to

Number of Required Qubits

depending on the encoding architecture.

For example:

16 Features

↓

16 Input Parameters

↓

Potentially a circuit using approximately 16 qubits for a straightforward one-feature-per-qubit encoding.

Reducing this to:

4 Components

can significantly simplify the corresponding quantum circuit.

The exact qubit requirement depends on the encoding method and circuit design.

12.9 Quantum Feature Representation

After dimensionality reduction, the resulting vector may look like:

$[z_1, z_2, z_3, z_4]$

These values can then be scaled appropriately and encoded into a quantum circuit.

For example:

Reduced Feature Vector

$[z_1, z_2, z_3, z_4]$



Scaling

$[z_1', z_2', z_3', z_4']$



Quantum Encoding



Quantum State



Parameterized Quantum Circuit



Measurement

This creates the connection between classical dimensionality reduction and the quantum learning component.

12.10 Scaling Before Quantum Encoding

The output of dimensionality reduction may have numerical values that are not directly suitable for a particular quantum encoding scheme.

Therefore, the platform should apply appropriate scaling before encoding.

For example:

PCA Components



Scaling / Normalization



Quantum-Compatible Values

↓

Feature Encoding

The exact transformation depends on the selected quantum feature map.

12.11 Avoiding Data Leakage

Dimensionality reduction must be performed without using information from the test set during model development.

For PCA, the correct workflow is:

Training Data

↓

Fit PCA

↓

Learn Principal Components

↓

Transform Training Data

↓

Transform Validation/Test Data Using Same PCA

This prevents test-set information from influencing the learned representation.

The same principle applies to any data-driven dimensionality-reduction method.

12.12 Dimensionality Reduction and Fair Benchmarking

For a meaningful comparison between classical ML and hybrid QML, the platform should maintain a consistent data-processing strategy.

For example:

Same Dataset



Same Preprocessing



Same Feature Engineering



Same Feature Selection



Same Dimensionality Reduction



✓ **Classical ML**

✗ **Hybrid QML**

This helps isolate the effect of the model architecture rather than allowing different preprocessing pipelines to create an unfair comparison.

12.13 Testing Different Dimensions

The platform should support experiments using different reduced dimensions.

For example:

Reduced Dimensions	Classical ML	Hybrid QML
2	Evaluate	Evaluate
4	Evaluate	Evaluate
6	Evaluate	Evaluate
8	Evaluate	Evaluate
10	Evaluate	Evaluate

The system can then compare:

- **Accuracy**
- **Sensitivity**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Training time**
- **Inference time**
- **Quantum resource requirements**

This helps identify an experimentally suitable representation.

12.14 Trade-Off Between Information and Quantum Cost

Dimensionality reduction creates an important trade-off.

Too Many Dimensions

Advantages:

- **More original information may be retained.**

Disadvantages:

- **Larger quantum circuit.**
- **More qubits.**
- **Greater computational cost.**
- **Potentially deeper circuits.**
- **More difficult execution on near-term hardware.**

Too Few Dimensions

Advantages:

- **Smaller quantum circuit.**
- **Lower computational requirements.**
- **Easier simulator/hardware execution.**

Disadvantages:

- **Important disease-related information may be lost.**

- Prediction performance may decrease.

Therefore, the goal is not simply to minimize the number of dimensions.

The goal is:

Find a compact representation that provides a useful balance between predictive performance and quantum resource requirements.

12.15 Alternative Dimensionality-Reduction Methods

Although PCA is a practical baseline, the platform can be designed so that other suitable methods can be evaluated.

Possible approaches include:

- PCA
- Truncated SVD
- Autoencoder-based representations
- Other dataset-appropriate reduction techniques

The initial SIH prototype can prioritize PCA because it is relatively simple, reproducible, and suitable for structured numerical biomedical data.

Advanced techniques can be added later if experimental requirements justify them.

12.16 Example: Cancer Dataset

Suppose a cancer dataset contains:

2,000 genomic features

After preprocessing:

2,000 Features

↓

After feature selection:

100 Selected Features

↓

After PCA:

8 Principal Components

↓

After scaling:

8 Quantum-Compatible Values

↓

Quantum feature encoding:

8 Input Parameters

↓

Hybrid QML model

The purpose of this process is to make the high-dimensional biomedical problem feasible for a small-scale QML experiment.

The example does not imply that eight components will always be optimal. That must be determined experimentally.

12.17 Example: Cardiovascular Dataset

Suppose a cardiovascular dataset contains:

30 clinical features

The platform may perform:

30 Features

↓

15 Selected Features



6 Principal Components



6 Quantum-Compatible Values



Hybrid QML

The same processed representation can also be supplied to classical models for comparison.

12.18 Explainability Considerations

Dimensionality reduction can make direct interpretation of individual model inputs more difficult.

For example:

PC1

may contain information from multiple original biomedical features.

Therefore, the platform should retain the relationship between:

Original Features

and

Reduced Components

This metadata can later be used during explainability analysis.

For example, the system can report which original features contribute strongly to a particular principal component.

This helps researchers understand what information was retained during dimensionality reduction.

12.19 Experiment Tracking

For every dimensionality-reduction experiment, the platform should record:

- Original feature count.
- Selected feature count.
- Reduction technique.
- Number of components.
- Explained variance where applicable.
- Scaling method.
- Dataset version.
- Random seed where applicable.
- Model configuration.

This allows different experiments to be reproduced and compared.

12.20 Expected Output

The Dimensionality Reduction Module should produce:

1. Reduced feature matrix.
 2. Number of retained dimensions.
 3. Reduction-method configuration.
 4. Explained-variance information where applicable.
 5. Transformation parameters.
 6. Mapping/metadata between original features and reduced representation.
 7. Quantum-compatible feature vector.
 8. Experiment metadata.
-

12.21 Integration with Quantum Machine Learning

The complete quantum preparation workflow is:

Biomedical Data



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Scaling



Quantum Feature Encoding



Quantum Circuit



Measurement



Classical Optimization



Disease Prediction

This allows the platform to use quantum models without requiring the entire high-dimensional biomedical dataset to be directly processed by the quantum circuit.

12.22 Key Design Principle

Dimensionality reduction should be treated as a controlled experimental stage, not as a guarantee of improved model performance.

The platform should evaluate different reduced dimensions and determine their effect on:

- **Predictive accuracy.**
- **Sensitivity.**
- **Specificity.**
- **F1-score.**
- **ROC-AUC.**
- **Generalization.**
- **Training time.**
- **Inference time.**
- **Number of qubits.**
- **Circuit complexity.**

This provides an evidence-based way to determine whether a particular reduced representation is suitable for hybrid QML.

12.23 Summary

The Dimensionality Reduction Module addresses one of the major practical challenges in applying QML to biomedical data: high dimensionality versus limited quantum resources.

The proposed approach is:

High-Dimensional Biomedical Data

→ Feature Selection

→ Dimensionality Reduction

→ Compact Feature Representation

→ Quantum Feature Encoding

→ Hybrid Quantum Model

At the same time, the same reduced representation can be supplied to classical ML models, allowing the platform to perform a controlled and fair comparison.

The platform therefore does not assume that dimensionality reduction or quantum processing will automatically provide an advantage. Instead, it uses

controlled experiments to determine whether hybrid QML can provide useful performance while remaining computationally practical.

13. Classical Machine Learning Baselines

The Classical Machine Learning Baseline Module provides conventional machine-learning models that serve as the reference point for evaluating the proposed Hybrid Quantum Machine Learning approach.

The objective of the project is not simply to build a quantum model. The platform must determine whether quantum-enhanced learning provides a measurable benefit compared with established classical approaches.

Therefore, every hybrid QML experiment should be compared with one or more classical baseline models using the same dataset, preprocessing strategy, feature representation, and evaluation procedure wherever appropriate.

13.1 Purpose

The Classical ML Baseline Module is designed to:

- Provide reference models for disease prediction.
- Establish the performance of conventional ML.
- Enable fair comparison with hybrid QML.
- Evaluate accuracy, sensitivity, specificity, and other metrics.
- Measure training and inference efficiency.
- Identify whether the QML approach provides a measurable improvement.
- Provide a reliable fallback model when the quantum approach does not outperform classical methods.

The baseline models should be strong and appropriately configured rather than intentionally weak models.

13.2 Position in the Overall Architecture

The classical baseline pipeline is:

Biomedical Dataset

↓

Preprocessing

↓

Feature Engineering

↓

Feature Selection

↓

Dimensionality Reduction

↓

Classical ML Models

↓

Disease Prediction

↓

Performance Evaluation

The same processed data can also be passed to:

Hybrid Quantum ML Models

This creates the basis for a controlled comparison.

13.3 Why Classical Baselines Are Necessary

A QML model producing 90% accuracy does not automatically demonstrate an advantage.

For example:

Model	Accuracy
--------------	-----------------

Logistic Regression	91%
SVM	92%
Random Forest	93%
Hybrid QML	90%

In this example, the hybrid model does not outperform the classical approaches.

Therefore, the platform must answer:

Does the hybrid QML model provide a measurable advantage over suitable classical ML models for the selected biomedical dataset?

This is one of the central research questions of the proposed platform.

13.4 Selected Classical Models

The platform can support several classical ML algorithms.

The initial implementation can focus on:

1. Logistic Regression
2. Support Vector Machine
3. Random Forest
4. Gradient-Boosting-based models such as XGBoost
5. Neural Network

Different algorithms provide different types of decision boundaries and learning behavior.

13.5 Logistic Regression

Logistic Regression is a simple and interpretable classification model.

For binary disease prediction, it estimates the probability that a sample belongs to a particular class.

Conceptually:

Biomedical Features

→

Logistic Regression

→

Disease Probability

→

Disease Class

For example:

$$P(\text{Disease}) = 0.82$$

The system may classify the sample as positive based on the selected decision threshold.

Advantages

- Simple.
- Fast to train.
- Easy to interpret.
- Useful as a baseline.
- Suitable for binary classification.

Role in the Project

Logistic Regression provides a strong, simple reference against which more complex models can be compared.

13.6 Support Vector Machine

A Support Vector Machine (SVM) is particularly relevant because its quantum counterpart can be implemented through quantum kernel methods or quantum-enhanced SVM approaches.

The classical SVM attempts to identify a decision boundary that separates different classes.

For example:

Disease-Negative Samples

vs.

Disease-Positive Samples

The model finds a suitable boundary between them.

Advantages

- **Effective for classification.**
- **Can work well in high-dimensional feature spaces.**
- **Supports nonlinear kernels.**
- **Provides a useful comparison with quantum kernel methods.**

Role in the Project

Classical SVM is an important baseline because the platform may later compare:

Classical SVM

vs.

Quantum Kernel / QSVM Approach

This provides a direct and meaningful experimental comparison.

13.7 Random Forest

Random Forest combines multiple decision trees to produce a final prediction.

Conceptually:

Biomedical Features

↓

Multiple Decision Trees



Combined Prediction



Disease Classification

Advantages

- **Handles nonlinear relationships.**
- **Can work with mixed feature patterns.**
- **Provides feature-importance information.**
- **Generally robust to different data characteristics.**
- **Useful for benchmarking.**

Random Forest can therefore provide a different type of baseline compared with linear models such as Logistic Regression.

13.8 Gradient Boosting / XGBoost

Gradient-boosting methods build models sequentially, with later models attempting to improve the errors made by previous models.

A suitable implementation such as XGBoost can be included as a strong classical benchmark.

Advantages

- **Strong performance on structured/tabular datasets.**
- **Captures nonlinear relationships.**
- **Can model complex feature interactions.**
- **Widely useful for structured biomedical data.**

Because biomedical datasets may contain complex relationships between variables, a boosting-based model can provide a competitive classical reference.

13.9 Neural Network

A classical neural network can also be included as a baseline.

A simplified architecture is:

Input Features



Dense Layer



Hidden Layer



Output Layer



Disease Probability

Neural networks can learn nonlinear relationships between biomedical variables.

The classical neural network can also serve as a useful comparison because both classical and quantum neural models can use parameter optimization.

13.10 Model Training

Each baseline model is trained using the training dataset.

The general workflow is:

Training Data



Model Initialization



Model Training

↓

Validation

↓

Hyperparameter Tuning

↓

Final Model

↓

Test Evaluation

The test dataset must remain isolated until the final evaluation stage.

13.11 Hyperparameter Optimization

Different classical models have different hyperparameters.

Examples include:

SVM

- **Kernel type**
- **Regularization parameter**
- **Kernel parameters**

Random Forest

- **Number of trees**
- **Maximum tree depth**
- **Minimum samples per split**

Neural Network

- **Number of layers**
- **Number of neurons**
- **Learning rate**
- **Batch size**

- **Number of training epochs**

The platform can use a suitable validation strategy or cross-validation to select model configurations.

Hyperparameter optimization should be performed without using the final test set.

13.12 Same Data for Fair Comparison

A critical design principle is to use comparable data for classical and hybrid QML experiments.

The pipeline can be:

Same Biomedical Dataset

↓

Same Data Split

↓

Same Preprocessing

↓

Same Feature Engineering

↓

Same Feature Selection

↓

Same Dimensionality Reduction

↓

✓ **Classical ML**

✗ **Hybrid QML**

This ensures that the comparison focuses primarily on the model architecture.

13.13 Handling Class Imbalance

Disease datasets may have unequal class distributions.

For example:

Disease Negative = 90%

Disease Positive = 10%

A model predicting "negative" for almost every patient could achieve high accuracy while failing to identify disease-positive cases.

Therefore, the classical baseline module should support techniques such as:

- Class weighting.
- Stratified splitting.
- Appropriate sampling strategies where justified.
- Threshold analysis.

The same considerations should be applied when evaluating QML models.

13.14 Evaluation Metrics

The classical models should be evaluated using multiple metrics.

Accuracy

Measures the proportion of correct predictions.

Precision

Measures how many predicted positive cases are actually positive.

Recall / Sensitivity

Measures how many actual positive cases are correctly identified.

For early disease detection, sensitivity is particularly important because missed positive cases can be significant.

Specificity

Measures how effectively the model identifies negative cases.

F1-Score

Provides a balance between precision and recall.

ROC-AUC

Measures the model's ability to distinguish between classes across different classification thresholds.

The platform should not rely only on accuracy.

13.15 Confusion Matrix

The platform should generate a confusion matrix for each model.

For binary disease classification:

	Actual Negative	Actual Positive
Predicted Negative	TN	FN
Predicted Positive	FP	TP

Where:

- **TP = True Positive**
- **TN = True Negative**
- **FP = False Positive**
- **FN = False Negative**

This allows researchers to understand the types of errors made by each model.

13.16 Computational Performance

The platform should evaluate not only prediction quality but also computational efficiency.

Measurements can include:

- Training time.
- Inference time.
- Memory usage where practical.
- Number of model parameters where relevant.
- Quantum circuit resources for the QML side.

For classical models, training and inference can be measured directly on the selected computing environment.

This information can later be compared with the hybrid QML pipeline.

13.17 Generalization Testing

A model should perform well not only on the training data but also on unseen data.

The platform can compare:

Training Performance

vs.

Validation Performance

vs.

Test Performance

Large differences may indicate:

- Overfitting.

- Data leakage.
- Dataset limitations.
- Excessive model complexity.

Cross-validation can also be considered for suitable datasets.

13.18 Model Comparison Table

The platform can automatically generate a comparison table.

Example:

Model	Accuracy	Sensitivity	Specificity	F1	ROC-AUC	Training Time
Logistic Regression	—	—	—	—	—	—
SVM	—	—	—	—	—	—
Random Forest	—	—	—	—	—	—
XGBoost	—	—	—	—	—	—
Neural Network	—	—	—	—	—	—
Hybrid QML	—	—	—	—	—	—

The actual values should come from experiments rather than being predetermined.

13.19 Classical Baselines and Explainability

Classical models can also contribute to the explainability component.

For example:

Logistic Regression

Model coefficients can indicate the direction and relative influence of features.

Random Forest / Tree Models

Feature-importance methods can indicate which features contribute to predictions.

Other Models

Model-agnostic methods such as SHAP can be considered where appropriate.

These explanations can be presented alongside hybrid QML results.

13.20 Baseline Experiment Design

For each selected biomedical dataset, the platform can perform:

Experiment 1

Train Logistic Regression.

Experiment 2

Train SVM.

Experiment 3

Train Random Forest.

Experiment 4

Train XGBoost.

Experiment 5

Train Neural Network.

Experiment 6

Train Hybrid QML model.

The system then evaluates all models using the same evaluation framework.

13.21 Baseline Selection Strategy

The platform does not need to train every possible classical algorithm.

A practical SIH prototype can begin with:

- Logistic Regression
- SVM
- Random Forest

and optionally include:

- XGBoost
- Neural Network

This provides diversity across:

Linear Models → Kernel Models → Tree Models → Neural Models

The selected baselines can then be expanded depending on the dataset and computational resources.

13.22 Classical Baselines as a Research Control

The classical models act as a control group in the experiment.

The research question becomes:

Given the same biomedical dataset and comparable preprocessing,
how does the hybrid quantum model perform relative to established
classical approaches?

Possible outcomes include:

QML Better

The hybrid model performs better on one or more important metrics.

Classical Better

The classical models perform better.

Comparable

The two approaches provide similar predictive performance.

All three outcomes are scientifically useful.

The project should report the measured result rather than assuming that quantum models will always win.

13.23 Example

Consider a cardiovascular disease dataset.

After preprocessing and dimensionality reduction, the platform produces six model inputs.

These six inputs are supplied to:

- **Logistic Regression**
- **SVM**
- **Random Forest**
- **Hybrid QML model**

The platform records:

- **Accuracy**
- **Sensitivity**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Training time**
- **Inference time**

The results are displayed together.

This allows the researcher to determine whether the hybrid QML approach provides a meaningful improvement for that dataset.

13.24 Expected Output

The Classical ML Baseline Module should produce:

1. Trained classical models.
 2. Model predictions.
 3. Evaluation metrics.
 4. Confusion matrices.
 5. ROC curves where applicable.
 6. Training and inference measurements.
 7. Feature-importance information where supported.
 8. Model configuration.
 9. Experiment metadata.
 10. Benchmark results for comparison with hybrid QML.
-

13.25 Role in the Overall Platform

The classical baseline module is not a separate system from the quantum pipeline.

It is an essential part of the benchmarking framework.

The complete experimental structure is:

Biomedical Dataset

↓

Common Preprocessing

↓

Feature Engineering

↓

Feature Selection



Dimensionality Reduction



✓ **Classical ML Baselines**

↘ **Hybrid QML Models**



Prediction



Performance Evaluation



Classical vs Hybrid Comparison



Explainability & Results Dashboard

13.26 Key Design Principle

The platform should treat classical ML as a serious benchmark, not merely a comparison model.

A successful hybrid QML system should demonstrate its value through measured experiments.

The evaluation should answer:

- **Does QML improve predictive performance?**
- **Does it improve sensitivity?**
- **Does it improve specificity?**
- **Does it generalize better?**
- **What is the computational cost?**
- **What quantum resources are required?**
- **Is any observed improvement consistent across experiments?**

Only experimental evidence should be used to make conclusions about the usefulness of QML.

13.27 Summary

The Classical Machine Learning Baseline Module establishes the reference performance of conventional machine-learning approaches.

The proposed platform compares:

Logistic Regression

SVM

Random Forest

XGBoost / Gradient Boosting

Neural Network

against:

Hybrid Quantum Machine Learning Models

using a controlled and reproducible evaluation pipeline.

This comparison is fundamental to the SIH problem statement because the purpose of the platform is not merely to demonstrate quantum computing, but to evaluate whether hybrid quantum-classical learning can provide useful improvements for early disease detection compared with classical approaches while remaining practical on current quantum technology.

14. Quantum Machine Learning Models

The Quantum Machine Learning (QML) Module is the core quantum component of the proposed Hybrid Quantum Machine Learning Platform.

Its purpose is to integrate quantum computation into the disease-detection pipeline after classical preprocessing, feature engineering, feature selection, and dimensionality reduction.

The platform will evaluate quantum-enhanced models such as:

- Quantum Support Vector Machine / Quantum Kernel Methods
- Variational Quantum Classifier (VQC)
- Quantum Neural Network (QNN)

These models will be evaluated against classical machine-learning baselines using the same biomedical datasets and experimental framework.

The platform will not assume that quantum models are automatically better. Their effectiveness will be determined through experimental benchmarking.

14.1 Objective

The objectives of the QML module are to:

- Implement quantum-enhanced classification models.
 - Process compact biomedical feature representations using quantum circuits.
 - Explore quantum feature maps and parameterized circuits.
 - Integrate quantum models with classical optimization.
 - Support execution on quantum simulators.
 - Maintain compatibility with near-term quantum hardware where feasible.
 - Compare QML models with classical ML baselines.
 - Measure predictive and computational performance.
 - Evaluate whether quantum-enhanced approaches provide measurable benefits.
-

14.2 Why Quantum Machine Learning?

Biomedical datasets can contain complex relationships between variables.

For example:

Patient Features

- Age
- Biomarkers
- Laboratory measurements
- Genomic measurements
- Imaging-derived features

These variables may interact in ways that are difficult to capture efficiently using a simple model.

QML provides an alternative learning framework in which classical biomedical features are encoded into quantum states and processed through quantum circuits.

The conceptual workflow is:

Biomedical Features



Classical Preprocessing



Feature Selection



Dimensionality Reduction



Quantum Feature Encoding



Quantum Circuit



Measurement



Classical Optimization



14.3 Why a Hybrid Approach?

The proposed platform does not attempt to perform the entire machine-learning pipeline on a quantum computer.

Instead:

Classical Computing Handles

- Data ingestion
- Data cleaning
- Missing-value handling
- Feature engineering
- Feature selection
- Dimensionality reduction
- Data splitting
- Classical baseline models
- Optimization
- Evaluation
- Visualization

Quantum Computing Handles

- Quantum feature encoding
- Quantum kernels
- Parameterized quantum circuits
- Quantum state transformations
- Measurements used by QML models

This approach is more practical for current quantum technology.

14.4 QML Model 1 — Quantum Support Vector Machine / Quantum Kernel

A Quantum Support Vector Machine (QSVM) approach can use a quantum circuit to construct a similarity or kernel representation of the input data.

A simplified workflow is:

Input Feature Vector



Quantum Feature Map



Quantum State



Quantum Kernel Evaluation



Classical SVM



Disease Classification

The quantum circuit is used to calculate relationships between data points, while the classical SVM performs the final classification.

14.5 Quantum Feature Map

Suppose the reduced biomedical feature vector is:

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$$

A quantum feature map transforms these classical values into a quantum state.

Conceptually:

$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4$



Quantum Encoding

↓

$|\psi(\mathbf{x})\rangle$

The quantum state can then be processed by a circuit designed to capture relationships between features.

The exact feature map will depend on the experiment and available quantum resources.

14.6 Quantum Kernel

A quantum kernel measures a similarity between quantum-encoded data points.

Conceptually:

$K(\mathbf{x}_i, \mathbf{x}_j)$

represents the similarity between two feature vectors after quantum encoding.

The kernel matrix can then be supplied to a classical SVM.

This creates a natural hybrid structure:

Quantum Computer

→ Computes quantum feature relationships

Classical Computer

→ Performs SVM optimization and classification

This is particularly useful for comparing:

Classical SVM

vs.

Quantum Kernel SVM

using the same biomedical feature representation.

14.7 Advantages of Quantum Kernel Methods

Potential advantages include:

- Ability to use quantum feature spaces.
- Natural integration with classical SVM algorithms.
- Suitable for small-dimensional datasets.
- Relatively clear separation between quantum and classical components.
- Useful for benchmarking quantum feature representations.

However, these are potential advantages, not guaranteed improvements.

The platform must experimentally determine whether the quantum kernel improves prediction for the selected biomedical dataset.

14.8 QML Model 2 — Variational Quantum Classifier

The Variational Quantum Classifier (VQC) is another important QML model for classification.

A VQC combines:

1. Quantum feature encoding.
2. Parameterized quantum circuits.
3. Quantum measurements.
4. Classical optimization.

The workflow is:

Input Features

↓

Quantum Feature Map

↓

Parameterized Quantum Circuit



Measurement



Prediction



Loss Calculation



Classical Optimizer



Parameter Update



Repeat

This is one of the main hybrid quantum-classical learning approaches proposed for the platform.

14.9 Parameterized Quantum Circuit

A VQC contains trainable parameters.

For example:

$$\theta = [\theta_1, \theta_2, \theta_3, \dots]$$

The circuit contains quantum gates whose behavior depends on these parameters.

During training:

Initial Parameters

↓

Quantum Circuit

↓

Measurement

↓

Prediction

↓

Calculate Loss

↓

Classical Optimizer

↓

Updated Parameters

↓

Run Circuit Again

This loop continues until the selected training criterion is reached.

14.10 Quantum Neural Network

A Quantum Neural Network (QNN) applies neural-network-like learning concepts using parameterized quantum circuits.

A simplified QNN consists of:

Input Features

↓

Quantum Encoding

↓

Parameterized Quantum Layers

↓

Entanglement Layers

↓

Measurement

↓

Output

The parameters are optimized using a classical optimization algorithm.

A QNN can therefore be viewed as a hybrid model in which the trainable representation includes a quantum circuit.

14.11 Quantum Circuit Structure

A simplified quantum circuit used by the platform can contain:

1. Initialization

Qubits are initialized into a known state.

2. Feature Encoding

Classical biomedical values are encoded into the quantum system.

3. Parameterized Gates

Trainable quantum gates are applied.

4. Entanglement

Selected qubits are connected using entangling operations.

5. Measurement

The circuit is measured to obtain classical outputs.

Conceptually:

Input



Feature Map



Trainable Quantum Layers



Entanglement



Measurement



Classical Output

14.12 Quantum Feature Encoding

The quantum model requires classical data to be converted into a quantum representation.

A practical approach for the prototype is angle encoding.

For example:

$x_1 \rightarrow \text{Rotation Angle}$

$x_2 \rightarrow \text{Rotation Angle}$

$x_3 \rightarrow \text{Rotation Angle}$

$x_4 \rightarrow \text{Rotation Angle}$

The values may first be scaled into an appropriate range.

The actual encoding strategy can be configured depending on the model and quantum backend.

14.13 Entanglement

Quantum circuits can include entangling gates to create correlations between qubits.

For example:

Qubit 1

↕

Qubit 2

↕

Qubit 3

This allows the circuit to represent relationships between multiple encoded features.

The entanglement structure should be selected carefully because deeper or more complicated circuits can increase computational requirements and make execution on noisy hardware more difficult.

14.14 Measurement

Quantum circuits produce measurement results that are converted into classical values.

For example, the model may use expectation values such as:

$\langle Z \rangle$

or measurement probabilities.

These outputs can then be used to generate a disease-classification score.

Conceptually:

Quantum Circuit

↓

Measurement

↓

Expectation / Probability

↓

Classification Score

↓

Disease Prediction

14.15 Hybrid Training Loop

The most important characteristic of the QML module is the interaction between classical and quantum computation.

The training process is:

Step 1

Take preprocessed biomedical data.

Step 2

Encode the selected features into a quantum circuit.

Step 3

Execute the parameterized quantum circuit.

Step 4

Measure the quantum output.

Step 5

Calculate the prediction loss on the classical computer.

Step 6

Use a classical optimizer to update quantum parameters.

Step 7

Run the circuit again.

Step 8

Repeat until the training criterion is satisfied.

Therefore:

Classical Data

→ **Quantum Circuit**

→ **Measurement**

→ **Classical Loss**

→ **Classical Optimization**

→ **Updated Quantum Circuit**

This is the central hybrid learning mechanism.

14.16 Classical Optimization

Quantum circuit parameters can be optimized using classical optimization algorithms.

Possible optimizers include suitable gradient-based or gradient-free methods depending on the circuit and training setup.

The optimizer receives the model's loss information and updates the trainable parameters.

The platform should allow different optimization configurations to be evaluated experimentally.

14.17 Avoiding Excessive Circuit Depth

Current quantum hardware is affected by noise and limited coherence.

Therefore, the platform should attempt to keep circuits reasonably compact.

The design can prioritize:

- Limited number of qubits.
- Shallow circuits.
- Efficient feature encoding.
- Controlled entanglement.
- Limited number of trainable parameters.

This makes the models more suitable for near-term quantum devices.

14.18 Quantum Simulator Support

The QML module should initially operate using quantum simulators.

This provides several advantages:

- No dependency on physical quantum hardware.
- Easier debugging.
- Reproducible experiments.
- Ability to test different circuits.
- Lower experimental cost.
- Faster development.

The simulator can provide an environment for testing quantum kernels, VQC, and QNN models before attempting execution on physical hardware.

14.19 Near-Term Hardware Compatibility

The platform should use a backend abstraction so that the same QML workflow can potentially target:

Quantum Simulator

or

Near-Term Quantum Hardware

without redesigning the entire application.

Conceptually:

QML Model

↓

Quantum Backend Interface

↙

↘

Simulator

Quantum Hardware

This allows the platform to begin with simulation and later experiment with compatible quantum devices.

Actual hardware execution will depend on device availability, qubit count, noise, queue time, and circuit compatibility.

14.20 Model Selection

The platform can provide multiple QML model choices:

Model

Main Quantum Role

**Classical
Component**

Quantum Kernel / QSVM	Quantum similarity representation	SVM
VQC	Trainable quantum classifier	Optimizer
QNN	Quantum neural representation	Optimizer

The researcher can train multiple models and compare their results.

14.21 QML Hyperparameters

The platform should track relevant QML configuration parameters.

Examples include:

- Number of qubits.
- Number of quantum layers.
- Feature-map type.
- Encoding method.
- Entanglement structure.
- Number of shots.
- Circuit parameters.
- Optimizer.
- Learning rate where applicable.
- Number of training iterations.
- Classification threshold.

These values should be stored with the experiment.

14.22 QML Training Example

Suppose dimensionality reduction produces:

$[\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$

The VQC workflow may be:

Four Reduced Features



Scale Values



Encode into Four Qubits



Apply Parameterized Quantum Layers



Apply Entanglement



Measure



Calculate Disease Probability



Calculate Loss



Update Parameters



Repeat

After training, the optimized circuit can be used for inference on unseen samples.

14.23 Prediction

After training, the QML model receives an unseen feature vector.

The inference workflow is:

New Biomedical Sample



Same Preprocessing



Same Feature Engineering



Same Feature Selection



Same Dimensionality Reduction



Quantum Encoding



Trained Quantum Circuit



Measurement



Disease Prediction

The preprocessing transformations used during training must be applied consistently during inference.

14.24 QML and Explainability

Quantum models can be more difficult to interpret than simple classical models.

Therefore, the platform should not claim complete interpretability of the quantum circuit.

Instead, explainability can be approached at multiple levels:

Input-Level Analysis

Identify which original biomedical features contribute to the reduced representation.

Feature-Level Analysis

Analyze the relationship between selected features and model outputs.

Model-Level Analysis

Record quantum circuit structure, feature map, parameters, and measurements.

Comparative Analysis

Compare predictions and feature effects with classical models.

The platform can therefore provide transparent experimental information without overclaiming that every quantum decision can be fully explained.

14.25 QML Performance Evaluation

Each QML model should be evaluated using the same metrics used for classical models.

These may include:

- **Accuracy**
- **Precision**
- **Recall**
- **Sensitivity**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Training time**
- **Inference time**

- Generalization performance

Additional quantum-specific measurements can include:

- Number of qubits
 - Circuit depth
 - Number of quantum gates
 - Number of circuit executions/shots
 - Backend type
-

14.26 Classical vs QML Comparison

The platform should automatically compare the models.

Example:

Model	Accuracy	Sensitivity	Specificity	F1	ROC-AUC
Logistic Regression	—	—	—	—	—
Classical SVM	—	—	—	—	—
Random Forest	—	—	—	—	—
Quantum Kernel	—	—	—	—	—
VQC	—	—	—	—	—
QNN	—	—	—	—	—

The actual values will be obtained from experiments.

14.27 Avoiding the Claim of Automatic Quantum Advantage

A major principle of the project is:

Quantum machine learning is being evaluated, not assumed to be superior.

A hybrid QML model may:

- **Outperform classical models.**
- **Match classical models.**
- **Perform worse than classical models.**

All of these outcomes are useful for understanding the applicability of QML to the selected biomedical problem.

The platform should therefore report experimental evidence rather than making an unsupported claim of quantum advantage.

14.28 Model Reproducibility

Every QML experiment should record:

- **Dataset version.**
- **Selected features.**
- **Number of dimensions.**
- **Quantum model type.**
- **Number of qubits.**
- **Feature-map configuration.**
- **Circuit architecture.**
- **Optimizer.**
- **Training iterations.**
- **Backend.**
- **Random seed where applicable.**
- **Evaluation metrics.**

This allows researchers to reproduce and compare experiments.

14.29 Expected Output

The Quantum Machine Learning Module should produce:

1. Trained quantum-enhanced models.
 2. Quantum circuit configurations.
 3. Disease predictions.
 4. Prediction probabilities/scores where supported.
 5. Training-loss information.
 6. Evaluation metrics.
 7. Quantum resource measurements.
 8. Simulator execution results.
 9. Hardware execution results where available.
 10. Experiment metadata.
 11. Comparison results against classical ML.
-

14.30 Role in the Complete Platform

The QML module receives the compact biomedical representation created by the classical pipeline.

The complete workflow is:

Biomedical Data

↓

Preprocessing

↓

Feature Engineering

↓

Feature Selection

↓

Dimensionality Reduction

↓

Quantum Feature Encoding

↓

Quantum Kernel / VQC / QNN



Measurement



Classical Optimization



Disease Prediction



Explainability



Performance Evaluation



Classical vs Hybrid QML Benchmark

14.31 Key Design Principle

The proposed QML module follows a hybrid-first, simulator-first, benchmark-driven approach.

It uses classical computing for tasks that are currently efficient on classical hardware and introduces quantum computation specifically into the machine-learning component.

The platform will experimentally evaluate whether quantum-enhanced models can provide improvements in disease-detection performance, efficiency, or generalization.

No quantum advantage will be assumed without measured evidence.

14.32 Summary

The Quantum Machine Learning Module provides the quantum-enhanced learning capability of the proposed platform.

It supports three major model families:

Quantum Kernel / QSVM

→ Quantum-based feature similarity + classical SVM

Variational Quantum Classifier

→ Parameterized quantum circuit + classical optimization

Quantum Neural Network

→ Quantum neural representation + classical optimization

These models operate on compact, carefully prepared biomedical feature representations and can initially run on quantum simulators before being tested on compatible near-term quantum hardware.

The resulting QML models are evaluated against strong classical baselines using common biomedical datasets and consistent evaluation procedures.

The ultimate purpose is to determine, through measurable experiments, whether hybrid quantum-classical machine learning can provide useful improvements for early disease detection while remaining practical, interpretable, scalable, and compatible with current quantum computing technology.

15. Quantum Feature Encoding

Quantum Feature Encoding is the process of converting classical biomedical data into a quantum representation that can be processed by a quantum machine-learning model.

In the proposed Hybrid Quantum Machine Learning Platform, the biomedical data is first processed using classical techniques. After feature selection and dimensionality reduction, the resulting numerical feature vector is converted into parameters of a quantum circuit.

The basic flow is:

Biomedical Data



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Scaling



Quantum Feature Encoding



Quantum Circuit



Measurement



Disease Prediction

This module forms the direct connection between the classical machine-learning pipeline and the quantum learning component.

15.1 Objective

The main objectives of quantum feature encoding are to:

- Convert classical biomedical features into quantum-compatible representations.
 - Map numerical features to quantum circuit parameters.
 - Efficiently use the available qubits.
 - Support different quantum feature-map strategies.
 - Maintain compatibility with quantum simulators.
 - Support execution on suitable near-term quantum hardware.
 - Preserve useful information during encoding.
 - Provide consistent input representation across QML experiments.
-

15.2 Why Feature Encoding Is Required

A quantum computer does not directly process a normal machine-learning feature vector in the same way as a classical ML algorithm.

For example, after dimensionality reduction, the platform may produce:

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$$

These values must be mapped into a quantum state.

The process becomes:

Classical Feature Vector

$$[\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$$

↓

Quantum Encoding

↓

Quantum State

↓

Quantum Circuit

Therefore, quantum feature encoding is an essential step in any QML pipeline.

15.3 Input to the Encoding Module

The encoding module receives a compact numerical feature vector from the dimensionality-reduction stage.

For example:

[0.24, -0.61, 0.83, 0.17]

Before encoding, the values may be transformed into a suitable range depending on the selected feature map.

The platform should ensure that the same transformation is used consistently during training and inference.

15.4 Feature Scaling Before Encoding

Raw biomedical values may have very different numerical ranges.

For example:

- Age → 20–90
- Blood pressure → 80–200
- Biomarker → 0.001–10

Directly using these values as quantum rotation parameters may not be appropriate.

Therefore:

Raw Features

↓

Scaling / Normalization

↓

Quantum-Compatible Values

↓

Feature Encoding

The scaling parameters must be learned from the training data and then applied to validation and test samples.

15.5 Angle Encoding

A practical encoding technique for the proposed prototype is angle encoding.

In angle encoding, classical feature values are converted into rotation angles for quantum gates.

For example:

$$x_1 \rightarrow RY(x_1)$$

$$x_2 \rightarrow RY(x_2)$$

$$x_3 \rightarrow RY(x_3)$$

$$x_4 \rightarrow RY(x_4)$$

A simplified circuit can therefore represent:

Feature 1 $\rightarrow$ Qubit 1

Feature 2 $\rightarrow$ Qubit 2

Feature 3 $\rightarrow$ Qubit 3

Feature 4 $\rightarrow$ Qubit 4

The exact rotation gate and scaling strategy can be configured based on the selected quantum model.

15.6 Simple Example

Suppose dimensionality reduction produces:

$X = [0.2, 0.5, 0.7, 0.3]$

The platform may map these values to quantum rotations:

$x_1 \rightarrow \text{Qubit 1}$

$x_2 \rightarrow \text{Qubit 2}$

$x_3 \rightarrow \text{Qubit 3}$

$x_4 \rightarrow \text{Qubit 4}$

The quantum circuit then processes the resulting state.

Conceptually:

$[0.2, 0.5, 0.7, 0.3]$

↓

Angle Encoding

↓

$|\psi(x)\rangle$

↓

Parameterized Quantum Circuit

↓

Measurement

15.7 Qubit Mapping

The platform needs to determine how classical features are mapped to available qubits.

For a simple one-feature-per-qubit design:

Feature	Qubit
---------	-------

Feature	Qubit
1	0

Feature	Qubit
2	1

Feature	Qubit
3	2

Feature	Qubit
4	3

Therefore:

4 Features → 4 Qubits

This relationship is only a simplified example. Other encoding architectures may use different relationships between features and qubits.

15.8 Why Dimensionality Reduction Comes Before Encoding

Suppose the original biomedical dataset contains:

1,000 features

Using one qubit for every feature would be impractical for a small quantum device.

The platform therefore performs:

1,000 Features

↓

Feature Selection

↓

100 Candidate Features



Dimensionality Reduction



8 Components



Quantum Encoding



8-Input Quantum Representation

This significantly reduces the quantum resource requirement.

15.9 Quantum Feature Map

A quantum feature map defines how classical input features are embedded into quantum states.

A feature map can contain:

- **Single-qubit rotation gates.**
- **Entangling gates.**
- **Multiple encoding layers.**
- **Feature-dependent parameters.**

Conceptually:

Classical Vector



Feature Map



Quantum State Representation

The feature map is an important experimental component because different feature maps may produce different learning behavior.

15.10 Entangling Feature Maps

A feature map may also include interactions between qubits.

For example:

Qubit 1



Qubit 2



Qubit 3

Entangling operations allow the circuit to represent relationships between encoded features.

This can be useful when disease-related patterns depend on combinations of biomedical variables.

However, additional entanglement can increase circuit complexity.

Therefore, the platform should balance:

Representation Capability

against

Circuit Complexity

15.11 Encoding for Quantum Kernel Models

For a quantum kernel approach, the feature vector is encoded into a quantum state.

For two samples:

$\mathbf{x}_i$

and

$\mathbf{x}$

the quantum feature map produces:

$|\psi(\mathbf{x}_i)\rangle$

and

$|\psi(\mathbf{x})\rangle$

A quantum kernel can then estimate their similarity.

Conceptually:

Sample A

→ **Quantum Encoding**

→ $|\psi(\mathbf{A})\rangle$

Sample B

→ **Quantum Encoding**

→ $|\psi(\mathbf{B})\rangle$

↓

Quantum Kernel

↓

Classical SVM

↓

Disease Prediction

15.12 Encoding for Variational Quantum Classifiers

For a VQC, encoding is followed by a trainable quantum circuit.

The workflow is:

Feature Vector



Quantum Feature Map



Parameterized Quantum Circuit



Measurement



Prediction



Loss



Classical Optimizer



Updated Parameters

The encoding itself may remain fixed while the parameters of the variational circuit are trained.

15.13 Encoding for Quantum Neural Networks

A QNN also requires the classical features to be mapped into a quantum state.

The workflow is:

Reduced Biomedical Features



Quantum Encoding



Trainable Quantum Layers



Entanglement



Measurement



Classical Output

The QNN parameters are optimized during training.

15.14 Encoding and Information Preservation

A major challenge is that dimensionality reduction and quantum encoding can alter the representation of the original biomedical data.

Therefore, the platform should evaluate whether the encoded representation retains sufficient information for disease prediction.

This can be tested experimentally by comparing:

- Different feature counts.
- Different dimensionality-reduction configurations.
- Different feature maps.
- Different encoding strategies.

The best configuration should be selected based on measured validation performance and resource requirements.

15.15 Different Encoding Strategies

Although angle encoding can be the primary prototype approach, the platform can be designed to support additional encoding methods when appropriate.

Potential approaches include:

Angle Encoding

Maps features to rotation angles.

Advantages:

- Relatively straightforward.
- Suitable for small numerical feature vectors.
- Practical for prototype experiments.

Amplitude Encoding

Attempts to encode a vector into amplitudes of a quantum state.

It can provide a compact representation in terms of qubit count, but state preparation can be more complex.

Other Feature Maps

Additional quantum feature maps can be evaluated depending on:

- Dataset size.
- Available qubits.
- Circuit depth.
- Backend limitations.
- Experimental requirements.

For the initial SIH prototype, the implementation can prioritize a practical and reproducible encoding method.

15.16 Encoding and Quantum Resource Requirements

Quantum feature encoding directly affects quantum resource usage.

Important factors include:

- Number of qubits.
- Number of gates.
- Circuit depth.
- Number of encoding layers.
- Number of measurements/shots.
- Entanglement structure.

For example:

More Features

→ More encoding operations

→ Potentially more qubits

→ Larger circuit

→ Greater computational cost

Therefore, feature selection and dimensionality reduction are critical preparation stages.

15.17 Circuit Depth

The encoding circuit should remain reasonably shallow where possible.

A simplified circuit may contain:

Feature Encoding Layer



Entanglement Layer



Trainable Layer



Measurement

Adding many repeated layers can increase circuit depth.

On noisy near-term hardware, deeper circuits can experience greater effects from noise and limited coherence.

Therefore, the platform should allow the number of layers to be configured and experimentally evaluated.

15.18 Simulator-Based Encoding

During development, quantum feature encoding can first be tested using a quantum simulator.

This allows the team to:

- **Verify feature-to-qubit mapping.**
- **Test circuit construction.**
- **Compare feature maps.**
- **Debug encoding errors.**
- **Evaluate different circuit depths.**
- **Reproduce experiments.**

After validation, compatible circuits can potentially be executed on near-term quantum hardware.

15.19 Near-Term Hardware Considerations

The encoding strategy should consider the limitations of current quantum devices.

Important constraints include:

- Limited qubit availability.
- Gate noise.
- Measurement noise.
- Circuit depth.
- Connectivity between physical qubits.
- Execution/queue time.

Therefore, the platform should avoid unnecessarily complicated feature maps.

The objective is to design an encoding process that is small enough to execute realistically while still representing useful biomedical information.

15.20 Training and Inference Consistency

The encoding process must be identical during training and inference.

For example:

Training

Raw Data

→ **Scaling**

→ **Feature Reduction**

→ **Quantum Encoding**

→ **Model Training**

Inference

New Data

- Same Scaling
- Same Feature Reduction
- Same Quantum Encoding
- Trained Model

The transformation parameters learned during training must be reused during inference.

This ensures that the model receives data in the representation it was trained on.

15.21 Avoiding Data Leakage

The encoding stage itself may not usually learn parameters, but the transformations before encoding can.

For example, scaling and dimensionality reduction must not be fitted using the test dataset.

Correct process:

Training Data

- Fit preprocessing
- Fit dimensionality reduction
- Encode

Validation/Test Data

- Apply learned transformations
- Encode

This maintains a clean experimental evaluation.

15.22 Encoding Metadata

For reproducibility, the platform should store:

- Encoding method.
- Feature-to-qubit mapping.
- Scaling method.
- Number of qubits.
- Quantum feature-map configuration.
- Number of encoding layers.
- Rotation gates.
- Entanglement structure.
- Circuit depth.
- Backend.
- Number of shots where applicable.

This information should be associated with each experiment.

15.23 Example: Cardiovascular Dataset

Suppose the platform starts with:

30 Clinical Features

After feature selection:

12 Features

After dimensionality reduction:

4 Components

The resulting vector is:

$[z_1, z_2, z_3, z_4]$

The platform can use:

$z_1 \rightarrow \text{Qubit 0}$

$z_2 \rightarrow \text{Qubit 1}$

$z_3 \rightarrow$ Qubit 2

$z_4 \rightarrow$ Qubit 3

Then:

Angle Encoding



Entanglement Layer



Variational Layer



Measurement



Disease Classification

The same four-dimensional representation can also be provided to a classical baseline model for comparison.

15.24 Example: Genomic Dataset

Suppose a genomic dataset contains:

5,000 features

The platform can perform:

5,000 Genomic Features



Feature Selection



Candidate Feature Set

↓

PCA

↓

8 Components

↓

Scaling

↓

Quantum Encoding

↓

Hybrid QML

This demonstrates how the classical preprocessing pipeline makes high-dimensional biomedical information compatible with a small-scale quantum experiment.

15.25 Encoding Quality Evaluation

The platform should evaluate different encoding configurations using measurable results.

Possible experiments include:

Configuration n	Qubit s	Circuit Depth	Accurac y	ROC-AU C
Encoding A	—	—	—	—
Encoding B	—	—	—	—
Encoding C	—	—	—	—

The actual values should be generated from experiments.

This allows the researchers to identify whether a particular encoding method provides a useful balance between predictive performance and quantum resource requirements.

15.26 Expected Output

The Quantum Feature Encoding Module should produce:

1. Quantum-compatible feature representation.
 2. Feature-to-qubit mapping.
 3. Quantum feature-map configuration.
 4. Quantum circuit representation.
 5. Encoding metadata.
 6. Number of qubits used.
 7. Circuit-depth information.
 8. Backend configuration.
 9. Encoded input for the selected QML model.
-

15.27 Role in the Complete System

Quantum feature encoding connects the classical and quantum parts of the platform:

Biomedical Data



Classical Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Scaling



Quantum Feature Encoding



Quantum Kernel / VQC / QNN



Measurement



Classical Optimization



Disease Prediction



Explainability



Performance Evaluation

15.28 Key Design Principle

The proposed platform should use resource-aware quantum feature encoding.

The objective is not to encode as many features as possible. Instead, the goal is to find a compact representation that:

- **Preserves useful disease-related information.**

- Uses a practical number of qubits.
- Maintains manageable circuit depth.
- Can run on simulators.
- Can potentially run on compatible near-term quantum hardware.
- Produces measurable predictive performance.

The effectiveness of an encoding strategy should therefore be determined through experimentation.

15.29 Summary

Quantum Feature Encoding transforms the compact classical biomedical feature vector into a quantum representation that can be processed by QML models.

The proposed workflow is:

High-Dimensional Biomedical Data

→ Feature Selection

→ Dimensionality Reduction

→ Scaling

→ Quantum Feature Encoding

→ Quantum Circuit

→ Measurement

→ Disease Prediction

A practical initial implementation can use angle encoding with compact feature vectors and shallow circuits.

The platform can then experimentally compare different encoding configurations while measuring both predictive performance and quantum resource requirements.

This ensures that the quantum component remains practical, reproducible, simulator-compatible, and suitable for experimentation on near-term quantum hardware, without assuming quantum advantage in advance.

16. Hybrid Quantum-Classical Training Workflow

The Hybrid Quantum-Classical Training Workflow is the core training mechanism of the proposed platform. It combines the strengths of classical computing with quantum machine learning to train disease-detection models.

Instead of attempting to perform the complete machine-learning process on a quantum computer, the platform divides the workload between classical computing resources and the quantum processing layer.

The basic concept is:

Classical Data Processing



Quantum Feature Encoding



Quantum Circuit



Measurement



Classical Loss Calculation



Classical Optimization



Updated Quantum Parameters



Repeat Until Convergence

This approach is particularly suitable for current quantum technology because near-term quantum devices have limited qubits, noise, and circuit-depth capabilities.

16.1 Objective

The main objectives of the hybrid training workflow are to:

- Combine classical ML and QML in a single training pipeline.
 - Use classical computing for data preparation and optimization.
 - Use quantum circuits for quantum-enhanced learning.
 - Train models such as VQC and QNN using a hybrid optimization loop.
 - Support quantum kernel-based approaches.
 - Work with quantum simulators.
 - Maintain compatibility with suitable near-term quantum hardware.
 - Compare hybrid models against classical ML baselines.
 - Track training performance and resource usage.
 - Provide reproducible experiments.
-

16.2 Why a Hybrid Training Approach?

Current quantum computers have several practical limitations:

- Limited number of usable qubits.
- Noise in quantum operations.
- Limited circuit depth.
- Measurement overhead.
- Hardware connectivity constraints.
- Limited availability of quantum processors.

At the same time, classical computers are highly effective at:

- Data preprocessing.
- Feature engineering.
- Feature selection.
- Dimensionality reduction.
- Optimization.
- Evaluation.
- Visualization.
- Data storage.

Therefore, the proposed platform follows the principle:

Use classical computing where it is currently strongest and quantum computing where it can provide a meaningful learning component.

16.3 Overall Training Pipeline

The complete workflow is:

Biomedical Dataset



Data Validation



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Train / Validation / Test Split



Quantum Feature Encoding



Quantum Model



Measurement



Prediction



Loss Calculation



Classical Optimizer



Parameter Update



Next Training Iteration



Validation



Final Model

16.4 Step 1: Biomedical Data Input

The training process starts with a biomedical dataset.

Possible datasets include:

- Cancer datasets.
- Cardiovascular datasets.
- Neurological datasets.
- Genomic datasets.
- Medical imaging-derived datasets.

- **Electronic health record datasets.**
- **Other benchmark biomedical datasets.**

Each dataset contains input features and a target variable representing the disease class or outcome.

Example:

Feature	Value
Age	56
Blood Pressure	148
Biomarker 1	0.72
Biomarker 2	1.31
Feature 5	0.41
Disease	1

16.5 Step 2: Classical Preprocessing

Before entering the quantum system, the data is processed classically.

This stage can include:

- **Missing-value handling.**
- **Duplicate removal.**
- **Categorical encoding.**
- **Feature scaling.**
- **Outlier analysis.**
- **Target preparation.**
- **Class-imbalance handling.**
- **Train/validation/test splitting.**

The objective is to produce clean and consistent numerical input.

16.6 Step 3: Feature Engineering

Domain-relevant features can then be generated from the available biomedical information.

For example, clinical data may be transformed into:

- Age-related features.
- Ratios.
- Differences.
- Statistical measurements.
- Biomarker combinations.
- Interaction features.

For genomic or imaging-derived data, appropriate numerical representations can be used.

Feature engineering should remain controlled because generating a very large number of unnecessary features can increase computational and quantum resource requirements.

16.7 Step 4: Feature Selection

The platform then identifies the most useful features.

Possible techniques include:

- Correlation-based selection.
- Mutual information.
- Statistical tests.
- Recursive Feature Elimination.
- Tree-based feature importance.

This step reduces irrelevant and redundant information.

It also helps make the quantum model more practical.

16.8 Step 5: Dimensionality Reduction

Biomedical datasets can contain hundreds or thousands of features.

Quantum circuits available for near-term experimentation cannot generally handle such large feature vectors directly.

Therefore, dimensionality reduction can be applied.

For example:

2,000 Features

↓

Feature Selection

↓

100 Important Features

↓

PCA

↓

8 Components

↓

Quantum Model

This produces a compact representation suitable for the available quantum resources.

16.9 Step 6: Quantum Feature Encoding

The reduced feature vector is converted into a quantum representation.

For example:

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$$

may be mapped to quantum rotations:

$$\mathbf{x}_1 \rightarrow \text{Qubit } 0$$

$$\mathbf{x}_2 \rightarrow \text{Qubit } 1$$

$$\mathbf{x}_3 \rightarrow \text{Qubit } 2$$

$$\mathbf{x}_4 \rightarrow \text{Qubit } 3$$

The resulting quantum state is then processed by the selected QML model.

16.10 Step 7: Quantum Circuit Execution

The encoded input enters the quantum circuit.

A simplified circuit structure is:

Input State



Feature Encoding



Parameterized Quantum Gates



Entanglement



Measurement

The trainable parameters of the circuit are represented by:

$$\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3, \dots, \theta]$$

These parameters are initially initialized and subsequently optimized during training.

16.11 Step 8: Quantum Measurement

Quantum computation does not directly return a normal classical prediction.

The circuit is measured to obtain classical information.

For example, the platform may calculate an expectation value:

$\langle Z \rangle$

or another measurement observable.

This measurement is converted into a value that can be used by the classical part of the training pipeline.

Conceptually:

Quantum Circuit

↓

Measurement

↓

Classical Output

16.12 Step 9: Prediction Calculation

The measurement result is transformed into a prediction.

For binary disease classification, the output can be interpreted as a probability or classification score.

For example:

Prediction Score = 0.82

The platform may interpret this as a high likelihood of the positive class according to the model's defined output mapping.

The exact interpretation depends on the model and calibration strategy.

16.13 Step 10: Loss Calculation

The predicted result is compared with the actual training label.

For example:

Actual Label: **1**

Predicted Score: **0.82**

A loss function measures the difference between the prediction and the actual target.

Conceptually:

Loss = L(prediction, actual label)

The purpose of the loss function is to tell the optimization process how well the current quantum model is performing.

16.14 Step 11: Classical Optimization

The loss value is processed by the classical optimization layer.

The optimizer determines how the quantum circuit parameters should change.

Conceptually:

$\theta_{\text{new}} = \theta_{\text{old}} - \text{learning_step}$

The exact optimization method depends on the selected algorithm.

Possible optimizers can include suitable gradient-based or gradient-free optimizers supported by the selected QML framework.

16.15 Step 12: Parameter Update

After optimization, the updated parameters are sent back to the quantum circuit.

The cycle becomes:

Parameters



Quantum Circuit



Measurement



Prediction



Loss



Classical Optimizer



Updated Parameters



Quantum Circuit

This process is repeated over multiple training iterations.

16.16 Complete Hybrid Training Loop

The core loop can be represented as:

Initialize model parameters θ



Load training batch



Preprocess features



Encode features into quantum circuit



Execute quantum circuit



Measure quantum output



Calculate prediction



Calculate loss



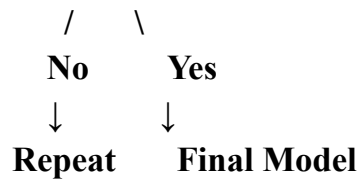
Classical optimizer calculates parameter update



Update θ



Check stopping condition



This is the central mechanism of the hybrid QML system.

16.17 Batch-Based Training

For larger datasets, the platform can process samples in batches.

For example:

Dataset

→ 1,000 samples



Batch 1

→ 32 samples



Batch 2

→ 32 samples



Batch 3

→ 32 samples

↓

...

Each batch can be processed through the quantum circuit and used to update the model parameters.

Batch size should be configurable because quantum execution can be significantly more expensive than ordinary classical computation.

16.18 Epoch-Based Training

The platform can organize training into epochs.

One epoch represents processing the training dataset according to the selected batching strategy.

Example:

Epoch 1

→ Training

→ Loss = 0.48

Epoch 2

→ Training

→ Loss = 0.37

Epoch 3

→ Training

→ Loss = 0.29

...

Training continues until a predefined stopping condition is reached.

16.19 Validation During Training

The validation dataset should not be used to update the model parameters.

Instead, it is used to monitor generalization.

For example:

Epoch	Training Loss	Validation Loss
1	0.48	0.51
2	0.37	0.42
3	0.29	0.35
4	0.24	0.34

If validation performance stops improving, the platform can use an appropriate stopping strategy.

This helps reduce overfitting.

16.20 Test Dataset

The test dataset should remain isolated until final evaluation.

The correct structure is:

Training Set

→ Model training

Validation Set

→ Model selection/tuning

Test Set

→ Final unbiased evaluation

The test set should not be repeatedly used to select the best model.

16.21 Hybrid Workflow for Quantum Kernel Models

Quantum kernel methods have a slightly different workflow.

The platform first performs quantum encoding for samples and estimates quantum similarities.

The process is:

Training Features



Quantum Feature Map



Quantum Kernel Evaluation



Kernel Matrix



Classical SVM



Disease Prediction

In this approach, the quantum circuit contributes to the kernel calculation while the final classifier remains classical.

16.22 Hybrid Workflow for VQC

For a Variational Quantum Classifier:

Input Features



Quantum Feature Map



Parameterized Quantum Circuit



Measurement



Prediction



Loss



Classical Optimizer



Parameter Update



Repeat

This creates a direct quantum-classical training loop.

16.23 Hybrid Workflow for QNN

For a Quantum Neural Network:

Input Features

↓

Quantum Encoding

↓

Quantum Neural Layers

↓

Entanglement

↓

Measurement

↓

Classical Output Layer

↓

Loss

↓

Optimization

↓

Parameter Update

↓

Repeat

The architecture can be configured according to the selected QNN design.

16.24 Gradient and Parameter Optimization

Training a variational quantum model requires estimating how changes in circuit parameters affect the output.

The platform can use suitable techniques supported by the chosen quantum framework.

Examples include:

- Parameter-shift based gradients.
- Automatic differentiation where supported.
- Gradient-free optimization.

The appropriate approach can depend on:

- Quantum simulator.
 - Quantum hardware.
 - Circuit architecture.
 - Noise level.
 - Number of parameters.
 - Computational cost.
-

16.25 Noise-Aware Training

When running on real quantum hardware, quantum noise can affect measurements.

Therefore, the platform should distinguish between:

Ideal Simulation

and

Noisy Simulation / Hardware Execution

This allows researchers to study how model performance changes under realistic quantum conditions.

For example:

Backend	Accuracy	ROC-AUC
Ideal Simulator	—	—

Noisy Simulator	—	—
Quantum Hardware	—	—

Actual values must come from experiments.

16.26 Simulator-First Training

The initial prototype should primarily use quantum simulators.

This provides a controlled environment for:

- Model development.
- Circuit debugging.
- Hyperparameter testing.
- Feature-map comparison.
- Reproducibility.
- Benchmarking.

After successful simulation experiments, selected configurations can be tested on compatible quantum hardware.

16.27 Hardware-Aware Training

When a near-term quantum backend is available, the platform should consider:

- Number of available qubits.
- Circuit depth.
- Gate connectivity.
- Noise.
- Measurement shots.
- Execution time.
- Backend availability.

The platform should therefore keep the quantum backend separate from the main ML pipeline.

Conceptually:

Hybrid QML Model



Backend Interface



Simulator Quantum Hardware

This allows the same experiment to be tested across different backends where technically compatible.

16.28 Hyperparameter Configuration

The training workflow should allow configuration of parameters such as:

- **Number of qubits.**
- **Number of quantum layers.**
- **Circuit depth.**
- **Learning rate where applicable.**
- **Optimizer.**
- **Batch size.**
- **Number of epochs.**
- **Encoding method.**
- **Entanglement pattern.**
- **Measurement strategy.**
- **Backend.**
- **Number of shots.**

These configurations should be recorded with every experiment.

16.29 Early Stopping

The training process should have a configurable stopping condition.

Possible conditions include:

- Maximum number of epochs reached.
- Validation performance stops improving.
- Loss reaches a target threshold.
- Parameter updates become sufficiently small.
- Resource/time limit reached.

This prevents unnecessary quantum circuit execution.

16.30 Experiment Tracking

Every training run should store important information such as:

- Dataset version.
- Preprocessing configuration.
- Selected features.
- Dimensionality-reduction configuration.
- Encoding method.
- Quantum model.
- Number of qubits.
- Circuit depth.
- Optimizer.
- Hyperparameters.
- Training loss.
- Validation performance.
- Test performance.
- Backend.
- Execution time.

This enables reproducible research.

16.31 Classical and Quantum Training Comparison

The platform should train classical and hybrid models using comparable data splits.

For example:

Same Dataset



Same Train/Test Split



Same Preprocessing



Classical Model Hybrid QML Model



Performance Comparison

This is essential for a scientifically meaningful evaluation.

16.32 Computational Efficiency

The platform should not evaluate only predictive accuracy.

It should also measure:

- **Training time.**
- **Inference time.**
- **Number of quantum circuit executions.**
- **Number of qubits.**
- **Circuit depth.**
- **Simulator resources.**
- **Hardware execution cost/time where measurable.**

A hybrid model that produces similar accuracy but requires substantially greater computational resources should be reported honestly.

16.33 Generalization Testing

The trained model should be evaluated on unseen data.

Where appropriate, the platform can also evaluate:

- Different random train/test splits.
- Cross-validation.
- Multiple benchmark datasets.
- Different feature dimensions.
- Different quantum backends.

This helps determine whether the observed performance is consistent rather than the result of one particular split.

16.34 Explainability Integration

The hybrid training workflow should provide information that can later support explainability.

For example:

- Selected input features.
- Feature transformations.
- Model configuration.
- Prediction score.
- Classical feature importance where applicable.
- Quantum model configuration.
- Experimental comparison.

The platform should clearly distinguish between model explanation and clinical interpretation.

16.35 Failure Handling

Quantum experiments may fail because of:

- **Backend unavailability.**
- **Circuit-size limitations.**
- **Unsupported operations.**
- **Execution timeout.**
- **Simulation memory limits.**
- **Invalid parameter configurations.**

The platform should provide clear error messages rather than silently producing incomplete results.

For example:

“Selected configuration requires more qubits than the selected backend provides.”

This makes the platform easier to use and debug.

16.36 Example End-to-End Training Scenario

Consider a cardiovascular dataset.

Input

1,000 patient records

↓

Preprocessing

Missing values handled and numerical features scaled.

↓

Feature Selection

40 relevant features selected

↓

Dimensionality Reduction

40 → 6 components



Quantum Encoding

6 components → quantum representation



VQC

Quantum circuit processes the encoded data.



Measurement

Produces prediction values.



Loss

Predictions compared with actual disease labels.



Classical Optimization

Quantum parameters updated.



Repeat

Training continues for multiple iterations.



Validation

Best model configuration identified.



Final Testing

Performance evaluated on the untouched test set.



Benchmark

Compared with:

- **Logistic Regression.**
- **SVM.**
- **Random Forest.**
- **XGBoost.**



Final Result

The platform reports predictive performance, computational cost, and quantum resource requirements.

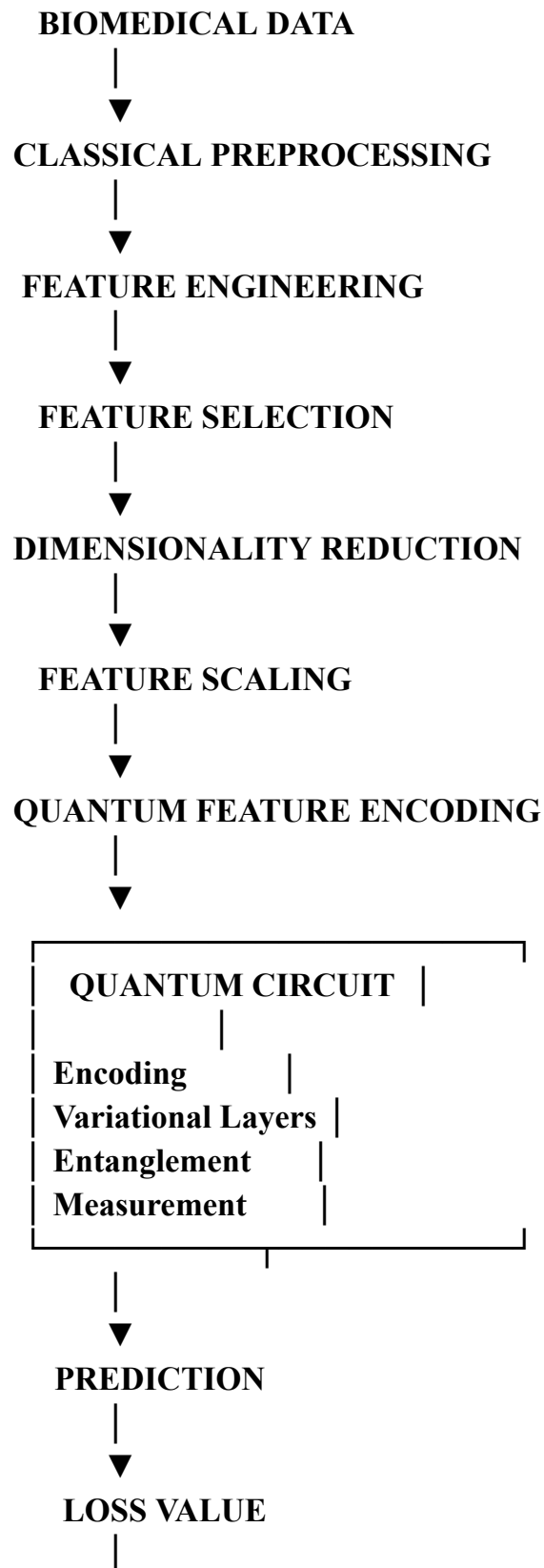
16.37 Expected Output of the Training Module

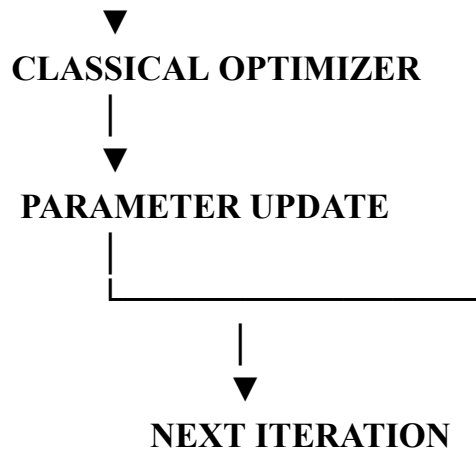
The Hybrid Training Module should produce:

- 1. Trained QML model.**
 - 2. Optimized quantum parameters.**
 - 3. Training and validation history.**
 - 4. Final prediction scores.**
 - 5. Performance metrics.**
 - 6. Quantum circuit configuration.**
 - 7. Backend information.**
 - 8. Resource usage information.**
 - 9. Experiment configuration.**
 - 10. Model artifact for inference.**
-

16.38 Core Hybrid Training Architecture

The complete mechanism can be summarized as:





16.39 Key Advantages of the Hybrid Workflow

1. Practical

It does not require the entire ML pipeline to run on quantum hardware.

2. Resource Efficient

Classical preprocessing reduces the amount of information entering the quantum circuit.

3. Flexible

Different QML models and quantum backends can be tested.

4. Benchmarkable

The same biomedical data can be evaluated using classical and hybrid approaches.

5. Simulator Compatible

The complete workflow can initially run using quantum simulators.

6. Hardware Ready

The architecture can later connect compatible quantum hardware.

7. Research Friendly

Different feature maps, models, dimensions, and optimizers can be experimentally compared.

16.40 Important Research Principle

The platform should not assume that the hybrid quantum model will always outperform classical ML.

Instead, the workflow should answer an experimental question:

Can a hybrid quantum-classical model provide useful predictive performance, sensitivity, specificity, generalization, or computational benefits for early disease detection compared with established classical models under comparable experimental conditions?

The final conclusion should be based on measured results.

16.41 Summary

The Hybrid Quantum-Classical Training Workflow is the central engine of the proposed platform.

It combines:

Classical Computing

for:

- **Data processing.**
- **Feature engineering.**
- **Feature selection.**
- **Dimensionality reduction.**
- **Loss calculation.**
- **Optimization.**
- **Evaluation.**

with:

Quantum Computing

for:

- Quantum feature encoding.
- Quantum feature maps.
- Quantum kernels.
- Parameterized quantum circuits.
- Quantum measurements.

The complete cycle is:

Biomedical Data → Classical Processing → Quantum Encoding → Quantum Circuit → Measurement → Prediction → Loss → Classical Optimization → Parameter Update → Repeat

This hybrid design makes the proposed solution suitable for current quantum simulators and potentially compatible with near-term quantum hardware while enabling a fair comparison against classical machine-learning approaches.

17. Disease Prediction

The Disease Prediction Module is responsible for using the trained machine-learning and hybrid quantum-classical models to classify unseen biomedical samples.

After preprocessing, feature engineering, feature selection, dimensionality reduction, and model training, the platform receives a new patient/sample record and passes it through the same transformation pipeline used during training.

The prediction workflow is:

New Biomedical Data



Data Validation



Preprocessing



Feature Transformation



Feature Selection



Dimensionality Reduction



Quantum Feature Encoding / Classical Representation



Trained Model



Prediction Score



Disease Classification



Explainability



Prediction Report

The module is designed for early disease detection research and decision support, not as a replacement for professional medical diagnosis.

17.1 Objective

The main objectives of the Disease Prediction Module are to:

- **Generate predictions from trained models.**
- **Support both classical and hybrid QML models.**

- **Process previously unseen biomedical samples.**
 - **Produce disease-class predictions.**
 - **Provide prediction scores or probabilities where supported.**
 - **Maintain consistency between training and inference.**
 - **Compare predictions from classical and hybrid models.**
 - **Provide interpretable prediction information.**
 - **Store prediction results for evaluation and experimentation.**
 - **Clearly communicate that model output is not a clinically validated diagnosis.**
-

17.2 Input to the Prediction Module

The module receives a biomedical sample containing the features required by the selected model.

For example, a cardiovascular dataset may contain:

Feature	Example Value
Age	57
Blood Pressure	145
Cholesterol	220
Heart Rate	88
Biomarker 1	0.73
Biomarker 2	1.12

The target disease label is not available when making a real prediction.

The trained model uses the input features to estimate the corresponding class.

17.3 Training and Inference Must Use the Same Pipeline

A critical requirement is that the prediction data must undergo the same transformations used during training.

For example:

Training

Raw Data

- Missing-value handling
- Scaling
- Feature selection
- PCA
- Quantum encoding
- Model training

Prediction

New Data

- Same missing-value handling
- Same scaling parameters
- Same selected features
- Same PCA transformation
- Same quantum encoding
- Trained model

This prevents inconsistencies between training and inference.

17.4 Prediction Pipeline

The complete inference process can be represented as:

New Biomedical Sample



Data Validation



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Scaling

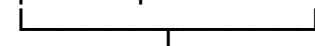


Model Type Selection



**Classical
Model**

**Hybrid QML
Model**



Prediction Score



Classification



17.5 Classical Model Prediction

For classical models such as:

- Logistic Regression.
- SVM.
- Random Forest.
- XGBoost.
- Neural Network.

the processed input is passed directly to the trained model.

Conceptually:

$X_{\text{new}} \rightarrow \text{Classical Model} \rightarrow \text{Prediction}$

For binary classification:

$\text{Prediction} \in \{0, 1\}$

Depending on the model, a probability or decision score may also be available.

17.6 Hybrid QML Prediction

For a hybrid QML model, the new sample follows the quantum inference pipeline.

The process is:

New Sample

↓

Classical Preprocessing

↓

Feature Selection

↓

Dimensionality Reduction

↓

Quantum Feature Encoding

↓

Quantum Circuit

↓

Measurement

↓

Prediction Score

↓

Disease Classification

The quantum circuit uses the trained parameters obtained during the hybrid training process.

17.7 Binary Disease Classification

The initial prototype can focus on binary classification because it provides a straightforward demonstration of early disease detection.

For example:

Class 0 → No detected disease pattern

Class 1 → Disease pattern detected

The exact meaning of the classes depends on the selected benchmark dataset.

The platform should obtain the class definitions from the dataset metadata rather than assuming that every dataset uses the same labels.

17.8 Prediction Score

Instead of displaying only:

Disease = Yes/No

the platform can provide a model output score where the selected model supports an interpretable probability-like output.

For example:

Prediction Score: 0.82

This can be presented as:

Predicted Class: Positive

However, the platform should clearly distinguish a model probability/score from a clinically validated probability of disease.

17.9 Classification Threshold

For models producing continuous scores, a classification threshold can be used.

Conceptually:

If score $\geq$ threshold

↓

Positive Class

If score $<$ threshold

↓
Negative Class

A default threshold such as 0.5 may be used where appropriate, but the platform should also support threshold selection based on validation data.

This is particularly important for medical screening because the preferred threshold may depend on the desired balance between sensitivity and specificity.

17.10 Sensitivity-Oriented Prediction

For early disease detection, missing a positive case can be particularly important.

Therefore, the platform should allow researchers to study different classification thresholds and their effects on:

- Sensitivity.
- Specificity.
- Precision.
- F1-score.
- False positives.
- False negatives.

This does not mean that the platform automatically chooses a clinically optimal threshold.

Instead, it provides an experimental framework for evaluating threshold behavior.

17.11 Confusion Matrix

Prediction results can be summarized using a confusion matrix.

For binary classification:

	Actual Positive	Actual Negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

Where:

- TP = True Positive.
- TN = True Negative.
- FP = False Positive.
- FN = False Negative.

These values are important for evaluating disease-detection models.

17.12 Sensitivity

Sensitivity measures how effectively the model identifies positive cases.

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

A higher sensitivity means fewer positive cases are missed.

This metric is particularly relevant when evaluating models intended for early disease screening.

17.13 Specificity

Specificity measures how effectively the model identifies negative cases.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

A higher specificity means fewer negative cases are incorrectly classified as positive.

17.14 Precision

Precision measures how many predicted positive cases are actually positive.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

This helps evaluate the number of false positive predictions.

17.15 F1-Score

The F1-score combines precision and recall/sensitivity.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

It can be useful when the dataset has class imbalance.

17.16 Accuracy

Accuracy represents the overall proportion of correctly classified samples.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

However, accuracy should not be used as the only metric, especially when disease classes are imbalanced.

17.17 ROC-AUC

The platform can also calculate ROC-AUC when suitable prediction scores are available.

ROC-AUC provides a threshold-independent evaluation of the model's ability to distinguish between the two classes.

It can be particularly useful when comparing classical and hybrid QML models.

17.18 Classical vs Hybrid Prediction

A major feature of the platform is that the same sample can be evaluated by multiple models.

For example:

New Sample

↓

├— **Logistic Regression**

├— **SVM**

├— **Random Forest**

├— **XGBoost**

└— **Hybrid QML**

↓

Prediction Comparison

This allows the researcher to determine whether the hybrid QML approach provides useful predictive performance relative to established classical methods.

17.19 Prediction Comparison Table

The dashboard can present results such as:

Model	Predicted Class	Score	Sensitivity	Specificity
Logistic Regression	—	—	—	—
SVM	—	—	—	—
Random Forest	—	—	—	—
XGBoost	—	—	—	—
Hybrid QML	—	—	—	—

The actual values should be generated from the trained models.

17.20 Model Selection

The platform may contain multiple trained models.

For example:

- Classical SVM.
- Random Forest.
- XGBoost.
- Quantum Kernel/SVM.
- VQC.
- QNN.

The user can select a model for inference.

Alternatively, the platform can identify the best-performing model based on a predefined validation criterion.

The selection criterion should be explicitly recorded.

17.21 Prediction Confidence

Where the underlying model provides an appropriate confidence or probability-like score, the platform can display it.

Example:

Prediction: Positive Class

Model Score: 0.84

The interface should avoid presenting this as:

“84% medically certain.”

Instead, it should be described as a model-generated score/probability estimate, depending on the model and calibration.

17.22 Model Calibration

Prediction scores are not automatically well-calibrated probabilities.

Therefore, if the platform presents probability-like outputs, calibration can be evaluated using suitable validation techniques.

This is important because:

High prediction score $\neq$ guaranteed high clinical probability

The platform should therefore report calibration information where appropriate.

17.23 Early Detection Use Case

The platform is designed to investigate whether machine-learning models can identify disease-related patterns at an early stage.

For example:

Patient/Sample Features

↓

Pretrained Model



Disease Risk Classification



Early Detection Signal

The output can then be reviewed by a qualified healthcare professional in a real-world setting after appropriate clinical validation.

The prototype itself should not claim to independently diagnose a patient.

17.24 Multi-Disease Extension

The architecture can be extended beyond a single disease.

For example:

Cancer

Input:

- Clinical features.
- Biomarkers.
- Genomic features.
- Imaging-derived features.

Output:

Cancer-related classification

Cardiovascular Disease

Input:

- Age.
- Blood pressure.
- Cholesterol.
- Heart-related measurements.

Output:

Cardiovascular disease classification

Neurological Conditions

Input:

- **Clinical features.**
- **Neurological measurements.**
- **Imaging-derived features.**

Output:

Neurological-condition classification

The exact datasets and labels determine the actual prediction task.

17.25 Multi-Class Prediction

Although binary classification is a practical starting point, the platform can support multi-class problems where the selected model allows it.

For example:

Class 0 → Condition A

Class 1 → Condition B

Class 2 → Condition C

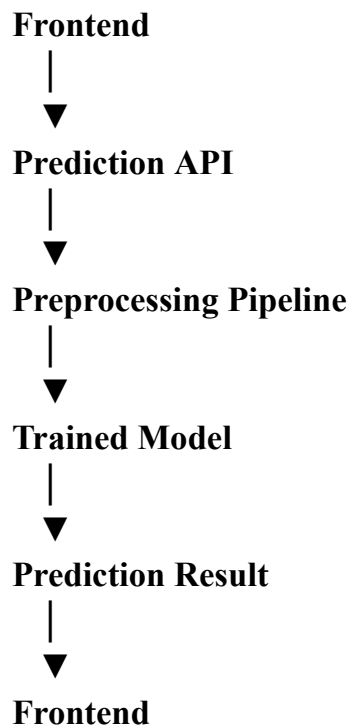
The prediction module can return:

- **Predicted class.**
 - **Class scores.**
 - **Evaluation metrics appropriate for multi-class classification.**
-

17.26 Prediction API

The prediction engine can be exposed through an API.

Conceptually:



A request can contain the required biomedical features and model identifier.

The response can contain:

- Model name.
- Predicted class.
- Prediction score.
- Model version.
- Timestamp.
- Explainability information where available.

17.27 Model Versioning

Every prediction should ideally be associated with a specific model version.

For example:

Model: VQC

Version: v1.2

Dataset: Cardiovascular Dataset v1

Features: PCA-6

Backend: Simulator

This makes experimental results reproducible.

17.28 Handling Invalid Input

The prediction module should validate incoming data before running inference.

Examples of invalid input include:

- **Missing required feature.**
- **Incorrect data type.**
- **Impossible value.**
- **Incorrect number of features.**
- **Unsupported categorical value.**
- **Incompatible model configuration.**

Instead of producing a potentially misleading result, the system should return a clear validation error.

Example:

Prediction unavailable: required feature "age" is missing.

17.29 Handling Distribution Changes

A new sample may have values very different from the training dataset.

The platform can monitor basic input distribution characteristics and flag unusual inputs.

For example:

Warning: input contains values outside the observed training range.

This does not automatically mean that the prediction is incorrect, but it alerts the user that the sample may differ from the data used to train the model.

17.30 Prediction Explainability

The prediction module should connect directly with the Explainable AI module.

After producing a prediction:

Prediction



Explanation

The explanation may identify:

- **Important classical features.**
- **Feature contributions where the selected explanation method supports them.**
- **Transformation information.**
- **Model used.**
- **Prediction score.**
- **Relevant experiment configuration.**

For hybrid QML models, explanations should be presented carefully because interpreting quantum circuit behavior is more complex than simply assigning direct clinical meaning to individual quantum operations.

17.31 Prediction Report

For every inference request, the platform can generate a structured result.

Example:

Prediction Report

Model: Hybrid QML - VQC

Model Version: 1.0

Dataset: Cardiovascular Benchmark

Features Used: 6

Backend: Quantum Simulator

Predicted Class: Positive

Model Score: 0.82

Evaluation:

Sensitivity: —

Specificity: —

ROC-AUC: —

Explanation:

**Top contributing processed features
and model information**

Note:

Research/decision-support output.

Not a clinically validated diagnosis.

**The evaluation metrics shown above should come from the appropriate
evaluation experiment rather than from a single unseen sample.**

17.32 Prediction Latency

The platform can also measure inference time.

Important measurements include:

- **Classical preprocessing time.**
- **Feature transformation time.**
- **Quantum circuit execution time.**
- **Measurement time where applicable.**
- **Total inference time.**

This is especially important when comparing simulator execution with real quantum hardware.

17.33 Prediction on Quantum Simulator

During development, prediction can be performed using a simulator.

The workflow is:

New Sample

→ Preprocessing

→ Quantum Encoding

→ Simulator

→ Measurement

→ Prediction

This enables repeatable testing without requiring continuous access to quantum hardware.

17.34 Prediction on Near-Term Quantum Hardware

When a compatible backend is available, the same model architecture can potentially be executed on quantum hardware.

The platform can compare:

Simulator Prediction

vs.

Hardware Prediction

This can reveal the effect of hardware noise and execution constraints.

The system should record the backend and configuration for every experiment.

17.35 Prediction Result Dashboard

The dashboard can display:

Patient/Sample Input

Selected features and processed representation.

Model

Selected classical or hybrid QML model.

Prediction

Predicted class and score.

Model Comparison

Results from multiple models.

Explainability

Available feature/model explanation.

Performance

Sensitivity, specificity, F1, ROC-AUC, and other experiment-level metrics.

Backend

Simulator or quantum hardware information.

17.36 End-to-End Prediction Example

Consider a new cardiovascular sample.

Step 1

User provides required clinical features.



Step 2

System validates the input.



Step 3

The same preprocessing pipeline used during training is applied.



Step 4

The selected features are extracted.



Step 5

The trained dimensionality-reduction transformation is applied.



Step 6

The resulting features are quantum encoded.



Step 7

The trained hybrid QML model executes.



Step 8

Quantum measurement produces the model output.

↓

Step 9

The output is converted into a classification score.

↓

Step 10

The platform displays the prediction and available explanation.

17.37 Expected Outputs

The Disease Prediction Module should provide:

- 1. Predicted disease class.**
 - 2. Prediction score/probability where supported.**
 - 3. Model identifier and version.**
 - 4. Preprocessing configuration.**
 - 5. Quantum backend information where applicable.**
 - 6. Explainability information.**
 - 7. Prediction timestamp.**
 - 8. Model comparison results where requested.**
 - 9. Input validation status.**
 - 10. Research/decision-support disclaimer.**
-

17.38 Important Medical Safety Principle

The platform should clearly distinguish between:

Machine-Learning Prediction

and

Clinical Diagnosis

The system is intended to demonstrate and evaluate a computational approach to early disease detection.

It should not claim:

- **Definitive diagnosis.**
- **Guaranteed disease detection.**
- **Guaranteed treatment recommendation.**
- **Clinical effectiveness without appropriate validation.**

A suitable prototype message is:

“This result is generated by an experimental machine-learning model and is intended for research/decision-support evaluation. It is not a clinically validated diagnosis.”

17.39 Key Design Principle

The Disease Prediction Module should prioritize:

Consistency + Transparency + Measurable Performance

The same preprocessing and feature-transformation pipeline must be applied during training and prediction.

The system should also show which model generated the prediction and provide the relevant model score and explanation.

Most importantly, the platform should evaluate whether hybrid QML actually provides an advantage rather than assuming that quantum models are automatically superior.

17.40 Summary

The Disease Prediction Module converts a new biomedical sample into a model-generated disease classification.

The complete workflow is:

New Biomedical Data

- **Validation**
- **Preprocessing**
- **Feature Engineering**
- **Feature Selection**
- **Dimensionality Reduction**
- **Quantum Encoding / Classical Representation**
- **Trained Model**
- **Prediction Score**
- **Disease Classification**
- **Explainability**
- **Prediction Report**

The module supports both classical ML and hybrid QML models, allowing direct comparison of their predictions and performance.

The final system is intended as a research and decision-support platform for evaluating early disease detection models, not as an independently validated clinical diagnostic system.

18. Explainable AI (XAI)

Explainable Artificial Intelligence (XAI) is a major component of the proposed Hybrid Quantum Machine Learning Platform. Its purpose is to make model predictions easier to understand by showing what information influenced the prediction and how the model reached its output.

In healthcare-related applications, a prediction alone is often insufficient. A researcher or healthcare professional needs to understand why a model classified a sample as belonging to a particular disease class.

The proposed platform therefore integrates explainability with both the classical preprocessing pipeline and the model prediction pipeline.

The overall concept is:

Biomedical Data



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Classical / Hybrid QML Model



Prediction



Explainability Layer



Feature Contribution + Model Information + Prediction Reasoning

The goal is not simply to produce a prediction, but to provide useful information about the factors associated with that prediction.

18.1 Objective of Explainable AI

The main objectives of the XAI module are to:

- Make model predictions easier to understand.
 - Identify important input features.
 - Show how features contribute to model outputs where supported.
 - Improve transparency of the ML pipeline.
 - Help researchers analyze model behavior.
 - Support comparison between classical and hybrid QML models.
 - Detect potentially misleading model behavior.
 - Help identify data or preprocessing issues.
 - Provide visual explanations through the platform dashboard.
 - Maintain a clear distinction between computational explanation and clinical interpretation.
-

18.2 Why Explainability Is Important in Disease Detection

Consider a model that produces:

Prediction: Positive

A user may naturally ask:

“Why did the model make this prediction?”

A useful XAI system can provide information such as:

- Which processed features were important.
- Whether those features increased or decreased the model output.
- Which features had relatively larger contributions.
- Which model generated the prediction.
- What preprocessing was applied.
- What model configuration was used.

This creates a more transparent workflow.

Without explainability:

Input → Prediction

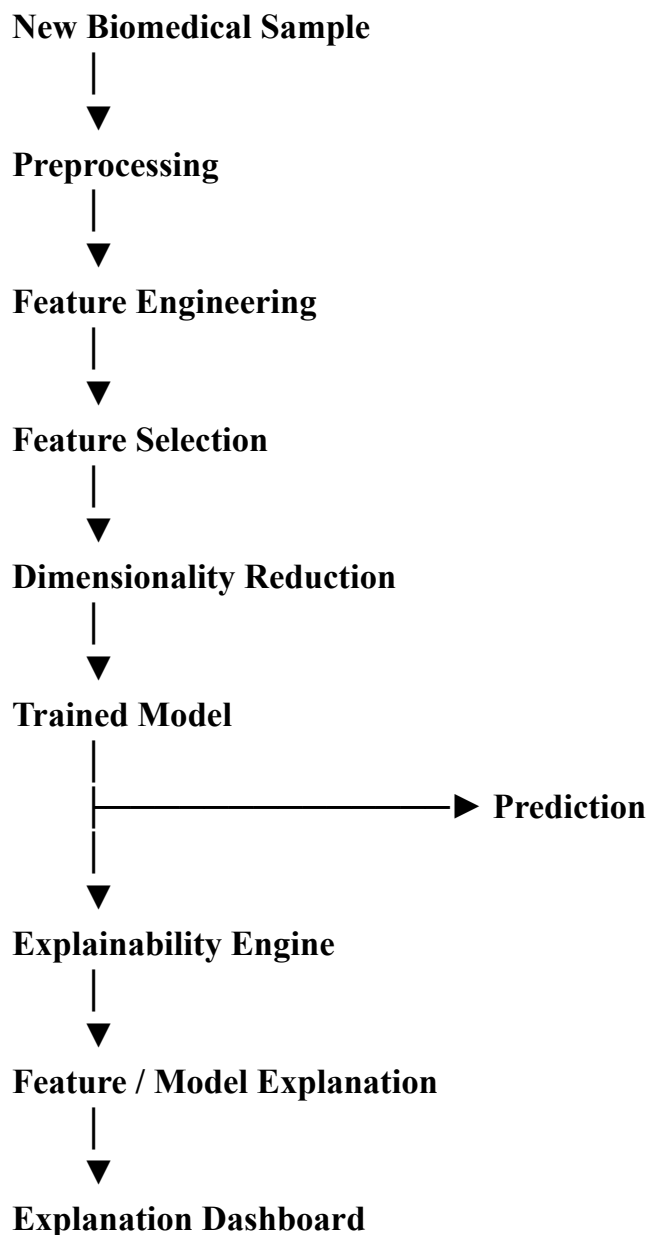
With explainability:

Input → Model → Prediction + Explanation

18.3 Explainability in the Proposed Platform

The XAI module is connected to the prediction pipeline.

The workflow is:



The explanation should always be associated with the specific model version and preprocessing configuration used to generate the prediction.

18.4 Types of Explainability

The platform can use two major types of explanation.

A. Global Explainability

Global explanations describe the overall behavior of the model across a dataset.

For example:

“Feature X was consistently important across the validation samples.”

Global explanations help researchers understand:

- Overall feature importance.
 - Model behavior.
 - Dataset-level patterns.
 - Differences between models.
-

B. Local Explainability

Local explanations describe a particular prediction.

For example:

“For this sample, the model output was strongly influenced by these processed features.”

Local explanations are useful for understanding individual model predictions.

The platform can therefore provide:

Global Explanation

→ How the model behaves overall.

Local Explanation

→ Why a particular sample received its prediction.

18.5 Feature Importance

One of the simplest forms of explainability is feature importance.

For a classical model, the platform can calculate or obtain feature importance using methods supported by the selected algorithm.

Example:

Feature	Relative Importance
---------	------------------------

Feature A	0.31
-----------	------

Feature B	0.24
-----------	------

Feature C	0.18
-----------	------

Feature D	0.15
-----------	------

Feature E	0.12
-----------	------

The dashboard can display these values using a ranking or visualization.

18.6 SHAP-Based Explainability

For suitable classical models, the platform can use SHAP (SHapley Additive exPlanations) to analyze feature contributions.

SHAP can provide both:

Global Explanation

Which features are generally important across the dataset.

Local Explanation

Which features influenced a particular prediction.

Conceptually:

Model Prediction



SHAP Analysis



Feature Contributions



Explanation

For example:

Prediction: Positive



The exact values should be generated from the trained model.

18.7 Positive and Negative Contributions

Where the selected explanation method supports directional contributions, the platform can show whether a feature moved the prediction toward or away from a particular class.

Conceptually:

Feature A → increases positive-class score
Feature B → increases positive-class score
Feature C → decreases positive-class score
Feature D → small contribution

This is more informative than simply showing feature importance.

18.8 Explainability After Dimensionality Reduction

A major challenge in the proposed platform is that quantum-compatible processing may involve dimensionality reduction.

For example:

50 Original Features

↓

PCA

↓

6 Principal Components

↓

Quantum Model

The quantum model may therefore operate on:

PC1, PC2, ..., PC6

rather than directly on the original clinical variables.

The platform should make this transformation visible.

Instead of presenting:

“PC1 caused the disease.”

the system should clearly explain that PC1 is a transformed representation of multiple original features.

18.9 Mapping Components Back to Original Features

Where possible, the platform can provide information about which original features contribute strongly to the transformed components.

For example:

Principal Component 1

- Biomarker A
- Biomarker B
- Age
- Clinical Feature D

This helps users understand the relationship between the original biomedical variables and the reduced representation.

However, this should be presented as an interpretation of the transformation, not as a direct causal explanation.

18.10 Explainability for Classical Models

Different classical algorithms provide different forms of interpretability.

Logistic Regression

Coefficient-based interpretation can indicate the direction and magnitude of feature effects within the model.

Decision Trees / Random Forest

Tree-based feature importance can identify influential features.

XGBoost

Feature importance and suitable post-hoc explanation methods can be used.

SVM

Direct interpretation is more limited, so post-hoc explanation methods may be used where appropriate.

Neural Networks

Post-hoc explanation methods can be used to investigate feature contributions.

The platform should use the explanation technique appropriate for the selected model rather than forcing one explanation method onto every model.

18.11 Explainability for Hybrid QML

Explainability becomes more challenging for quantum models.

A hybrid QML model may contain:

Classical Features

↓

Quantum Feature Map

↓

Parameterized Quantum Circuit

↓

Entanglement

↓

Measurement

↓

Classical Prediction

The model's internal quantum state is not directly equivalent to a conventional feature-weight system.

Therefore, the platform should avoid making unsupported statements such as:

“Qubit 2 directly represents disease severity.”

Instead, it should provide explanations based on experimentally supported model behavior.

18.12 Perturbation-Based Explanation for Hybrid Models

One practical approach is to study how the model output changes when input features are modified.

For example:

Original Input

→ Prediction = 0.82

Then modify Feature A:

Modified Input

→ Prediction = 0.69

This indicates that the model output changed when Feature A changed.

Repeated systematically across features, this can provide a model-behavior-based explanation.

Conceptually:

Feature A



Modify Feature A



Run Hybrid QML Model



Measure Prediction Change

The result can be used to rank feature influence.

18.13 Feature Perturbation

For a given sample:

$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$

the system can evaluate:

$\mathbf{X}_1 = [\mathbf{x}_1', \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4]$

Then compare:

$\text{Prediction}(\mathbf{X})$

with

$\text{Prediction}(\mathbf{X}_1)$

The difference provides an experimental measure of the model's sensitivity to that feature.

This method can be applied to selected processed features.

18.14 Quantum Circuit-Level Information

For hybrid QML models, the dashboard can also expose model configuration such as:

- Number of qubits.
- Feature map.

- Number of layers.
- Entanglement structure.
- Measurement observable.
- Trainable parameters.
- Backend.
- Circuit depth.

This does not automatically constitute a complete explanation of the prediction, but it improves transparency about how the quantum component is constructed.

18.15 Quantum Model Explanation vs Clinical Explanation

This distinction is extremely important.

The platform can explain:

Why the computational model produced a particular output.

It cannot automatically conclude:

Why the patient medically has the disease.

For example:

Model Explanation:

“The prediction was strongly influenced by the processed features associated with Features A, B, and C.”

This is different from:

Clinical Explanation:

“The patient has the disease because of Feature A.”

The second statement would require clinical evidence and validation beyond the ML model.

18.16 Explainability Dashboard

The platform can provide a dedicated XAI dashboard.

Possible sections include:

Prediction

Predicted Class: Positive

Model Score: 0.82

Top Influential Features

1. Feature A
2. Feature B
3. Feature C
4. Feature D

Contribution Visualization

Graph showing the relative contribution of features.

Transformation Information

- Features selected.
- PCA components used.
- Encoding method.

Quantum Model Information

- Qubits.
- Circuit depth.
- Feature map.
- Backend.

Model Comparison

Explanation differences between classical and hybrid models.

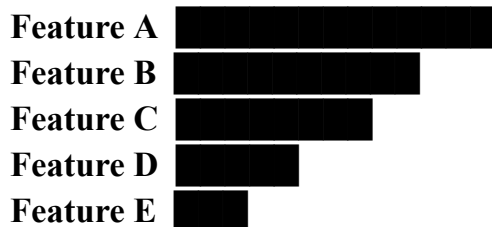
18.17 Global Explanation Dashboard

For dataset-level analysis, the platform can show:

- Overall feature importance.
- Feature contribution distributions.
- Important transformed components.
- Model comparison.
- Error patterns.
- Class-specific feature behavior.

For example:

Top Dataset-Level Features



This helps researchers understand the model beyond individual predictions.

18.18 Local Explanation Dashboard

For a single prediction:

Sample Prediction

Predicted Class: Positive
Model Score: 0.82

Important Processed Features

Feature A +++++++
Feature B +++++
Feature C +++
Feature D --

Model:
Hybrid QML - VQC

Qubits:
6

Backend:
Quantum Simulator

The exact explanation format depends on the selected model and explanation method.

18.19 Explainability and Model Debugging

XAI is also useful for detecting model problems.

Suppose the model claims high performance but consistently relies on an unexpected feature.

This may indicate:

- **Data leakage.**
- **Dataset bias.**
- **Incorrect preprocessing.**
- **Spurious correlation.**
- **An irrelevant feature dominating predictions.**

Therefore:

Explainability → Model Inspection → Potential Error Detection

This makes XAI useful not only for presentation but also for model development.

18.20 Explainability and Data Leakage

Consider a dataset containing a feature that indirectly reveals the target label.

A model may obtain very high accuracy by relying heavily on that feature.

Feature-importance analysis may reveal:

“Feature X dominates model predictions.”

This can trigger further investigation.

Therefore, explainability can complement the platform's data-leakage prevention mechanisms.

It does not replace formal leakage checks, but it provides an additional diagnostic layer.

18.21 Explainability and Bias

Biomedical datasets can contain sampling biases.

The platform can investigate whether model behavior differs across available subgroups when such metadata is legitimately available and appropriate.

For example, researchers may compare:

- **Prediction performance.**
- **Error rates.**
- **Feature importance.**

across relevant dataset-defined groups.

Any such analysis should be performed carefully and should not infer sensitive characteristics that are not present or appropriate for the dataset.

18.22 Explainability for Model Comparison

One important feature of the platform is comparing not only model performance but also model behavior.

For example:

Model	Accuracy	ROC-AUC	Top Feature
Logistic Regression	—	—	Feature A
Random Forest	—	—	Feature B
XGBoost	—	—	Feature A
Hybrid QML	—	—	Processed Feature C

The actual values should come from experiments.

This allows the research team to ask:

“Are the models making predictions based on similar information, or are they learning different patterns?”

18.23 Explainability and Error Analysis

The platform should also connect explanations with incorrect predictions.

For example:

False Positive

→ Model predicted positive

→ Actual class was negative

The system can examine which features influenced that prediction.

Similarly:

False Negative

→ Model predicted negative

→ Actual class was positive

The platform can inspect the corresponding feature contributions.

This can help researchers understand model weaknesses.

18.24 Explainability and Sensitivity

For disease detection, sensitivity is important.

If the model produces a false negative, researchers can inspect:

- Input features.
- Feature transformations.
- Prediction score.
- Model configuration.
- Explanation output.

This can help identify patterns in missed cases.

However, the explanation does not prove why the disease was missed medically; it only describes model behavior.

18.25 Explanation Consistency

The explanation system should be reproducible.

The platform should record:

- Model version.
- Dataset version.
- Preprocessing configuration.
- Selected features.
- Dimensionality-reduction method.
- Encoding method.
- Quantum circuit configuration.
- Explanation method.
- Random seed where applicable.

This allows the same experiment to be reproduced later.

18.26 Explainability Storage

Each experiment can store an explanation record containing:

Experiment ID

Model ID

Dataset Version

Selected Features

Transformation Method

Encoding Method

Prediction

Prediction Score

Explanation Method

Feature Contributions

Quantum Configuration

Backend

Timestamp

This makes the platform suitable for systematic research rather than one-time demonstrations.

18.27 Explainability Workflow

The complete XAI workflow is:

Prediction Generated

↓

Identify Model

↓

Load Model Configuration



Load Feature Transformation Information



Run Appropriate Explanation Method



Calculate Feature Contributions / Importance



Generate Explanation



Display Explanation



Store Explanation Metadata

18.28 Example: Cancer Detection

Suppose a cancer benchmark dataset is processed as:

30 Original Features



Feature Selection



15 Features



PCA



6 Components



Hybrid QML Model



Prediction: Positive

The explanation dashboard could show:

Prediction

Positive Class

Model Score

0.81

Important Processed Components

- **PC2**
- **PC1**
- **PC5**

Original Feature Associations

PC2 → Features A, C, F

PC1 → Features B, D

PC5 → Features E, G

This helps the researcher understand how the final model input relates to the original dataset.

18.29 Example: Cardiovascular Detection

For a cardiovascular benchmark:

Input Features

↓

Preprocessing

↓

Feature Selection

↓

PCA

↓

Hybrid VQC

↓

Prediction

The XAI module could show:

Prediction: Positive

Important Processed Inputs:

- 1. Component 1**
- 2. Component 4**
- 3. Component 2**

Model:

VQC

Backend:

Simulator

Circuit:

6 Qubits

Explanation:

Feature perturbation analysis

Again, these outputs are computational explanations rather than clinical conclusions.

18.30 Explainability Across Quantum Backends

The same model can potentially be evaluated on:

- Ideal simulator.
- Noisy simulator.
- Near-term quantum hardware.

The platform can compare explanation behavior across backends.

For example:

Backend	Predictio n	Scor e	Explanation Stability
Ideal Simulator	—	—	—
Noisy Simulator	—	—	—
Hardware	—	—	—

Actual results should be obtained experimentally.

This can help investigate how quantum noise affects model behavior.

18.31 Explanation Stability

An explanation should ideally not change dramatically due to insignificant changes in the input or execution environment.

Therefore, the platform can experimentally evaluate explanation stability.

For example:

Original Sample

→ **Explanation A**

Small Input Variation

→ **Explanation B**

Researchers can compare the explanations to determine whether the model behaves consistently.

This is an optional research evaluation component but can strengthen the platform.

18.32 Limitations of Explainability

The platform should acknowledge several limitations.

1. Post-hoc explanations are approximations

An explanation method may approximate model behavior rather than reveal the model's complete internal reasoning.

2. Feature importance is not causality

A feature being important does not mean that it causes the disease.

3. Dimensionality reduction complicates interpretation

Quantum models may operate on transformed components rather than original features.

4. Quantum models are more difficult to interpret

Quantum states and circuit operations do not map directly to conventional feature weights.

5. Clinical interpretation requires domain expertise

A computational explanation should not be treated as medical reasoning.

18.33 XAI Technology Approach

A practical implementation can use:

Classical Models

- SHAP where appropriate.
- Model-specific feature importance.
- Coefficients.
- Permutation-based analysis.

Hybrid QML Models

- Input perturbation.
- Prediction sensitivity analysis.
- Feature-ablation experiments.
- Circuit/model configuration visualization.
- Quantum-output analysis where appropriate.

The platform should select explanation methods according to the model rather than claiming that one method universally explains every model.

18.34 XAI and the SIH Problem Statement

Explainability directly satisfies the requirement that the proposed platform should support:

Data ingestion, hybrid model training, prediction, explainability, and performance evaluation.

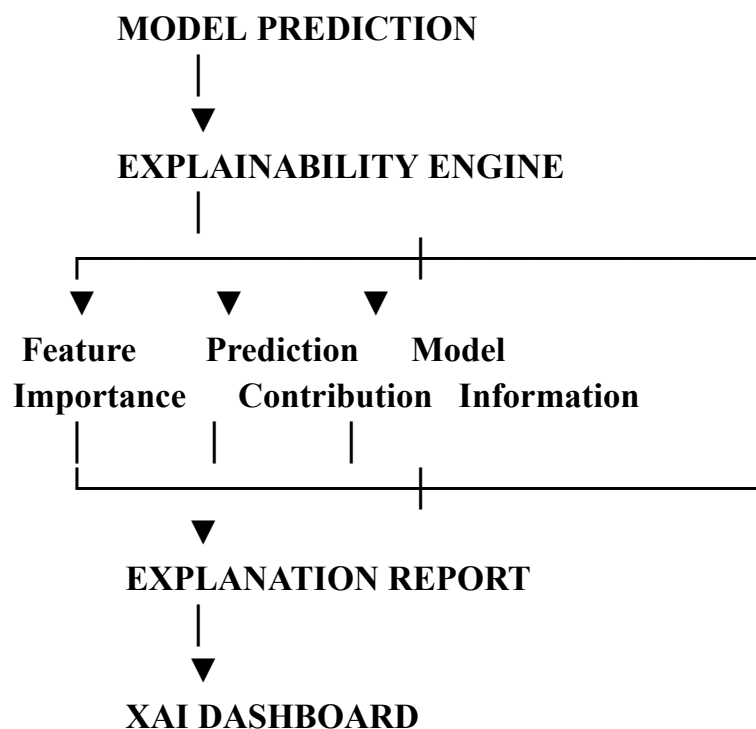
The XAI module therefore forms an essential part of the complete platform rather than being an optional visualization feature.

18.35 Expected Outputs

The Explainable AI module should provide:

1. Global feature importance.
 2. Local prediction explanations.
 3. Feature contribution information where supported.
 4. Model configuration information.
 5. Transformation information.
 6. Quantum circuit configuration.
 7. Prediction explanation visualization.
 8. Error-case explanations.
 9. Classical vs hybrid explanation comparison.
 10. Explanation metadata for reproducibility.
-

18.36 Core XAI Architecture



18.37 Key Design Principle

The key principle of the XAI module is:

Explain model behavior without overstating clinical meaning.

The platform should clearly communicate:

What the model predicted

-

Which processed information influenced the prediction

-

How the model was configured

-

How reliable the model is according to measured evaluation

This produces a more transparent and scientifically responsible system.

18.38 Complete Explainability Flow

The final workflow is:

Biomedical Input



Classical Processing



Quantum / Classical Model



Prediction



Feature Contribution Analysis



Model Behavior Analysis

↓

Explanation

↓

Visualization

↓

Researcher / Healthcare Professional Review

The final interpretation remains subject to appropriate clinical validation and expert review.

18.39 Summary

Explainable AI makes the proposed Hybrid Quantum Machine Learning Platform more transparent and useful for research.

The module provides:

- **Global explanations for understanding overall model behavior.**
- **Local explanations for understanding individual predictions.**
- **Feature importance for identifying influential variables.**
- **SHAP or suitable classical explanation methods for classical models.**
- **Perturbation and sensitivity analysis for hybrid QML models.**
- **Quantum circuit information for transparency.**
- **Error analysis for understanding false positives and false negatives.**
- **Model comparison for studying differences between classical and hybrid approaches.**

The final principle is:

Prediction alone is not enough → Prediction + Evidence of Model Behavior + Performance Evaluation

The platform should therefore provide explanations that help researchers understand and evaluate model behavior while clearly avoiding unsupported claims of causality or clinical diagnosis.

19. Performance Evaluation

The Performance Evaluation Module is responsible for measuring how effectively the proposed hybrid quantum-classical machine learning models perform on biomedical disease-detection tasks.

The platform should not assume that a quantum model is automatically better than a classical model. Instead, every model must be evaluated using the same experimental conditions and compared using objective metrics.

The evaluation process is:

Dataset



Preprocessing



Train / Validation / Test Split



Classical Baseline Models



Hybrid QML Models



Prediction



Performance Metrics



Computational Efficiency



Generalization Analysis



Classical vs Hybrid Comparison



Final Experimental Report

The purpose is to determine whether the hybrid QML approach provides measurable benefits for early disease detection while remaining practical with current quantum simulators and near-term quantum hardware.

19.1 Objectives

The main objectives of performance evaluation are to:

- **Measure disease-classification performance.**
 - **Evaluate accuracy.**
 - **Measure sensitivity/recall.**
 - **Measure specificity.**
 - **Measure precision.**
 - **Measure F1-score.**
 - **Measure ROC-AUC where applicable.**
 - **Analyze false positives and false negatives.**
 - **Compare classical ML and hybrid QML models.**
 - **Measure training and inference time.**
 - **Measure quantum resource requirements.**
 - **Evaluate model generalization.**
 - **Analyze performance under different feature dimensions.**
 - **Compare simulator and hardware execution where possible.**
 - **Produce reproducible experimental results.**
-

19.2 Why Performance Evaluation Is Important

A model producing a prediction is not enough.

For example, suppose two models produce:

Model A

Accuracy = 92%

Model B

Accuracy = 90%

It may appear that Model A is better.

However, if Model A has significantly lower sensitivity, it may miss more disease-positive cases.

Therefore, healthcare-related machine-learning evaluation should consider multiple metrics rather than accuracy alone.

The platform should evaluate:

Predictive Performance

-

Clinical-Relevant Classification Metrics

-

Computational Efficiency

-

Generalization

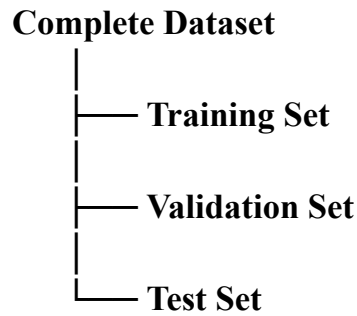
-

Quantum Resource Usage

19.3 Evaluation Dataset Split

The platform should divide the dataset into separate subsets.

A typical structure is:



Training Set

Used to train model parameters.

Validation Set

Used for model selection and hyperparameter tuning.

Test Set

Used for final performance evaluation.

The test set should remain isolated until final evaluation.

19.4 Fair Evaluation

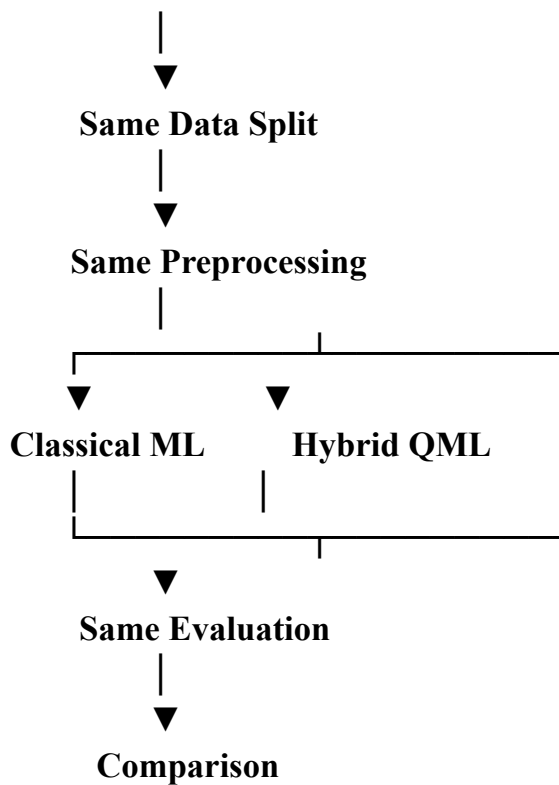
A fair comparison between classical ML and hybrid QML requires comparable experimental conditions.

The platform should use:

- **Same dataset.**
- **Same target definition.**
- **Same train/validation/test split.**
- **Same preprocessing where applicable.**
- **Same feature-selection strategy where appropriate.**
- **Same dimensionality-reduction configuration where appropriate.**
- **Same evaluation metrics.**

For example:

Same Dataset



This reduces the possibility that one model receives an unfair advantage from a different data-processing pipeline.

19.5 Accuracy

Accuracy measures the proportion of correctly classified samples.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Where:

- **TP = True Positive.**
- **TN = True Negative.**
- **FP = False Positive.**
- **FN = False Negative.**

Accuracy provides a general overview of model performance.

However, it should not be considered sufficient by itself when the dataset contains class imbalance.

19.6 Sensitivity

Sensitivity measures how effectively the model identifies positive cases.

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

A high sensitivity indicates that the model identifies a large proportion of positive samples.

For an early-detection research problem, sensitivity is particularly important because false negatives represent positive samples that the model failed to identify.

19.7 Specificity

Specificity measures how effectively the model identifies negative cases.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

A high specificity means that fewer negative samples are incorrectly classified as positive.

Sensitivity and specificity should be analyzed together.

19.8 Precision

Precision measures the proportion of predicted positive samples that are actually positive.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

It provides information about false-positive behavior.

19.9 Recall

For binary classification:

$\text{Recall} = \text{Sensitivity}$

Therefore, recall provides the same measure as sensitivity when evaluating the positive class.

19.10 F1-Score

F1-score combines precision and recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

It can be useful when there is an imbalance between disease-positive and disease-negative samples.

19.11 ROC Curve

The Receiver Operating Characteristic (ROC) curve evaluates model behavior across different classification thresholds.

It plots:

True Positive Rate

against

False Positive Rate

for different thresholds.

This allows researchers to understand how the model behaves beyond a single classification threshold.

19.12 ROC-AUC

The Area Under the ROC Curve (ROC-AUC) provides a summary measure of discrimination performance.

A larger ROC-AUC generally indicates better separation between the two classes.

The platform can calculate ROC-AUC when the model provides an appropriate continuous prediction score.

19.13 Precision-Recall Analysis

For imbalanced datasets, precision-recall analysis can provide additional information.

The platform can evaluate:

- Precision.
- Recall.
- Precision-recall curve.
- Average precision where appropriate.

This is useful when the positive disease class is relatively uncommon.

19.14 Confusion Matrix

The platform should generate a confusion matrix for each classification model.

Example:

	Actual Positive	Actual Negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

This makes it easy to identify:

- Correct positive predictions.
- Correct negative predictions.
- False positives.
- False negatives.

The confusion matrix can be displayed on the dashboard.

19.15 False Positive Analysis

A false positive occurs when the model predicts the positive disease class for a sample that belongs to the negative class.

The platform should track false-positive counts and rates.

This is important because excessive false positives can reduce the practical usefulness of a screening model.

Researchers can analyze:

- Which samples are misclassified.
 - Prediction scores.
 - Feature patterns.
 - Model-specific behavior.
-

19.16 False Negative Analysis

A false negative occurs when the model predicts the negative class for a sample belonging to the positive class.

False negatives are particularly important in early-detection research.

The platform should therefore provide:

- False-negative count.
- False-negative rate.
- Sensitivity.
- Examples of incorrectly classified samples where appropriate.
- Model comparison.

The goal is to understand where the model fails, rather than reporting only its overall accuracy.

19.17 Model Comparison Metrics

The platform can generate a comparison table:

Model	Accuracy	Sensitivity	Specificity	Precision	F1	ROC-AUC
Logistic Regression	—	—	—	—	—	—
SVM	—	—	—	—	—	—
Random Forest	—	—	—	—	—	—
XGBoost	—	—	—	—	—	—
Quantum Kernel / QSVM	—	—	—	—	—	—
VQC	—	—	—	—	—	—
QNN	—	—	—	—	—	—

The values must be generated from actual experiments.

19.18 Training Time

Predictive performance is only one part of evaluation.

The platform should also measure training time.

For example:

Model	Training Time
Logistic Regression	—
Random Forest	—
XGBoost	—
VQC	—
QNN	—

This helps determine the computational cost of each approach.

For quantum models, the time may include repeated quantum circuit execution.

19.19 Inference Time

The platform should also measure how long it takes to generate predictions.

Total inference time may include:

Preprocessing

-

Feature Transformation

-

Quantum Encoding

-

Quantum Execution

-

Post-processing

This is particularly useful when comparing:

- Classical inference.
- Simulator inference.
- Near-term quantum hardware inference.

19.20 Quantum Resource Evaluation

For hybrid QML models, the platform should record quantum-specific resources.

Important measurements include:

- Number of qubits.
- Number of quantum gates.
- Circuit depth.
- Number of circuit executions.
- Number of measurement shots where applicable.
- Backend type.
- Simulation resources.
- Hardware execution time where available.

Example:

Model	Qubits	Circuit Depth	Execution s
Quantum Kernel	—	—	—
VQC	—	—	—
QNN	—	—	—

Actual values should be obtained from implementation and experiments.

19.21 Classical Computational Resources

The platform can also record classical resource usage such as:

- CPU time.
- Memory consumption.
- GPU usage where applicable.
- Training duration.
- Storage requirements.

This allows a more complete comparison between classical and hybrid approaches.

19.22 Generalization Evaluation

A model should perform well not only on training data but also on unseen data.

The platform should compare:

Training Performance

vs.

Validation Performance

vs.

Test Performance

For example:

Dataset	Accurac y	ROC-AU C
Training	—	—

Validation — —
n
Test — —

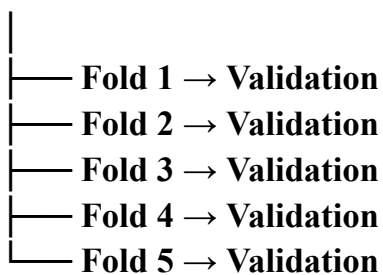
A large difference between training and test performance may indicate overfitting.

19.23 Cross-Validation

Where dataset size permits, cross-validation can be used to obtain a more robust estimate of model performance.

For example, with k-fold cross-validation:

Dataset



The model is trained and evaluated across different splits.

The platform can report:

- Mean performance.
- Standard deviation.
- Fold-level results.

This reduces dependence on a single train/test split.

19.24 Repeated Experiments

Quantum and ML models can be sensitive to initialization and random seeds.

Therefore, the platform can repeat experiments with different random seeds.

For example:

Experiment 1

→ Seed 1

Experiment 2

→ Seed 2

Experiment 3

→ Seed 3

The platform can then calculate:

- **Mean score.**
- **Standard deviation.**
- **Best score.**
- **Worst score.**

This provides a better estimate of model stability.

19.25 Statistical Comparison

If sufficient experiments are available, the platform can perform statistical comparisons between model results.

For example:

Classical Model Performance

vs.

Hybrid QML Performance

The objective is to determine whether an observed difference is consistent across repeated experiments rather than simply the result of random variation.

Statistical testing should be selected according to the experimental design and data characteristics.

19.26 Feature-Dimension Evaluation

An important experiment for this project is to evaluate how model performance changes as the number of features is reduced.

For example:

Component s	Classical Accuracy	Hybrid QML Accuracy
4	—	—
6	—	—
8	—	—
10	—	—

This helps investigate the trade-off between:

Information Content

and

Quantum Resource Requirement

19.27 Encoding Comparison

Different quantum feature maps can also be evaluated.

For example:

Feature Map	Qubit s	Depth	Accuracy	ROC-AUC
----------------	------------	-------	----------	---------

Map A	—	—	—	—
Map B	—	—	—	—
Map C	—	—	—	—

The platform can select the best experimentally supported configuration.

19.28 Quantum Model Comparison

The proposed platform can compare:

Quantum Kernel / QSVM

vs.

VQC

vs.

QNN

The comparison should consider:

- Predictive performance.
- Circuit complexity.
- Training time.
- Number of qubits.
- Number of executions.
- Stability.
- Generalization.

This prevents selecting a quantum model based only on accuracy.

19.29 Simulator vs Hardware Evaluation

When suitable hardware is available, the same model can be evaluated on:

Ideal Simulator



Noisy Simulator



Near-Term Quantum Hardware

The platform can compare:

- Accuracy.
- Sensitivity.
- Specificity.
- ROC-AUC.
- Circuit execution time.
- Noise effects.

This provides evidence about how the model behaves outside an ideal simulation environment.

19.30 Noise Impact Analysis

Quantum noise can change the measured output of a circuit.

The platform can therefore evaluate:

Ideal Circuit Performance

vs.

Noisy Circuit Performance

The difference can be reported as:

Performance Degradation=Ideal Performance–Noisy Performance
Degradation = Ideal\ Performance - Noisy\ Performance

This helps identify whether the selected QML approach is robust enough for near-term experimentation.

19.31 Model Stability

A good model should ideally produce reasonably consistent results across repeated experiments.

The platform can evaluate stability across:

- Random seeds.
- Train/test splits.
- Cross-validation folds.
- Feature dimensions.
- Quantum backends.

Example:

Run	Accuracy
-----	----------

1	—
---	---

2	—
---	---

3	—
---	---

4	—
---	---

5	—
---	---

Then calculate mean and variation.

19.32 Calibration Evaluation

If the model produces probability-like outputs, calibration can be evaluated.

A well-calibrated model should have predicted probabilities that reasonably correspond to observed frequencies.

For example:

If a group of predictions is assigned approximately 0.8 probability, the observed positive frequency should be reasonably close to that value over a sufficiently large evaluation set.

Calibration should be evaluated on suitable validation data and not assumed from raw model scores.

19.33 Threshold Analysis

The prediction threshold can affect sensitivity and specificity.

The platform can therefore evaluate multiple thresholds.

Example:

Threshold	Sensitivity	Specificity
0.30	—	—
0.40	—	—
0.50	—	—
0.60	—	—
0.70	—	—

This provides a clear picture of the trade-off between detecting positive cases and reducing false positives.

19.34 Learning Curve Analysis

The platform can optionally evaluate model behavior as the training dataset size increases.

For example:

20% Training Data

→ Performance

40% Training Data

→ Performance

60% Training Data

→ Performance

80% Training Data

→ Performance

100% Training Data

→ Performance

This can help researchers understand whether the model benefits from additional data.

19.35 Error Analysis

The performance module should not only calculate metrics but also identify where models fail.

Error analysis can include:

- **False positives.**
- **False negatives.**
- **Difficult samples.**
- **Misclassification patterns.**
- **Feature distributions.**
- **Model disagreement.**

For example:

Classical Model → Positive

Hybrid QML → Negative

This disagreement can be flagged for investigation.

19.36 Model Disagreement Analysis

The platform can compare predictions from multiple models.

For example:

Sample	SVM	Random Forest	VQC	Actual
A	Positive	Positive	Positive	Positive
B	Negative	Positive	Negative	Positive
C	Negative	Negative	Positive	Negative

Samples with disagreement can be highlighted for detailed analysis.

This is particularly useful for understanding how different algorithms learn different decision boundaries.

19.37 Explainability-Performance Relationship

The platform can analyze performance together with explainability.

For example:

Model A

High accuracy

-

High explainability

Model B

Similar accuracy

-

More complex explanation

This allows researchers to consider both performance and transparency.

19.38 Evaluation Dashboard

The Performance Evaluation dashboard can contain:

Model Metrics

- **Accuracy.**
- **Precision.**
- **Sensitivity.**
- **Specificity.**
- **F1-score.**
- **ROC-AUC.**

Confusion Matrix

Shows TP, TN, FP, FN.

ROC Curve

Shows threshold behavior.

Precision-Recall Curve

Useful for imbalanced datasets.

Computational Metrics

- Training time.
- Inference time.
- Memory usage.

Quantum Metrics

- Qubits.
- Circuit depth.
- Gate count.
- Circuit executions.
- Backend.

Generalization

- Training performance.
- Validation performance.
- Test performance.

Comparison

Classical vs Hybrid QML.

19.39 Automated Evaluation Report

After completing an experiment, the platform can automatically generate a structured report.

Example:

Performance Evaluation Report

Dataset:

Cardiovascular Benchmark

Models:

Logistic Regression

SVM

Random Forest

XGBoost

Quantum Kernel
VQC

Metrics:
Accuracy
Sensitivity
Specificity
Precision
F1
ROC-AUC

Computational:
Training Time
Inference Time

Quantum:
Qubits
Circuit Depth
Executions
Backend

Generalization:
Validation Performance
Test Performance

Conclusion:
Performance comparison based on
measured experimental results.

19.40 Example Evaluation Workflow

Suppose a cardiovascular dataset is selected.

Step 1

Load dataset.

↓

Step 2

Apply preprocessing.



Step 3

Perform feature selection.



Step 4

Reduce features to a quantum-compatible dimension.



Step 5

Train classical baselines.



Step 6

Train hybrid QML models.



Step 7

Evaluate all models on the same test set.



Step 8

Calculate:

- **Accuracy.**
- **Sensitivity.**
- **Specificity.**
- **Precision.**
- **F1.**
- **ROC-AUC.**

↓

Step 9

Measure computational and quantum resources.

↓

Step 10

Compare results.

↓

Step 11

Generate evaluation report.

19.41 Example Benchmark Table

The final SIH demonstration can contain a table like:

Model	Accuracy	Sensitivity	Specificity	F1	ROC-AUC	Training Time
Logistic Regression	—	—	—	—	—	—
SVM	—	—	—	—	—	—
Random Forest	—	—	—	—	—	—
XGBoost	—	—	—	—	—	—
Quantum Kernel	—	—	—	—	—	—
VQC	—	—	—	—	—	—
QNN	—	—	—	—	—	—

Important: Do not put invented performance percentages into the SIH presentation. Replace the dashes with actual experimental results.

19.42 Evaluation of the Main Research Question

The performance module ultimately addresses the project's central research question:

Does the hybrid quantum-classical approach provide useful performance for early disease detection compared with established classical machine-learning models under comparable experimental conditions?

The answer should be based on:

Predictive Metrics

-

Efficiency

-

Generalization

-

Quantum Resource Requirements

-

Experimental Reproducibility

19.43 No Assumption of Quantum Advantage

The platform should explicitly avoid claiming:

“Quantum ML is better than classical ML.”

before experiments are conducted.

Instead:

“The platform experimentally evaluates whether hybrid QML provides measurable benefits over classical baselines for selected biomedical datasets.”

Possible outcomes include:

Hybrid QML performs better

The experiment provides evidence supporting the tested configuration.

Classical ML performs better

The result is still scientifically valuable because it shows the limitations of the tested quantum approach.

Performance is similar

The platform can investigate whether the hybrid approach provides other benefits or whether additional optimization is necessary.

This makes the project scientifically credible.

19.44 Reproducibility

Every evaluation should store:

- **Dataset version.**
- **Data split.**
- **Random seed.**
- **Preprocessing configuration.**
- **Selected features.**
- **Dimensionality-reduction method.**
- **Model.**
- **Hyperparameters.**
- **Quantum circuit configuration.**

- Backend.
- Evaluation metrics.
- Execution time.

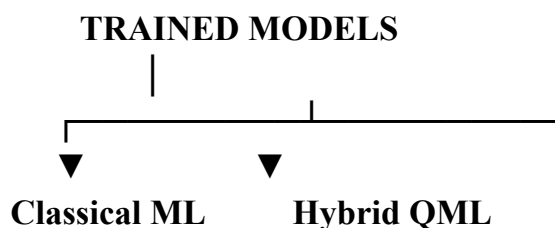
This allows the experiment to be reproduced.

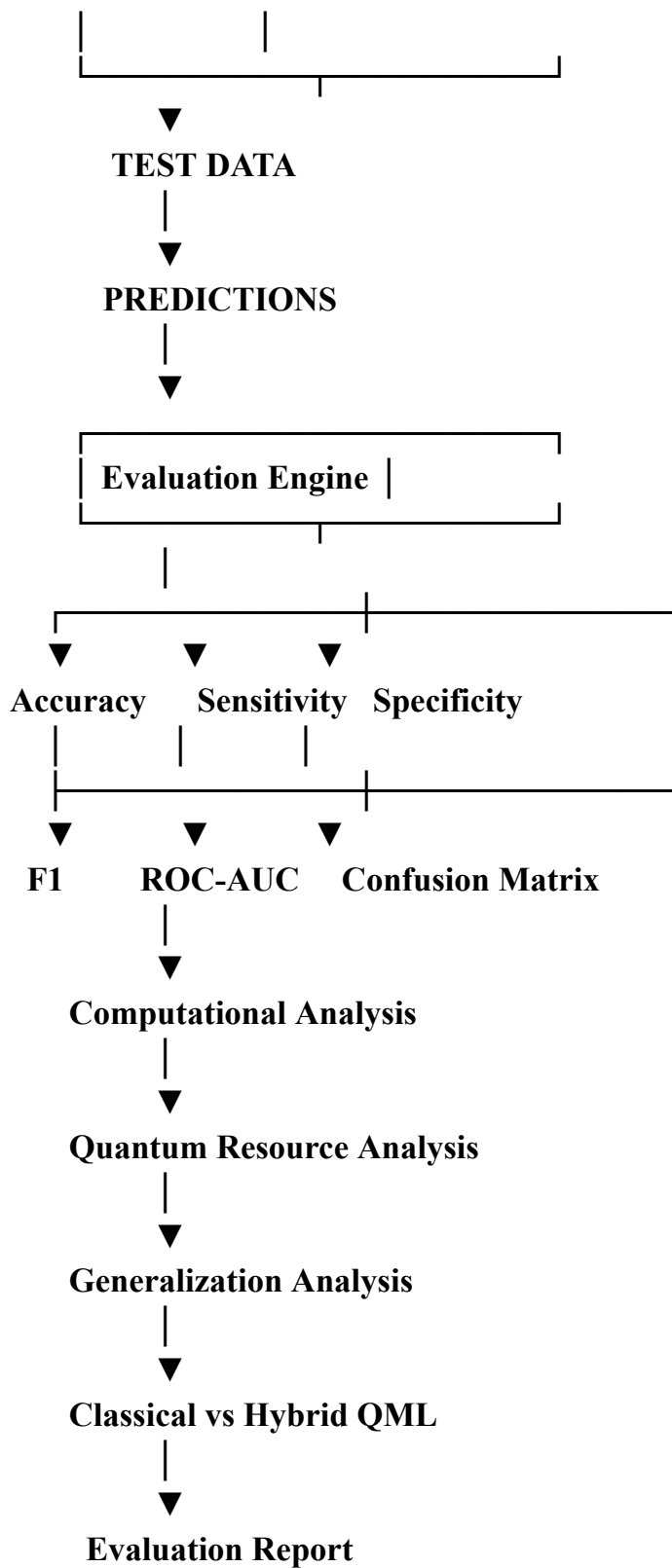
19.45 Expected Outputs

The Performance Evaluation Module should provide:

1. Accuracy.
 2. Precision.
 3. Sensitivity/Recall.
 4. Specificity.
 5. F1-score.
 6. ROC-AUC.
 7. Confusion matrix.
 8. ROC curve.
 9. Precision-recall analysis where appropriate.
 10. Training time.
 11. Inference time.
 12. Quantum resource usage.
 13. Generalization results.
 14. Model stability results.
 15. Classical vs hybrid comparison.
 16. Automated performance report.
-

19.46 Key Performance Evaluation Architecture





19.47 Key Design Principle

The key principle of the Performance Evaluation Module is:

Measure first, claim later.

The platform should not select or promote a quantum model simply because it is quantum.

Instead, the model should be evaluated using:

Accuracy

-

Sensitivity

-

Specificity

-

Precision

-

F1-score

-

ROC-AUC

-

Generalization

-

Computational Efficiency

-

Quantum Resource Requirements

The final conclusion should be based entirely on experimental evidence.

19.48 Summary

Performance Evaluation provides the evidence required to determine whether the proposed hybrid QML approach is useful for early disease detection.

The platform evaluates both predictive performance and practical computational requirements.

The complete evaluation flow is:

Train Models

- Test on Unseen Data**
- Calculate Classification Metrics**
- Analyze Errors**
- Measure Computational Cost**
- Measure Quantum Resources**
- Evaluate Generalization**
- Compare Classical vs Hybrid QML**
- Generate Experimental Report**

This ensures that the proposed system remains benchmark-driven, scientifically measurable, reproducible, and aligned with the SIH problem statement.

Most importantly, the platform does not assume quantum advantage. It provides an experimental framework to determine whether a hybrid quantum-classical approach actually offers a meaningful benefit for the selected biomedical disease-detection task.

20. Classical vs Hybrid QML Benchmarking

The Classical vs Hybrid QML Benchmarking Module provides a systematic framework for comparing traditional machine-learning models with hybrid quantum-classical models on the same biomedical disease-detection task.

The objective is not to assume that quantum machine learning is superior. Instead, the platform should provide an objective, reproducible, and fair experimental comparison.

The benchmarking process is:

Biomedical Dataset



Common Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Same Train / Validation / Test Split



Classical ML Pipeline Hybrid QML Pipeline



Predictions



Common Evaluation Framework



Performance + Efficiency + Generalization



Benchmark Comparison



Experimental Conclusion

20.1 Objective

The main objectives of benchmarking are to:

- Compare classical ML and hybrid QML fairly.
 - Use identical or appropriately controlled datasets.
 - Compare accuracy.
 - Compare sensitivity.
 - Compare specificity.
 - Compare precision and F1-score.
 - Compare ROC-AUC.
 - Compare false positives and false negatives.
 - Compare training and inference time.
 - Compare quantum resource requirements.
 - Compare generalization.
 - Compare model stability.
 - Analyze performance under different feature dimensions.
 - Evaluate simulator and hardware behavior where possible.
 - Determine whether observed differences are meaningful.
-

20.2 Why Benchmarking Is Essential

The central research question of the project is not simply:

“Can we build a quantum machine-learning model?”

The more important question is:

“Does the hybrid quantum-classical model provide useful advantages for early disease detection compared with established classical approaches?”

A quantum model may have:

- **Higher accuracy.**
- **Similar accuracy.**
- **Lower accuracy.**

All three outcomes are meaningful research results.

Therefore, benchmarking must be designed before claiming any quantum advantage.

20.3 Models Included in the Benchmark

The platform can compare multiple classical and hybrid models.

Classical Models

- 1. Logistic Regression.**
- 2. SVM.**
- 3. Random Forest.**
- 4. XGBoost / Gradient Boosting.**
- 5. Neural Network where appropriate.**

Hybrid QML Models

- 1. Quantum Kernel / QSVM.**
- 2. Variational Quantum Classifier.**
- 3. Quantum Neural Network.**

The exact models used in the prototype can depend on dataset size and available computational resources.

20.4 Common Dataset

All models should be evaluated on the same biomedical dataset for a direct comparison.

Possible datasets include:

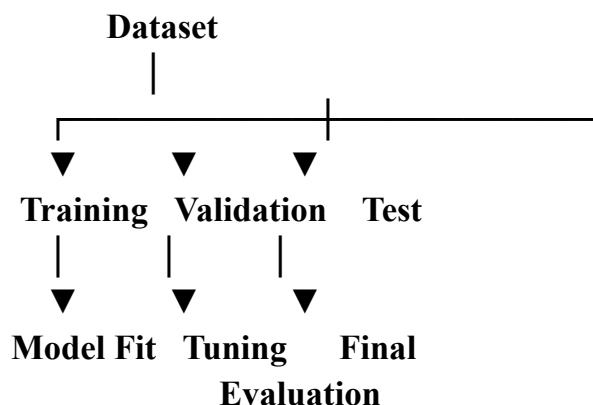
- Cancer benchmark datasets.
- Cardiovascular datasets.
- Neurological datasets.
- Genomic datasets.
- Medical imaging-derived datasets.
- EHR datasets.

For the initial SIH prototype, selecting one well-understood benchmark dataset is often more practical than attempting to demonstrate every disease category simultaneously.

20.5 Common Data Split

A common train/validation/test split should be created before model training.

For example:



The same split should be used for classical and hybrid models whenever technically appropriate.

This prevents differences in model performance from being caused by different test samples.

20.6 Common Preprocessing

The platform should maintain a common preprocessing pipeline wherever possible.

For example:

Raw Dataset



Missing-value handling



Categorical encoding



Scaling



Feature selection



Dimensionality reduction



Model-specific input

The goal is to ensure that the models are evaluated on comparable information.

20.7 Quantum-Compatible Feature Pipeline

Quantum models may require additional processing because of limited qubit availability.

For example:

100 Features



Feature Selection



30 Features



PCA



6 Components



Quantum Encoding



Hybrid QML

The classical baseline should receive a comparable six-component representation when the experiment is specifically intended to isolate the effect of the quantum model.

This creates a controlled comparison.

20.8 Two Important Benchmarking Strategies

There are two useful ways to structure the comparison.

Strategy A: Common Compact Representation

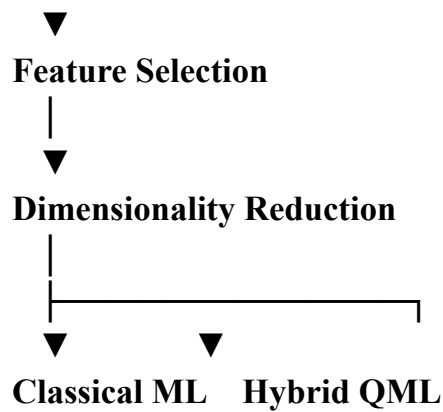
Both classical and hybrid models receive the same reduced feature representation.

Dataset



Common Preprocessing

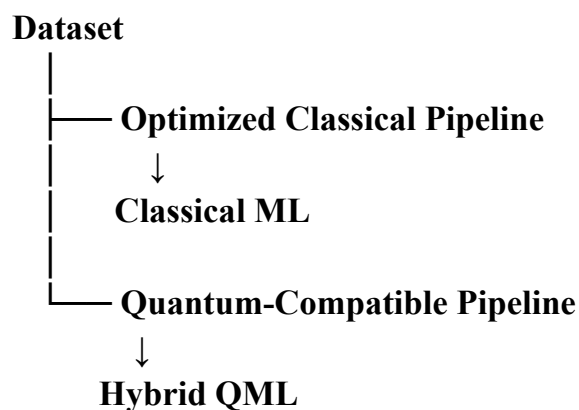




This is useful when the goal is to compare the learning models themselves.

Strategy B: Best Practical Pipeline

Each model can use its best experimentally justified preprocessing configuration.



This evaluates the best practical performance of each approach.

Both strategies can be valuable, but the experimental setup must clearly state which one was used.

20.9 Primary Performance Metrics

The benchmark should compare:

- Accuracy.
- Sensitivity.
- Specificity.
- Precision.
- F1-score.
- ROC-AUC.

These metrics should be calculated on the same evaluation dataset.

20.10 Accuracy Comparison

The platform can compare:

AccuracyClassicalAccuracy_{Classical}

with:

AccuracyHybridAccuracy_{Hybrid}

The difference can be reported as:

$\Delta\text{Accuracy} = \text{AccuracyHybrid} - \text{AccuracyClassical}$ \Delta Accuracy =
Accuracy_{Hybrid} - Accuracy_{Classical}

A positive value indicates higher measured accuracy for the hybrid model in that experiment.

It does not by itself establish general quantum advantage.

20.11 Sensitivity Comparison

Sensitivity is particularly important for early disease detection.

The platform can calculate:

$\Delta\text{Sensitivity} = \text{SensitivityHybrid} - \text{SensitivityClassical}$ \Delta Sensitivity =
Sensitivity_{Hybrid} - Sensitivity_{Classical}

This helps determine whether the hybrid approach identifies more positive samples.

20.12 Specificity Comparison

Similarly:

$$\Delta \text{Specificity} = \text{Specificity}_{\text{Hybrid}} - \text{Specificity}_{\text{Classical}} \backslash \Delta \text{Specificity} = \text{Specificity}_{\{\text{Hybrid}\}} - \text{Specificity}_{\{\text{Classical}\}}$$

This shows how the hybrid approach compares in identifying negative samples.

Sensitivity and specificity should be evaluated together because improving one can sometimes affect the other.

20.13 F1-Score Comparison

The platform can compare F1-score across models.

Example:

Model	F1-Score
Logistic Regression	—
SVM	—
Random Forest	—
XGBoost	—
Quantum Kernel	—
VQC	—
QNN	—

The actual values must come from experimental results.

20.14 ROC-AUC Comparison

ROC-AUC can provide an additional measure of class discrimination.

The dashboard can display ROC curves for multiple models.

Conceptually:

ROC Curve

Hybrid QML

Classical SVM

Random Forest

Logistic Regression

The actual curves should be generated from model predictions.

20.15 Confusion Matrix Comparison

Each model can have its own confusion matrix.

For example:

Classical SVM

→ TP, TN, FP, FN

Hybrid VQC

→ TP, TN, FP, FN

Researchers can compare the types of errors made by each model.

This is more informative than comparing accuracy alone.

20.16 False Negative Comparison

Since early disease detection can place importance on identifying positive cases, false negatives should be explicitly compared.

For example:

Model	False Negatives
Logistic Regression	—
SVM	—
Random Forest	—
XGBoost	—
Hybrid QML	—

A model with lower false-negative count on the same test set may have higher sensitivity.

However, this should always be interpreted alongside the full set of evaluation metrics.

20.17 False Positive Comparison

False positives should also be evaluated.

For example:

Model	False Positives
Logistic Regression	—
SVM	—

Random Forest —

XGBoost —

Hybrid QML —

This helps evaluate the trade-off between sensitivity and specificity.

20.18 Computational Efficiency

The benchmark should also measure computational requirements.

Important metrics include:

- Training time.
- Inference time.
- CPU usage.
- Memory usage.
- GPU usage where applicable.
- Number of quantum circuit executions.
- Simulator runtime.
- Quantum hardware execution time where measurable.

This is important because a model with slightly better predictive performance may not be practically useful if its computational requirements are substantially higher.

20.19 Quantum Resource Comparison

For QML models, the platform should report:

- Number of qubits.
- Circuit depth.
- Number of gates.
- Number of trainable parameters.
- Number of circuit executions.
- Number of measurement shots where applicable.

- Backend type.

Example:

QML Model	Qubit s	Depth	Execution s
Quantum Kernel	—	—	—
VQC	—	—	—
QNN	—	—	—

This makes the benchmark more meaningful than simply reporting accuracy.

20.20 Training-Time Comparison

The platform can compare:

Classical Training Time

vs.

Hybrid QML Training Time

For example:

Model	Training Time
Logistic Regression	—
Random Forest	—
XGBoost	—
Quantum Kernel	—
VQC	—

QNN —

Actual values should be measured on the selected execution environment.

20.21 Inference-Time Comparison

The platform should also compare how quickly models produce predictions.

For classical ML:

Preprocessing → Model → Prediction

For QML:

Preprocessing → Encoding → Quantum Execution → Measurement → Prediction

The difference can be measured experimentally.

This is especially important when comparing simulator execution with hardware execution.

20.22 Generalization Comparison

A strong benchmark should evaluate how well models perform on unseen data.

For each model:

Training Performance

vs.

Validation Performance

vs.

Test Performance

This helps identify overfitting.

A model with very high training accuracy but substantially lower test accuracy may not generalize well.

20.23 Cross-Validation Benchmark

Where dataset size permits, the platform can perform cross-validation.

For example:

5-Fold Cross-Validation

Each model is evaluated across the same folds.

The platform can report:

- **Mean accuracy.**
- **Mean sensitivity.**
- **Mean specificity.**
- **Mean F1.**
- **Mean ROC-AUC.**
- **Standard deviation.**

This provides a more robust comparison.

20.24 Stability Comparison

A model may perform differently depending on initialization or data split.

Therefore, repeated experiments can be performed.

For example:

Random Seed 1



Classical + Hybrid Evaluation

Random Seed 2



Classical + Hybrid Evaluation

Random Seed 3



Classical + Hybrid Evaluation

The platform can report average performance and variation.

20.25 Feature-Dimension Benchmark

One of the most valuable experiments for the proposed project is studying how performance changes with the number of features.

For example:

Features / Components	Classical ML	Hybrid QML
4	—	—
6	—	—
8	—	—
10	—	—

This allows the research team to investigate:

How much biomedical information can be compressed before prediction performance degrades?

It also reveals the trade-off between model performance and quantum resource requirements.

20.26 Quantum Model Benchmark

Different quantum models can be compared.

Quantum Kernel

Potentially useful for similarity-based classification.

VQC

Uses trainable quantum circuit parameters.

QNN

Provides a quantum neural-network-style architecture.

Comparison:

Model	Accuracy	Sensitivity	Specificity	Qubits	Depth
Quantum Kernel	—	—	—	—	—
VQC	—	—	—	—	—
QNN	—	—	—	—	—

The best model should be determined experimentally.

20.27 Encoding Benchmark

The platform can also compare quantum feature maps.

For example:

Encoding A

vs.

Encoding B

vs.

Encoding C

Metrics:

- Accuracy.
- Sensitivity.
- Specificity.
- ROC-AUC.
- Circuit depth.
- Execution count.

This helps identify whether the chosen feature representation is suitable for the dataset.

20.28 Simulator Benchmark

The same QML model can be evaluated under different simulator configurations.

For example:

Ideal Simulation

→ Baseline quantum behavior.

Noisy Simulation

→ Approximation of hardware imperfections.

The performance difference can help estimate the robustness of the model.

20.29 Hardware Benchmark

If access to a suitable quantum processor is available, the selected QML model can be tested on near-term hardware.

The platform can compare:

Backend	Accuracy	Sensitivity	Specificity	Runtime
	y	y	y	e

Ideal Simulator	—	—	—	—
Noisy Simulator	—	—	—	—
Quantum Hardware	—	—	—	—

This is an optional extension depending on hardware availability.

20.30 Noise Impact

A useful experiment is:

Performance_{ideal}Performance_{ideal}

vs.

PerformancenoisyPerformance_{noisy}

The platform can calculate:

$$\text{Noise Impact} = \text{Performance}_{\text{ideal}} - \text{Performance}_{\text{noisy}}$$

The metric can be applied to accuracy or other evaluation measures.

This allows researchers to understand how sensitive the QML model is to quantum noise.

20.31 Explainability Comparison

Benchmarking should not be limited to predictive performance.

The platform can compare how easily the models can be analyzed.

For example:

Model	Performance	Explanation Method	Interpretability
Logistic Regression	—	Coefficients / SHAP	—
Random Forest	—	Feature Importance / SHAP	—
XGBoost	—	SHAP	—
VQC	—	Perturbation / Model Analysis	—

The interpretation category should be based on the actual implemented explanation method rather than subjective claims.

20.32 Model Agreement Analysis

The platform can identify cases where classical and hybrid models agree or disagree.

For example:

Sample A

Classical → Positive

Hybrid QML → Positive

Agreement ✓

Sample B

Classical → Negative

Hybrid QML → Positive

Disagreement !

Disagreement cases can be sent to the error-analysis module for further investigation.

20.33 Benchmarking Dashboard

The SIH prototype can provide a dedicated benchmarking screen.

Model Ranking

Displays all tested models.

Performance Cards

- Accuracy.
- Sensitivity.
- Specificity.
- F1.
- ROC-AUC.

Confusion Matrix

For the selected model.

ROC Curve

For model comparison.

Efficiency

- Training time.
- Inference time.

Quantum Resources

- Qubits.
- Circuit depth.
- Executions.

Generalization

- Validation performance.
- Test performance.

Model Agreement

Shows where models disagree.

20.34 Example Benchmark Dashboard

A simplified dashboard could look like:

CLASSICAL vs HYBRID QML						
Model	Accuracy	Sens.	Spec.	F1	AUC	
Logistic	—	—	—	—	—	
SVM	—	—	—	—	—	
Random Forest	—	—	—	—	—	
XGBoost	—	—	—	—	—	
Quantum Kernel	—	—	—	—	—	
VQC	—	—	—	—	—	
QNN	—	—	—	—	—	
Training Time:	—					
Inference Time:	—					
Qubits:	—					
Circuit Depth:	—					
Backend:	Simulator					

The final dashboard should populate these fields using actual experimental results.

20.35 Benchmarking Protocol

To make the comparison scientifically meaningful, the platform can follow a fixed protocol.

Step 1

Select dataset.

Step 2

Define target variable.

Step 3

Create train/validation/test split.

Step 4

Fit preprocessing only on training data.

Step 5

Apply common preprocessing.

Step 6

Perform feature selection.

Step 7

Perform dimensionality reduction where required.

Step 8

Train classical models.

Step 9

Train hybrid QML models.

Step 10

Evaluate all models on the same test data.

Step 11

Calculate metrics.

Step 12

Measure computational resources.

Step 13

Repeat experiments where required.

Step 14

Generate comparison report.

20.36 Avoiding Unfair Benchmarking

Several mistakes must be avoided.

Mistake 1: Different test sets

This makes performance comparison unreliable.

Mistake 2: Using the test set for tuning

This can cause optimistic results.

Mistake 3: Weak classical baseline

A quantum model should not be compared against an intentionally poor classical model.

Mistake 4: Reporting only accuracy

Medical classification requires additional metrics.

Mistake 5: Ignoring computational cost

Higher accuracy does not automatically mean better practical performance.

Mistake 6: Claiming quantum advantage from one experiment

A single result is not sufficient to establish general superiority.

20.37 Benchmarking Experimental Matrix

A structured experiment matrix can be used.

Experiment	Dataset	Features	Classical Model	QML Model	Backend
E1	Cancer	6	SVM	VQC	Simulator
E2	Cancer	8	SVM	VQC	Simulator
E3	Cancer	6	RF	QNN	Simulator
E4	Cancer	8	RF	QNN	Simulator

This makes the research process organized and reproducible.

20.38 Result Interpretation

The platform should categorize results carefully.

Case 1: Hybrid QML performs better

The tested hybrid configuration shows higher measured performance.

Case 2: Classical ML performs better

The classical model performs better under the tested conditions.

This is a valid result and may indicate that the selected QML architecture or dataset is not favorable for the current quantum approach.

Case 3: Similar performance

The models perform similarly.

Further comparison can focus on:

- Efficiency.
- Resource usage.
- Stability.

- Generalization.
 - Backend behavior.
-

20.39 What Counts as a Meaningful Result?

A meaningful result should consider multiple dimensions.

For example:

Hybrid QML

Accuracy ↑

Sensitivity ↑

Specificity ≈

Training Cost ↑↑

Qubit Requirement ↑

This is not automatically a clear overall improvement.

The platform should present the complete trade-off rather than highlighting only the best metric.

20.40 Benchmark Result Report

At the end of an experiment, the platform can generate:

Benchmark Report

Dataset: [Dataset Name]

Classical Models:

- Logistic Regression
- SVM
- Random Forest

- XGBoost

Hybrid Models:

- Quantum Kernel**
- VQC**
- QNN**

Metrics:

- Accuracy**
- Sensitivity**
- Specificity**
- Precision**
- F1**
- ROC-AUC**

Efficiency:

- Training Time**
- Inference Time**

Quantum Resources:

- Qubits**
- Circuit Depth**
- Executions**
- Backend**

Generalization:

- Validation**
- Test**

Conclusion:

Based on measured experimental results.

20.41 Reproducibility

Every benchmark experiment should store:

- Dataset version.**
- Data split.**

- **Random seed.**
- **Preprocessing configuration.**
- **Feature-selection method.**
- **Number of selected features.**
- **Dimensionality-reduction configuration.**
- **Model.**
- **Hyperparameters.**
- **Quantum encoding.**
- **Circuit architecture.**
- **Backend.**
- **Evaluation metrics.**
- **Runtime.**

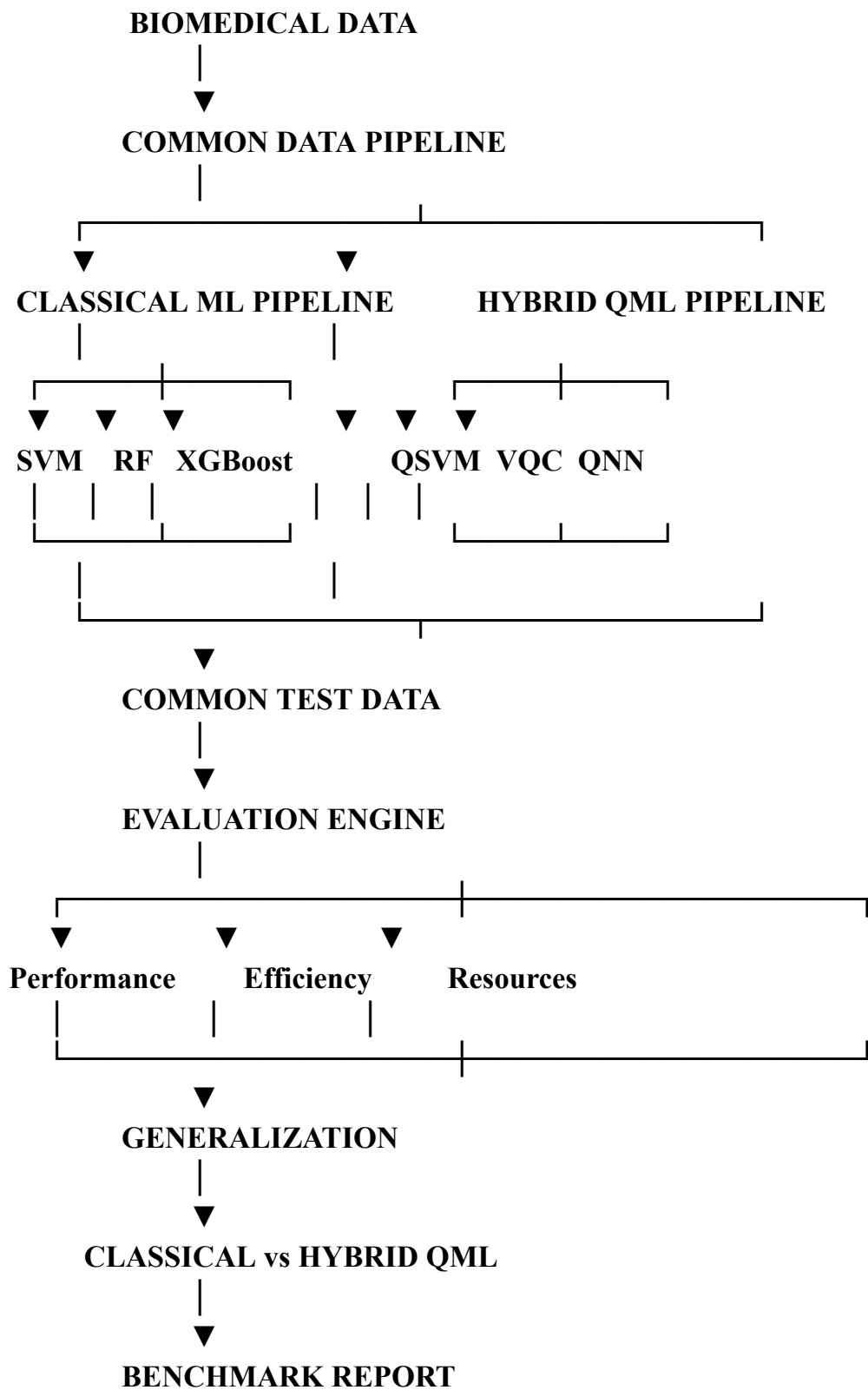
This makes it possible to reproduce the experiment.

20.42 Expected Outputs

The benchmarking module should provide:

- 1. Classical model results.**
 - 2. Hybrid QML model results.**
 - 3. Accuracy comparison.**
 - 4. Sensitivity comparison.**
 - 5. Specificity comparison.**
 - 6. Precision and F1 comparison.**
 - 7. ROC-AUC comparison.**
 - 8. Confusion matrices.**
 - 9. ROC curves.**
 - 10. Training-time comparison.**
 - 11. Inference-time comparison.**
 - 12. Quantum-resource analysis.**
 - 13. Generalization analysis.**
 - 14. Stability analysis.**
 - 15. Feature-dimension analysis.**
 - 16. Simulator/hardware comparison where available.**
 - 17. Model disagreement analysis.**
 - 18. Automated benchmark report.**
-

20.43 Core Benchmarking Architecture



20.44 Key Research Question

The benchmarking module ultimately answers:

Under comparable experimental conditions, does the hybrid quantum-classical approach provide measurable benefits for early disease detection compared with established classical machine-learning models?

The answer must be derived from experimental evidence.

20.45 Key Design Principle

The most important principle is:

Fair comparison is more important than claiming quantum superiority.

The platform should treat classical ML as a strong baseline rather than a competitor that is intentionally weakened.

The benchmark should evaluate:

Predictive Performance

-

Sensitivity / Specificity

-

Generalization

-

Computational Efficiency

-

Quantum Resource Requirements

-

Model Stability

This creates a scientifically credible comparison.

20.46 Summary

The Classical vs Hybrid QML Benchmarking Module is the experimental backbone of the proposed platform.

It allows the same biomedical problem to be evaluated using:

Classical ML

- **Logistic Regression.**
- **SVM.**
- **Random Forest.**
- **XGBoost.**
- **Neural Networks where appropriate.**

and:

Hybrid QML

- **Quantum Kernel / QSVM.**
- **VQC.**
- **QNN.**

Both approaches are evaluated using:

Accuracy

→ **Sensitivity**

→ **Specificity**

→ **Precision**

→ **F1**

→ **ROC-AUC**

→ **Generalization**

→ **Training/Inference Time**

→ **Quantum Resources**

→ **Stability**

The final outcome is not predetermined.

The platform may demonstrate that hybrid QML performs better, similarly, or worse than classical ML for a particular dataset and configuration. All outcomes are useful because the purpose of the system is to experimentally determine the effectiveness and practicality of hybrid QML for early disease detection.

This benchmark-driven approach directly supports the SIH requirement to evaluate the hybrid solution against purely classical baselines.

21. Quantum Simulator Support

Quantum simulator support is a core component of the proposed Hybrid Quantum Machine Learning Platform for Early Disease Detection.

Current quantum computers have limitations such as a small number of usable qubits, noise, limited availability, queue times, and restricted circuit execution. Therefore, the platform should primarily use quantum simulators during development, experimentation, testing, and demonstration.

The simulator allows the project team to design and evaluate quantum machine-learning circuits using classical computing infrastructure while keeping the software architecture compatible with real quantum hardware.

The basic approach is:

Biomedical Dataset



Classical Preprocessing



Feature Selection



Dimensionality Reduction



Quantum Feature Encoding



Quantum Simulator



Quantum Circuit Execution



Measurement



Hybrid Model Prediction



Performance Evaluation



Classical vs Hybrid QML Comparison

21.1 Objective

The Quantum Simulator Module should:

- **Allow QML models to run without physical quantum hardware.**
- **Support development and testing of quantum circuits.**
- **Enable repeatable experiments.**
- **Support ideal quantum simulation.**
- **Support noisy simulation where possible.**
- **Allow different QML architectures to be tested.**
- **Measure quantum resources such as qubits and circuit depth.**
- **Support benchmarking.**
- **Provide a path from simulator execution to real quantum hardware.**

- **Make the complete SIH prototype demonstrable on conventional computing infrastructure.**
-

21.2 Why Quantum Simulation Is Required

Physical quantum computers are currently not always practical for continuous development.

Challenges include:

- **Limited quantum hardware availability.**
- **Limited number of usable qubits.**
- **Hardware noise.**
- **Quantum decoherence.**
- **Execution queues.**
- **Hardware-dependent errors.**
- **Limited circuit depth.**
- **Execution cost.**
- **Connectivity constraints.**

A simulator provides a controlled environment for developing and testing the proposed QML models.

Therefore, the project follows a:

Simulator-first, hardware-ready architecture.

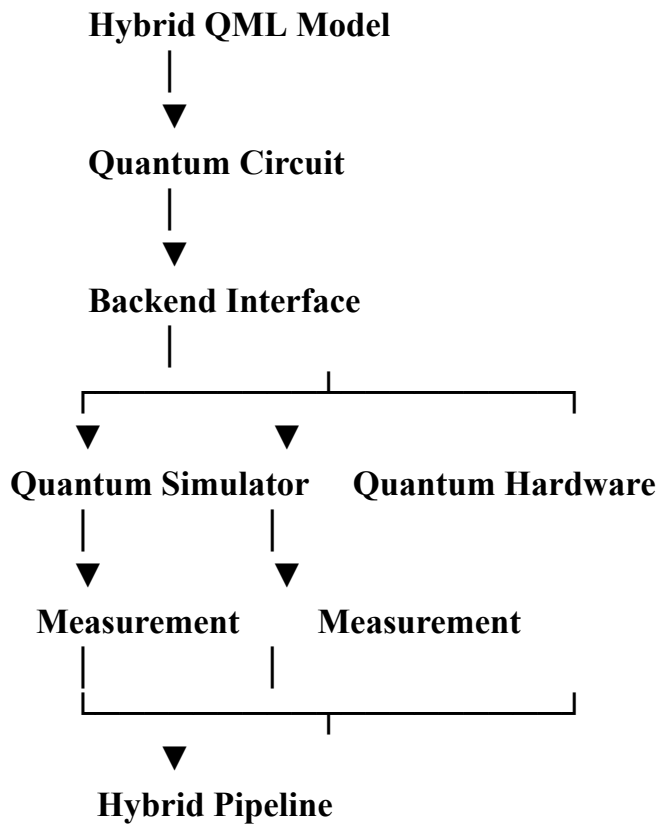
This means the initial system can run entirely on classical infrastructure while maintaining compatibility with near-term quantum processors.

21.3 Role of the Simulator in the Platform

The simulator is not a separate experimental tool.

It becomes part of the QML execution layer.

The architecture can be:

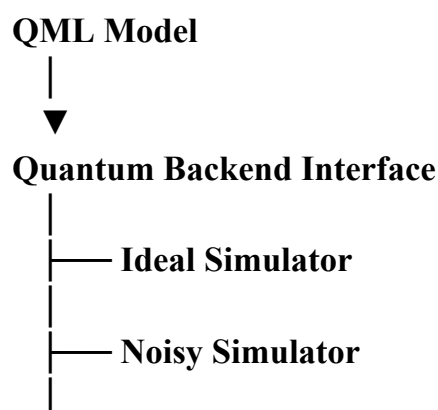


The model should interact with a common backend interface instead of directly depending on a single quantum device.

21.4 Backend Abstraction

A backend abstraction layer allows the same QML model to run using different execution environments.

For example:



└─ Real Quantum Backend

The model therefore does not need to be completely redesigned when moving from simulation to hardware.

This improves:

- Portability.
 - Maintainability.
 - Experimentation.
 - Hardware compatibility.
 - Reproducibility.
-

21.5 Ideal Quantum Simulation

The first execution mode should be ideal simulation.

An ideal simulator attempts to reproduce quantum circuit behavior without intentionally introducing hardware noise.

It can be used for:

- Testing quantum circuits.
- Debugging circuits.
- Verifying feature encoding.
- Testing QML models.
- Checking circuit measurements.
- Comparing model architectures.
- Establishing a baseline.

For example:

Biomedical Features



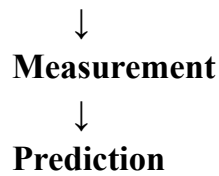
Angle Encoding



Quantum Circuit



Ideal Simulator



This provides a controlled environment for initial experiments.

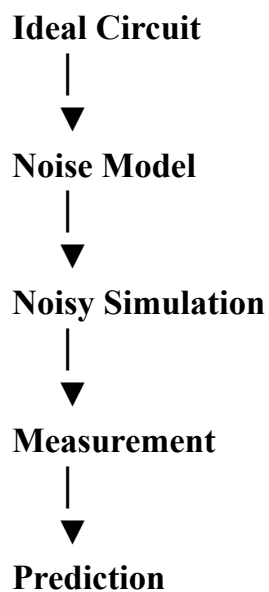
21.6 Noisy Quantum Simulation

The second execution mode can introduce simulated noise.

Real quantum hardware is affected by different types of imperfections.

A noisy simulator can approximate these effects.

Conceptually:



This allows the team to investigate whether a QML model remains useful when quantum noise is present.

21.7 Why Noise Testing Matters

A QML model may perform well under ideal simulation but degrade under noisy conditions.

For example:

Ideal Simulation

Accuracy = A

↓

Noisy Simulation

Accuracy = B

The difference can be analyzed as:

Noise Impact = A - B

The actual values should come from experiments.

This provides information about the robustness of the quantum model.

21.8 Simulator-Supported QML Models

The simulator should support the main quantum models proposed by the project.

1. Quantum Kernel / QSVM

Input Features

↓

Quantum Feature Map

↓

Quantum Kernel

↓

Classical SVM

↓

Prediction

The simulator executes the quantum feature map and estimates the required kernel values.

2. Variational Quantum Classifier

Input Features



Feature Encoding



Parameterized Circuit



Measurement



Classical Loss



Optimizer



Updated Parameters



Repeat

The simulator executes the circuit during the training loop.

3. Quantum Neural Network

A QNN can use:

- Feature encoding.
- Trainable quantum layers.
- Entanglement.
- Measurements.
- Classical optimization.

The simulator executes these circuits during training and inference.

21.9 Quantum Circuit Execution

For each sample or batch, the system performs:

1. Receive processed features.
 2. Normalize features if required.
 3. Convert features into quantum parameters.
 4. Construct the quantum circuit.
 5. Apply the feature map.
 6. Apply trainable quantum layers.
 7. Apply entangling operations.
 8. Measure selected qubits.
 9. Convert measurements into model outputs.
 10. Pass the output to the classical optimization or prediction layer.
-

21.10 Measurement

The quantum simulator produces measurement results.

These results can be converted into numerical values used by the hybrid model.

For example:

Quantum Circuit



Measurement



Expectation Values / Probabilities



Classical Processing



Disease Prediction

The exact measurement strategy depends on the selected QML model.

21.11 Simulator Configuration

The platform can provide configurable simulator settings.

Possible configuration parameters include:

Parameter	Purpose
Number of Qubits	Controls quantum circuit size
Circuit Depth	Controls number of layers
Shots	Number of measurement repetitions
Noise Model	Simulates hardware imperfections
Backend	Selects simulation environment
Random Seed	Improves reproducibility
Encoding Method	Defines feature representation
Ansatz	Defines trainable circuit structure

Not every parameter needs to be exposed to the end user; advanced parameters can remain in an experiment configuration panel.

21.12 Number of Qubits

The number of qubits should correspond to the size of the quantum representation.

For example:

Selected Features



Dimensionality Reduction



6 Components



6-Qubit Encoding

The exact mapping depends on the chosen encoding strategy.

Because current quantum systems have resource constraints, dimensionality reduction remains an important part of the architecture.

21.13 Circuit Depth

Circuit depth represents how many sequential layers of quantum operations are performed.

A deeper circuit can provide greater representational complexity but may also become more difficult to execute reliably on near-term hardware.

Therefore, the platform should record:

- Circuit depth.
- Number of gates.
- Entangling operations.
- Trainable parameters.

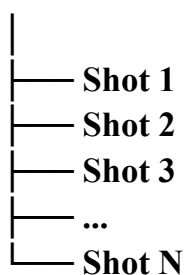
This allows the model's predictive performance to be evaluated alongside its quantum complexity.

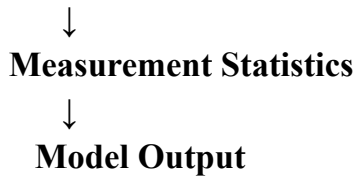
21.14 Measurement Shots

When using shot-based simulation, the circuit can be executed repeatedly to estimate measurement probabilities or expectation values.

For example:

Quantum Circuit





Increasing the number of shots can improve the statistical estimation but also increases computation.

Therefore, shots can be treated as an experimental parameter.

21.15 Simulator Performance

The platform should monitor simulator execution time.

Important measurements include:

- Circuit construction time.
- Circuit execution time.
- Number of circuits executed.
- Number of shots.
- Training time.
- Inference time.
- Memory usage where practical.

This is important because simulation itself can become computationally expensive as circuit size increases.

21.16 Simulator Scalability

Quantum simulation can become increasingly expensive as the number of qubits grows.

Therefore, the platform should use resource-aware quantum model design.

Instead of immediately using large circuits, the prototype should focus on:

- **Compact feature representations.**
- **Reduced feature dimensions.**
- **Shallow circuits.**
- **Limited qubit counts.**
- **Efficient circuit execution.**
- **Experimentally selected configurations.**

This is consistent with the near-term quantum computing environment.

21.17 Simulator-Based Development Workflow

The development workflow can be:

Step 1 — Dataset Selection

Select a biomedical benchmark dataset.

Step 2 — Classical Processing

Clean and transform the data.

Step 3 — Feature Selection

Select useful features.

Step 4 — Dimensionality Reduction

Reduce the feature space to a quantum-compatible representation.

Step 5 — Quantum Encoding

Encode the selected features into a quantum circuit.

Step 6 — Ideal Simulation

Test the QML circuit under ideal conditions.

Step 7 — Model Training

Train the hybrid model.

Step 8 — Validation

Evaluate on validation data.

Step 9 — Noisy Simulation

Evaluate robustness under noise.

Step 10 — Test Evaluation

Evaluate on unseen test data.

Step 11 — Benchmark

Compare against classical models.

Step 12 — Hardware Transition

If a suitable quantum processor is available, run the same model using the hardware backend.

21.18 Simulator vs Hardware

The platform should clearly distinguish simulation results from hardware results.

Property	Simulator	Quantum Hardware
Availability	High	Limited
Noise	Configurable	Physical
Execution	Classical computer	Quantum processor
Reproducibility	Generally easier	Hardware-dependent
Cost	Depends on simulator	May involve access cost/limits
Development	Suitable	Suitable for final hardware experiments

Scalability

**Limited by classical
simulation cost**

Limited by current hardware

The project should never present simulator results as actual quantum-hardware results.

21.19 Experiment Tracking

Every simulation experiment should store its configuration.

Example:

Experiment ID: EXP-001

Dataset: Cardiovascular Dataset

Features: 8

Encoding: Angle Encoding

Qubits: 8

Circuit Depth: 3

Model: VQC

Backend: Ideal Simulator

Shots: 1024

Random Seed: 42

The results can then be stored with the experiment configuration.

This makes experiments reproducible.

21.20 Reproducibility

The simulator module should support reproducible experiments through:

- **Fixed random seeds.**
- **Dataset versioning.**
- **Model configuration storage.**

- **Circuit configuration storage.**
- **Backend configuration.**
- **Preprocessing configuration.**
- **Feature-selection configuration.**
- **Training parameters.**

This allows researchers to repeat an experiment and verify the result.

21.21 Simulator Error Handling

The platform should handle common execution problems.

Possible cases:

- **Invalid circuit configuration.**
- **Feature dimension mismatch.**
- **Unsupported operation.**
- **Excessive circuit size.**
- **Simulator memory limitation.**
- **Invalid measurement configuration.**
- **Training timeout.**
- **Backend failure.**

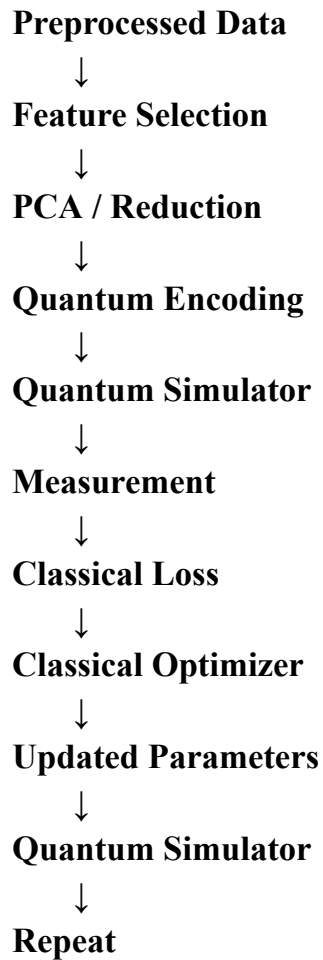
Instead of silently failing, the platform should provide meaningful error messages.

For example:

“The selected feature dimension requires more quantum resources than the configured simulator can efficiently execute. Reduce the number of components or select another configuration.”

21.22 Integration With the Hybrid Training Pipeline

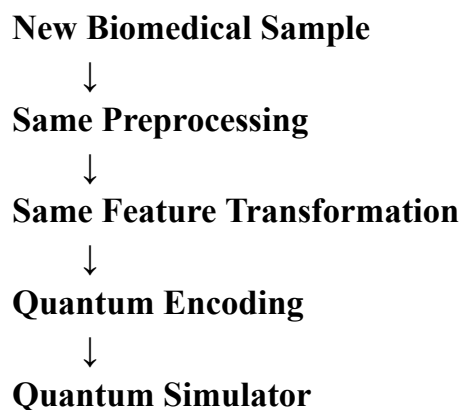
The simulator connects directly to the hybrid training workflow.

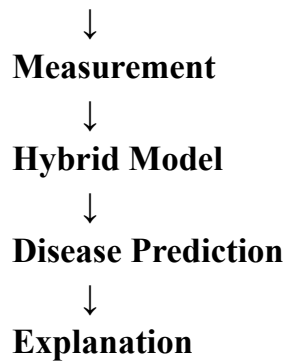


Thus, the simulator becomes an execution backend inside the training loop.

21.23 Integration With Disease Prediction

During inference:





The same preprocessing and encoding configuration used during training must be used during inference.

21.24 Simulator Dashboard

The platform can provide a dedicated simulator panel.

Backend

Ideal Simulator

Qubits

8

Circuit Depth

3

Shots

1024

Model

VQC

Encoding

Angle Encoding

Execution Time

—

Noise

Disabled

Status

Completed

The values shown in the actual prototype should be generated from real experiment configurations.

21.25 Noisy Simulation Dashboard

A second experiment can show:

Noise Analysis

Ideal Accuracy: —

Noisy Accuracy: —

Performance Change: —

Circuit Depth: —

Qubits: —

Shots: —

Noise Configuration:

[Selected Model]

This helps demonstrate robustness.

21.26 Simulator-Based Benchmarking

The simulator should integrate with the benchmarking module.

For example:

Model	Backend	Accuracy	Sensitivity	Specificity	Runtime
SVM	Classical	—	—	—	—
Random Forest	Classical	—	—	—	—
Quantum Kernel	Ideal Simulator	—	—	—	—
VQC	Ideal Simulator	—	—	—	—
QNN	Ideal Simulator	—	—	—	—

This provides a complete comparison.

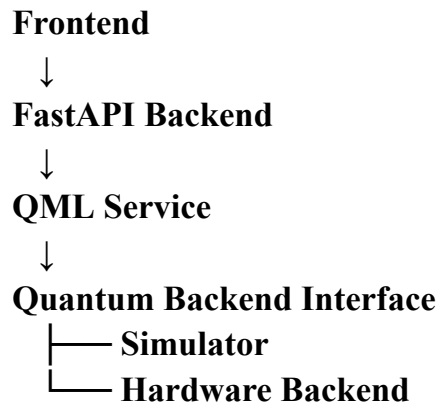
21.27 Technology Approach

The implementation can use a quantum machine-learning framework that supports:

- Quantum circuit construction.
- Quantum simulation.
- Variational circuits.
- Quantum kernels.
- Automatic differentiation where supported.
- Backend abstraction.
- Hardware integration.

The platform should keep the quantum framework behind a service/interface layer so that the rest of the application is not tightly coupled to one backend.

A possible architecture is:



The exact framework can be selected based on implementation feasibility and available hardware access.

21.28 Security and Data Isolation

Biomedical data should remain separate from the simulator infrastructure wherever possible.

The simulator does not require patient-identifying information for its quantum computation.

Therefore, the platform should preferably perform:

De-identification



Preprocessing



Feature Transformation



Quantum-Compatible Representation



Quantum Execution

Only the required numerical representation should be passed into the QML execution layer.

21.29 Advantages of Simulator-First Development

This approach provides several benefits:

Accessibility

The prototype can be developed without continuous access to quantum hardware.

Cost Efficiency

Large-scale hardware access is not required for initial development.

Reproducibility

Controlled simulator configurations make experiments easier to repeat.

Debugging

Quantum circuits can be tested before hardware execution.

Rapid Experimentation

Different feature maps and QML architectures can be evaluated quickly.

Hardware Readiness

A backend abstraction allows later transition to real quantum processors.

21.30 Important Limitation

Quantum simulation is not equivalent to running on a real quantum computer.

A simulator runs using classical computational resources.

Therefore:

Simulator performance should not be interpreted as proof of practical quantum-hardware advantage.

The project should clearly distinguish:

- **Ideal simulation.**
- **Noisy simulation.**
- **Real quantum hardware.**

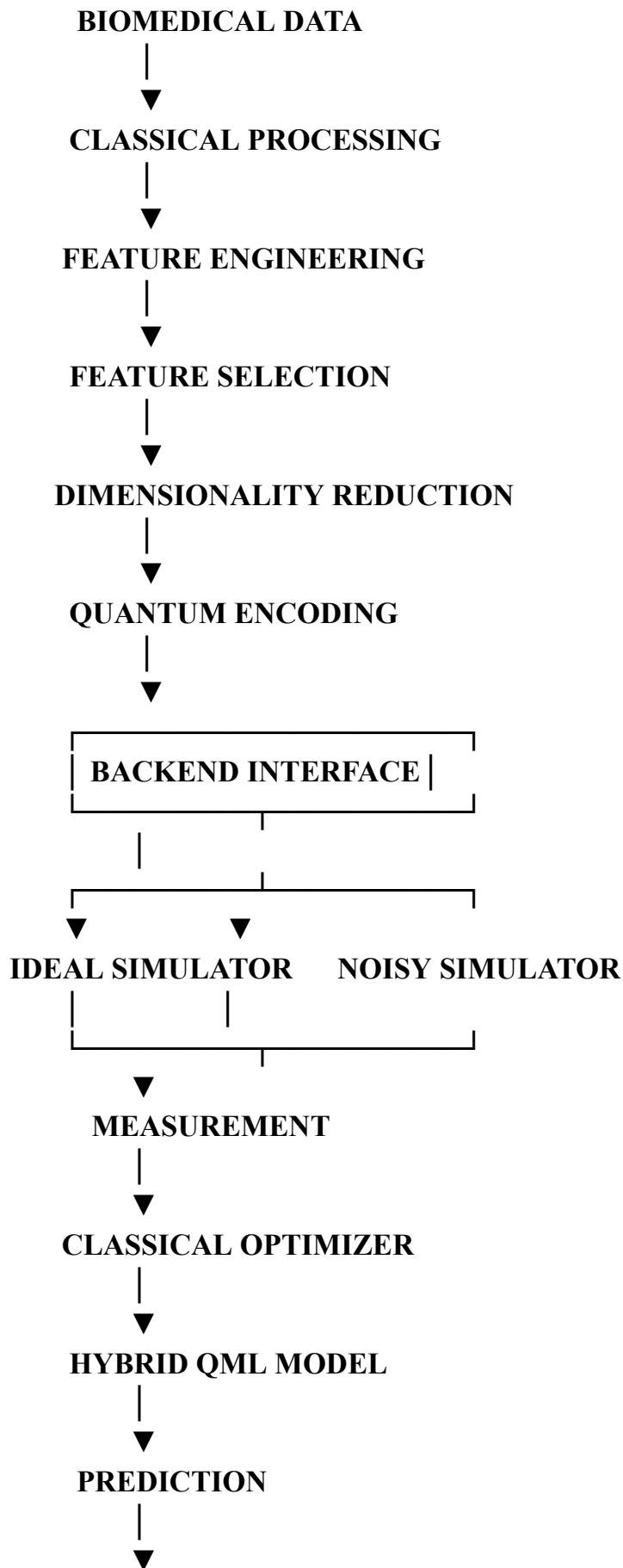
This is especially important when presenting results to judges.

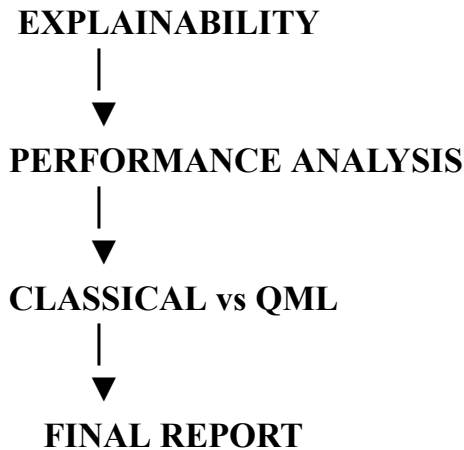
21.31 Expected Outputs

The Quantum Simulator Module should provide:

- 1. Quantum circuit execution.**
 - 2. Ideal simulation.**
 - 3. Optional noisy simulation.**
 - 4. QML model execution.**
 - 5. Measurement results.**
 - 6. Configurable qubit count.**
 - 7. Configurable circuit depth.**
 - 8. Configurable shots.**
 - 9. Quantum resource reporting.**
 - 10. Simulator execution-time measurement.**
 - 11. Experiment tracking.**
 - 12. Reproducible configurations.**
 - 13. Error handling.**
 - 14. Integration with hybrid training.**
 - 15. Integration with disease prediction.**
 - 16. Integration with benchmarking.**
 - 17. Optional transition to quantum hardware.**
-

21.32 Complete Simulator Architecture





21.33 Key Design Principle

The key principle of this module is:

Build and validate the complete QML workflow using simulators first, while keeping the architecture ready for near-term quantum hardware.

This provides a practical balance between current quantum limitations and the research objective of the project.

21.34 Summary

Quantum Simulator Support makes the proposed platform practical for current quantum-computing conditions.

The platform uses simulators to:

- **Develop quantum circuits.**
- **Train QML models.**
- **Test quantum feature encoding.**
- **Evaluate ideal behavior.**
- **Study noise effects.**
- **Measure quantum resources.**
- **Benchmark QML against classical ML.**
- **Reproduce experiments.**

- **Prepare models for future hardware execution.**

The overall strategy is:

Classical Biomedical Processing

- **Quantum Feature Encoding**
- **Quantum Simulation**
- **Hybrid Training**
- **Disease Prediction**
- **Explainability**
- **Benchmarking**
- **Optional Quantum Hardware Execution**

This ensures that the SIH prototype remains implementable today while being compatible with the near-term evolution of quantum computing.

22. Near-Term Quantum Hardware Compatibility

The proposed Hybrid Quantum Machine Learning Platform should be designed to work not only with quantum simulators but also with near-term quantum hardware.

Current quantum processors are still limited in terms of qubit count, noise, circuit depth, connectivity, and execution availability. Therefore, the platform should not depend on large fault-tolerant quantum computers.

Instead, it should follow a NISQ-compatible and hardware-aware design that uses relatively small, shallow, and optimized quantum circuits.

The overall approach is:

Biomedical Data

↓

Classical Preprocessing

↓

Feature Selection



Dimensionality Reduction



Quantum-Compatible Representation



Hardware-Aware Quantum Circuit



Near-Term Quantum Processor



Measurement



Classical Processing



Disease Prediction



Explainability + Evaluation

22.1 Objective

The main objectives of hardware compatibility are to:

- **Make QML models executable on near-term quantum processors.**
- **Minimize quantum resource requirements.**
- **Reduce circuit depth where possible.**
- **Account for hardware noise.**
- **Respect available qubit counts.**

- **Consider hardware connectivity.**
 - **Support measurement-based execution.**
 - **Maintain the same model architecture between simulation and hardware as far as practical.**
 - **Compare simulator and hardware results.**
 - **Provide a backend-independent architecture.**
 - **Prepare the platform for future quantum hardware improvements.**
-

22.2 Why Near-Term Hardware Compatibility Matters

A purely simulator-based project demonstrates quantum algorithms theoretically, but connecting the system to actual quantum hardware provides an additional level of practical experimentation.

The project therefore aims for:

Simulator-first development + hardware-ready architecture + optional real-device execution.

This allows the prototype to function even when physical hardware is unavailable while still supporting real quantum experiments when access is available.

22.3 NISQ-Oriented Design

Near-term quantum processors are commonly associated with the Noisy Intermediate-Scale Quantum (NISQ) era.

Such systems have practical constraints:

- **Limited qubit counts.**
- **Imperfect gates.**
- **Measurement errors.**
- **Decoherence.**
- **Noise.**
- **Limited connectivity.**

- **Restricted circuit depth.**
- **Execution overhead.**

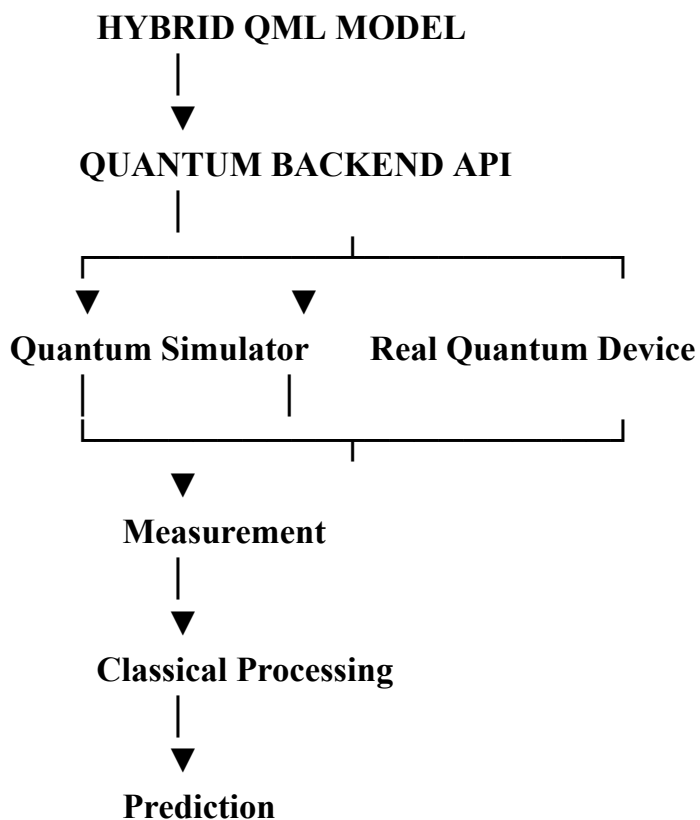
The proposed platform should therefore avoid unnecessarily large quantum circuits.

The design principle is:

Use the smallest quantum circuit that can adequately represent and evaluate the selected biomedical learning problem.

22.4 Hardware-Aware Architecture

The platform should contain a backend abstraction layer.



This prevents the application from becoming tightly coupled to a particular quantum processor.

22.5 Backend Abstraction Layer

The backend layer can provide a common interface such as:

```
submit_circuit()
run_circuit()
get_result()
get_backend_info()
estimate_resources()
check_availability()
```

The application can therefore communicate with:

- Ideal simulators.
- Noisy simulators.
- Quantum hardware.

The exact API depends on the selected quantum framework.

22.6 Hardware Selection

Before executing a model, the platform should check whether the selected backend satisfies the model's requirements.

Possible checks include:

- Available qubits.
- Supported gates.
- Connectivity.
- Maximum circuit depth.
- Measurement support.
- Backend availability.
- Execution limitations.

For example:

Required Qubits = 8

Available Qubits = 5



Configuration Not Compatible



Reduce Features / Select Another Backend

This prevents invalid hardware execution.

22.7 Qubit Resource Management

The number of quantum features must be compatible with the available qubits.

For example:

8 selected quantum components

may require approximately:

8 qubits

under a straightforward one-feature-per-qubit encoding.

Therefore, dimensionality reduction becomes critical.

The pipeline becomes:

High-Dimensional Biomedical Data



Feature Selection



Dimensionality Reduction



Compact Representation



Quantum Encoding



Available Qubits

This directly connects the classical preprocessing pipeline with quantum hardware limitations.

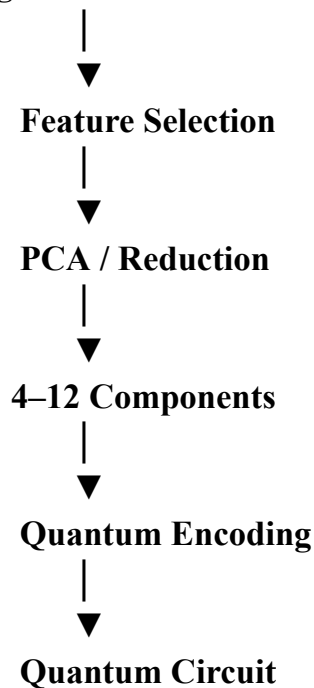
22.8 Feature Compression for Hardware

Biomedical datasets can contain hundreds, thousands, or even millions of potential measurements.

A near-term quantum processor cannot directly process arbitrarily large feature spaces.

Therefore:

High-Dimensional Biomedical Data

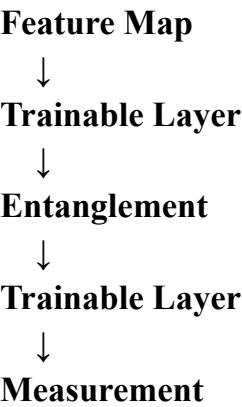


The exact number of components should be experimentally determined based on the dataset and hardware resources.

22.9 Circuit Depth Optimization

Circuit depth should be kept as low as reasonably possible.

A typical QML circuit may contain:



Adding more layers may increase model complexity but can also increase exposure to noise.

Therefore, the platform should allow experiments with different depths.

For example:

Circuit	Depth	Performance
Circuit A	2	—
Circuit B	3	—
Circuit C	4	—

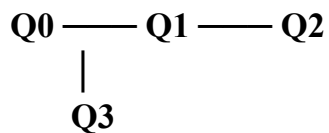
Circuit 5 —
D

The optimal configuration should be selected based on experimental results.

22.10 Hardware Connectivity

Real quantum processors may not allow every qubit to directly interact with every other qubit.

For example:



If a circuit requires an interaction between physically disconnected qubits, additional operations may be required to map the logical circuit onto the hardware.

This can increase circuit complexity.

Therefore, the platform should consider hardware connectivity during circuit compilation.

22.11 Qubit Mapping

The platform can distinguish between:

Logical Qubits

and

Physical Qubits

The QML model defines logical qubits.

The hardware backend maps them to available physical qubits.

Logical Qubits

Q0 Q1 Q2 Q3



Hardware Mapping



Physical Qubits

P2 P4 P5 P7

The exact mapping depends on the selected backend.

22.12 Gate Compatibility

Different quantum processors may support different native gate sets.

Therefore, a circuit may require compilation or decomposition before execution.

Conceptually:

High-Level Quantum Circuit



Circuit Compilation



Hardware-Compatible Gates



Quantum Processor

This allows the same high-level QML model to target different hardware backends.

22.13 Noise-Aware Execution

Noise is one of the most important considerations for near-term quantum hardware.

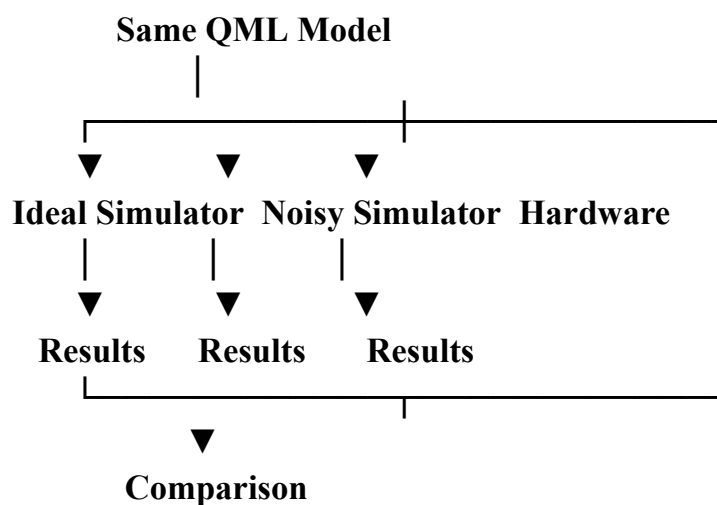
Possible sources include:

- Gate errors.
- Measurement errors.
- Decoherence.
- Environmental interactions.
- Hardware-specific imperfections.

The platform should record the backend and relevant execution configuration so that hardware results can be interpreted correctly.

22.14 Simulator vs Hardware Experiment

The same QML model can ideally be tested in multiple environments.



This allows the team to measure how hardware noise affects model performance.

22.15 Hardware Performance Metrics

The platform should evaluate hardware execution using the same primary metrics as the simulator.

These include:

- Accuracy.
- Sensitivity.
- Specificity.
- Precision.
- F1-score.
- ROC-AUC.
- Confusion matrix.

Additional hardware metrics include:

- Execution time.
 - Circuit depth.
 - Number of gates.
 - Number of qubits.
 - Number of shots.
 - Number of circuit executions.
 - Backend information.
-

22.16 Hardware Noise Impact

The platform can calculate the difference between ideal simulation and hardware execution.

For example:

$$\text{Hardware Degradation} = \text{Performance}_{\text{Ideal}} - \text{Performance}_{\text{Hardware}}$$
$$\text{Degradation} = \text{Performance}_{\{\text{Ideal}\}} - \text{Performance}_{\{\text{Hardware}\}}$$

The metric can be calculated for:

- Accuracy.
- Sensitivity.
- Specificity.
- F1-score.
- ROC-AUC.

The result should be treated as an experimental observation, not as a universal property of quantum computing.

22.17 Hardware-Aware Quantum Feature Encoding

Feature encoding should be designed with hardware constraints in mind.

For example:

Input

8 reduced biomedical components.

↓

8-Qubit Encoding

↓

Shallow Feature Map

↓

Limited Entanglement

↓

Measurement

This reduces unnecessary circuit complexity.

Angle encoding can be used as a practical starting point because it maps numerical features to rotation parameters.

22.18 Entanglement Strategy

Entanglement can be useful for capturing relationships between encoded features.

However, excessive entanglement may increase circuit complexity.

Therefore, the platform can support configurable entanglement structures.

For example:

Linear Entanglement

Q0 – Q1 – Q2 – Q3

Circular Entanglement

Q0 – Q1
| |
Q3 – Q2

The selected structure should be evaluated experimentally.

22.19 Shallow Circuit Design

The project should prioritize shallow circuits for near-term hardware.

A simplified circuit could be:

q0 — Encoding — RY —●—— Measure
 |
q1 — Encoding — RY —X—— Measure

The actual circuit depends on the selected QML architecture.

The goal is to balance:

Representational Capability

against

Hardware Feasibility

22.20 Hardware-Compatible VQC

For a VQC, the workflow becomes:

Biomedical Features



Feature Encoding



Parameterized Quantum Circuit



Hardware Execution



Measurement



Loss Calculation



Classical Optimizer



Updated Parameters



Hardware Execution



Repeat

The classical optimizer remains outside the quantum processor.

This is the core hybrid architecture.

22.21 Hardware-Compatible Quantum Kernel

For quantum kernel methods:

Sample A



Quantum Feature Map



Hardware Execution



Kernel Information

Sample B



Quantum Feature Map



Hardware Execution



Kernel Information



Quantum Kernel Matrix



Classical SVM



Prediction

The quantum processor performs the quantum portion while classical ML performs the final classification.

22.22 Execution Overhead

Quantum hardware execution can involve significant overhead.

The platform should therefore track:

- **Number of circuit submissions.**
- **Number of shots.**
- **Number of optimization iterations.**
- **Total execution time.**
- **Backend queue/execution time where available.**

This is important when evaluating computational efficiency.

22.23 Batch Processing Considerations

Classical ML can process large batches efficiently.

Quantum execution may require individual circuit evaluations or repeated executions.

Therefore, the platform should use compact datasets and efficient circuit execution during the prototype stage.

The system should avoid assuming that quantum hardware automatically provides faster execution.

22.24 Hardware Availability

Hardware execution should be treated as an optional backend.

If no physical quantum processor is available:

Simulator → Complete Prototype

If hardware is available:

Simulator → Hardware Experiment

This ensures that the project remains functional regardless of quantum hardware access.

22.25 Hardware Backend Configuration

A configuration panel can contain:

Quantum Backend

Backend: [Simulator / Hardware]

Qubits Required: 8

Circuit Depth: 3

Shots: 1024

Encoding:
[Angle Encoding]

Entanglement:
[Linear]

Noise:
[Hardware]

Run Experiment

Advanced settings can be hidden from normal users.

22.26 Hardware Validation Workflow

The platform can follow this process:

Step 1

Train and validate the model using simulation.

Step 2

Verify the circuit.

Step 3

Check hardware compatibility.

Step 4

Compile the circuit for the selected backend.

Step 5

Execute a small test circuit.

Step 6

Execute the complete experiment.

Step 7

Collect measurements.

Step 8

Generate predictions.

Step 9

Evaluate performance.

Step 10

Compare hardware results with simulator results.

22.27 Hardware Experiment Logging

Every hardware experiment should record:

Experiment ID

Dataset Version

Model

Feature Count

Qubit Count

Circuit Depth

Encoding

Ansatz

Backend

Shots

Execution Time

Random Seed
Performance Metrics

This creates a reproducible experimental record.

22.28 Hardware Failure Handling

The system should handle cases such as:

- Backend unavailable.
- Insufficient qubits.
- Unsupported gates.
- Circuit too deep.
- Execution failure.
- Queue timeout.
- Authentication/configuration failure.
- Hardware result unavailable.

The platform should provide a fallback:

Hardware Failure



Run on Compatible Simulator

where appropriate.

The fallback should be clearly labeled as simulation.

22.29 Hardware vs Simulator Dashboard

The platform can provide a comparison panel:

Metric	Ideal Simulator	Noisy Simulator	Hardware
--------	--------------------	--------------------	----------

Accuracy	—	—	—
Sensitivity	—	—	—
Specificity	—	—	—
F1	—	—	—
ROC-AUC	—	—	—
Runtime	—	—	—
Qubits	—	—	—
Circuit Depth	—	—	—

This creates a strong demonstration of the complete hybrid pipeline.

22.30 Hardware Readiness Score

The platform could provide a simple technical compatibility assessment before execution.

For example:

Hardware Compatibility

Qubit Requirement	✓
Supported Gates	✓
Connectivity	✓
Circuit Depth	✓
Measurement	✓

Overall:
READY FOR EXECUTION

This is a software compatibility check, not a measure of clinical or predictive quality.

22.31 Resource-Aware Model Selection

The platform can select models based on available resources.

For example:

Available Hardware



Resource Analysis



Candidate QML Models



Filter Incompatible Models



Benchmark Compatible Models

This prevents attempting unnecessarily expensive circuits.

22.32 Future Hardware Scalability

As quantum hardware improves, the platform should be able to support:

- More qubits.
- Better connectivity.
- Lower noise.
- Greater circuit depth.
- Larger quantum feature representations.
- More complex QML models.

The classical architecture should remain largely unchanged.

Only the quantum backend and resource configuration may need to evolve.

22.33 Important Scientific Limitation

Running a QML model on real quantum hardware does not automatically establish quantum advantage.

Hardware results must still be compared against:

- Strong classical baselines.
- Ideal simulation.
- Noisy simulation where relevant.
- Comparable preprocessing.
- Appropriate evaluation metrics.

The project should therefore report measured results rather than making predetermined claims.

22.34 Medical Context Limitation

The hardware-compatible QML system is a research and experimental platform.

A successful hardware execution does not mean the resulting model is ready for clinical deployment.

Clinical use would require additional:

- Clinical validation.
- Independent datasets.
- Robustness testing.
- Safety evaluation.
- Regulatory assessment.
- Domain-expert review.

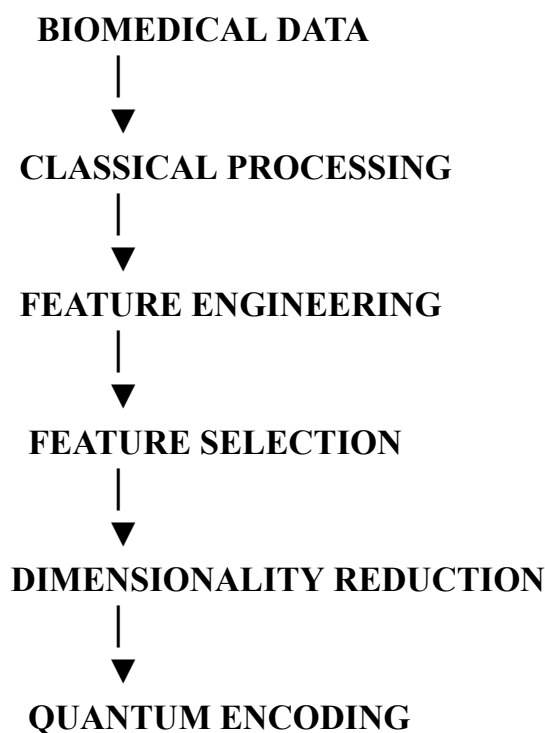
Therefore, the prototype should be presented as a research/decision-support system, not as an independently validated medical diagnostic device.

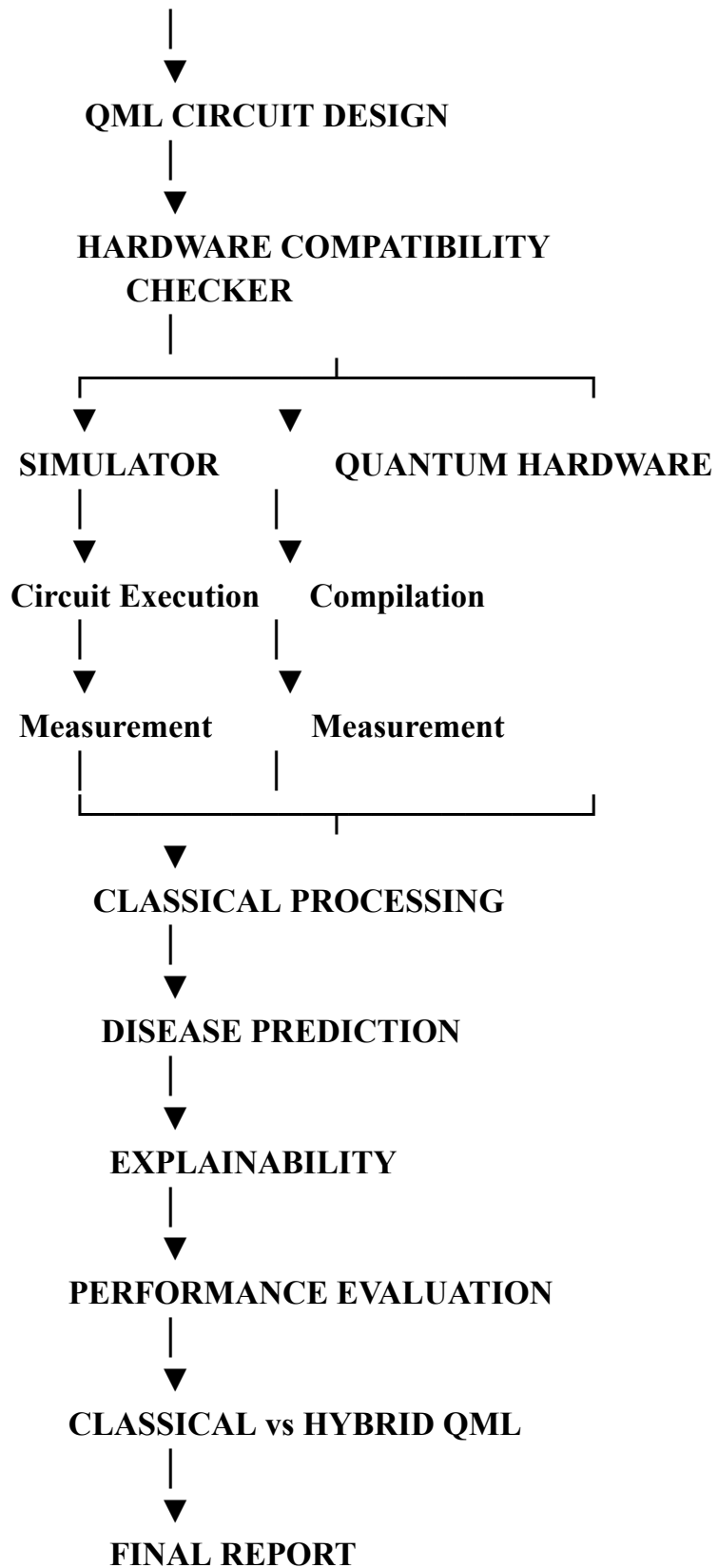
22.35 Expected Outputs

The Near-Term Quantum Hardware Compatibility Module should provide:

- 1. Hardware-compatible quantum circuits.**
 - 2. Backend abstraction.**
 - 3. Qubit resource checking.**
 - 4. Circuit-depth checking.**
 - 5. Hardware connectivity consideration.**
 - 6. Circuit compilation support.**
 - 7. Hardware execution support where available.**
 - 8. Noise-aware evaluation.**
 - 9. Simulator-vs-hardware comparison.**
 - 10. Quantum resource reporting.**
 - 11. Hardware execution logging.**
 - 12. Failure handling.**
 - 13. Simulator fallback.**
 - 14. Performance benchmarking.**
 - 15. Reproducible experiment records.**
 - 16. Future hardware scalability.**
-

22.36 Complete Hardware-Compatible Architecture





22.37 Key Design Principle

The core principle is:

Design the QML pipeline for the constraints of today's quantum hardware rather than assuming future large-scale quantum computers.

This means:

Compact Features

-

Few Qubits

-

Shallow Circuits

-

Hardware-Aware Compilation

-

Noise-Aware Evaluation

-

Classical Optimization

-

Strong Classical Benchmarking

22.38 Summary

Near-Term Quantum Hardware Compatibility extends the platform from a simulator-based research environment toward real quantum execution.

The platform achieves this through:

Classical Preprocessing

- **Feature Reduction**
- **Quantum Encoding**
- **Hardware-Aware Circuit**
- **Quantum Execution**
- **Measurement**
- **Classical Optimization**
- **Disease Prediction**
- **Explainability**
- **Benchmarking**

The architecture remains simulator-first so that the complete SIH prototype can function without physical quantum hardware. At the same time, the backend abstraction makes it possible to execute compatible models on near-term quantum processors when access is available.

Most importantly, the project will measure actual hardware performance rather than assuming quantum advantage.

This makes the proposed platform practical for current quantum technology while preserving a clear path toward more capable quantum systems.

23. Model Optimization

Model optimization is an important component of the proposed Hybrid Quantum Machine Learning Platform for Early Disease Detection.

Biomedical datasets can contain many features, noise, class imbalance, and complex relationships. At the same time, quantum models have additional constraints such as limited qubits, circuit depth, noise, and execution cost.

Therefore, simply training a model is not sufficient. The platform should systematically search for a configuration that provides a good balance between:

Predictive Performance

-

Generalization

-

Computational Efficiency

-

Quantum Resource Requirements

The optimization process applies to both classical and hybrid QML models.

The overall workflow is:

Biomedical Data



Preprocessing



Feature Selection



Dimensionality Reduction



Model Configuration



Hyperparameter Optimization



Model Training



Validation



Performance Evaluation



Resource Evaluation



Best Configuration



Final Test Evaluation

23.1 Objective

The objectives of the Model Optimization Module are to:

- Improve model performance through systematic parameter tuning.
 - Reduce overfitting.
 - Select appropriate model hyperparameters.
 - Optimize the number of input features.
 - Optimize quantum circuit configuration.
 - Optimize circuit depth.
 - Optimize the number of trainable parameters.
 - Select suitable classical optimizers for QML.
 - Improve computational efficiency.
 - Evaluate different quantum feature encodings.
 - Improve robustness against noise.
 - Maintain good generalization.
 - Find a practical performance/resource trade-off.
-

23.2 Why Model Optimization Is Required

Different configurations of the same model can produce very different results.

For example, a VQC may use:

- 4 qubits or 8 qubits.
- 2 circuit layers or 5 layers.
- Different learning rates.
- Different optimizers.
- Different feature maps.
- Different entanglement structures.

Similarly, a classical SVM may perform differently depending on:

- Kernel.
- Regularization.
- Kernel parameters.
- Feature scaling.

Therefore, optimization is necessary to determine which configuration works best for the selected biomedical dataset.

23.3 Optimization Should Not Use the Test Set

One of the most important rules is:

The test dataset must remain unseen during model optimization.

The optimization process should use:

Training Data

for learning parameters,

and:

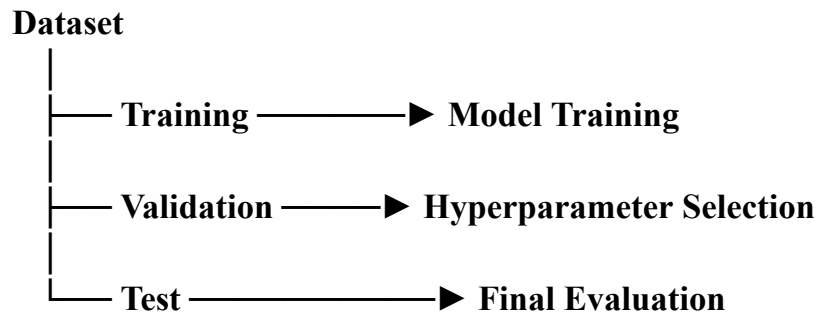
Validation Data

for selecting the best configuration.

The final:

Test Data

should only be used after optimization is complete.



This prevents overly optimistic performance estimates.

23.4 Classical Model Optimization

Classical models can be optimized using appropriate hyperparameter-search techniques.

Logistic Regression

Possible parameters:

- Regularization strength.
- Regularization type.
- Solver.

SVM

Possible parameters:

- Kernel.
- C.
- Gamma.
- Kernel parameters.

Random Forest

Possible parameters:

- Number of trees.
- Maximum depth.
- Minimum samples per split.

- **Maximum features.**

XGBoost / Gradient Boosting

Possible parameters:

- **Number of estimators.**
- **Learning rate.**
- **Maximum depth.**
- **Subsampling.**
- **Regularization.**

Neural Network

Possible parameters:

- **Number of layers.**
- **Hidden units.**
- **Learning rate.**
- **Batch size.**
- **Number of epochs.**
- **Dropout.**

Only relevant parameters need to be exposed in the prototype.

23.5 Quantum Model Optimization

Quantum models require optimization at multiple levels.

These include:

Data-Level Optimization

- **Number of features.**
- **Feature selection.**
- **Dimensionality reduction.**
- **Feature scaling.**

Circuit-Level Optimization

- **Number of qubits.**

- **Circuit depth.**
- **Ansatz structure.**
- **Entanglement pattern.**
- **Feature map.**

Training-Level Optimization

- **Learning rate.**
- **Classical optimizer.**
- **Number of iterations.**
- **Batch size where applicable.**
- **Early stopping.**

This makes QML optimization more complex than simply changing one model parameter.

23.6 Quantum Circuit Optimization

The platform should attempt to keep quantum circuits as compact as reasonably possible.

A circuit can be represented as:

Feature Encoding



Trainable Layer



Entanglement



Trainable Layer



Measurement

Optimization can explore:

- **Fewer layers.**
- **Fewer gates.**
- **Simpler entanglement.**
- **Fewer trainable parameters.**

- Smaller feature dimensions.

The objective is to preserve useful predictive capability while reducing quantum resource requirements.

23.7 Circuit Depth Optimization

Circuit depth is an important optimization parameter.

For example:

Circuit Depth	Accuracy	Runtime	Qubit Count
2	—	—	—
3	—	—	—
4	—	—	—
5	—	—	—

The platform can experimentally identify whether additional circuit layers actually improve performance.

If deeper circuits provide little improvement while significantly increasing computational cost, the shallower configuration may be preferred.

23.8 Number of Qubits Optimization

The platform can evaluate different quantum feature dimensions.

For example:

High-Dimensional Dataset



Feature Selection



4 Components —► 4-Qubit Model



6 Components —► 6-Qubit Model



8 Components —► 8-Qubit Model

Each configuration can be benchmarked.

The goal is not to maximize the number of qubits.

Instead:

Use sufficient quantum resources to achieve useful predictive performance.

23.9 Feature-Dimension Optimization

Feature dimension is particularly important for biomedical datasets.

For example:

Component s	Classical Performance	Hybrid QML Performance
4	—	—
6	—	—
8	—	—
10	—	—

This experiment can reveal whether increasing the number of features improves predictive performance enough to justify additional quantum resources.

23.10 Quantum Feature Map Optimization

Different quantum feature maps can produce different representations.

The platform can compare experimentally:

- Feature map A.
- Feature map B.
- Feature map C.

For each configuration, it can measure:

- Accuracy.
- Sensitivity.
- Specificity.
- F1-score.
- ROC-AUC.
- Circuit depth.
- Number of gates.
- Execution time.

The best feature map should therefore be selected based on measured results.

23.11 Entanglement Optimization

Entanglement structure can also be optimized.

Possible structures include:

Linear

Q0 — Q1 — Q2 — Q3

Circular

Q0 — Q1

| |

Q3 — Q2

Other hardware-compatible structures

The platform can compare these structures experimentally.

The objective is to determine whether additional connectivity improves learning enough to justify the associated circuit complexity.

23.12 Classical Optimizer for QML

Variational quantum models require classical optimization.

The optimizer updates quantum circuit parameters based on the measured loss.

The workflow is:

Quantum Parameters



Quantum Circuit



Measurement



Loss



Classical Optimizer



Updated Parameters



Quantum Circuit

Possible optimizers include suitable gradient-based or gradient-free methods depending on the selected QML framework and objective.

The platform can treat the optimizer as an experiment parameter.

23.13 Learning Rate Optimization

For trainable hybrid models, the learning rate can strongly affect convergence.

A learning rate that is too high may cause unstable training.

A learning rate that is too low may make training unnecessarily slow.

The platform can therefore evaluate multiple configurations.

For example:

Learning Rate	Validation F1	Training Time
0.001	—	—
0.01	—	—
0.05	—	—
0.1	—	—

The actual search range should depend on the selected optimizer and model.

23.14 Epoch and Iteration Optimization

For VQC and QNN models, training can require repeated optimization iterations.

Too few iterations may result in underfitting.

Too many iterations may:

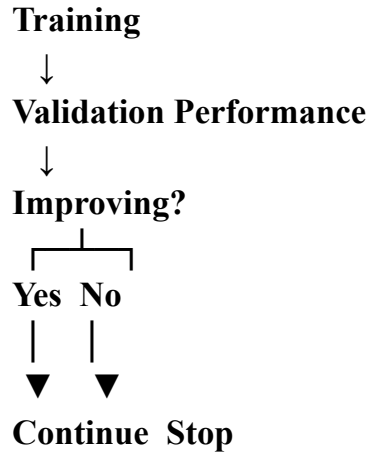
- Increase execution time.
- Increase quantum circuit evaluations.
- Increase computational cost.
- Provide little additional benefit.
- Increase the risk of overfitting.

Therefore, the platform can use validation performance and early stopping where appropriate.

23.15 Early Stopping

Early stopping can terminate training when validation performance stops improving.

Conceptually:



This can reduce unnecessary computation.

23.16 Overfitting Control

Biomedical datasets may have many features compared with the number of available samples.

This creates a risk of overfitting.

The platform can reduce this risk through:

- **Feature selection.**
- **Dimensionality reduction.**
- **Regularization.**
- **Cross-validation.**
- **Early stopping.**
- **Controlled circuit complexity.**
- **Separate test data.**
- **Repeated experiments.**

The optimization process should prioritize generalization rather than simply maximizing training accuracy.

23.17 Optimization Objective Function

A simple optimization objective can focus on validation performance.

For example:

$$\text{Best Model} = \arg\max(\text{Model Performance}_{\text{Validation}})$$
$$\text{Best Model} = \arg\max(\text{Model Performance}_{\{\text{Validation}\}})$$

However, because quantum models also have resource costs, the platform can evaluate a broader trade-off.

Conceptually:

$$\text{Utility} = \text{Performance} - \lambda(\text{Resource Cost})$$
$$\text{Utility} = \text{Performance} - \lambda(\text{Resource Cost})$$

where λ represents how strongly resource cost is considered.

The exact formulation can be selected based on the experimental objective.

The platform should report the individual performance and resource metrics rather than hiding them inside a single score.

23.18 Multi-Objective Optimization

The project can treat optimization as a multi-objective problem involving:

Objective 1

Maximize predictive performance.

Objective 2

Maximize sensitivity.

Objective 3

Maximize specificity.

Objective 4

Improve generalization.

Objective 5

Minimize computational cost.

Objective 6

Minimize quantum resource usage.

This produces a more realistic model-selection process.

23.19 Performance vs Resource Trade-Off

Consider two configurations:

Configuration	Accuracy	Qubits	Depth	Runtime
Model A	91%	4	2	Low
Model B	92%	10	7	High

Model B has slightly higher accuracy but requires substantially more resources.

The platform should allow researchers to decide whether the additional performance justifies the additional cost.

This is especially important for near-term quantum hardware.

23.20 Optimization Search Methods

The platform can support suitable search strategies such as:

Grid Search

Tests predefined parameter combinations.

Useful when the search space is small.

Random Search

Samples combinations from predefined ranges.

Useful when many parameters are available.

Bayesian Optimization

Can intelligently explore promising configurations when optimization becomes expensive.

For the SIH prototype, grid or randomized search may be sufficient for a controlled demonstration, while more advanced optimization can be added later.

23.21 Cross-Validation During Optimization

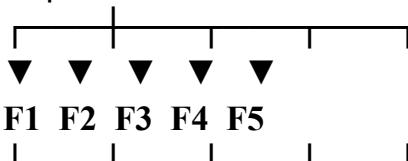
When the dataset size permits, cross-validation can be used during model selection.

For example:

Training Data



5-Fold Cross Validation



Mean Validation Score



Parameter Selection

This can provide a more stable estimate than relying on a single validation split.

23.22 Repeated Optimization

Optimization results can depend on:

- Random initialization.
- Data split.
- Parameter initialization.
- Quantum measurement statistics.

Therefore, important experiments can be repeated using different random seeds.

The platform can report:

- Mean performance.
- Standard deviation.
- Best performance.
- Worst performance.

This provides a better understanding of model stability.

23.23 Noise-Aware Optimization

For hardware-oriented QML, the model can also be optimized under noisy conditions.

For example:

Circuit Configuration A



Noisy Simulation



Performance

Circuit Configuration B



Noisy Simulation



Performance

Circuit Configuration C



Noisy Simulation



Performance

A configuration that performs well under noise may be more useful for near-term hardware than one that performs well only under ideal simulation.

23.24 Hardware-Aware Optimization

If hardware access is available, optimization can consider:

- **Qubit connectivity.**
- **Native gates.**
- **Circuit depth.**
- **Hardware error rates where available.**
- **Number of circuit executions.**
- **Measurement shots.**
- **Execution time.**

This can help identify configurations that are practically executable.

23.25 Optimization Pipeline

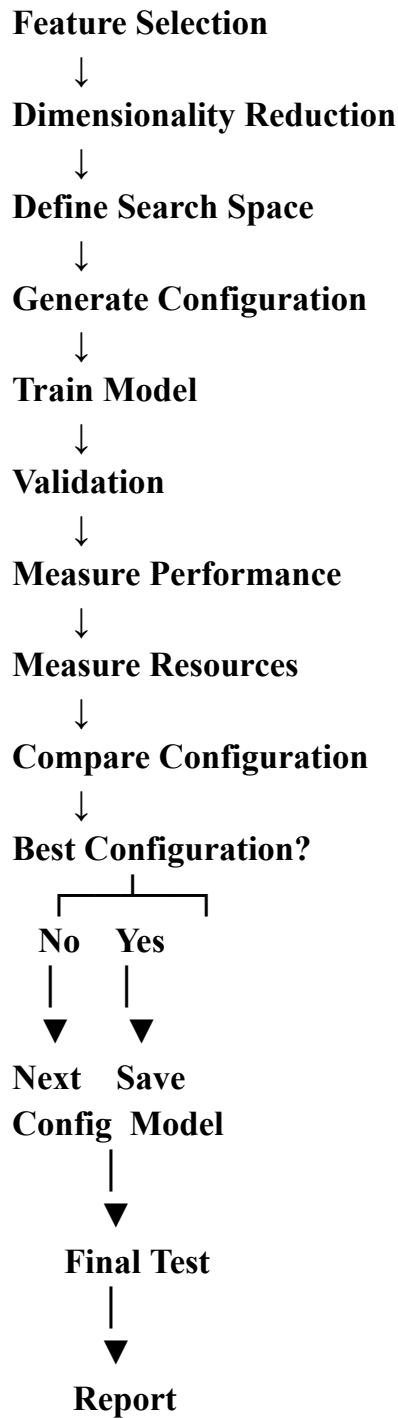
The complete optimization workflow can be:

Biomedical Dataset



Preprocessing





23.26 Optimization for Classical vs Hybrid Models

Optimization should be applied fairly to both model families.

The classical baseline should not be intentionally under-tuned.

For example:

Classical Models



Hyperparameter Optimization



Best Classical Configuration

Hybrid QML Models



Quantum + Classical Optimization



Best Hybrid Configuration



Final Benchmark

This supports a credible comparison.

23.27 Experiment Tracking

Every optimization experiment should record:

- **Experiment ID.**
- **Dataset version.**
- **Feature count.**
- **Preprocessing configuration.**
- **Model type.**
- **Hyperparameters.**
- **Quantum encoding.**
- **Qubit count.**
- **Circuit depth.**
- **Entanglement structure.**
- **Optimizer.**
- **Learning rate.**
- **Number of iterations.**

- Backend.
- Performance metrics.
- Runtime.
- Resource usage.

This makes optimization reproducible.

23.28 Optimization Dashboard

A dashboard can display:

Best Model

—

Best Accuracy

—

Best Sensitivity

—

Best Specificity

—

Best F1

—

Best ROC-AUC

—

Qubits

—

Circuit Depth

—
Training Time

—
Backend

—

23.29 Hyperparameter Search Dashboard

The platform can also show:

Optimization

Model: VQC

Feature Count: [4, 6, 8]

Circuit Depth: [2, 3, 4]

Learning Rate: [0.01, 0.05, 0.1]

Optimizer: [Selected Options]

[Start Optimization]

Experiments Completed: —

Best Validation Score: —

Best Configuration: —

This makes the optimization process visible during the SIH demonstration.

23.30 Model Selection

After optimization, the platform should select the best configuration according to the predefined evaluation criteria.

The selection should consider:

- Predictive performance.
- Sensitivity.
- Specificity.
- Generalization.
- Stability.
- Computational efficiency.
- Quantum resources.

The final selection criteria should be documented.

23.31 Avoiding Optimization Bias

The platform must avoid repeatedly tuning the model based on the final test results.

Correct workflow:

Training → Validation → Optimization → Freeze Configuration → Final Test

Incorrect workflow:

Training → Test → Change Model → Test Again → Change Model → Test Again

The second approach can produce overly optimistic results.

23.32 Optimization Output

The module should produce:

1. Best classical model configuration.
2. Best hybrid QML configuration.
3. Best feature dimension.
4. Best quantum encoding.
5. Best circuit depth.
6. Best qubit configuration.

7. Best optimizer configuration.
 8. Best learning rate where applicable.
 9. Validation performance.
 10. Final test performance.
 11. Training time.
 12. Inference time.
 13. Quantum resource usage.
 14. Optimization history.
 15. Reproducibility information.
-

23.33 Example Optimization Report

MODEL OPTIMIZATION REPORT

Dataset:

[Dataset Name]

Selected Model:

[VQC / Quantum Kernel / QNN]

Features:

8

Qubits:

8

Circuit Depth:

3

Encoding:

Angle Encoding

Optimizer:

[Selected Optimizer]

Learning Rate:

[Value]

Validation Accuracy:

—

Validation Sensitivity:

—

Validation Specificity:

—

Validation F1:

—

Final Test Accuracy:

—

Training Time:

—

Inference Time:

—

Backend:

Ideal / Noisy / Hardware

All performance values should be populated from actual experiments.

23.34 Optimization and Explainability

Optimization should not create an unnecessarily complicated model that becomes impossible to analyze.

Therefore, the platform can consider explainability alongside performance.

For example:

Model A

High performance + simpler architecture.

Model B

Slightly higher performance + much greater complexity.

Model A may be preferable depending on the research objective.

This reinforces the project's emphasis on interpretable and practical AI.

23.35 Optimization and Medical Reliability

The optimization process should not simply maximize accuracy.

For disease detection, important considerations include:

- **Sensitivity.**
- **Specificity.**
- **False negatives.**
- **False positives.**
- **Calibration where applicable.**
- **Generalization.**
- **Stability.**

A model with slightly lower accuracy but substantially better sensitivity may be more relevant for an early-detection research scenario.

The final choice should depend on the defined evaluation criteria.

23.36 Optimization and Quantum Advantage

Optimization should never be designed to guarantee that the QML model wins.

Instead, it should answer:

What is the best experimentally supported configuration for each model family under comparable conditions?

The optimized classical model and optimized hybrid QML model can then be benchmarked fairly.

This avoids biased experimentation.

23.37 Expected Outputs

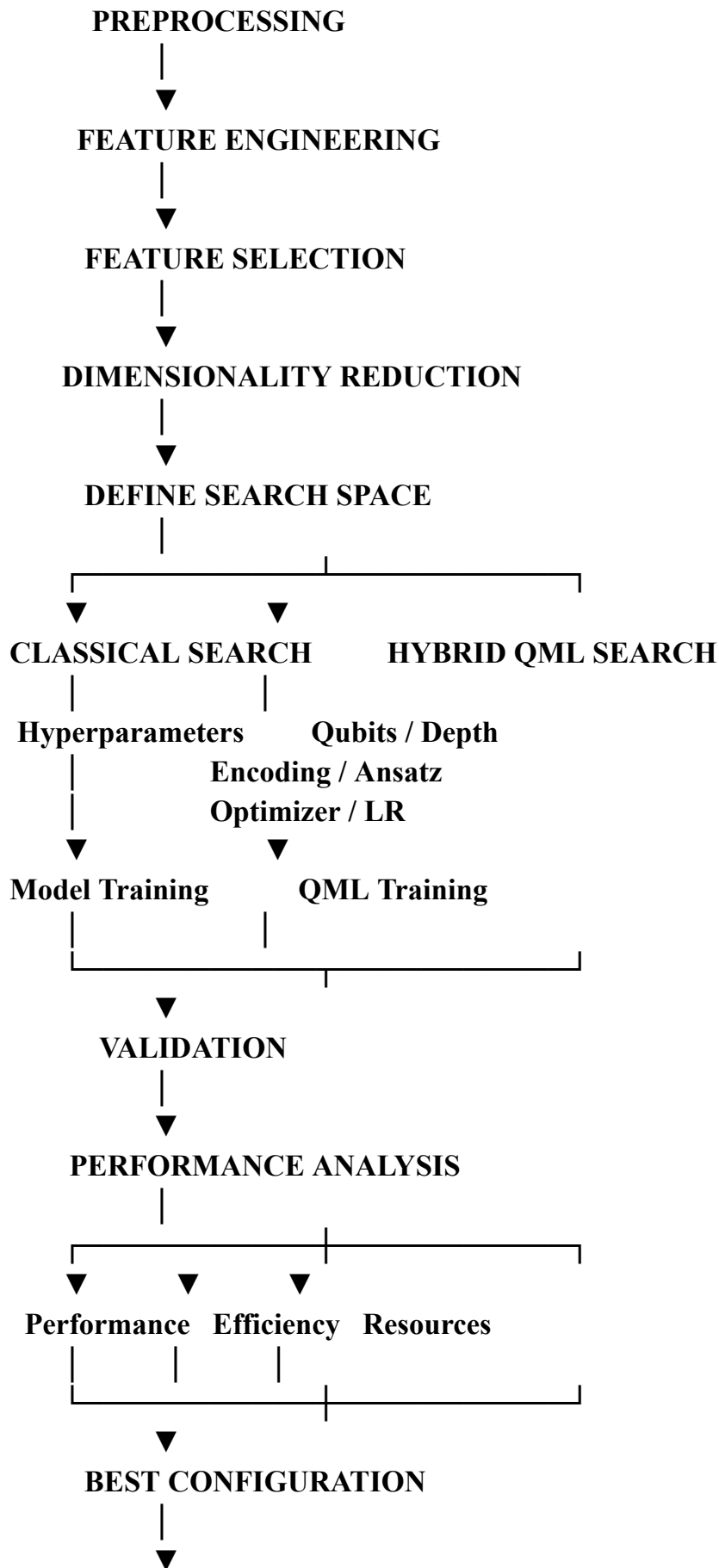
The complete Model Optimization Module should provide:

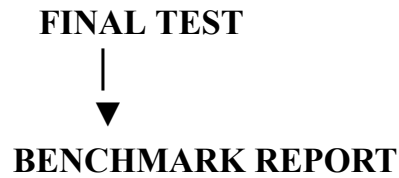
- Automated hyperparameter search.
 - Classical model optimization.
 - Quantum model optimization.
 - Feature-dimension optimization.
 - Circuit-depth optimization.
 - Qubit-count optimization.
 - Encoding comparison.
 - Entanglement comparison.
 - Classical optimizer selection.
 - Learning-rate tuning.
 - Early stopping.
 - Cross-validation.
 - Repeated experiments.
 - Noise-aware optimization.
 - Hardware-aware optimization.
 - Resource-performance analysis.
 - Optimization history.
 - Best-model selection.
 - Reproducible experiment configuration.
-

23.38 Complete Model Optimization Architecture

BIOMEDICAL DATA







23.39 Key Design Principle

The central principle of model optimization is:

Find the best-performing practical model, not simply the most complex model.

For the proposed platform, optimization must balance:

Performance

-

Sensitivity

-

Specificity

-

Generalization

-

Interpretability

-

Computational Efficiency

-

Quantum Resource Requirements

This makes the optimization process aligned with the actual objectives of the problem statement.

23.40 Summary

The Model Optimization Module systematically searches for effective configurations of both classical and hybrid QML models.

It optimizes:

Classical Hyperparameters

→ **Feature Dimensions**

→ **Quantum Encoding**

→ **Qubit Count**

→ **Circuit Depth**

→ **Entanglement**

→ **Classical Optimizer**

→ **Training Parameters**

→ **Noise Robustness**

→ **Hardware Compatibility**

The optimized models are then evaluated on unseen test data and compared using the project's common benchmarking framework.

Most importantly, optimization does not assume that the quantum model must outperform classical ML.

Instead:

The platform experimentally determines which configuration provides the best balance between disease-detection performance, generalization, interpretability, computational efficiency, and quantum resource requirements.

24. Platform Dashboard

24.1 Purpose of the Platform Dashboard

The Platform Dashboard is the main user interface of the Hybrid Quantum Machine Learning Platform. It provides a single place where users can manage biomedical datasets, configure preprocessing, train classical and hybrid quantum models, evaluate results, view explanations, and compare model performance.

Instead of requiring users to interact separately with data-processing scripts, machine-learning libraries, quantum simulators, and evaluation tools, the dashboard integrates the complete workflow into one structured platform.

The dashboard follows the complete project pipeline:

Dataset → Preprocessing → Feature Engineering → Feature Selection → Dimensionality Reduction → Classical/QML Model → Training → Prediction → Explainability → Benchmarking → Final Results

The primary goal is to make the proposed QML system usable, interpretable, reproducible, and easy to demonstrate, while still exposing the important technical parameters required for experimentation.

24.2 Dashboard Design Philosophy

The dashboard should be designed around five principles:

1. **Simple** – users should be able to understand the workflow without needing to manually write quantum-circuit code.
2. **Configurable** – researchers should be able to change preprocessing, features, models, and quantum parameters.
3. **Transparent** – the system should clearly show how data moves through the pipeline.
4. **Benchmark-driven** – classical and hybrid QML results should be displayed side-by-side.
5. **Experiment-oriented** – every experiment should record its configuration, results, and resource usage.

The dashboard should not present the quantum model as automatically superior. Instead, it should allow the measured results to determine whether the hybrid approach provides a useful improvement.

24.3 Main Dashboard Layout

The main dashboard can be divided into the following sections:

1. Overview

Displays the current project and experiment status.

Example information:

- Selected dataset
- Number of samples
- Number of features
- Target disease
- Selected preprocessing method
- Number of selected features
- Number of PCA components
- Selected classical models
- Selected QML models
- Current quantum backend
- Training status
- Best-performing model
- Current experiment ID

A simple status flow can be displayed as:

Data ✓ → Preprocessing ✓ → Features ✓ → Training ⊙ → Evaluation ○ → Explainability ○

This allows the user to immediately understand where the experiment currently stands.

24.4 Dataset Upload and Data Management Screen

The first major dashboard screen is the Dataset Management section.

Users can upload or select a biomedical dataset.

Supported data sources can include:

- **Clinical datasets**
- **Electronic Health Records**
- **Genomic datasets**
- **Medical imaging-derived numerical features**
- **Benchmark biomedical datasets**

For the prototype, CSV-based datasets can provide a simple and reliable starting point.

After uploading a dataset, the dashboard automatically displays:

- **Dataset name**
- **Number of samples**
- **Number of features**
- **Numerical columns**
- **Categorical columns**
- **Missing values**
- **Duplicate records**
- **Target column**
- **Class distribution**
- **Potential invalid values**

A Data Quality Report can be generated before training begins.

This prevents poor-quality data from silently entering the machine-learning pipeline.

24.5 Preprocessing Configuration Screen

The preprocessing screen allows users to configure the data-cleaning pipeline.

Possible controls include:

Missing Values

- **Mean imputation**
- **Median imputation**
- **Mode imputation**

- **Configurable handling of missing records**

Categorical Features

- **One-hot encoding**
- **Label encoding where appropriate**

Numerical Features

- **Standardization**
- **Min-max scaling**

Duplicate Handling

- **Detect duplicates**
- **Remove duplicates where appropriate**

Outlier Analysis

The dashboard can identify potential outliers and display their distribution.

Data Splitting

Users can configure:

- **Training percentage**
- **Validation percentage**
- **Testing percentage**
- **Random seed**

The test dataset remains isolated from model optimization.

This is particularly important because the platform is intended for fair benchmarking between classical and hybrid QML models.

24.6 Feature Engineering Screen

The dashboard provides controls for creating meaningful features from the processed biomedical data.

Depending on the dataset, users can configure:

- **Statistical features**
- **Ratios**
- **Differences**
- **Interaction features**
- **Log transformations**
- **Domain-specific derived features**

For example, a cardiovascular dataset may contain features related to:

- **Age**
- **Blood pressure**
- **Cholesterol**
- **Heart rate**
- **Other clinical measurements**

The dashboard can show both the original and engineered features.

Each generated feature should have metadata describing how it was created.

This improves experiment reproducibility and helps users understand the input given to the models.

24.7 Feature Selection Screen

The Feature Selection section allows users to reduce unnecessary features before model training.

Available techniques may include:

- **Correlation-based selection**
- **Mutual information**
- **Statistical feature selection**
- **Recursive Feature Elimination**
- **Tree-based feature importance**

The dashboard can provide a feature-ranking visualization.

Example:

Feature	Importance	Selected
Feature A	0.91	✓
Feature B	0.83	✓
Feature C	0.42	✓
Feature D	0.08	✗

Users can configure the desired number of features.

This becomes especially important for QML because quantum models may have limited qubit and circuit resources.

24.8 Dimensionality Reduction Screen

Biomedical datasets can contain hundreds or thousands of features, while current quantum systems generally work with much smaller input dimensions.

The dashboard therefore provides a dimensionality-reduction stage.

The primary prototype method can be:

Principal Component Analysis (PCA)

Users can configure:

- Number of components
- Variance retention target
- Scaling method

The dashboard can display:

- Original feature count
- Reduced feature count

- Explained variance
- Cumulative explained variance

Example:

2,000 features → 100 features → 20 features → 8 quantum-compatible components

This gives the user a visual understanding of how the original biomedical data is compressed before quantum processing.

24.9 Classical Model Selection Screen

The platform should provide a dedicated section for classical baseline models.

Possible models include:

- Logistic Regression
- Support Vector Machine
- Random Forest
- XGBoost / Gradient Boosting
- Neural Network

Users can select one or more models for benchmarking.

For each model, the dashboard can expose relevant configuration parameters.

For example:

SVM

- Kernel
- Regularization parameter
- Gamma

Random Forest

- Number of trees
- Maximum depth
- Minimum samples per split

Neural Network

- **Number of layers**
- **Hidden units**
- **Learning rate**
- **Epochs**

The same prepared dataset should be used for comparable experiments.

24.10 Quantum Model Configuration Screen

The Quantum ML section is one of the key parts of the dashboard.

Users can select:

Quantum Kernel / QSVM

Configuration can include:

- **Feature map**
- **Number of qubits**
- **Kernel settings**
- **Quantum backend**

Variational Quantum Classifier

Configuration can include:

- **Number of qubits**
- **Feature map**
- **Ansatz**
- **Number of layers**
- **Entanglement structure**
- **Classical optimizer**
- **Learning rate**
- **Number of iterations**

Quantum Neural Network

Configuration can include:

- **Number of qubits**
- **Circuit depth**

- **Parameterized gates**
- **Entanglement configuration**
- **Optimizer**
- **Training iterations**

The dashboard should also show a quantum resource estimate before training.

For example:

Features: 8

Qubits: 8

Circuit depth: 6

Trainable parameters: 48

This allows users to understand the computational requirements of the selected QML configuration.

24.11 Quantum Backend Selection

The dashboard should provide a backend selector.

Possible options:

- **Ideal Quantum Simulator**
- **Noisy Quantum Simulator**
- **Near-Term Quantum Hardware**

The default prototype mode can be the simulator.

A backend status panel can display:

Backend: Ideal Simulator

Qubits: 8

Noise: Disabled

Shots: 1,024

Status: Available

When a real quantum backend is available, the same QML model can be routed through the backend abstraction layer.

This allows the platform to maintain the same overall workflow while changing the execution environment.

24.12 Training Control Screen

The training screen provides control over the complete experiment.

Users can start:

Classical Training

or

Hybrid QML Training

The dashboard should display real-time or periodically updated information such as:

- Current epoch/iteration
- Training loss
- Validation loss
- Validation performance
- Current model parameters
- Training time
- Quantum circuit executions
- Number of shots
- Backend status

For VQC/QNN training, the conceptual loop displayed to the user is:

Input → Quantum Encoding → Quantum Circuit → Measurement → Loss →
Classical Optimizer → Parameter Update → Repeat

This makes the hybrid nature of the proposed solution visible during the demonstration.

24.13 Prediction Screen

After training, users can use the Prediction section to evaluate unseen samples.

The workflow is:

New Biomedical Data → Same Preprocessing → Feature Transformation → Trained Model → Prediction

The dashboard can display:

- Predicted class
- Prediction score
- Selected model
- Model version
- Processing time
- Explanation

For example:

Prediction: Higher-risk class

Model: Hybrid VQC

Confidence/score: Model output

Backend: Quantum Simulator

The interface should clearly state that the output is a model prediction for research/decision-support purposes, not a clinically validated diagnosis.

24.14 Explainability Screen

The dashboard should contain a dedicated Explainable AI section.

This section helps users understand why a model produced a particular prediction.

For classical models, the platform can display:

- Feature importance
- SHAP-based explanations where appropriate
- Local feature contributions
- Global feature importance

For hybrid QML models, the dashboard can provide suitable model-level and perturbation-based analyses, such as:

- Feature sensitivity
- Feature ablation
- Prediction changes after controlled input perturbation
- Quantum circuit configuration
- Feature-map information
- Number of qubits
- Circuit depth
- Measurement configuration

The interface should clearly distinguish model explanation from clinical interpretation.

For example:

“Feature X contributed strongly to this model prediction.”

should not automatically be interpreted as:

“Feature X caused the disease.”

This distinction is important for responsible biomedical AI.

24.15 Performance Evaluation Screen

The evaluation screen displays the performance of each trained model.

Important metrics include:

- Accuracy
- Precision
- Recall
- Sensitivity
- Specificity
- F1-score
- ROC-AUC

The dashboard can include:

Confusion Matrix

Shows:

- **True Positives**
- **True Negatives**
- **False Positives**
- **False Negatives**

ROC Curve

Allows comparison of classification performance across models.

Precision-Recall Curve

Useful when class imbalance is important.

Error Analysis

The dashboard can identify where models make incorrect predictions.

24.16 Classical vs Hybrid QML Comparison

This is one of the most important dashboard components for the SIH demonstration.

A comparison table can be generated automatically:

Model	Accuracy	Sensitivity	Specificity	F1	ROC-AUC	Training Time
Logistic Regression	—	—	—	—	—	—
SVM	—	—	—	—	—	—
Random Forest	—	—	—	—	—	—
XGBoost	—	—	—	—	—	—
Quantum Kernel	—	—	—	—	—	—

VQC	—	—	—	—	—	—
QNN	—	—	—	—	—	—

The values should come from actual experiments rather than predefined claims.

The dashboard can highlight:

- Best accuracy
- Best sensitivity
- Best specificity
- Best ROC-AUC
- Fastest model
- Lowest resource usage
- Best generalization

This ensures that the platform evaluates the quantum approach objectively.

24.17 Quantum Resource Dashboard

Because this project specifically focuses on QML, the dashboard should also report quantum resource usage.

Important measurements include:

- Number of qubits
- Circuit depth
- Number of gates
- Trainable parameters
- Number of circuit executions
- Number of shots
- Simulator execution time
- Hardware execution time when available
- Noise configuration

This provides a more complete comparison than accuracy alone.

A model achieving slightly better accuracy but requiring substantially greater quantum resources should be identified accordingly.

24.18 Experiment Tracking

Every experiment should have a unique experiment record.

The system can store:

- Dataset version
- Dataset size
- Preprocessing configuration
- Selected features
- PCA configuration
- Classical model configuration
- Quantum model configuration
- Backend
- Number of qubits
- Circuit depth
- Training parameters
- Random seed
- Evaluation metrics
- Explainability results
- Execution time

Example:

Experiment ID: EXP-024

Dataset: Cardiovascular Benchmark

Features: 8

Model: VQC

Backend: Noisy Simulator

Qubits: 8

Accuracy: Measured result

Sensitivity: Measured result

ROC-AUC: Measured result

This makes the platform reproducible and suitable for research experimentation.

24.19 Results Dashboard

The final results page summarizes the complete experiment.

It can contain four major panels:

Model Performance

Displays:

- Accuracy
- Sensitivity
- Specificity
- F1
- ROC-AUC

Computational Performance

Displays:

- Training time
- Inference time
- Memory/resource usage

Quantum Resources

Displays:

- Qubits
- Circuit depth
- Gates
- Shots
- Circuit executions
- Backend

Explainability

Displays:

- Important features
- Feature contributions
- Error patterns
- Model behavior

The dashboard therefore provides both prediction quality and computational/quantum resource information.

24.20 Architecture Behind the Dashboard

The dashboard can follow a modular software architecture:

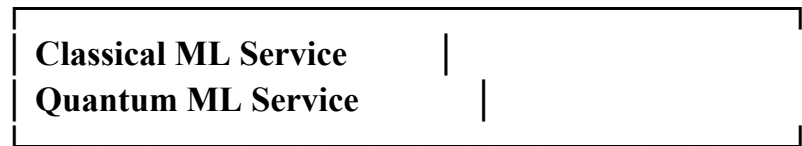
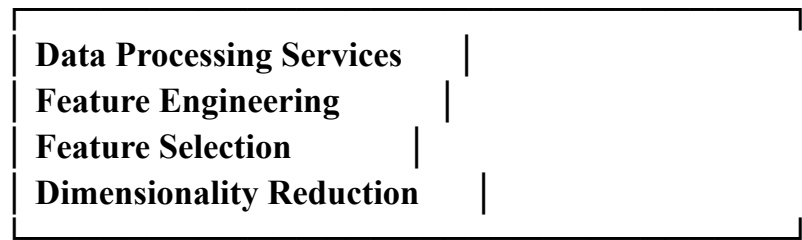
Frontend Dashboard



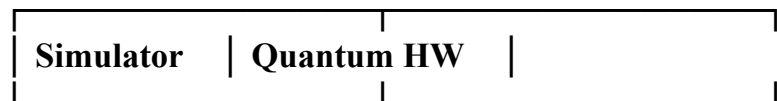
FastAPI Backend



Experiment Management Layer



Quantum Backend Abstraction



Evaluation + XAI



Results Database



Dashboard Visualization

A possible implementation stack is:

Frontend: React / Next.js

Backend: Python + FastAPI

ML: Scikit-learn / XGBoost

QML: Qiskit / PennyLane or another suitable quantum framework

Data Processing: Pandas / NumPy

Visualization: Plotly / suitable charting library

Database: PostgreSQL or equivalent

Execution: Local/cloud simulator and compatible quantum hardware backend

The exact libraries can be selected based on implementation requirements.

24.21 User Experience and Navigation

A simple sidebar can organize the platform into:

Dashboard

- |**
- | — Dataset**
- | — Preprocessing**
- | — Feature Engineering**
- | — Feature Selection**
- | — Dimensionality Reduction**
- | — Classical Models**
- | — Quantum Models**
- | — Training**
- | — Prediction**
- | — Explainability**
- | — Benchmarking**
- | — Experiments**
- | — Results**

This follows the same sequence as the underlying ML/QML pipeline, making the system easy to understand during an SIH presentation.

24.22 Validation and Error Messages

The dashboard should prevent invalid experiments before training.

Examples:

- Target column not selected
- Missing target values
- Insufficient samples
- Unsupported feature type
- Number of features greater than selected quantum resources
- Invalid quantum circuit configuration
- Unsupported hardware gate
- Backend unavailable
- Training failure
- Invalid prediction input

Instead of silently failing, the platform should display a clear explanation and possible corrective action.

24.23 Security and Responsible Use

If sensitive biomedical data is used, the dashboard should support appropriate security practices such as:

- Authentication
- Role-based access where required
- Secure API communication
- Controlled dataset access
- Experiment access control
- Audit logging
- Secure storage
- De-identified data for prototype demonstrations

The dashboard should also display a clear notice:

“This platform is intended for research and decision-support experimentation. Model predictions should not be treated as clinically validated diagnoses without appropriate medical validation.”

24.24 Expected Dashboard Output

At the end of an experiment, the dashboard should allow the user to obtain a complete result containing:

1. Dataset summary
2. Data-quality report
3. Preprocessing configuration
4. Feature-engineering configuration
5. Selected features
6. Dimensionality-reduction results
7. Classical model results
8. Quantum model results
9. Hybrid training information
10. Prediction results
11. Explainability results
12. Classical vs QML benchmark
13. Quantum resource usage
14. Simulator/hardware information
15. Generalization results
16. Experiment configuration
17. Final performance summary

This creates a complete and reproducible experimental record.

24.25 Role in the Overall Solution

The Platform Dashboard acts as the control and visualization layer of the entire Hybrid Quantum Machine Learning system.

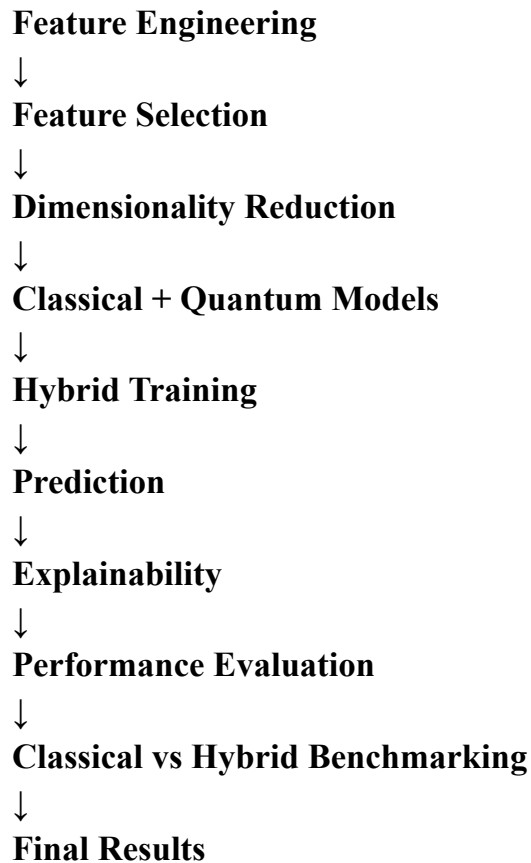
It connects all major modules:

Data Ingestion

↓

Preprocessing

↓



Therefore, the dashboard is not merely a visualization page. It is the interface through which the complete proposed solution can be configured, executed, monitored, evaluated, and demonstrated.

24.26 Key Design Principle

“One platform, complete biomedical ML-to-QML workflow, measurable results.”

The dashboard should make the complete research process visible while allowing fair experimentation between classical and hybrid quantum approaches.

Most importantly, the platform should measure the actual performance of QML rather than assuming quantum advantage. If a classical model performs better, the dashboard should report that result honestly. If a hybrid model performs better under the tested conditions, the system should provide the experimental evidence supporting that conclusion.

This makes the proposed platform practical, transparent, research-oriented, and closely aligned with the SIH problem statement.

25. End-to-End System Workflow

25.1 Overview

The End-to-End System Workflow describes how the proposed Hybrid Quantum Machine Learning Platform processes biomedical data from the initial dataset upload to the final disease prediction, explainability, and performance comparison.

The workflow combines classical machine-learning operations with quantum-enhanced learning in a single pipeline.

The complete workflow is:

Biomedical Dataset → Data Ingestion → Data Validation → Preprocessing → Feature Engineering → Feature Selection → Dimensionality Reduction → Classical Baseline + Quantum Feature Encoding → Hybrid QML Training → Prediction → Explainability → Performance Evaluation → Classical vs Hybrid Benchmarking → Final Report

The important design principle is that classical computing performs the data-intensive preparation tasks, while quantum computing is introduced at the model-learning stage where it can be experimentally evaluated.

25.2 Step 1 – Biomedical Dataset Input

The workflow begins with a biomedical dataset.

The platform can work with datasets related to:

- Cancer
- Cardiovascular disorders
- Neurological conditions
- Genomics
- Medical imaging-derived features
- Electronic Health Records
- Other suitable benchmark biomedical datasets

For the prototype, a structured dataset such as CSV can be used.

The user selects:

- **Dataset**
- **Target/disease column**
- **Feature columns**
- **Classification objective**

The system then creates a dataset profile.

Example

Dataset: Cardiovascular Disease Dataset

Samples: 1,000

Original Features: 20

Target: Disease

Classes: 0 / 1

25.3 Step 2 – Data Ingestion and Validation

After dataset upload, the Data Ingestion Module validates the input.

The system checks:

- **File format**
- **Dataset structure**
- **Column names**
- **Data types**
- **Missing values**
- **Duplicate records**
- **Invalid values**
- **Target availability**
- **Class distribution**
- **Number of samples**
- **Number of features**

A data-quality report is generated.

For example:

Dataset Validation

Samples : 1,000
Features : 20
Missing Values : 32
Duplicate Rows : 4
Target Column : Disease
Class 0 : 540
Class 1 : 460
Status : Valid

If critical problems are detected, the platform should stop the pipeline before model training.

25.4 Step 3 – Data Preprocessing

The validated dataset enters the preprocessing stage.

The system performs operations such as:

Missing-Value Handling

Depending on the dataset:

- **Mean imputation**
- **Median imputation**
- **Mode imputation**
- **Appropriate row/feature removal**

Categorical Encoding

Categorical variables are converted into numerical representations where required.

Scaling

Numerical features can be standardized or normalized.

This is particularly important before quantum feature encoding because quantum rotation angles require controlled numerical ranges.

Duplicate Handling

Duplicate records can be identified and handled.

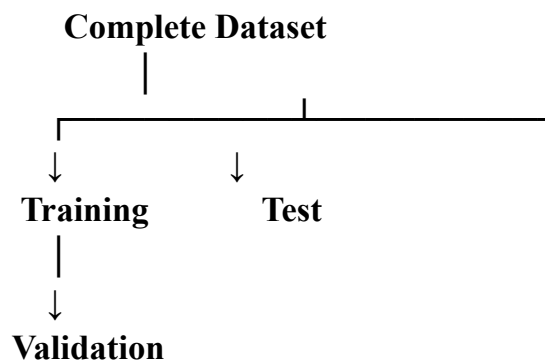
Outlier Analysis

Potential outliers are detected and analyzed without automatically deleting medically meaningful observations.

25.5 Step 4 – Train, Validation, and Test Split

After preprocessing, the data is divided into independent subsets.

A typical structure is:



For example:

- 70% Training
- 15% Validation
- 15% Testing

The exact split can be configurable.

The important rule is:

The test set must remain unseen during preprocessing decisions, model optimization, and hyperparameter tuning.

This prevents data leakage and produces a more reliable final evaluation.

25.6 Step 5 – Feature Engineering

The system transforms the cleaned features into more useful representations.

Depending on the biomedical domain, this may include:

- Statistical features
- Ratios
- Differences
- Interaction features
- Log transformations
- Domain-specific derived features
- Imaging-derived numerical features
- Genomic feature transformations

For example:

Original Features



Clinical Features



Derived Features



Interaction Features

The objective is not to create as many features as possible.

Instead, the platform should create meaningful and experimentally justified features.

25.7 Step 6 – Feature Selection

The engineered dataset may contain redundant or irrelevant features.

The feature-selection stage reduces this problem.

Possible techniques include:

- Correlation-based selection
- Mutual information
- Statistical selection
- Recursive Feature Elimination
- Tree-based feature importance

The system generates a ranked feature list.

Example:

2,000 Features



500 Relevant Features



50 Selected Features

Feature selection is especially important for the quantum branch because current quantum systems have limited qubit and circuit resources.

25.8 Step 7 – Dimensionality Reduction

The selected feature set may still be too large for practical QML processing.

The platform therefore performs dimensionality reduction.

PCA can be used as the primary method.

Example:

50 Selected Features



PCA



8 Components

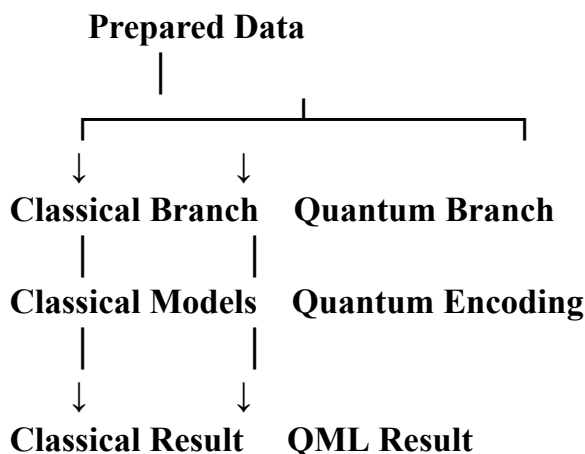
The system records:

- Number of original features
- Number of components
- Explained variance
- Transformation parameters

The transformation learned from training data must be applied consistently to validation, test, and future prediction data.

25.9 Step 8 – Parallel Classical and Quantum Model Preparation

After preprocessing and dimensionality reduction, the data can enter two model branches.



This parallel structure is essential for fair benchmarking.

The classical models provide the reference performance against which the hybrid QML models are evaluated.

25.10 Step 9 – Classical Baseline Training

The classical branch can train models such as:

- **Logistic Regression**
- **SVM**
- **Random Forest**
- **XGBoost**
- **Neural Network**

The models use the same prepared dataset.

The validation set is used for model selection and hyperparameter tuning.

The test set remains untouched until final evaluation.

The system records:

- **Training time**
- **Validation performance**
- **Model configuration**
- **Prediction performance**
- **Resource usage**

These results become the classical benchmark.

25.11 Step 10 – Quantum Feature Encoding

The quantum branch converts the reduced classical feature vector into quantum states.

For example:

8 Classical Features



Scaling



Quantum Feature Map



8 Qubits

One practical approach is angle encoding, where numerical feature values are mapped to quantum rotation angles.

The platform should record:

- **Encoding method**
- **Number of qubits**
- **Feature-to-qubit mapping**
- **Feature scaling**
- **Circuit configuration**

The encoding must remain consistent between training and inference.

25.12 Step 11 – Quantum Circuit Execution

The encoded data enters a quantum circuit.

A typical circuit structure is:

Input Features



Feature Encoding



Parameterized Gates



Entanglement



Measurement



Quantum Output

Depending on the selected model, the circuit may be used for:

- **Quantum kernels**
- **Quantum Support Vector Machines**
- **Variational Quantum Classifiers**
- **Quantum Neural Networks**

The circuit should be designed with current quantum resource limitations in mind.

25.13 Step 12 – Hybrid Quantum-Classical Training

For variational models such as VQC and QNN, the quantum circuit works together with a classical optimizer.

The training loop is:

Training Data



Quantum Encoding



Quantum Circuit



Measurement



Model Output



Loss Calculation



Classical Optimizer



Parameter Update



Quantum Circuit Again



Repeat

The classical optimizer updates the trainable parameters based on the calculated loss.

Training continues until:

- Maximum iterations are reached
- Validation performance stops improving
- Early stopping condition is reached
- Convergence is achieved

This is the central mechanism that makes the system hybrid quantum-classical.

25.14 Step 13 – Quantum Backend Execution

The quantum model can execute through a backend abstraction layer.

The backend may be:

Ideal Simulator

Used for initial development and circuit verification.

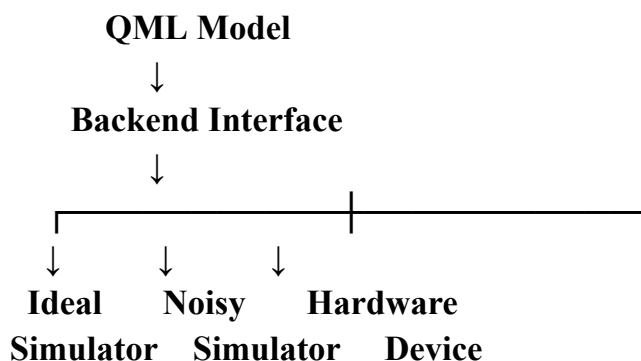
Noisy Simulator

Used to study the effect of realistic noise.

Near-Term Quantum Hardware

Used when compatible quantum hardware is available.

The overall application does not need to change when switching backends.



This architecture supports the project's requirement for compatibility with current simulators and near-term quantum hardware.

25.15 Step 14 – Model Optimization

The system evaluates different configurations using the validation dataset.

Parameters that may be optimized include:

Classical

- **Regularization**
- **Tree depth**
- **Number of estimators**
- **Learning rate**
- **Kernel parameters**
- **Neural-network architecture**

Quantum

- **Number of qubits**
- **Number of circuit layers**
- **Feature-map parameters**
- **Entanglement structure**
- **Ansatz configuration**
- **Classical optimizer**
- **Learning rate**
- **Training iterations**
- **Number of shots**

The optimization process must not use the final test set.

The goal is to identify a practical model configuration rather than simply the most complicated model.

25.16 Step 15 – Disease Prediction

After training and model selection, the final model is evaluated on unseen test samples.

The prediction process is:

New Biomedical Sample



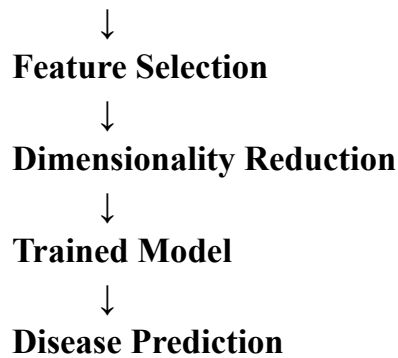
Input Validation



Same Preprocessing



Feature Engineering



For a binary disease classification task, the output can represent two classes.

Example:

Predicted Class: Disease / No Disease

Model Score: [model-generated score]

Model: Hybrid VQC

Backend: Quantum Simulator

The platform should avoid presenting this result as a clinically validated diagnosis.

25.17 Step 16 – Explainability

After prediction, the system generates an explanation where supported.

For classical models, this may include:

- **Feature importance**
- **SHAP values**
- **Local feature contribution**
- **Global feature contribution**

For hybrid QML models, suitable analyses can include:

- **Feature perturbation**
- **Feature ablation**
- **Sensitivity analysis**
- **Model-output changes**

- **Quantum circuit configuration**

The explanation answers:

“Which input information influenced the model's prediction?”

It should not claim:

“Which feature caused the disease?”

because model association is not equivalent to medical causation.

25.18 Step 17 – Performance Evaluation

The trained models are evaluated on the unseen test set.

The system calculates:

- **Accuracy**
- **Precision**
- **Sensitivity**
- **Recall**
- **Specificity**
- **F1-score**
- **ROC-AUC**

Additional analysis includes:

- **Confusion matrix**
- **ROC curve**
- **Precision-recall curve**
- **False-positive analysis**
- **False-negative analysis**
- **Error analysis**

For early disease detection, sensitivity and false-negative analysis are particularly important because missing a potentially diseased case can be more concerning than overall accuracy alone.

25.19 Step 18 – Computational and Quantum Resource Evaluation

Prediction quality is not the only evaluation criterion.

The system also records computational requirements.

Classical Resources

- Training time
- Inference time
- CPU/GPU usage
- Memory usage

Quantum Resources

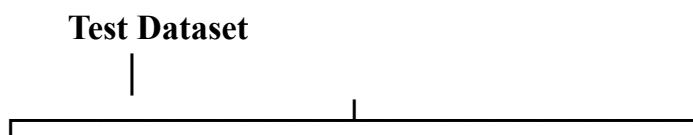
- Number of qubits
- Circuit depth
- Number of gates
- Trainable parameters
- Number of shots
- Circuit executions
- Simulator execution time
- Hardware execution time where available

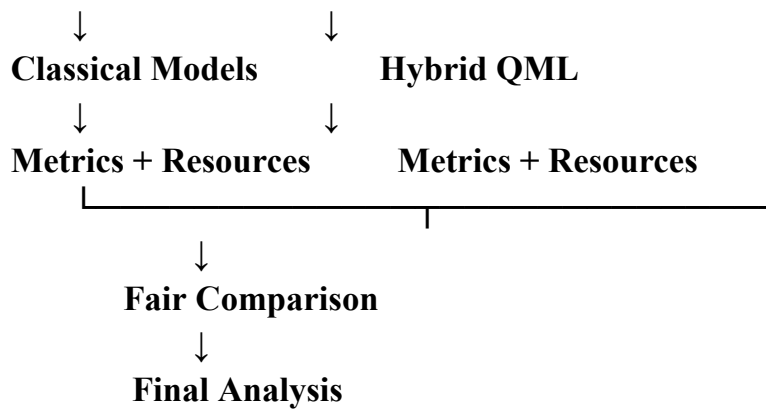
This allows the platform to evaluate the practical trade-off between performance and computational/quantum cost.

25.20 Step 19 – Classical vs Hybrid QML Benchmarking

The final comparison is performed using the collected experimental results.

Example structure:





The platform compares:

- Accuracy
- Sensitivity
- Specificity
- Precision
- F1-score
- ROC-AUC
- Generalization
- Training time
- Inference time
- Quantum resources

The system does not automatically declare the quantum approach superior.

Possible outcomes are:

Case 1 – Hybrid QML performs better

The experiment may show improved performance under the tested conditions.

Case 2 – Classical model performs better

The classical approach may provide better performance or efficiency.

Case 3 – Similar performance

Both approaches may provide comparable results.

All three outcomes are valid research findings.

25.21 Step 20 – Generalization Analysis

The platform evaluates whether the trained models generalize beyond their training data.

Possible approaches include:

- Cross-validation
- Repeated experiments
- Independent test evaluation
- Multiple random seeds
- Performance variance analysis

The system can report:

Mean ROC-AUC

Standard Deviation

Best Result

Worst Result

This helps determine whether an observed improvement is stable or simply the result of a particular data split or initialization.

25.22 Step 21 – Simulator vs Hardware Analysis

If a near-term quantum device is available, the same experiment can be executed in different environments.

Ideal Simulator



Noisy Simulator



Quantum Hardware

The system compares:

- **Predictive performance**
- **Execution time**
- **Circuit behavior**
- **Noise impact**
- **Resource requirements**

This helps identify how the QML model behaves when moving from an ideal computational environment toward real quantum hardware.

Simulator results should not be presented as equivalent to physical-hardware results.

25.23 Step 22 – Results Dashboard

All results are finally presented through the platform dashboard.

The final dashboard can contain:

Dataset Summary

- **Samples**
- **Features**
- **Classes**

Processing Summary

- **Selected features**
- **PCA components**
- **Encoding method**

Model Summary

- **Classical model**
- **QML model**
- **Backend**
- **Configuration**

Performance

- **Accuracy**

- Sensitivity
- Specificity
- F1
- ROC-AUC

Resource Usage

- Training time
- Inference time
- Qubits
- Circuit depth
- Shots

Explainability

- Important features
- Feature contributions
- Error patterns

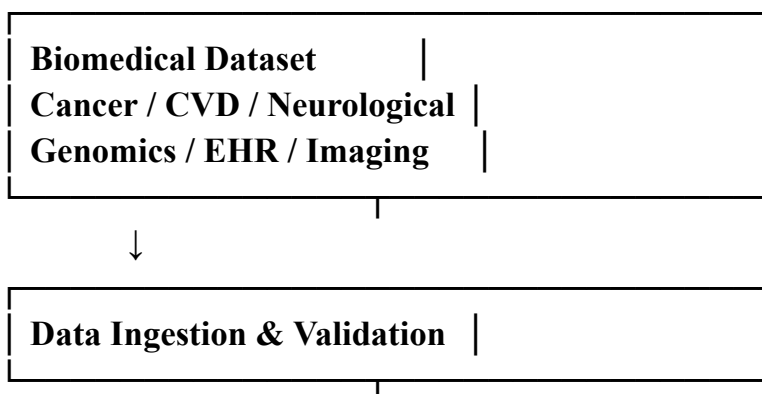
Benchmark

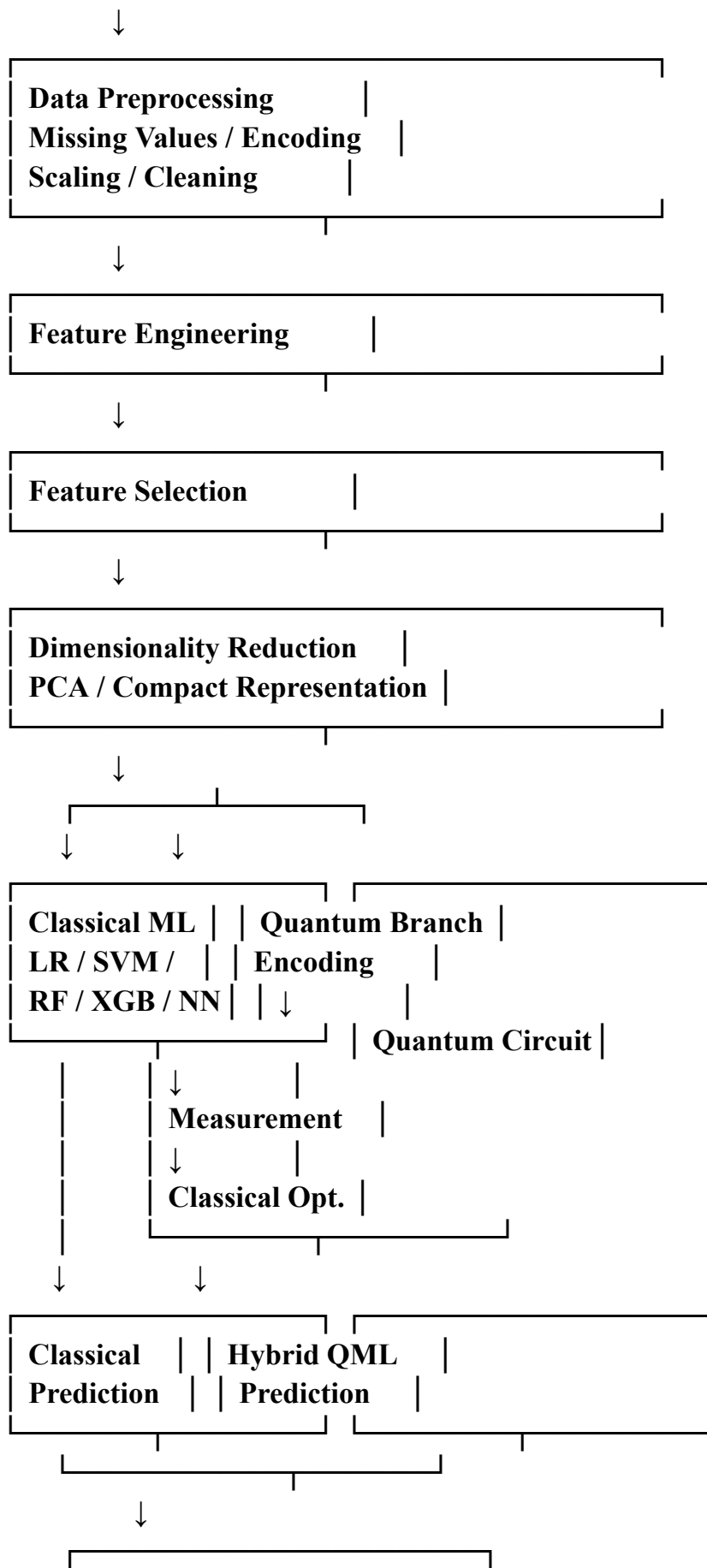
Classical vs Hybrid QML

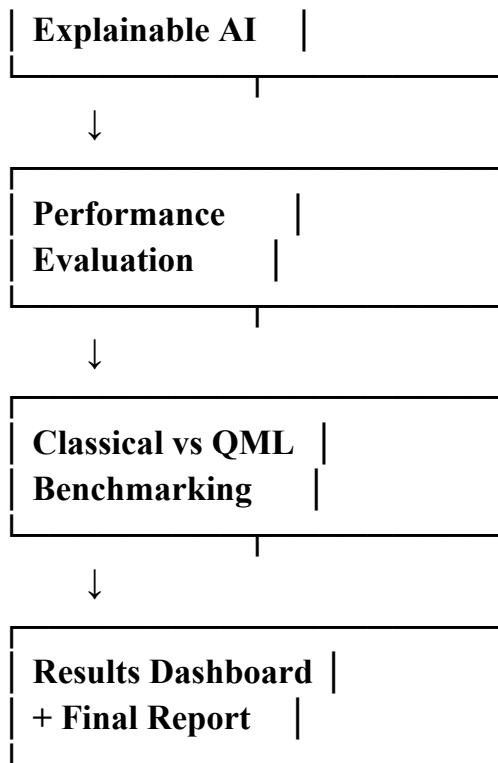
This provides a complete view of the experiment.

25.24 Complete End-to-End Workflow Diagram

The complete workflow can be represented as:







25.25 Example End-to-End Scenario

Consider a cardiovascular disease benchmark dataset.

Input

1,000 patient records

20 original features

Binary disease target

Processing

20 Features



Data Cleaning



Feature Engineering



Feature Selection



8 Selected Components

Classical Branch

8 Features



SVM / Random Forest / XGBoost



Validation



Best Classical Model

Quantum Branch

8 Features



Angle Encoding



8-Qubit Circuit



VQC / Quantum Kernel



Hybrid Optimization



Best QML Model

Final Evaluation

Both models are tested on the same unseen test set.

The platform then reports:

	Classical	Hybrid QML
Accuracy	—	—
Sensitivity	—	—
Specificity	—	—
F1-score	—	—
ROC-AUC	—	—
Training Time	—	—
Inference Time	—	—

Quantum Resources N/A —

The actual values are obtained experimentally.

25.26 Failure Handling and Recovery

The workflow should also handle failures safely.

Examples include:

Invalid Dataset

Stop the workflow and display a validation error.

Excessive Feature Dimension

Recommend feature selection or dimensionality reduction before quantum processing.

Quantum Circuit Failure

Return a clear backend error and allow execution on a simulator where appropriate.

Hardware Unavailable

Allow the experiment to continue using a compatible simulator if the experiment configuration permits it.

Model Training Failure

Store the error and preserve the experiment configuration for debugging.

Poor Model Performance

Report the result instead of artificially modifying the experiment to produce better numbers.

This makes the platform suitable for genuine experimentation.

25.27 Reproducibility

Every complete workflow should be reproducible.

The platform should store:

- Dataset version
- Preprocessing configuration
- Feature-engineering configuration
- Selected features
- PCA configuration
- Model parameters
- Quantum circuit configuration
- Backend
- Random seed
- Training configuration
- Evaluation metrics
- Experiment ID

A researcher should therefore be able to recreate a previous experiment using the stored configuration.

25.28 Final Workflow Principle

The proposed platform follows a classical-first, quantum-enhanced, benchmark-driven workflow.

Classical computing is used for:

Data ingestion → Cleaning → Feature engineering → Feature selection → Dimensionality reduction → Evaluation → Visualization

Quantum computing is introduced for:

Quantum feature encoding → Quantum kernels → Variational circuits → Quantum-enhanced learning

Both branches are then evaluated under comparable conditions.

The final goal is not to assume that quantum computing will outperform classical machine learning, but to experimentally determine whether the hybrid approach can provide useful improvements in:

- **Disease-detection performance**
- **Sensitivity**
- **Specificity**
- **Generalization**
- **Computational efficiency**
- **Or other measurable practical criteria**

while remaining compatible with current quantum simulators and near-term quantum hardware.

Core Principle: “Prepare classically, learn through hybrid quantum-classical models, evaluate fairly, explain the result, and let experimental evidence determine the benefit.”

26. Technology Stack

26.1 Overview

The proposed Hybrid Quantum Machine Learning Platform requires a technology stack that can support the complete workflow:

Biomedical Data → Classical Processing → Feature Engineering → Classical ML → Quantum ML → Hybrid Training → Prediction → Explainability → Evaluation → Benchmarking → Dashboard

The technology stack is therefore divided into multiple layers rather than depending on a single framework.

The main design principle is:

Use mature classical technologies for data processing and machine learning, and integrate quantum frameworks through a modular quantum backend layer.

This makes the platform practical to develop on current hardware while maintaining compatibility with quantum simulators and near-term quantum devices.

26.2 Technology Stack Overview

Layer	Recommended Technology	Purpose
Frontend	React / Next.js	Interactive dashboard
UI	HTML, CSS, JavaScript / TypeScript	User interface
Backend API	Python + FastAPI	Communication between frontend and ML services
Data Processing	Pandas, NumPy	Data loading and numerical processing
Classical ML	Scikit-learn	Classical baseline models
Gradient Boosting	XGBoost	Strong classical benchmark
Quantum ML	Qiskit / PennyLane	Quantum circuits and QML models
Quantum Simulation	Qiskit Aer / compatible simulator	Local quantum simulation
Explainability	SHAP + custom perturbation analysis	Model interpretation
Visualization	Plotly / Matplotlib	Metrics and experiment visualization
Database	PostgreSQL	Dataset/experiment/result metadata
Model Storage	Joblib / suitable model serialization	Classical model persistence
API Testing	Postman / automated tests	Backend validation

Version Control	Git + GitHub/GitLab	Source-code management
Deployment	Docker	Reproducible deployment
Hardware Backend	Compatible quantum cloud/backend	Optional real-device execution

The exact libraries can be adjusted according to availability, licensing, and implementation requirements.

26.3 Frontend Technology

React / Next.js

The frontend provides the main user interface for the platform.

It can contain pages such as:

- Dashboard**
- Dataset**
- Preprocessing**
- Feature Engineering**
- Feature Selection**
- Dimensionality Reduction**
- Classical Models**
- Quantum Models**
- Training**
- Prediction**
- Explainability**
- Benchmarking**
- Experiments**
- Results**

React is suitable because the dashboard contains many interactive components.

For example, changing the number of PCA components can immediately update the experiment configuration displayed on the screen.

Next.js can additionally provide:

- **Structured application routing**
- **Component-based development**
- **Efficient frontend architecture**
- **API integration**
- **Deployment flexibility**

TypeScript can be used to improve reliability for larger frontend components.

26.4 Backend Technology

Python + FastAPI

The backend acts as the central application layer.

It connects:

Frontend → Data Processing → ML Models → QML Models → Evaluation → Database

FastAPI can expose APIs such as:

POST /dataset/upload

POST /preprocess

POST /features/engineer

POST /features/select

POST /dimensions/reduce

POST /models/classical/train

POST /models/quantum/train

POST /prediction

GET /evaluation/{experiment_id}

GET /explainability/{experiment_id}

GET /benchmark/{experiment_id}

GET /experiments

FastAPI is useful because the QML and classical ML components can be exposed as modular services rather than being directly controlled by the frontend.

26.5 Data Processing Technology

Pandas

Pandas can handle structured biomedical datasets.

It can be used for:

- CSV loading
- Data inspection
- Missing-value analysis
- Duplicate detection
- Data filtering
- Feature transformation
- Statistical summaries
- Dataset preparation

Example workflow:

CSV Dataset



Pandas DataFrame



Validation



Cleaning



Feature Processing

NumPy

NumPy provides numerical operations required throughout the platform.

It can support:

- Numerical arrays
- Mathematical transformations
- Feature matrices
- Quantum input preparation
- Numerical optimization
- Evaluation calculations

Pandas and NumPy therefore form the basic data-processing layer.

26.6 Classical Machine Learning

Scikit-learn

Scikit-learn can provide the main classical ML baseline implementation.

Models can include:

Logistic Regression

Useful as a simple and interpretable baseline.

Support Vector Machine

Useful for comparison with kernel-based quantum approaches.

Random Forest

Provides a tree-based nonlinear baseline.

Neural Network

Provides a classical neural-learning reference.

Scikit-learn can also support:

- Data splitting
- Scaling
- PCA
- Feature selection
- Cross-validation

- **Hyperparameter optimization**
- **Metrics**
- **Confusion matrices**
- **ROC-AUC**

This makes it a central component of the classical branch.

26.7 XGBoost

XGBoost can be included as a stronger classical benchmark.

It is useful for structured biomedical datasets where nonlinear relationships may exist.

The system can therefore compare:

Simple Classical Models → Strong Classical Models → Hybrid QML Models

This prevents the quantum approach from being compared only against weak baselines.

26.8 Quantum Machine Learning Framework

Qiskit / PennyLane

A quantum framework is required for constructing and executing quantum circuits.

Depending on implementation requirements, the platform can use Qiskit, PennyLane, or another suitable QML framework.

The quantum layer should support:

- **Quantum circuits**
- **Feature maps**
- **Parameterized circuits**
- **Quantum kernels**

- **Measurements**
- **Variational optimization**
- **Quantum simulators**
- **Compatible quantum hardware backends**

The platform should isolate framework-specific implementation behind the Quantum Backend Layer wherever practical.

This makes it easier to change the quantum backend without redesigning the entire application.

26.9 Quantum ML Models

The quantum framework will implement the proposed QML approaches.

Quantum Kernel / QSVM

Used to construct quantum-based similarity representations and combine them with classical SVM learning.

Variational Quantum Classifier

Uses a parameterized quantum circuit trained with a classical optimization process.

Quantum Neural Network

Uses parameterized quantum circuits as a trainable model component.

The platform does not need to implement every QML model simultaneously for the initial prototype.

A practical SIH prototype can prioritize:

- 1. Quantum Kernel / QSVM**
- 2. VQC**

and then extend to QNN if implementation time and computational resources permit.

26.10 Quantum Simulation

Qiskit Aer / Compatible Simulator

Quantum simulation is important because physical quantum hardware may not always be available.

The simulator can support:

- Ideal circuit execution
- Measurement
- Parameterized circuits
- Quantum kernel experiments
- VQC training
- Noisy simulation where supported

The initial development workflow can therefore be:

QML Model



Ideal Simulator



Noisy Simulator



Compatible Quantum Hardware

This provides a practical development path from software experimentation toward real quantum devices.

26.11 Explainable AI Technology

SHAP

SHAP can be used for suitable classical models to analyze feature contributions.

It can provide:

- Global feature importance

- **Local explanations**
- **Feature contribution analysis**
- **Model comparison**

For example:

Feature A → strong positive contribution

Feature B → moderate contribution

Feature C → low contribution

For hybrid QML models, explainability should use appropriate techniques such as:

- **Feature perturbation**
- **Feature ablation**
- **Sensitivity analysis**
- **Prediction-change analysis**

Quantum circuit configuration can additionally be displayed to improve transparency.

The platform should not claim that these explanations establish medical causation.

26.12 Visualization Technology

Plotly

Plotly can be used for interactive dashboard visualizations.

Possible charts include:

- **Accuracy comparison**
- **Sensitivity comparison**
- **Specificity comparison**
- **ROC curves**
- **Precision-recall curves**
- **Confusion matrices**
- **Feature importance**

- **PCA explained variance**
- **Training loss**
- **Validation loss**
- **Resource comparison**

Interactive visualizations are particularly useful during the SIH demonstration.

Matplotlib

Matplotlib can be used for static plots and research reports.

It can support:

- **Training curves**
- **Performance graphs**
- **Distribution plots**
- **Experiment reports**

Plotly can primarily support the interactive dashboard, while Matplotlib can support report generation and offline analysis.

26.13 Database Technology

PostgreSQL

A relational database can store experiment metadata and results.

Possible tables include:

users

datasets

dataset_versions

experiments

preprocessing_configs

feature_configs

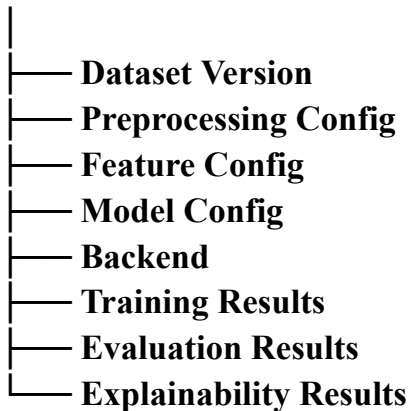
classical_models

quantum_models

training_runs
predictions
evaluation_results
explainability_results
quantum_resources

For example:

Experiment



This allows previous experiments to be retrieved and compared.

26.14 Model Storage

Classical trained models can be serialized using appropriate Python model-storage mechanisms such as Joblib where suitable.

The stored model should be associated with:

- **Model version**
- **Dataset version**
- **Feature transformation version**
- **Preprocessing configuration**
- **Training configuration**
- **Experiment ID**

For QML models, the platform should store the relevant:

- **Circuit definition**

- Trainable parameters
- Feature-map configuration
- Backend configuration
- Training metadata

This ensures that a prediction can be associated with the exact model configuration that generated it.

26.15 API Communication

The frontend communicates with the backend through REST APIs.

Example:

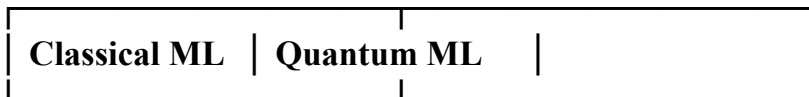
React / Next.js



REST API



FastAPI Backend



Evaluation + XAI



Database

This separation allows each module to be developed and tested independently.

26.16 Containerization

Docker

Docker can package the application and its dependencies into reproducible environments.

Possible containers include:

Frontend Container



Backend Container



ML/QML Container



Database Container

This reduces problems caused by differences between development environments.

For an SIH prototype, the complete system can potentially be started using a simple Docker Compose configuration.

26.17 Version Control

Git

Git should be used for source-code management.

The repository can be organized as:

```
project/
|
|— frontend/
|— backend/
|— data_pipeline/
|— classical_ml/
|— quantum_ml/
|— explainability/
|— evaluation/
|— database/
|— tests/
```

|— notebooks/
|— documentation/
|— docker/

Git also allows the team to track changes to:

- **Model implementations**
 - **Quantum circuits**
 - **API endpoints**
 - **Dashboard components**
 - **Experiment configuration**
 - **Documentation**
-

26.18 Testing Technology

The platform should contain automated testing for important components.

Testing can include:

Data Pipeline Tests

- **Missing-value handling**
- **Encoding**
- **Scaling**
- **Feature selection**

ML Tests

- **Model training**
- **Prediction**
- **Evaluation metrics**

QML Tests

- **Circuit construction**
- **Feature encoding**
- **Measurement**
- **Parameter updates**
- **Simulator execution**

API Tests

- **Dataset upload**
- **Training request**
- **Prediction request**
- **Result retrieval**

Frontend Tests

- **Dashboard navigation**
- **Configuration forms**
- **Result rendering**

This helps ensure that changes to one module do not break the complete pipeline.

26.19 Security Technologies and Practices

Biomedical datasets can contain sensitive information, so security should be considered throughout the platform.

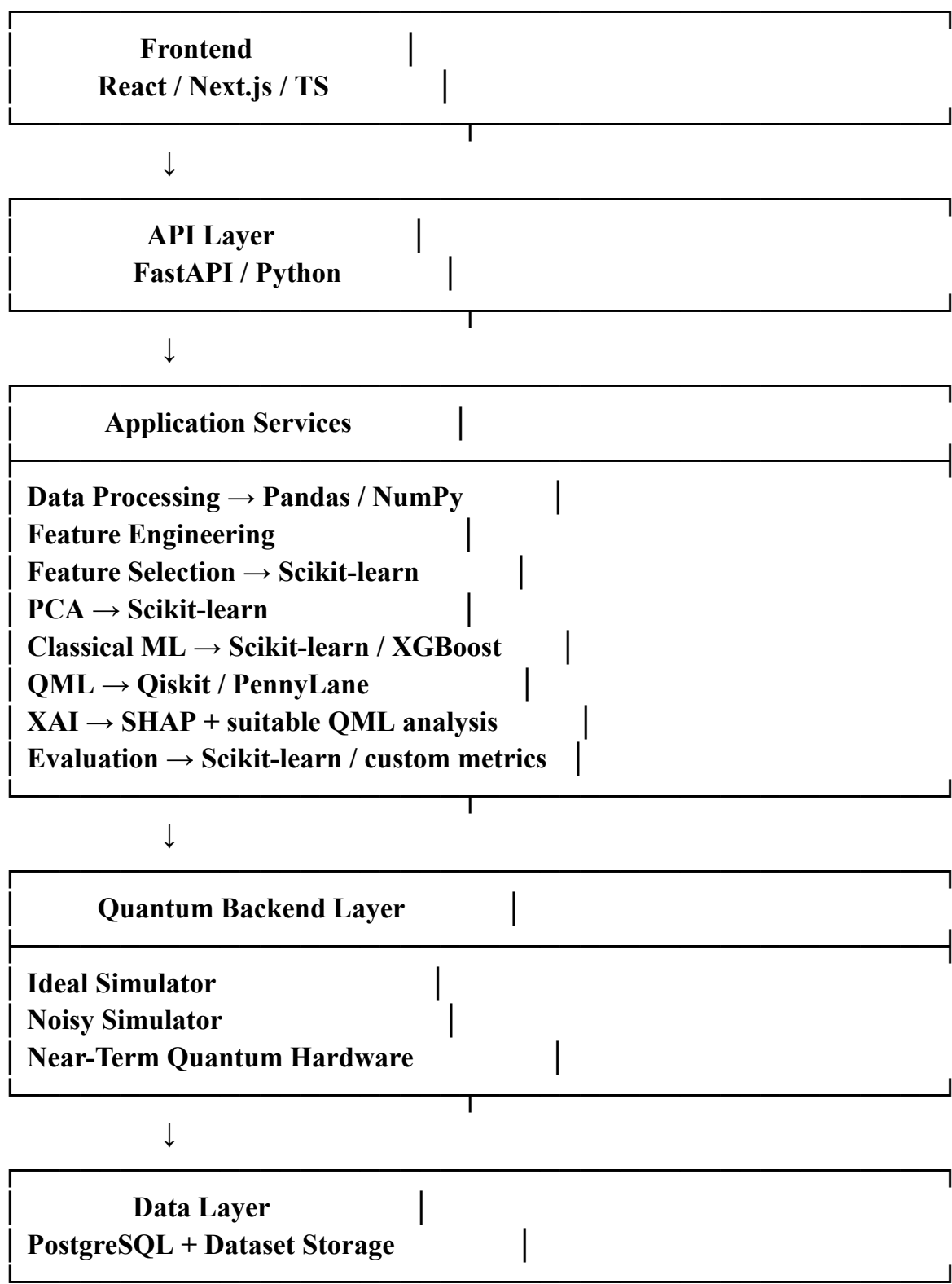
The system can implement:

- **HTTPS**
- **Authentication**
- **Role-based access**
- **Secure API endpoints**
- **Controlled dataset access**
- **Database access controls**
- **Audit logs**
- **Secure environment variables**
- **De-identified datasets for demonstrations**

Sensitive biomedical information should not be exposed unnecessarily through logs or dashboard outputs.

26.20 Proposed Technical Architecture

The complete technology stack can be represented as:



26.21 Why This Technology Stack Fits the Problem

The selected technologies directly support the requirements of the problem statement.

Problem Requirement	Technology Support
Biomedical data ingestion	Pandas / NumPy / FastAPI
Data preprocessing	Pandas / Scikit-learn
Feature engineering	Python / Pandas / NumPy
Feature selection	Scikit-learn
Dimensionality reduction	PCA / Scikit-learn
Classical ML baseline	Scikit-learn / XGBoost
Quantum-enhanced models	Qiskit / PennyLane
Quantum simulation	Quantum simulator
Hybrid training	QML framework + classical optimizer
Disease prediction	ML/QML models
Explainability	SHAP + perturbation analysis
Performance evaluation	Scikit-learn / custom evaluation
Classical vs QML comparison	Benchmarking module
Dashboard	React / Next.js
API integration	FastAPI
Experiment tracking	PostgreSQL
Reproducibility	Git + experiment metadata
Deployment	Docker

Near-term hardware

Backend abstraction

Therefore, the technology stack is directly connected to the required system functionality rather than adding technologies that are unrelated to the problem.

26.22 Prototype-Friendly Implementation

For the SIH prototype, the team does not need to build the entire production-scale platform immediately.

A practical minimum implementation can use:

Frontend

React / Next.js

Backend

Python + FastAPI

Data

Pandas + NumPy

Classical ML

Scikit-learn + XGBoost

QML

Qiskit or PennyLane

Simulator

Compatible quantum simulator

Explainability

SHAP + feature perturbation

Visualization

Plotly

Database

PostgreSQL

Deployment

Docker

The prototype can initially focus on one biomedical benchmark dataset and demonstrate the complete workflow.

Once the core pipeline is stable, additional disease datasets, QML models, and hardware backends can be added.

26.23 Technology Selection Principle

The technology stack follows three important rules:

1. Mature Classical Tools

Use established Python-based tools for biomedical data processing and classical ML.

2. Modular Quantum Integration

Keep QML components separated from the rest of the system through a backend abstraction layer.

3. Experiment-Based Evaluation

Do not select technologies based on an assumption of quantum advantage. The platform should provide the infrastructure needed to measure actual performance.

26.24 Final Technology Stack

The recommended stack for the proposed platform is:

Frontend

→ React / Next.js + TypeScript

Backend

→ **Python + FastAPI**

Data Processing

→ **Pandas + NumPy**

Classical ML

→ **Scikit-learn + XGBoost**

Quantum ML

→ **Qiskit / PennyLane**

Quantum Simulation

→ **Compatible quantum simulator**

Explainability

→ **SHAP + perturbation/sensitivity analysis**

Visualization

→ **Plotly + Matplotlib**

Database

→ **PostgreSQL**

Model/Experiment Storage

→ **Joblib + experiment metadata**

Testing

→ **Pytest + API testing**

Version Control

→ **Git**

Deployment

→ **Docker**

Quantum Execution

→ **Simulator + compatible near-term quantum hardware**

26.25 Key Design Principle

“Use the right technology for each part of the pipeline: classical computing for data-intensive preparation and evaluation, quantum frameworks for quantum-enhanced learning, and a modular architecture to connect both.”

This technology stack provides a realistic foundation for implementing the proposed Hybrid Quantum Machine Learning Platform while keeping the system scalable, explainable, simulator-compatible, and ready for near-term quantum hardware.

27. Software Architecture

27.1 Overview

The software architecture defines how the different components of the Hybrid Quantum Machine Learning Platform interact with each other.

The architecture is designed around the complete workflow:

Data Ingestion → Preprocessing → Feature Engineering → Feature Selection → Dimensionality Reduction → Classical/QML Training → Prediction → Explainability → Evaluation → Benchmarking → Results

The system follows a modular, service-oriented architecture so that individual components can be developed, tested, replaced, and scaled independently.

The most important architectural principle is:

Classical processing and quantum processing are separated into independent modules but connected through a common experiment pipeline.

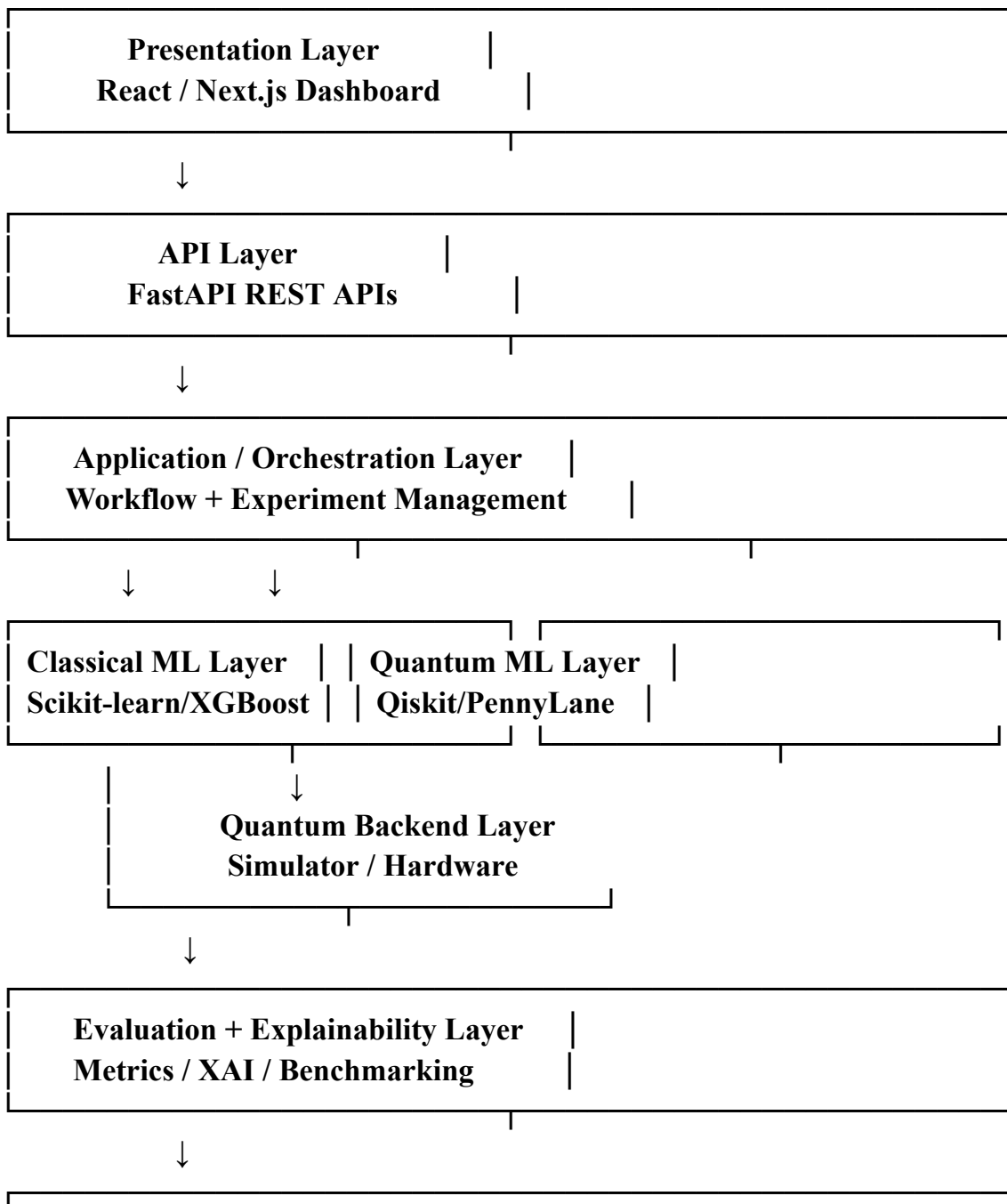
This allows the same biomedical dataset and preprocessing pipeline to be used for both classical and hybrid QML experiments.

27.2 High-Level Architecture

The proposed system can be divided into six major layers:

1. **Presentation Layer**
2. **API Layer**
3. **Application/Orchestration Layer**
4. **Machine Learning and Quantum Layer**
5. **Evaluation and Explainability Layer**
6. **Data and Storage Layer**

The overall structure is:



27.3 Presentation Layer

The presentation layer is the user-facing part of the application.

Technology

- **React / Next.js**
- **TypeScript**
- **HTML/CSS**
- **Plotly or equivalent visualization library**

Responsibilities

The frontend allows users to:

- **Upload datasets**
- **Select target variables**
- **Configure preprocessing**
- **Configure feature engineering**
- **Select features**
- **Configure dimensionality reduction**
- **Select classical models**
- **Select QML models**
- **Configure quantum parameters**
- **Start experiments**
- **Monitor training**
- **View predictions**
- **View explanations**
- **Compare models**
- **View experiment history**
- **Download results/reports**

The frontend should not directly execute ML or quantum operations.

Instead:

Frontend → API → Backend Services

This keeps computation and security-sensitive operations on the server side.

27.4 API Layer

The API layer acts as the communication bridge between the frontend and backend.

Technology

Python + FastAPI

The API exposes structured endpoints for different platform functions.

Example API structure:

Dataset APIs

POST /datasets/upload

GET /datasets

GET /datasets/{id}

Processing APIs

POST /preprocessing/run

POST /features/engineer

POST /features/select

POST /dimensions/reduce

Model APIs

POST /models/classical/train

POST /models/quantum/train

GET /models/{id}

Prediction APIs

POST /predict

Evaluation APIs

GET /evaluation/{experiment_id}
GET /benchmark/{experiment_id}

Explainability APIs

GET /explainability/{experiment_id}

Experiment APIs

POST /experiments
GET /experiments
GET /experiments/{id}

The API should validate incoming requests before sending them to the underlying services.

27.5 Application Orchestration Layer

The orchestration layer controls the complete ML/QML workflow.

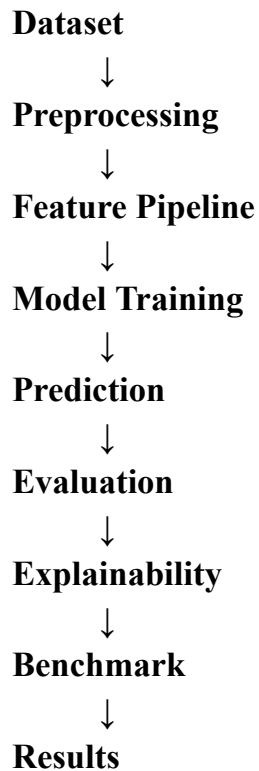
Its responsibility is to determine:

- **Which dataset is being used**
- **Which preprocessing configuration is selected**
- **Which features are selected**
- **Which dimensionality-reduction method is used**
- **Which models should be trained**
- **Which quantum backend should be used**
- **Which evaluation procedure should be performed**
- **Which experiment configuration should be stored**

The orchestration flow is:

Experiment Configuration





This prevents the frontend from having to understand the internal implementation of every ML or QML component.

27.6 Data Processing Module

The Data Processing Module is responsible for transforming raw biomedical data into a clean machine-learning-ready representation.

It contains components for:

- Data validation
- Missing-value handling
- Duplicate detection
- Categorical encoding
- Numerical scaling
- Outlier analysis
- Target preparation
- Train/validation/test splitting

The module should return both:

1. Processed data
2. Processing metadata

For example:

Processed Dataset

+

Preprocessing Configuration

+

Transformation Parameters

+

Data Quality Report

The transformation parameters are important because exactly the same transformations must be applied during inference.

27.7 Feature Engineering Module

This module creates meaningful features from the preprocessed data.

Possible operations include:

- Statistical transformations
- Ratios
- Differences
- Interaction features
- Log transformations
- Domain-specific derived features

The module should maintain a feature metadata record.

Example:

Feature: Cholesterol_Ratio

Created From:

Cholesterol / HDL

Transformation:

Ratio

Source Features:
Cholesterol, HDL

This makes the feature pipeline reproducible and easier to explain.

27.8 Feature Selection Module

The Feature Selection Module determines which features should be retained.

Supported methods may include:

- **Correlation**
- **Mutual information**
- **Statistical selection**
- **RFE**
- **Tree-based importance**

The module outputs:

Selected Features

+

Feature Ranking

+

Selection Method

+

Selection Parameters

The selected feature set is then passed to the dimensionality-reduction stage.

27.9 Dimensionality Reduction Module

This module reduces the feature space to a more manageable representation.

The primary method can be:

PCA

The module records:

- **Original feature count**
- **Number of components**
- **Explained variance**
- **PCA parameters**
- **Transformation matrix/model**

Example:

1,000 Features



Feature Selection



100 Features



PCA



8 Components

The final compact representation can then be used by both classical and quantum models.

27.10 Classical Machine Learning Service

The Classical ML Service contains the baseline machine-learning algorithms.

Possible models:

- **Logistic Regression**
- **SVM**
- **Random Forest**
- **XGBoost**
- **Neural Network**

Each model should follow a common interface:

```
train()
predict()
predict_score()
evaluate()
save()
load()
```

This makes it easy to add additional classical models later.

The service should also return standardized experiment information.

27.11 Quantum Machine Learning Service

The QML Service contains the quantum-enhanced learning models.

Possible implementations include:

- **Quantum Kernel**
- **QSVM**
- **Variational Quantum Classifier**
- **Quantum Neural Network**

A common interface can be used:

```
build_circuit()
encode_features()
train()
predict()
evaluate()
save_configuration()
```

The QML service should not directly depend on a single physical quantum device.

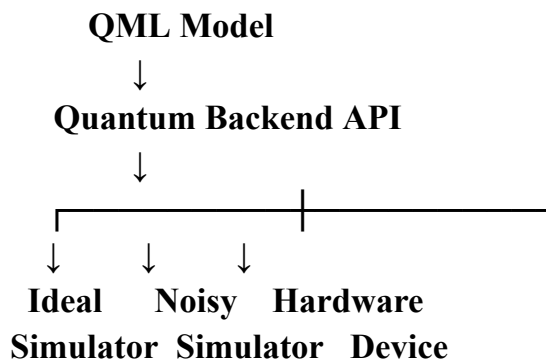
Instead, it communicates through the Quantum Backend Layer.

27.12 Quantum Backend Abstraction Layer

This is a critical part of the architecture.

The QML model should not be tightly coupled to one quantum backend.

Instead:



This allows the same QML workflow to be tested in multiple execution environments.

The backend interface can expose operations such as:

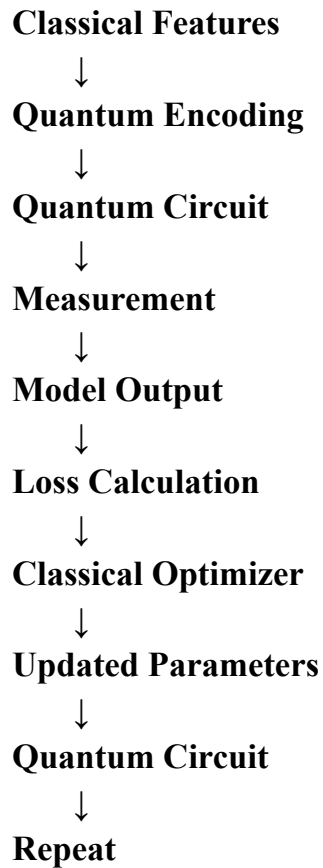
```
execute_circuit()
get_backend_info()
get_qubit_count()
get_supported_gates()
estimate_resources()
```

This abstraction is essential for the project's requirement of simulator compatibility and near-term hardware readiness.

27.13 Hybrid Training Controller

The Hybrid Training Controller manages the interaction between classical optimization and quantum circuit execution.

The workflow is:



The controller manages:

- **Epochs/iterations**
- **Batch processing where supported**
- **Loss calculation**
- **Parameter updates**
- **Validation**
- **Early stopping**
- **Training logs**
- **Backend execution**

This module is the main integration point between classical ML infrastructure and quantum computation.

27.14 Model Optimization Service

The optimization service searches for suitable model configurations.

For classical models, it can optimize parameters such as:

- Regularization
- Tree depth
- Number of trees
- Learning rate
- Kernel parameters

For QML models:

- Number of qubits
- Circuit layers
- Feature-map configuration
- Entanglement
- Ansatz
- Learning rate
- Optimizer
- Training iterations

The optimization service must operate on the validation data rather than the final test set.

27.15 Prediction Service

The Prediction Service handles inference after a model has been trained.

Its workflow is:

New Sample



Validation



Saved Preprocessing



Saved Feature Transformation



Saved Model



Prediction

↓
Explanation

The same preprocessing and feature transformations used during training must be reused during prediction.

This prevents inconsistent input processing.

27.16 Explainability Service

The Explainability Service generates model interpretation information.

For classical models, it may use:

- **SHAP**
- **Feature importance**
- **Model-specific explanations**

For QML models, it can provide suitable analysis such as:

- **Feature perturbation**
- **Feature ablation**
- **Sensitivity analysis**
- **Prediction changes**
- **Circuit configuration**

The service should return structured explanation data that can be visualized by the frontend.

27.17 Evaluation Service

The Evaluation Service calculates all required performance metrics.

It can generate:

- **Accuracy**
- **Precision**

- Recall
- Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion matrix
- ROC curve
- Precision-recall curve

It can additionally calculate:

- Training time
- Inference time
- Memory/resource usage
- Quantum circuit executions
- Number of shots
- Circuit depth
- Number of qubits

The output is stored against the corresponding experiment ID.

27.18 Benchmarking Service

The Benchmarking Service performs the final classical-vs-hybrid comparison.

It receives results from:

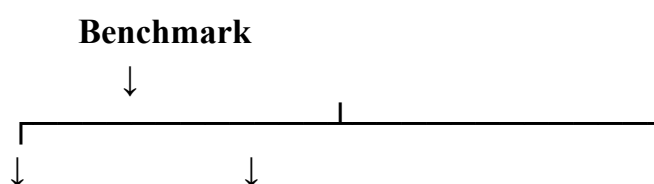
Classical ML Service

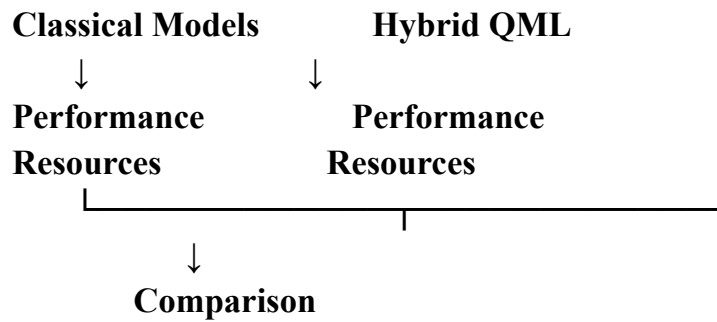
and

Quantum ML Service

and creates a standardized comparison.

Example:





The service should not assign an artificial winner.

It should report the measured results and allow the evidence to determine which model performs best under the tested conditions.

27.19 Experiment Management Service

Every training run should be treated as an independent experiment.

The Experiment Management Service creates an experiment ID.

Example:

EXP-001

Dataset: Cancer Dataset

Features: 8

Model: VQC

Backend: Simulator

Seed: 42

Status: Completed

The service records all configurations associated with the experiment.

This enables:

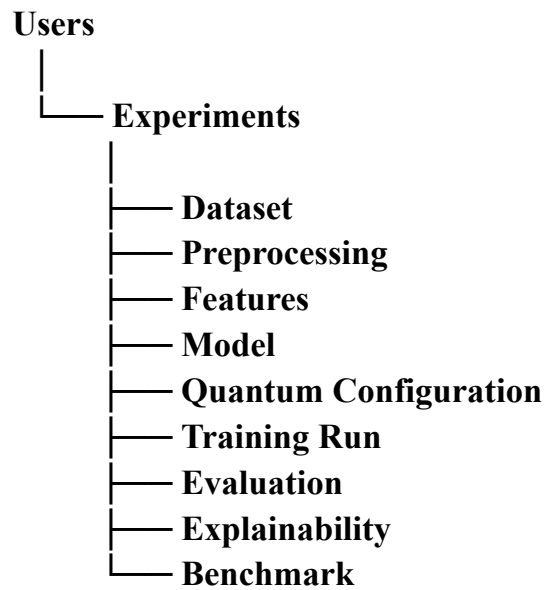
- Experiment history
- Reproducibility
- Model comparison
- Result tracking
- Debugging

- Report generation
-

27.20 Database Architecture

PostgreSQL can store the structured metadata.

A simplified relationship is:



Important database entities include:

Dataset

Stores dataset metadata.

Experiment

Stores experiment-level configuration.

Model

Stores classical or quantum model configuration.

Training Run

Stores training information.

Evaluation

Stores metrics.

Quantum Resource

Stores quantum execution information.

Explainability

Stores explanation results.

27.21 File and Dataset Storage

Large biomedical datasets should not necessarily be stored directly inside database tables.

A separate dataset-storage layer can maintain:

- **Raw dataset**
- **Processed dataset**
- **Dataset versions**
- **Model artifacts**
- **Experiment reports**

The database stores references and metadata.

This provides better separation between:

Metadata

and

Large data/model files

27.22 Authentication and Authorization

The application can include authentication if multiple users are supported.

Possible roles include:

Researcher

Can:

- **Upload datasets**
- **Configure experiments**
- **Train models**
- **View results**

Administrator

Can:

- **Manage users**
- **Manage system configuration**
- **Manage quantum backends**
- **Monitor system resources**

For the SIH prototype, a simplified authentication system may be sufficient.

27.23 Background Job Architecture

QML training and large ML experiments may take significant time.

Therefore, long-running jobs should not block the main API request.

The architecture can use a background job system:

User Starts Experiment



FastAPI



Job Queue



Training Worker



ML/QML Execution



Results



Database



Dashboard

For a prototype, background execution can initially be implemented using a lightweight task mechanism. A dedicated task queue can be introduced if the system needs to scale.

27.24 Experiment Status Tracking

Each experiment should have a status.

Example:

CREATED



VALIDATING



PREPROCESSING



FEATURE_PROCESSING



TRAINING



EVALUATING



EXPLAINING



BENCHMARKING



COMPLETED

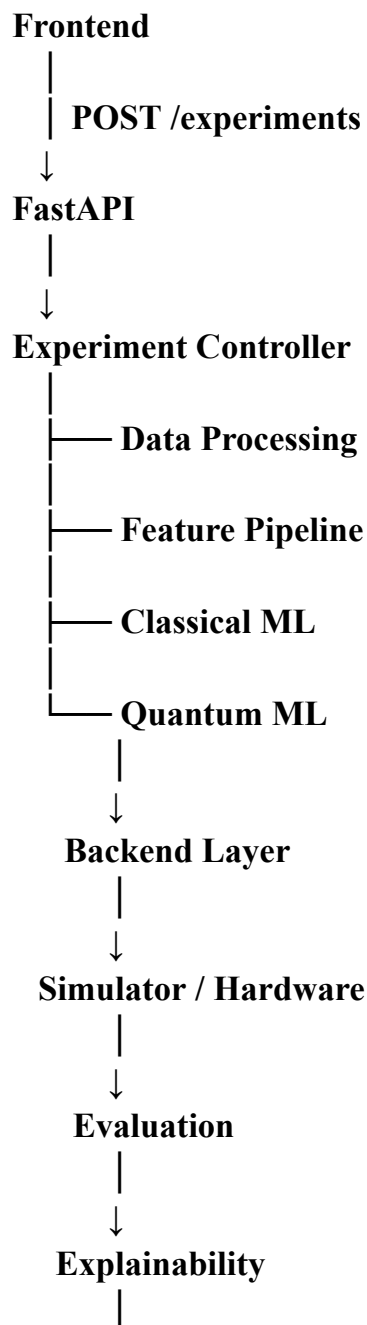
If an error occurs:

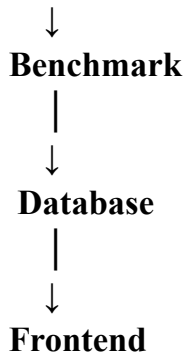
FAILED

The user can then view the error information without losing the experiment configuration.

27.25 API-to-Module Interaction

A typical training request can follow:





This creates a clear separation of responsibilities.

27.26 Error Handling Architecture

Each layer should handle its own errors and return meaningful information.

Examples:

Data Layer

Error: Invalid target column.

Preprocessing

Error: Unsupported data type.

QML

Error: Circuit configuration exceeds available resources.

Backend

Error: Quantum backend unavailable.

Training

Error: Model failed to converge.

Prediction

Error: Input features do not match the trained model.

The API converts these errors into structured responses for the frontend.

27.27 Logging and Monitoring

The system should maintain application and experiment logs.

Important information includes:

- API requests
- Experiment status
- Training errors
- Backend errors
- Model execution
- Quantum execution
- Processing time

Logs should not expose sensitive biomedical information unnecessarily.

The monitoring system can also track:

- CPU usage
 - Memory usage
 - Job status
 - Quantum backend status
 - Training duration
-

27.28 Reproducibility Architecture

Reproducibility is built into the architecture rather than added later.

Each experiment should preserve:

Dataset Version

+

Preprocessing Version

+

Feature Configuration

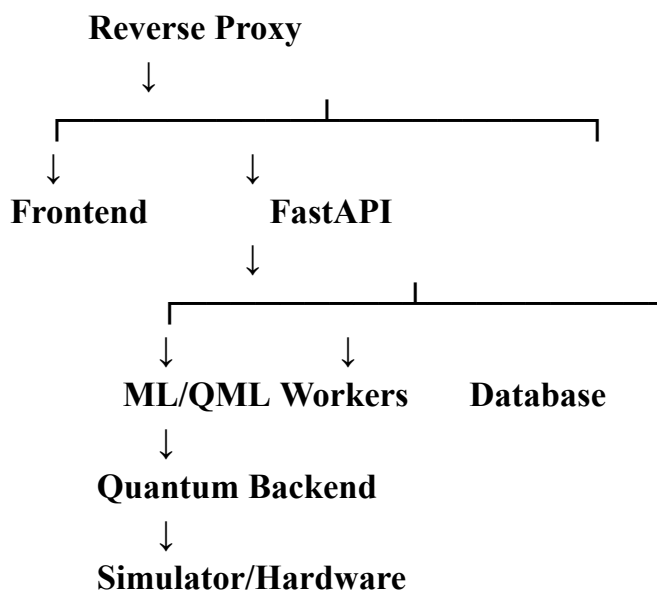
+
Model Configuration
+
Quantum Circuit Configuration
+
Backend
+
Random Seed
+
Training Parameters
+
Evaluation Results

This allows the same experiment to be recreated under the same conditions.

27.29 Deployment Architecture

The system can be deployed using containers.

A possible deployment structure is:

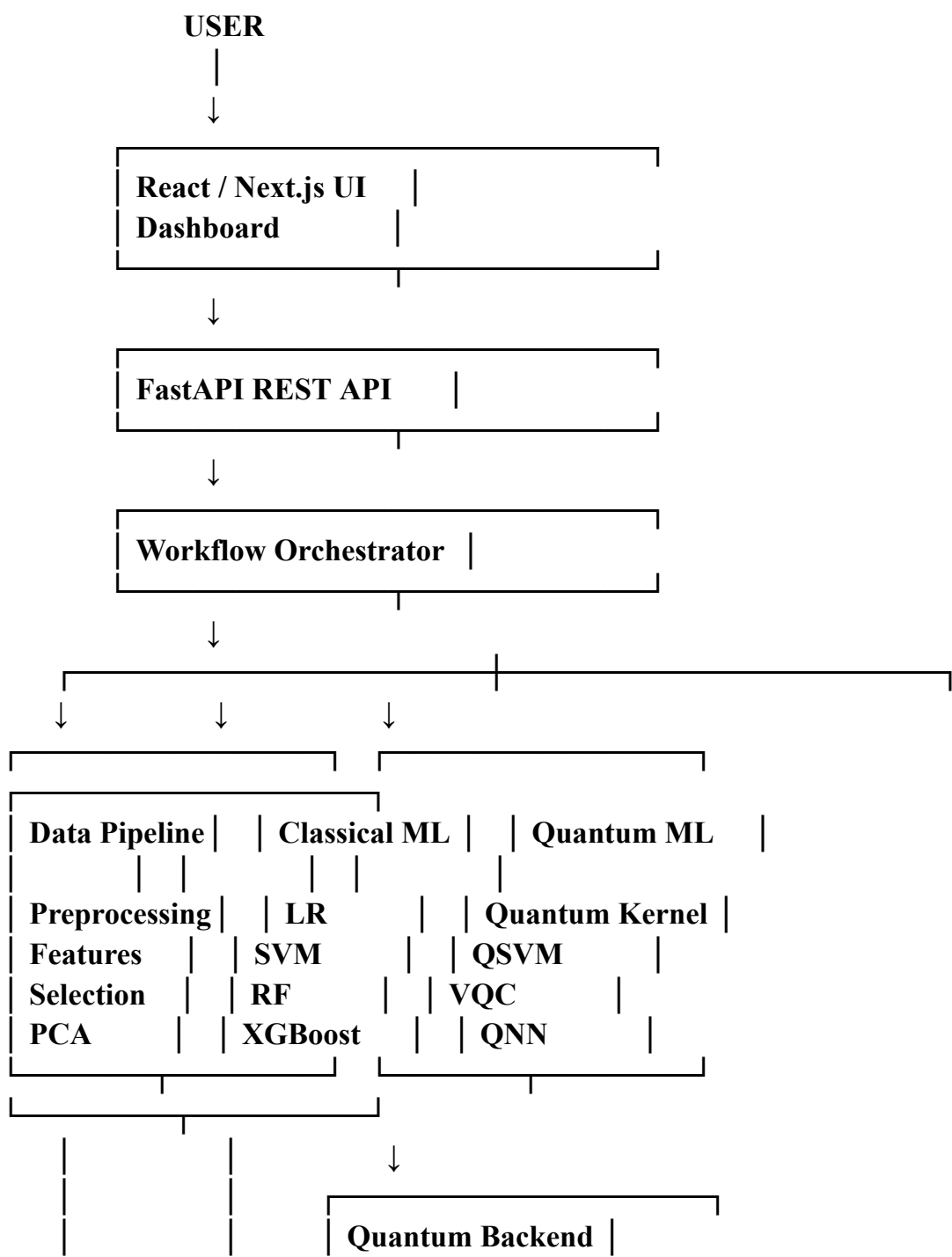


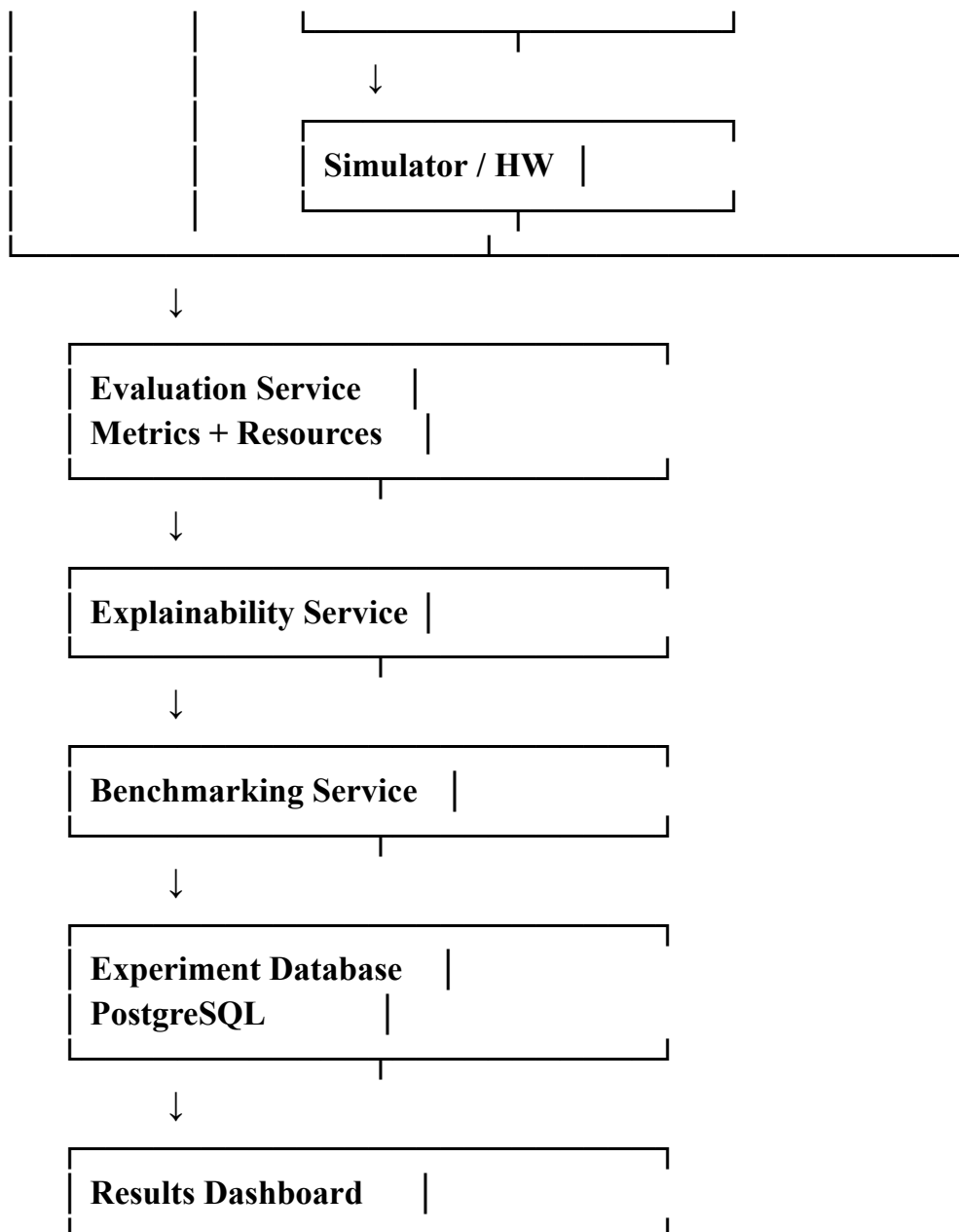
Docker can package the different components.

For the SIH prototype, the entire system can be deployed locally or on a suitable cloud environment.

27.30 Complete Software Architecture

The complete proposed architecture is:





27.31 Advantages of the Proposed Architecture

Modularity

Each component can be developed and tested independently.

Scalability

Additional disease datasets, ML models, and quantum backends can be added without redesigning the entire platform.

Reproducibility

Experiment configurations and results are stored systematically.

Quantum Flexibility

The backend abstraction allows simulator and hardware execution.

Fair Benchmarking

Classical and hybrid QML models can share the same data-processing pipeline.

Explainability

XAI is integrated into the workflow rather than treated as an afterthought.

Maintainability

The frontend, API, ML, QML, evaluation, and database components remain separated.

27.32 Final Architecture Principle

The proposed software architecture follows the principle:

“Modular by design, classical where practical, quantum where useful, and benchmarked end-to-end.”

The architecture ensures that the platform can take biomedical data through the complete machine-learning lifecycle while integrating quantum-enhanced models in a controlled and experimentally measurable manner.

It also ensures that the system remains compatible with current quantum simulators and can transition toward near-term quantum hardware without requiring a complete redesign of the platform.

Most importantly, the architecture supports the central objective of the project:

To build a practical hybrid quantum-classical platform for early disease detection and objectively evaluate its performance against established classical machine-learning approaches.

28. AI/ML Pipeline

28.1 Overview

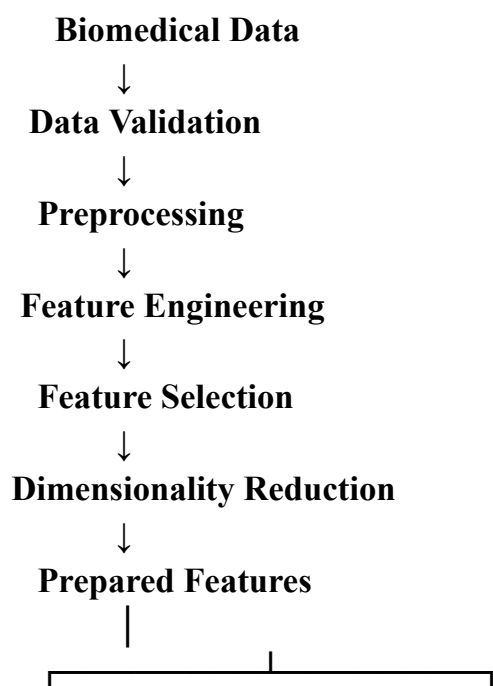
The AI/ML Pipeline is the computational core of the proposed Hybrid Quantum Machine Learning Platform. It converts raw biomedical data into a form that can be used by classical machine-learning and quantum-enhanced models for early disease detection.

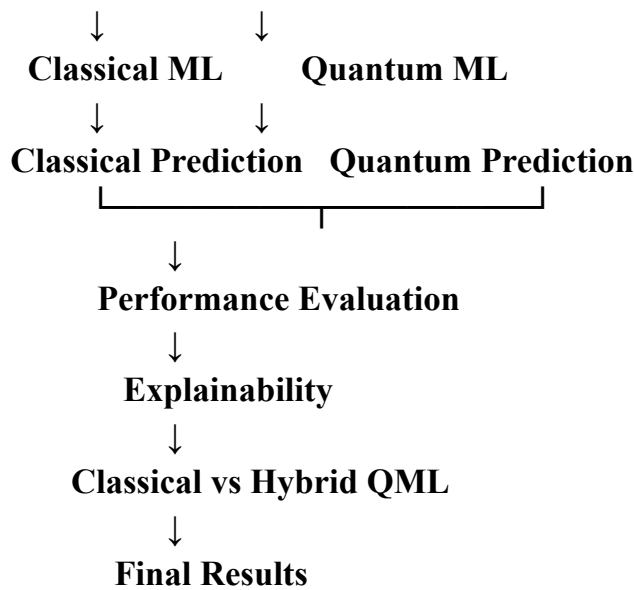
The pipeline is designed so that both classical and hybrid QML models receive a consistent and controlled representation of the biomedical data. This is essential for fair benchmarking.

The complete pipeline is:

Raw Biomedical Data → Validation → Preprocessing → Feature Engineering → Feature Selection → Dimensionality Reduction → Model Training → Prediction → Explainability → Evaluation → Benchmarking

The pipeline contains two model branches after the common data-preparation stage:





28.2 Pipeline Objectives

The AI/ML pipeline is designed to:

1. Convert raw biomedical data into reliable model inputs.
 2. Handle missing, noisy, and inconsistent data.
 3. Extract meaningful features.
 4. Reduce unnecessary dimensionality.
 5. Prepare compact representations for quantum models.
 6. Train strong classical baseline models.
 7. Train quantum-enhanced models.
 8. Generate early disease predictions.
 9. Explain model behavior.
 10. Measure predictive and computational performance.
 11. Compare classical and hybrid QML approaches fairly.
 12. Maintain reproducibility throughout the experiment.
-

28.3 Stage 1 – Raw Biomedical Data

The pipeline begins with a biomedical dataset.

Possible inputs include:

- **Cancer datasets**
- **Cardiovascular datasets**
- **Neurological datasets**
- **Genomic datasets**
- **Electronic Health Record data**
- **Medical imaging-derived numerical features**
- **Public benchmark datasets**

The platform initially focuses on structured numerical/categorical data because it provides a practical path for implementing and benchmarking QML models.

For example:

Samples = 1,000

Features = 50

Target = Disease

Classes = 0 / 1

28.4 Stage 2 – Data Validation

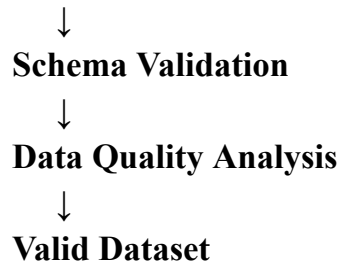
Before the data enters the ML pipeline, the system performs validation.

The validation layer checks:

- **Dataset structure**
- **Feature types**
- **Missing values**
- **Duplicate records**
- **Invalid values**
- **Target column**
- **Class labels**
- **Class distribution**
- **Number of samples**
- **Number of features**

The output is a data-quality report.

Raw Dataset



If a critical problem is detected, the pipeline should stop rather than producing unreliable model results.

28.5 Stage 3 – Data Preprocessing

The validated data is cleaned and transformed.

Major operations include:

Missing Values

Possible strategies:

- **Mean**
- **Median**
- **Mode**
- **Appropriate record removal**

Categorical Encoding

Categorical variables are converted into numerical representations.

Numerical Scaling

Features can be standardized or normalized.

Scaling is particularly important for QML because feature values are later mapped to quantum circuit parameters.

Duplicate Handling

Duplicate records are detected and handled appropriately.

Outlier Analysis

Potential outliers are identified and analyzed.

The pipeline should avoid blindly deleting unusual observations because biomedical outliers can sometimes contain meaningful information.

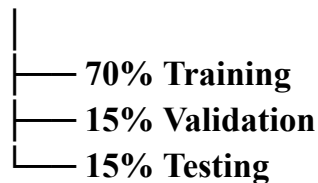
28.6 Stage 4 – Data Splitting

The dataset is separated into:

- **Training set**
- **Validation set**
- **Test set**

Example:

100% Dataset



The exact ratio can be configurable.

The test set remains isolated until final evaluation.

This prevents the model-selection process from indirectly learning from the test data.

28.7 Stage 5 – Feature Engineering

The pipeline transforms processed variables into meaningful features.

Possible operations include:

- **Ratios**

- Differences
- Statistical transformations
- Interaction features
- Log transformations
- Domain-specific features
- Imaging-derived features
- Genomic transformations

For example:

Age

Blood Pressure

Heart Rate

Cholesterol



Derived Features



Clinical Feature Representation

Feature engineering should be controlled because generating too many unnecessary features can increase overfitting and computational cost.

28.8 Stage 6 – Feature Selection

Feature selection identifies the most useful variables.

Methods may include:

- Correlation analysis
- Mutual information
- Statistical tests
- Recursive Feature Elimination
- Tree-based feature importance

Example:

1,000 Features



Feature Ranking

↓
100 Selected Features

This stage is especially important for QML because quantum models typically operate under constrained qubit and circuit resources.

28.9 Stage 7 – Dimensionality Reduction

After feature selection, the representation may still be too large for efficient quantum processing.

PCA can reduce the representation further.

Example:

100 Selected Features
↓
PCA
↓
8 Compact Components

The system records:

- **Number of components**
- **Explained variance**
- **PCA transformation**
- **Training parameters**

The same transformation must be applied to validation, testing, and inference data.

28.10 Stage 8 – Prepared ML Representation

At this point, the platform has a standardized representation suitable for both branches.

Raw Biomedical Data



Preprocessed Data



Engineered Features



Selected Features



Reduced Representation



Prepared ML Input

This common representation is important because it allows a fair comparison between classical and hybrid QML models.

28.11 Stage 9 – Classical ML Branch

The classical branch provides the baseline models.

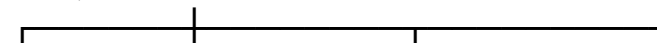
Possible models include:

- Logistic Regression
- SVM
- Random Forest
- XGBoost
- Neural Network

The models receive the prepared feature representation.

For example:

Prepared Features



↓ ↓ ↓ ↓
LR SVM RF XGBoost



Prediction + Evaluation

The best-performing classical models become reference points for the QML experiments.

28.12 Stage 10 – Quantum Feature Encoding

The QML branch transforms the prepared features into quantum states.

For example:

8 Reduced Features



Feature Scaling



Angle Encoding



8-Qubit Quantum Representation

The exact encoding depends on the selected QML model.

The system records:

- Encoding method
 - Number of features
 - Number of qubits
 - Feature-to-qubit mapping
 - Scaling parameters
-

28.13 Stage 11 – Quantum Model Branch

The encoded features are processed by quantum-enhanced models.

The proposed models include:

Quantum Kernel / QSVM

The quantum circuit generates a quantum-based similarity representation, which is then used by a classical SVM-style learning process.

Variational Quantum Classifier

The system uses:

Feature Map → Parameterized Circuit → Measurement → Classical Loss → Optimization

Quantum Neural Network

A parameterized quantum circuit acts as a trainable learning component.

The models can execute using:

- Ideal simulator
 - Noisy simulator
 - Compatible quantum hardware
-

28.14 Stage 12 – Hybrid Training

The central QML training loop combines quantum computation with classical optimization.

Input Features



Quantum Encoding



Quantum Circuit



Measurement



Prediction



Loss Calculation



Classical Optimizer



Parameter Update

↓
Repeat

The classical optimizer updates the trainable quantum parameters.

The process continues until:

- **Maximum iterations are reached**
 - **Convergence occurs**
 - **Validation performance stops improving**
 - **Early stopping is triggered**
-

28.15 Stage 13 – Model Optimization

Both branches can undergo hyperparameter optimization.

Classical Parameters

Examples:

- **Regularization**
- **Tree depth**
- **Number of estimators**
- **Learning rate**
- **Kernel parameters**

Quantum Parameters

Examples:

- **Number of qubits**
- **Circuit depth**
- **Number of layers**
- **Feature-map configuration**
- **Entanglement structure**
- **Ansatz**
- **Optimizer**
- **Learning rate**
- **Training iterations**

Optimization is performed using training/validation data only.

The test set is preserved for final evaluation.

28.16 Stage 14 – Model Selection

After optimization, the platform selects candidate models based on validation performance and practical resource requirements.

For example:

Classical Candidates



Validation



Best Classical Model

Quantum Candidates



Validation



Best Hybrid QML Model

The system can consider:

- **Sensitivity**
- **Specificity**
- **ROC-AUC**
- **F1-score**
- **Computational cost**
- **Quantum resource requirements**
- **Generalization**

The best model is not necessarily the model with the highest accuracy.

28.17 Stage 15 – Test Prediction

The selected models are finally evaluated on unseen test samples.

The inference pipeline is:

Unseen Test Sample



Same Preprocessing



Same Feature Engineering



Same Feature Selection



Same Dimensionality Reduction



Trained Model



Disease Prediction

For QML:

Prepared Features



Quantum Encoding



Quantum Circuit



Measurement



Prediction

28.18 Stage 16 – Prediction Output

The platform can produce:

- Predicted class
- Model score
- Selected model
- Experiment ID

- Backend
- Inference time
- Explanation

Example:

Prediction Result

Class : Disease
Model : Hybrid VQC
Backend : Simulator
Score : Model-generated score
Inference : Measured time

The output should be clearly identified as a model prediction rather than a clinically validated diagnosis.

28.19 Stage 17 – Explainability Pipeline

After prediction, the system can generate explanations.

For classical models:

Prediction



SHAP / Feature Importance



Important Features



Explanation

For QML:

Prediction



Feature Perturbation



Output Change

↓
Feature Sensitivity

The platform can also display:

- **Quantum feature map**
- **Number of qubits**
- **Circuit depth**
- **Parameterized layers**
- **Measurement configuration**

These explain the model's computational structure.

28.20 Stage 18 – Performance Evaluation

The pipeline calculates the required classification metrics.

Accuracy

Measures overall correct predictions.

Sensitivity

Measures the ability to identify positive disease cases.

Specificity

Measures the ability to identify negative cases.

Precision

Measures how many predicted positive cases are actually positive.

F1-score

Provides a combined measure of precision and recall.

ROC-AUC

Measures classification discrimination across thresholds.

The system also generates:

- Confusion matrix
 - ROC curve
 - Precision-recall curve
 - Error analysis
-

28.21 Stage 19 – Computational Evaluation

The pipeline also measures efficiency.

For classical models:

- Training time
- Inference time
- Memory usage
- CPU/GPU requirements

For QML:

- Number of qubits
- Circuit depth
- Gate count
- Shots
- Circuit executions
- Training time
- Inference time
- Backend

This provides a complete performance picture.

28.22 Stage 20 – Generalization Evaluation

The platform should evaluate whether model performance remains stable across different samples and data splits.

Methods can include:

- **Cross-validation**
- **Repeated experiments**
- **Multiple random seeds**
- **Independent test evaluation**

The system can report:

Mean Performance

Standard Deviation

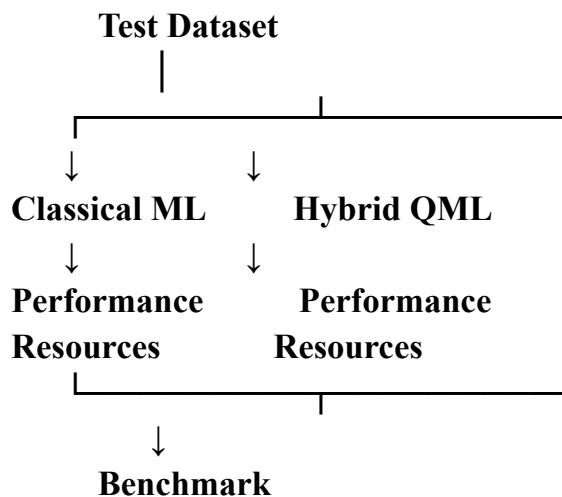
Best Performance

Worst Performance

This helps determine whether an observed performance difference is robust.

28.23 Stage 21 – Classical vs Hybrid QML Benchmark

The final AI/ML pipeline produces a direct comparison.



The comparison includes:

Category	Classical ML	Hybrid QML
----------	--------------	------------

Accuracy	Measured	Measured
Sensitivity	Measured	Measured
Specificity	Measured	Measured
Precision	Measured	Measured
F1-score	Measured	Measured
ROC-AUC	Measured	Measured
Training Time	Measured	Measured
Inference Time	Measured	Measured
Quantum Resources	N/A	Measured
Generalization	Measured	Measured

The values must come from actual experiments.

28.24 AI/ML Pipeline Configuration

The complete pipeline should be configurable.

An experiment configuration can contain:

Dataset

Target Column

Preprocessing Method

Feature Engineering

Feature Selection Method

Number of Selected Features

PCA Components

Classical Models

Quantum Models

Quantum Encoding

Number of Qubits
Circuit Depth
Entanglement

Optimizer
Learning Rate
Iterations
Shots
Backend

Evaluation Metrics
Random Seed

This configuration is stored with the experiment.

28.25 Pipeline Reproducibility

Reproducibility is essential for scientific comparison.

The pipeline records:

- **Dataset version**
- **Preprocessing parameters**
- **Feature transformations**
- **Selected features**
- **PCA parameters**
- **Model parameters**
- **Quantum circuit configuration**
- **Backend**
- **Random seed**
- **Training parameters**
- **Evaluation results**

Therefore:

**Same configuration + same dataset + same computational conditions →
reproducible experiment, within the expected variability of the execution
environment.**

28.26 Data Leakage Prevention

Data leakage is one of the major risks in biomedical ML.

The pipeline must ensure that operations such as:

- Scaling
- PCA
- Feature selection
- Feature engineering parameters
- Hyperparameter tuning

are learned from training data and then applied to validation/test data.

Incorrect:

Complete Dataset



PCA



Train/Test Split

Correct:

Dataset



Train/Test Split



Training Data → Fit PCA



Validation/Test → Apply fitted PCA

The same principle applies to feature selection and other learned transformations.

28.27 Handling Class Imbalance

Biomedical datasets may contain unequal numbers of positive and negative cases.

The pipeline can address imbalance using suitable methods such as:

- Class weighting
- Resampling
- Stratified splitting
- Appropriate evaluation metrics

Accuracy should not be used as the only metric when class imbalance is significant.

Sensitivity, specificity, F1-score, ROC-AUC, and confusion-matrix analysis should also be considered.

28.28 Handling Noisy Quantum Execution

Quantum hardware and noisy simulators may introduce errors.

The pipeline can therefore support:

Ideal Simulation



Noisy Simulation



Hardware Execution

The results can be compared to understand how noise affects:

- Prediction
- Sensitivity
- Specificity
- ROC-AUC
- Stability

This provides a realistic evaluation of near-term QML models.

28.29 AI/ML Pipeline Monitoring

The dashboard can monitor:

- Current pipeline stage
- Training progress
- Validation performance
- Loss
- Execution time
- Quantum backend
- Number of circuit executions
- Errors

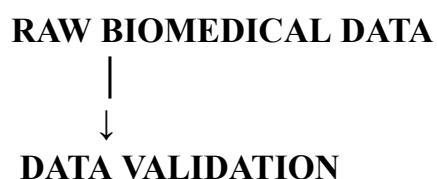
Example:

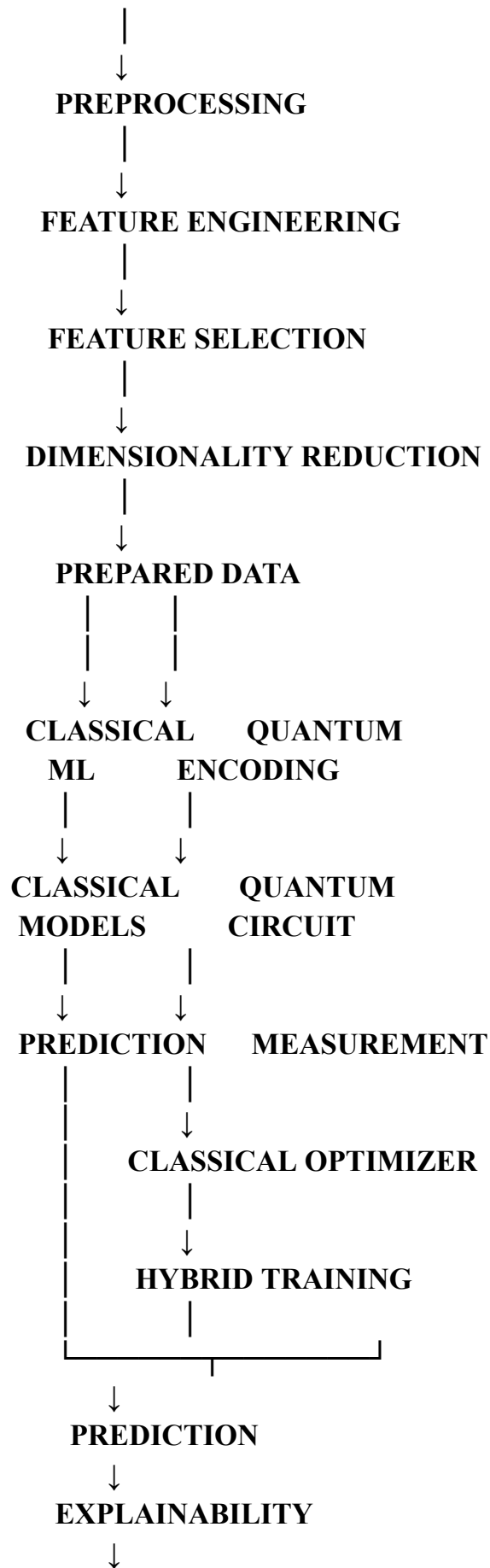
Dataset	✓
Preprocessing	✓
Feature Selection	✓
PCA	✓
Classical Training	✓
Quantum Training	⌚
Evaluation	○
Explainability	○
Benchmarking	○

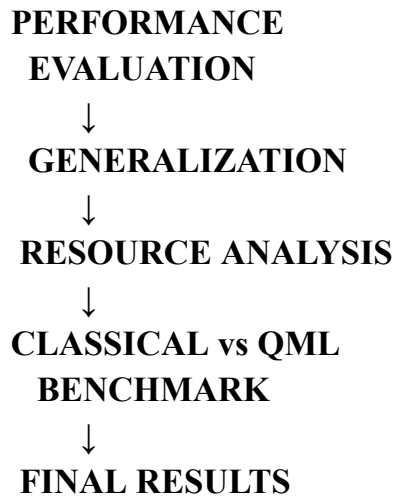
This makes long-running experiments easier to monitor.

28.30 Complete AI/ML Pipeline Architecture

The complete pipeline is:

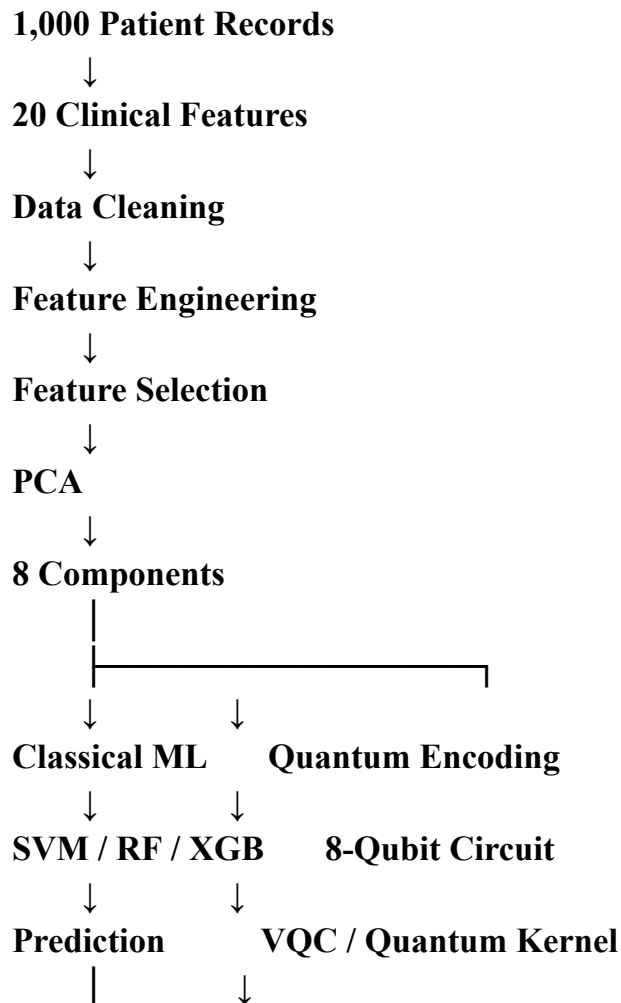


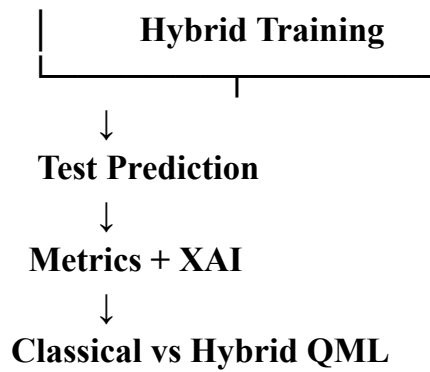




28.31 Example AI/ML Pipeline

For a cardiovascular disease dataset:





The final performance values are obtained experimentally.

28.32 Why the Pipeline is Hybrid

The pipeline is genuinely hybrid because different computational stages use the technology best suited to them.

Classical Computing

Used for:

- Data ingestion
- Data cleaning
- Feature engineering
- Feature selection
- PCA
- Classical ML
- Loss calculation
- Optimization
- Evaluation
- Visualization

Quantum Computing

Used for:

- Quantum feature encoding
- Quantum kernels
- Quantum circuits

- Variational quantum learning
- Quantum measurement

The two computing paradigms communicate during model training and inference.

28.33 Key Benefits

Complete Workflow

The entire process is integrated into one platform.

Fair Comparison

Classical and QML models can use the same prepared dataset.

Resource Awareness

Quantum resource requirements are measured.

Reproducibility

Every experiment can be tracked.

Explainability

Model behavior can be analyzed after prediction.

Hardware Readiness

The same QML pipeline can move from simulation toward compatible quantum hardware.

Research Transparency

The system reports measured results rather than assuming quantum superiority.

28.34 Final AI/ML Pipeline Principle

The proposed AI/ML pipeline follows:

“Clean the data classically, reduce it intelligently, learn through classical and quantum-enhanced models, evaluate on unseen data, explain the predictions, and benchmark both approaches under comparable conditions.”

The pipeline therefore directly supports the core objective of the project: developing and experimentally evaluating a practical hybrid quantum-classical machine-learning approach for early disease detection.

29. Dataset Strategy

29.1 Overview

The Dataset Strategy defines how biomedical datasets will be selected, prepared, validated, and used for developing and evaluating the Hybrid Quantum Machine Learning Platform.

Since the proposed system is intended for early disease detection, the quality and suitability of the dataset are critical. A sophisticated quantum model cannot compensate for poor-quality, biased, insufficient, or incorrectly prepared data.

The platform therefore follows a structured dataset strategy:

Dataset Selection → Data Validation → Quality Analysis → Preprocessing → Feature Engineering → Feature Selection → Dimensionality Reduction → Training → Testing → Benchmarking

The dataset strategy is designed to support:

- **Real or benchmark biomedical datasets**
- **Reproducible experiments**
- **Classical ML baselines**
- **Quantum ML experiments**
- **Fair benchmarking**
- **Explainability**
- **Privacy-aware development**
- **Simulator-based experimentation**
- **Future near-term quantum hardware execution**

29.2 Dataset Requirements

A suitable dataset should satisfy several requirements.

1. Biomedical Relevance

The dataset should represent a real biomedical problem such as:

- Cancer
- Cardiovascular disease
- Neurological conditions
- Genomics
- Medical imaging
- Electronic Health Records

2. Clear Target Variable

The dataset should have a clearly defined prediction target.

For example:

Target = Disease

Class 0 = No Disease

Class 1 = Disease

3. Sufficient Samples

The dataset should contain enough observations to train and evaluate machine-learning models reliably.

4. Meaningful Features

The input variables should contain useful biomedical information.

5. Suitable Data Format

For the prototype, structured datasets such as CSV are preferred because they simplify the initial implementation.

6. Reproducibility

The dataset should be publicly available or otherwise legally usable for research and demonstration.

29.3 Target Disease Categories

The problem statement specifically identifies disease applications such as:

- Cancer
- Cardiovascular disorders
- Neurological conditions

The platform should therefore be designed to support all three categories.

However, the SIH prototype does not necessarily need to train a separate large QML model for every disease category.

A more practical strategy is:

Build a disease-independent platform and demonstrate the complete workflow using one carefully selected benchmark dataset.

Additional datasets can then be used to demonstrate generalization of the platform.

29.4 Dataset Types

The platform can support several types of biomedical data.

A. Clinical Data

Examples include:

- Age
- Blood pressure
- Heart rate
- Cholesterol
- Laboratory measurements

- **Clinical indicators**

These datasets are particularly suitable for an initial prototype because they are usually structured.

B. Genomic Data

Genomic datasets may contain:

- **Gene-expression values**
- **Genetic markers**
- **Molecular measurements**
- **High-dimensional feature vectors**

These datasets are particularly relevant to the project's motivation because they can contain extremely high-dimensional information.

However, dimensionality reduction becomes especially important before QML processing.

C. Medical Imaging-Derived Data

Medical images themselves can be extremely large.

Instead of directly sending raw images into a quantum circuit, the platform can initially use numerical features extracted from images.

Example:

Medical Image



Feature Extraction



Numerical Feature Vector



Feature Selection



PCA



Quantum-Compatible Representation

This is a more practical approach for a near-term QML prototype.

D. Electronic Health Records

EHR datasets may contain:

- Demographics
- Clinical measurements
- Diagnoses
- Laboratory values
- Historical records

Structured EHR data can be transformed into numerical features suitable for the proposed pipeline.

29.5 Recommended Prototype Dataset Strategy

For the SIH prototype, a structured binary classification dataset is recommended.

The ideal initial dataset should have:

Moderate number of samples

+

Manageable feature count

+

Clear disease target

+

Limited preprocessing complexity

+

Public/research-appropriate availability

This allows the team to demonstrate the complete workflow rather than spending most of the project time solving dataset-engineering problems.

A practical prototype path is:

Cardiovascular disease dataset → classical preprocessing → feature reduction → 4–16 quantum-compatible features → QSVM/VQC → benchmark against classical models.

29.6 Dataset Selection Criteria

A dataset can be scored using a selection framework.

Criterion	Importance
Biomedical relevance	High
Clear prediction target	High
Data quality	High
Number of samples	High
Feature quality	High
Public/research availability	High
Class balance	Medium–High
Missing values	Medium
Dimensionality	High for QML
Reproducibility	High
Privacy suitability	High
Computational feasibility	High

The final dataset should provide a reasonable balance between biomedical relevance and QML feasibility.

29.7 Dataset Quality Assessment

Before training, the platform automatically generates a quality report.

The report includes:

Dataset Size

- Number of rows
- Number of columns

Data Types

- Numerical features
- Categorical features
- Boolean features

Missing Data

- Missing count
- Missing percentage
- Missing-value distribution

Duplicate Data

- Number of duplicate rows
- Duplicate percentage

Target Distribution

Example:

No Disease	540
Disease	460

Feature Distribution

The system can identify:

- Constant features
 - Near-constant features
 - Highly skewed features
 - Potential outliers
-

29.8 Dataset Versioning

Each dataset should have a version.

Example:

Dataset: Cardiovascular Benchmark

Version: 1.0

Samples: 1,000

Features: 20

Target: Disease

If preprocessing or dataset corrections create a new version:

Dataset v1.0



Validated Dataset



Dataset v1.1

Experiments should reference the exact dataset version used.

This is important for reproducibility.

29.9 Data Privacy Strategy

Biomedical data can contain sensitive information.

The prototype should therefore prioritize:

- **Public benchmark datasets**
- **De-identified datasets**
- **Research-authorized datasets**

If sensitive data is used in a future deployment, additional controls may be required.

The platform should avoid storing unnecessary personally identifiable information.

Security measures can include:

- **Access control**
 - **Secure storage**
 - **Authentication**
 - **Encryption**
 - **Audit logging**
-

29.10 Data Preprocessing Strategy

After selecting the dataset, preprocessing is performed.

The pipeline may include:

Raw Dataset



Missing Value Handling



Categorical Encoding



Duplicate Handling



Outlier Analysis



Feature Scaling



Train/Validation/Test Split

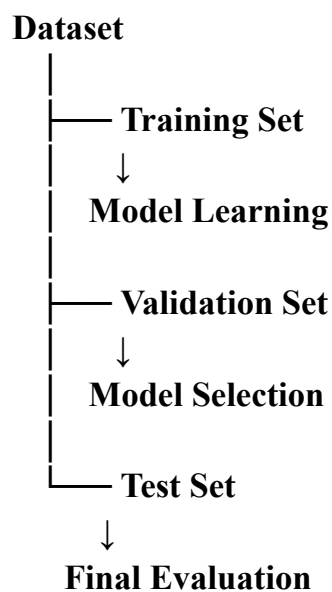
The exact preprocessing operations should depend on the dataset.

No single preprocessing technique should be assumed to be optimal for every biomedical dataset.

29.11 Train, Validation, and Test Data

The dataset is divided into independent subsets.

For example:



A representative split could be:

- 70% training
- 15% validation
- 15% testing

The exact split can be adjusted based on dataset size and experimental requirements.

29.12 Stratification

For classification tasks, the platform should use stratified splitting where appropriate.

This helps maintain similar class proportions across:

- **Training**
- **Validation**
- **Test**

For example:

Full Dataset

Disease = 46%

No Disease = 54%

Training

Disease $\approx$ 46%

No Disease $\approx$ 54%

Validation

Disease $\approx$ 46%

No Disease $\approx$ 54%

Test

Disease $\approx$ 46%

No Disease $\approx$ 54%

This helps avoid evaluation distortion caused by highly different class distributions.

29.13 Data Leakage Prevention

Data leakage is particularly dangerous in biomedical machine learning.

The dataset strategy therefore follows:

Split first, learn transformations from training data, then apply them to validation and test data.

For example, PCA should not be fitted using the complete dataset before splitting.

Incorrect:

Complete Dataset



PCA



Train/Test Split

Correct:

Dataset



Train/Test Split



Fit PCA on Training Data



Transform Validation/Test Data

The same rule applies to:

- **Scaling**
- **Feature selection**
- **Imputation parameters**
- **Feature engineering parameters**
- **Hyperparameter optimization**

29.14 Handling Class Imbalance

Some disease datasets may contain significantly more negative cases than positive cases.

The platform can use:

- **Class weights**
- **Appropriate resampling**
- **Stratified splitting**
- **Threshold analysis**

The evaluation should not depend only on accuracy.

Important metrics include:

- **Sensitivity**
- **Specificity**
- **Precision**
- **F1-score**
- **ROC-AUC**

This is especially important when the cost of missing positive disease cases is significant.

29.15 High-Dimensional Dataset Strategy

High-dimensional biomedical data is one of the main motivations for investigating QML.

For example:

10,000 Features



Feature Selection



500 Relevant Features



PCA



8–16 Components



Quantum Encoding

The platform should record the dimensionality at every stage.

This allows the researchers to study the trade-off between:

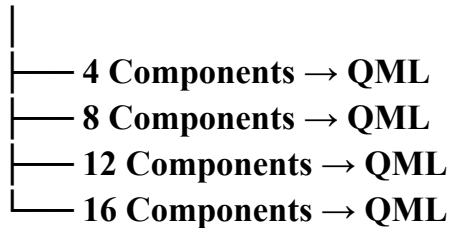
Information retained ↔ Number of features ↔ Quantum resource requirements

29.16 Feature-Dimension Experiments

The platform can evaluate different reduced dimensions.

For example:

Original Dataset



The resulting performance can then be compared.

This helps identify whether increasing the number of quantum inputs actually improves performance enough to justify the additional resource cost.

29.17 Dataset and Classical Baseline Consistency

The same dataset should be used for classical and hybrid QML experiments wherever the comparison is intended to be direct.

For example:

Same Dataset



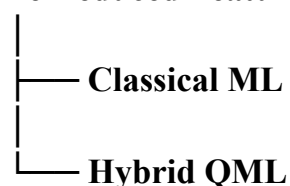
Same Split



Same Preprocessing



Same Reduced Features



This prevents differences in data preparation from being incorrectly attributed to the model architecture.

29.18 Dataset Strategy for QML

The dataset preparation for QML should consider quantum constraints.

Important factors include:

- Number of features
- Number of qubits
- Circuit depth
- Feature encoding
- Execution cost
- Noise
- Number of shots

The objective is to create a compact representation that preserves useful information without requiring impractical quantum resources.

29.19 Multiple Dataset Strategy

After validating the prototype on one primary dataset, additional datasets can be introduced.

Example:

Primary Dataset



Complete Pipeline Validation



Secondary Dataset



Pipeline Generalization



Third Dataset



Cross-Dataset Analysis

Possible categories:

Dataset	Domain	Purpose
Dataset A	Cardiovascular	Primary prototype
Dataset B	Cancer	Disease-domain extension
Dataset C	Neurological	Generalization study
Dataset D	Genomics	High-dimensional experiment

The platform itself remains disease-independent.

29.20 Dataset Selection vs Clinical Validation

A benchmark dataset can demonstrate technical feasibility, but it does not automatically establish clinical usefulness.

The distinction is:

Benchmark Dataset → Technical Evaluation

whereas:

Clinical Dataset + Clinical Study + External Validation → Potential Clinical Evidence

Therefore, the SIH prototype should focus on demonstrating:

- **Technical feasibility**
- **Model performance**

- **QML integration**
- **Explainability**
- **Benchmarking**
- **Resource evaluation**

It should not claim clinical diagnosis or clinical validation without appropriate evidence.

29.21 Dataset Documentation

Every dataset used by the platform should have associated metadata.

The system can store:

- **Dataset name**
- **Source**
- **Version**
- **Number of samples**
- **Number of features**
- **Target definition**
- **Feature descriptions**
- **Class distribution**
- **Missing-value information**
- **Preprocessing steps**
- **License/usage conditions where applicable**

This information should be accessible from the dashboard.

29.22 Dataset Experiment Record

A complete experiment can store:

Dataset



Version



Preprocessing

↓
Feature Engineering
↓
Feature Selection
↓
PCA
↓
Model
↓
Quantum Backend
↓
Results

Example:

Experiment ID: EXP-025

Dataset: Cardiovascular Benchmark

Version: 1.0

Samples: 1,000

Original Features: 20

Selected Features: 10

PCA Components: 8

Classical Model: SVM

Quantum Model: VQC

Backend: Noisy Simulator

This makes every result traceable to its dataset configuration.

29.23 Dataset Strategy for SIH Demonstration

For the SIH presentation, the dataset strategy should prioritize clarity and complete functionality over trying to support dozens of datasets.

A strong demonstration can follow:

Phase 1 – Dataset Selection

Choose one high-quality public biomedical benchmark dataset.

Phase 2 – Data Quality

Show automated validation.

Phase 3 – Processing

Demonstrate preprocessing and feature reduction.

Phase 4 – Classical Baseline

Train multiple classical models.

Phase 5 – QML

Train a quantum kernel or VQC model.

Phase 6 – Evaluation

Compare:

- **Accuracy**
- **Sensitivity**
- **Specificity**
- **F1**
- **ROC-AUC**

Phase 7 – Resource Analysis

Show:

- **Qubits**
- **Circuit depth**
- **Shots**
- **Execution time**

Phase 8 – Explainability

Show feature importance or suitable QML sensitivity analysis.

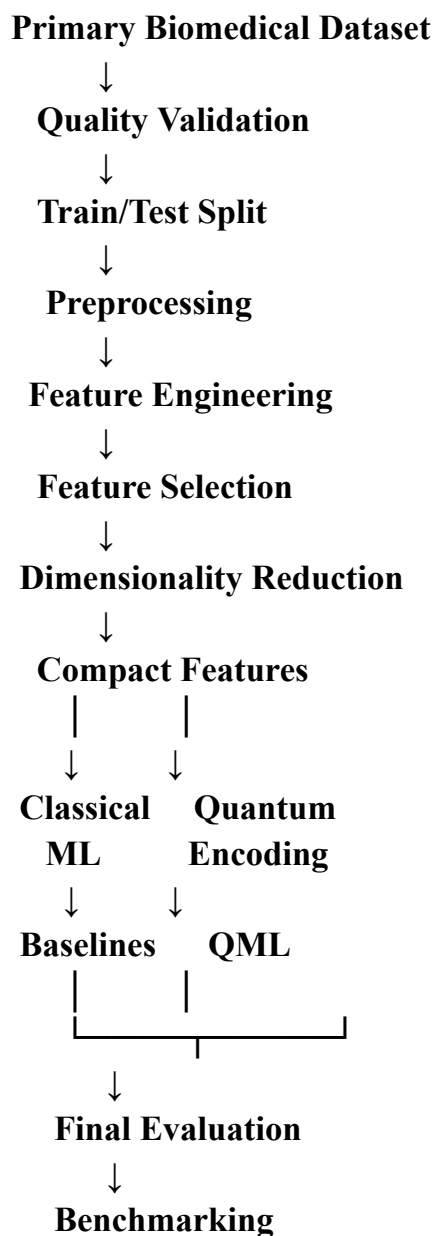
Phase 9 – Benchmark

Present the final classical-vs-QML comparison.

This demonstrates the complete problem-to-solution pipeline.

29.24 Recommended Data Strategy

The recommended implementation strategy is:



29.25 Dataset Strategy Principles

The project follows the following principles:

1. Quality Before Complexity

A reliable dataset is more important than a complex model.

2. Public and Reproducible Where Possible

The prototype should use datasets that can be legally and reproducibly accessed.

3. Leakage Prevention

No information from the final test set should influence model development.

4. Quantum-Aware Preparation

High-dimensional data should be reduced carefully before quantum processing.

5. Fair Benchmarking

Classical and QML models should use comparable data representations when directly compared.

6. Disease Independence

The platform architecture should not be hard-coded for one disease.

7. Clinical Responsibility

Benchmark performance should not be presented as clinical validation.

29.26 Final Dataset Strategy

The proposed strategy is:

Start with one high-quality, publicly usable biomedical benchmark dataset, validate and preprocess it systematically, create a compact and meaningful representation, use the same experimental foundation for classical and hybrid QML models, and gradually extend the platform to additional disease domains.

This strategy keeps the SIH prototype technically achievable while still demonstrating the broader capability of the proposed Hybrid Quantum Machine Learning Platform.

The dataset strategy therefore directly supports the project's objectives of:

- **Handling complex biomedical data**
- **Performing feature engineering**
- **Reducing high-dimensional information**
- **Training quantum-enhanced models**
- **Comparing against classical ML**
- **Evaluating sensitivity and specificity**
- **Supporting explainability**
- **Measuring computational and quantum resources**
- **Testing generalization**
- **Supporting simulator and near-term hardware workflows**

Core Principle: “Choose data that allows the complete experiment to be performed correctly before increasing the complexity of the model or quantum system.”

30. Training and Testing Strategy

30.1 Overview

The Training and Testing Strategy defines how classical machine-learning and hybrid quantum machine-learning models will be trained, optimized, validated, and tested.

The primary objective is to ensure that the reported performance is:

- **Reliable**
- **Reproducible**
- **Fair**
- **Free from data leakage**

- Comparable between classical and hybrid QML models
- Evaluated on unseen data

The platform follows the principle:

Training data is used for learning, validation data is used for model selection, and the test set is used only for final evaluation.

The complete strategy is:

Dataset → Split → Preprocessing → Training → Validation → Optimization → Final Model → Unseen Test Data → Evaluation → Benchmarking

30.2 Dataset Splitting

The dataset is divided into independent subsets.

A typical configuration is:

Dataset Portion	Example Percentage	Purpose
Training	70%	Model learning
Validation	15%	Model selection and tuning
Testing	15%	Final evaluation

The exact percentages can be adjusted according to dataset size.

For smaller datasets, cross-validation can be used to obtain more reliable estimates.

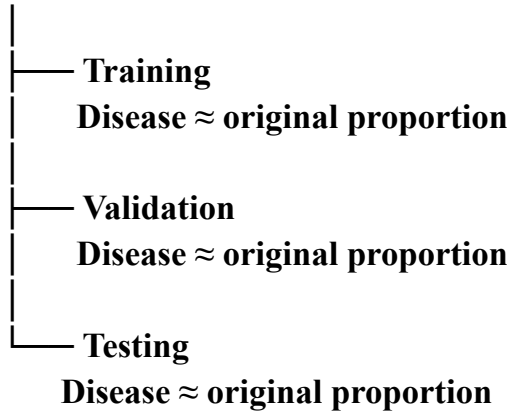
30.3 Stratified Splitting

For binary disease classification, stratified splitting should be used where appropriate.

The objective is to preserve the class distribution across the different subsets.

Example:

Complete Dataset



This reduces the possibility that one subset contains an unusually high or low proportion of disease-positive samples.

30.4 Training Pipeline

The training pipeline follows:

Training Dataset



Preprocessing



Feature Engineering



Feature Selection



Dimensionality Reduction



Model Training



Validation



Hyperparameter Optimization



Best Model

All transformations that require learning parameters should be fitted using the training data only.

30.5 Preprocessing During Training

Examples of learned preprocessing operations include:

- Mean/median imputation
- Standardization
- Normalization
- PCA
- Feature selection
- Encoding parameters

For example:

Training Data



Fit Scaler



Fit PCA



Train Model

The learned scaler and PCA transformation are then applied to validation and test data.

They are not refitted on the test set.

30.6 Classical Model Training

The platform trains multiple classical models as reference baselines.

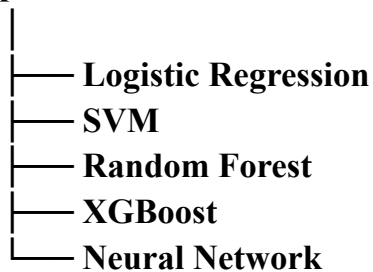
Possible models include:

- **Logistic Regression**
- **Support Vector Machine**
- **Random Forest**
- **XGBoost**
- **Neural Network**

Each model is trained using the same underlying dataset and preprocessing pipeline intended for the comparison.

Example:

Prepared Features



The best-performing classical models become reference points for the hybrid QML models.

30.7 Quantum Model Training

The quantum branch follows:

Prepared Features



Quantum Feature Encoding



Quantum Circuit



Measurement



Loss Calculation



Classical Optimizer



Parameter Update



Repeat

The quantum model can be implemented using approaches such as:

- **Quantum Kernel / QSVM**
- **Variational Quantum Classifier**
- **Quantum Neural Network**

The actual model selected depends on the dataset and experimental results.

30.8 Quantum Kernel Training

For a quantum-kernel approach, the system uses a quantum feature map to generate a similarity representation.

Conceptually:

Classical Features



Quantum Feature Map



Quantum Circuit



Kernel Matrix



Classical SVM



Prediction

The kernel approach allows the quantum circuit to contribute to the feature-space representation while classical optimization and classification remain part of the workflow.

30.9 VQC Training

For a Variational Quantum Classifier:

Input Features



Feature Encoding



Parameterized Quantum Circuit



Measurement



Prediction / Loss



Classical Optimizer



Update Circuit Parameters



Repeat

Training continues until:

- **Convergence**
 - **Maximum iterations**
 - **Early stopping condition**
 - **No meaningful validation improvement**
-

30.10 Hyperparameter Optimization

The platform should not simply use arbitrary model settings.

Important classical hyperparameters may include:

- **Learning rate**
- **Regularization**
- **Tree depth**
- **Number of estimators**
- **Kernel parameters**

- **Network architecture**

QML hyperparameters may include:

- **Number of qubits**
- **Number of circuit layers**
- **Feature-map configuration**
- **Entanglement structure**
- **Ansatz configuration**
- **Optimizer**
- **Learning rate**
- **Number of iterations**
- **Number of measurement shots**

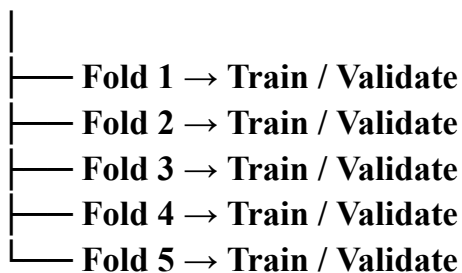
The optimization process should use the training/validation data and never use the final test set for tuning.

30.11 Cross-Validation

For datasets where appropriate, the platform can use k-fold cross-validation.

Example:

Dataset



The final validation score can be calculated across folds.

For example:

5-Fold Cross-Validation

Each fold becomes the validation set once while the remaining folds are used for training.

This gives a more robust estimate than relying on a single split.

30.12 Nested Validation for Model Selection

For rigorous experiments, nested cross-validation can be used when computationally feasible.

Conceptually:

Outer Loop



Generalization Evaluation



└ Inner Loop



Hyperparameter Selection

The inner loop selects model configurations.

The outer loop estimates generalization performance.

This is particularly useful when the dataset is relatively small.

30.13 Test Set Isolation

The final test set must remain isolated.

The test set should not be used for:

- Feature selection
- PCA fitting
- Hyperparameter tuning
- Model selection
- Threshold tuning
- Repeated experimentation

The test set should be accessed only after the final model configuration has been selected.

This ensures that final performance is a more realistic estimate of unseen-data performance.

30.14 Early Stopping

For iterative models, early stopping can help reduce overfitting.

The system can monitor validation performance.

Example:

Training



Validation Score Improves



Continue

Validation Score Stops Improving



Early Stopping



Best Model

For QML, this can also reduce unnecessary circuit evaluations.

30.15 Class Imbalance Strategy

Disease datasets may contain unequal numbers of positive and negative cases.

The platform can use:

- Class weighting
- Appropriate resampling

- Stratified splitting
- Threshold analysis

However, any resampling operation must be performed inside the training portion only.

Incorrect:

Entire Dataset



Oversampling



Train/Test Split

Correct:

Dataset



Train/Test Split



Training → Oversampling

Test → No Oversampling

This prevents synthetic or duplicated information from leaking into the test set.

30.16 Training the Classical and QML Models Fairly

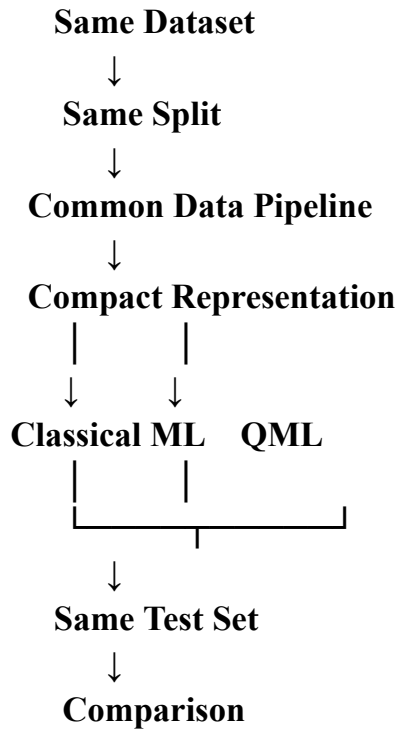
A central requirement of the project is fair comparison.

The system should maintain:

- Same dataset
- Same target definition
- Same train/test split
- Same evaluation metrics

- Comparable preprocessing
- Comparable feature representation where appropriate

Example:



This prevents an unfair comparison caused by giving one model a better dataset or a different test set.

30.17 Two Benchmarking Conditions

The platform can support two complementary comparison strategies.

Condition A — Common Representation

Both classical and QML models use the same reduced feature representation.

This provides a clean comparison of the learning approaches.

Condition B — Best Practical Pipeline

Each model can use its best reasonable configuration.

This answers a different question:

Which complete practical pipeline performs best under the available computational and quantum constraints?

Both results can be reported separately.

30.18 Performance Metrics

The models should be evaluated using multiple metrics.

Accuracy

Measures overall correct predictions.

Precision

Measures how many predicted positive cases are actually positive.

Sensitivity / Recall

Measures how many actual positive cases are detected.

Specificity

Measures how many actual negative cases are correctly identified.

F1-Score

Balances precision and recall.

ROC-AUC

Measures ranking/discrimination performance across classification thresholds.

Where class imbalance is significant, PR-AUC can also be considered.

30.19 Confusion Matrix

For binary disease classification:

	Predicted	
	Negative	Positive
Actual Negative	TN	FP
Actual Positive	FN	TP

This allows the system to analyze:

- **True Positives**
- **True Negatives**
- **False Positives**
- **False Negatives**

For early disease detection, false negatives deserve particular attention because they represent positive cases that were not detected.

30.20 Threshold Analysis

A classifier may produce a score that can be converted into a positive/negative decision using a threshold.

The platform can evaluate different thresholds.

Example:

Lower Threshold



Higher Sensitivity



Potentially More False Positives

Higher Threshold



Higher Specificity



Potentially More False Negatives

The chosen threshold should be determined using validation data and documented.

It should not be optimized directly on the final test set.

30.21 Generalization Testing

The platform should evaluate whether the model performs well on unseen samples.

Important indicators include:

- Training performance
- Validation performance
- Test performance
- Cross-validation performance
- Performance variance

Example:

Training AUC = High

Validation AUC = Similar

Test AUC = Similar

↓

Better indication of generalization

If training performance is much higher than test performance, the system should flag potential overfitting.

30.22 Multiple Random Seeds

QML and classical training can sometimes vary because of initialization or stochastic optimization.

Therefore, important experiments should be repeated using multiple random seeds where feasible.

Example:

Seed 1 → Model → Results

Seed 2 → Model → Results

Seed 3 → Model → Results

Seed 4 → Model → Results

Seed 5 → Model → Results



Mean ± Variability

This provides a better picture of model stability than reporting a single lucky run.

30.23 Quantum Resource Evaluation

The QML training process should record quantum-specific resources.

Examples:

- **Number of qubits**
- **Circuit depth**
- **Gate count**
- **Number of circuit evaluations**
- **Number of shots**
- **Simulator execution time**
- **Hardware execution time where applicable**
- **Backend type**
- **Noise configuration**

This allows performance to be evaluated together with resource requirements.

30.24 Classical Computational Evaluation

Classical resources should also be recorded.

Examples:

- **Training time**
- **Inference time**
- **Memory usage**
- **CPU utilization**
- **Dataset size**
- **Number of features**

This is important because the project is evaluating the overall hybrid system, not just predictive accuracy.

30.25 Simulator vs Hardware Testing

The training and testing strategy supports multiple execution environments.

Stage 1 — Ideal Simulator

Used for:

- **Initial development**
- **Debugging**
- **Algorithm verification**

Stage 2 — Noisy Simulator

Used for:

- **Noise sensitivity**
- **Robustness testing**
- **NISQ-style experimentation**

Stage 3 — Near-Term Quantum Hardware

Used where feasible for:

- **Hardware execution**
- **Real-device measurements**
- **Hardware-specific resource analysis**

Results from these environments should be reported separately.

A simulator result should never be presented as equivalent to real quantum-hardware performance.

30.26 Noise Robustness Testing

For QML models, noise can affect prediction performance.

The platform can experiment with:

- **Gate noise**
- **Measurement noise**
- **Readout errors**
- **Different noise strengths**

Example:

Ideal Simulator



Low Noise



Medium Noise



Higher Noise



Performance Analysis

This helps determine whether the QML model remains useful under realistic near-term conditions.

30.27 Model Stability Analysis

The platform should measure whether model performance is stable.

For repeated experiments, it can report:

- **Mean accuracy**

- Standard deviation
- Mean sensitivity
- Standard deviation
- Mean specificity
- Standard deviation
- Mean ROC-AUC
- Standard deviation

Example:

Model A

Accuracy = 91.2% ± 1.1%

Sensitivity = 89.7% ± 1.8%

Specificity = 92.5% ± 1.3%

The values above are illustrative; actual project results must come from experiments.

30.28 Overfitting Detection

The system should monitor training and validation performance.

Possible warning:

Training Performance

↑↑↑

Validation Performance

↑
↓

Potential Overfitting

Potential mitigation methods include:

- Regularization
- Simpler models
- Feature reduction
- Early stopping

- Reduced circuit depth
 - Reduced model complexity
 - Cross-validation
-

30.29 Underfitting Detection

The platform should also identify underfitting.

Example:

Training Performance → Low

Validation Performance → Low

↓

Potential Underfitting

Possible responses include:

- Better feature engineering
 - More informative features
 - More suitable model
 - Appropriate model complexity
 - Improved hyperparameters
-

30.30 Experiment Tracking

Every training experiment should have a unique identifier.

Example:

Experiment ID: EXP-031

Dataset: Biomedical Dataset v1.0

Split: 70/15/15

Features: 8 PCA components

Model: VQC

Qubits: 8
Layers: 3
Optimizer: Selected Configuration
Backend: Noisy Simulator

Metrics:
Accuracy
Sensitivity
Specificity
F1
ROC-AUC

This allows different experiments to be compared and reproduced.

30.31 Reproducibility

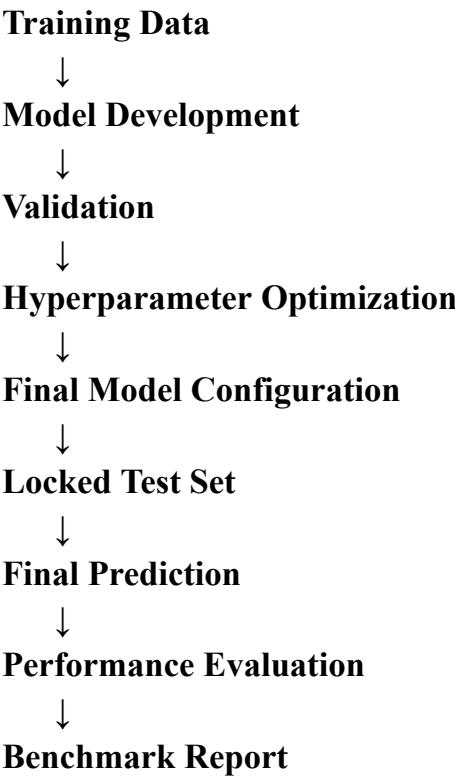
The platform should store:

- **Dataset version**
- **Dataset split**
- **Random seed**
- **Preprocessing configuration**
- **Feature-selection configuration**
- **PCA configuration**
- **Model configuration**
- **Hyperparameters**
- **Quantum backend**
- **Circuit configuration**
- **Number of shots**
- **Software environment**
- **Evaluation metrics**

This creates a reproducible experiment record.

30.32 Final Testing Workflow

After model selection is complete:



The test set is used only at this final stage.

30.33 Final Classical vs QML Comparison

The final report should contain a structured comparison.

Metric	Classical Model	Hybrid QML Model
Accuracy	Measured	Measured
Sensitivity	Measured	Measured
Specificity	Measured	Measured
Precision	Measured	Measured
F1	Measured	Measured

ROC-AUC	Measured	Measured
Training Time	Measured	Measured
Inference Time	Measured	Measured
Generalization	Measured	Measured
Quantum Resources	N/A	Measured

Actual values should only be entered after experiments are performed.

30.34 Statistical Interpretation

The platform should avoid concluding that one model is superior based on a very small difference in one test run.

Where feasible, repeated experiments and statistical analysis should be used.

The report should consider:

- **Mean performance**
- **Variability**
- **Confidence intervals where appropriate**
- **Cross-validation results**
- **Multiple random seeds**
- **Dataset limitations**

The objective is to distinguish meaningful improvement from experimental variation.

30.35 Model Selection Strategy

The final model should not be selected using accuracy alone.

A multi-factor decision can consider:

Predictive Performance

+

Sensitivity

+

Specificity

+

Generalization

+

Computational Cost

+

Quantum Resource Cost

+

Stability

The final choice should therefore be the best practical model for the experimental objective, not automatically the model with the highest single metric.

30.36 Training and Testing Dashboard

The dashboard should show:

Dataset

- **Dataset name**
- **Version**
- **Number of samples**
- **Number of features**

Training

- **Training progress**
- **Current epoch/iteration**
- **Loss**
- **Validation score**

Model

- **Classical/QML model**

- **Hyperparameters**
- **Feature dimension**

Quantum

- **Qubits**
- **Circuit depth**
- **Backend**
- **Shots**
- **Execution count**

Evaluation

- **Accuracy**
- **Sensitivity**
- **Specificity**
- **F1**
- **ROC-AUC**
- **Confusion matrix**

Generalization

- **Cross-validation results**
 - **Test performance**
 - **Stability**
-

30.37 Failure and Validation Handling

The system should detect problems before training.

Examples:

Invalid Dataset

Missing target column



Training blocked



User receives validation message

Insufficient Samples

Dataset too small



Warning



Recommend cross-validation

Excessive Quantum Dimension

Features > available qubits



Recommend feature reduction

Unstable Training

Validation performance unstable



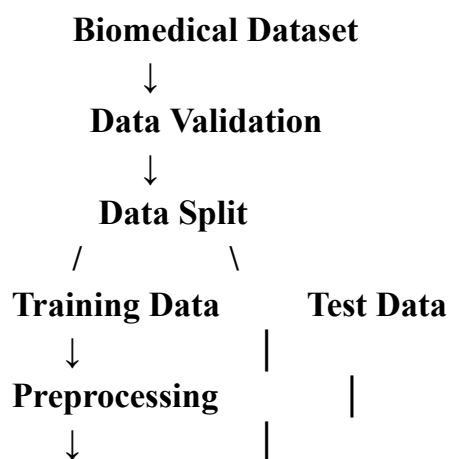
Flag experiment

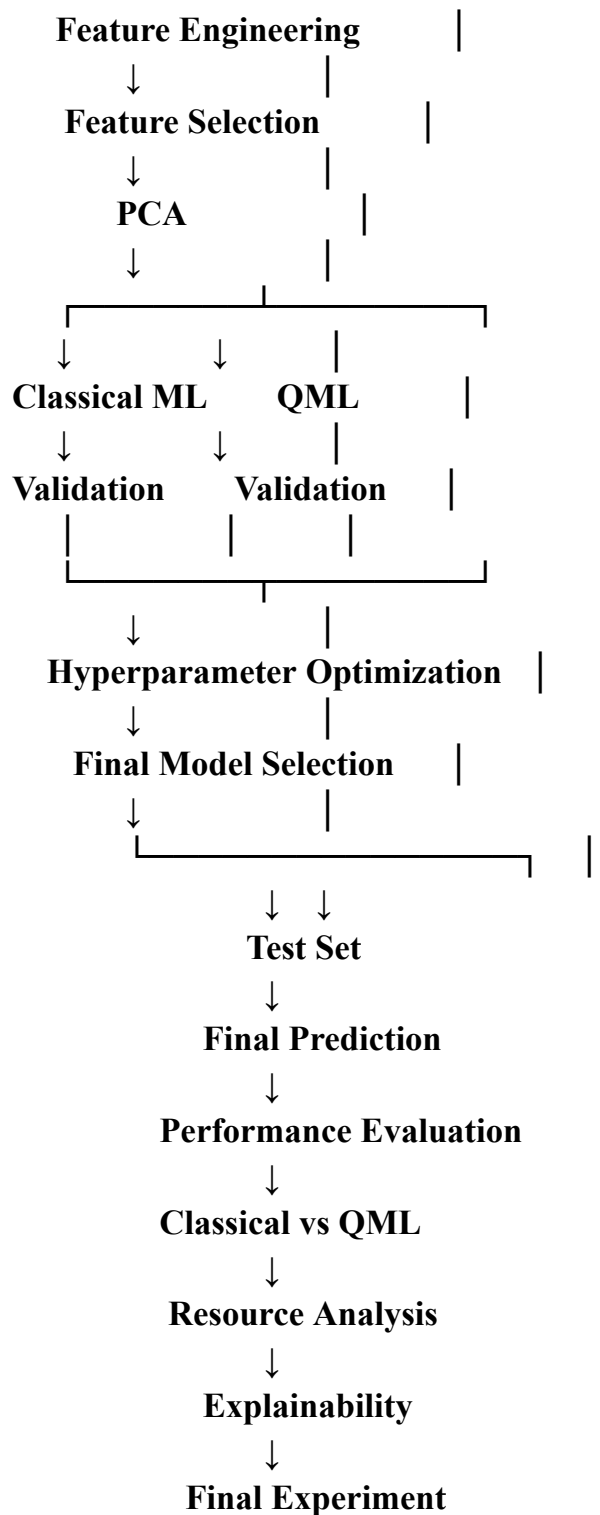


Recommend repeated runs

This makes the platform more robust and user-friendly.

30.38 Complete Training and Testing Architecture





30.39 Key Training and Testing Principles

The platform follows these principles:

Principle 1 — Test Data Is Sacred

The final test set must remain untouched during model development.

Principle 2 — Same Data, Fair Comparison

Classical and QML approaches should be compared using comparable experimental conditions.

Principle 3 — Optimize Without Leakage

Hyperparameter optimization must use training/validation data only.

Principle 4 — Repeat Important Experiments

Multiple seeds and cross-validation should be used where computationally feasible.

Principle 5 — Measure Resources

Accuracy alone is insufficient for evaluating a hybrid quantum system.

Principle 6 — Report Variability

A single result is not enough to establish robust performance.

Principle 7 — Separate Simulator and Hardware Results

Simulation and real-device execution must be clearly distinguished.

30.40 Final Training and Testing Strategy

The proposed platform follows:

Split the biomedical dataset correctly, perform all learned preprocessing using training data, train classical and quantum-enhanced models under comparable conditions, optimize using validation data, keep the final test set isolated, evaluate multiple medical-relevant metrics, repeat experiments where feasible, measure computational and quantum resources, and report the results transparently.

This strategy ensures that the project does not simply produce a high accuracy number, but instead produces a reproducible and scientifically defensible comparison between classical and hybrid quantum machine-learning approaches.

Core Principle

“Train fairly, validate carefully, test once, measure everything, and let experimental evidence determine whether the hybrid quantum approach provides an advantage.”

31. Data Leakage Prevention

31.1 Overview

Data leakage occurs when information that should be unavailable during model training or development is unintentionally used by the model.

In a disease-detection system, leakage can produce artificially high performance and make the model appear much better than it actually is.

Therefore, the proposed Hybrid Quantum Machine Learning Platform treats data leakage prevention as a core part of the ML pipeline, rather than as an optional post-processing step.

The fundamental rule is:

Information from validation and test data must not influence model training, feature selection, preprocessing, hyperparameter optimization, or model selection.

31.2 Why Data Leakage Is Critical

Consider a disease dataset containing:

Patient Information



Features



Disease Label

If information related to the disease label accidentally enters the training features, the model can learn the answer directly.

For example:

Target = Disease

Incorrect Feature:

"Final Diagnosis Code"

If the final diagnosis code directly reveals the target, the model may achieve extremely high accuracy without learning meaningful biomedical patterns.

This creates a misleading result.

31.3 Types of Data Leakage

The platform considers multiple leakage scenarios.

1. Target Leakage

A feature contains information that was generated after or because of the target outcome.

2. Train-Test Contamination

Information from the test set influences training.

3. Preprocessing Leakage

A preprocessing transformation is fitted using the complete dataset before splitting.

4. Feature Selection Leakage

Features are selected using the entire dataset, including test samples.

5. PCA Leakage

PCA is fitted on the complete dataset instead of training data.

6. Hyperparameter Leakage

The test set is repeatedly used to choose the best model configuration.

7. Duplicate Leakage

The same patient or highly similar sample appears in both training and test sets.

8. Temporal Leakage

Information from the future is used to predict an earlier event.

9. Group Leakage

Samples belonging to the same patient or related entity are split across training and test sets.

10. Resampling Leakage

Oversampling or synthetic data generation is performed before train/test splitting.

31.4 Correct Data Pipeline

The platform follows:

Raw Dataset



Initial Validation



Train / Validation / Test Split



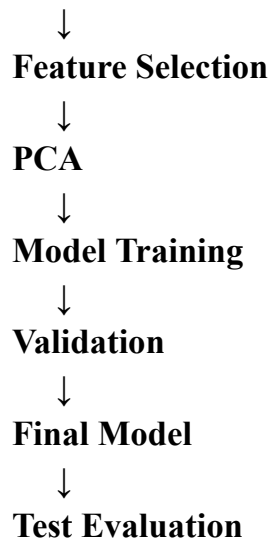
Training Data



Fit Preprocessing



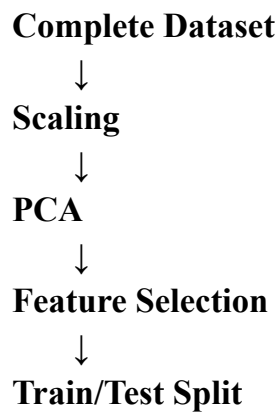
Feature Engineering



The test set remains isolated until final evaluation.

31.5 Incorrect Pipeline

The platform explicitly avoids workflows such as:

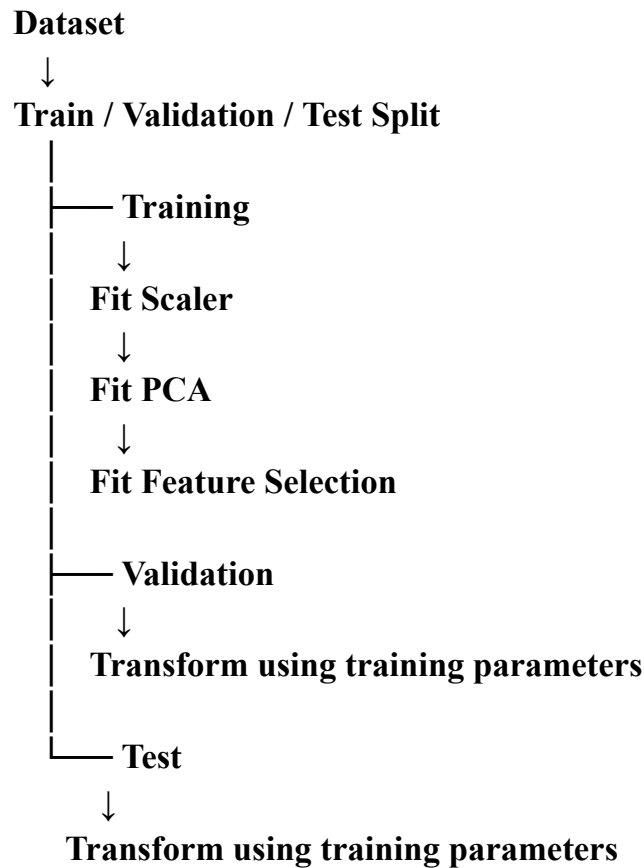


This can leak statistical information from the test set into the training process.

Even though the test labels may not be directly used, information from the test distribution has influenced the learned transformation.

31.6 Correct Preprocessing

The correct approach is:



The validation and test sets are transformed using parameters learned from the training data.

31.7 Scaling Leakage

Suppose the platform uses standardization:

$$x' = (x - \mu) / \sigma$$

The values of:

- μ = mean
- σ = standard deviation

must be learned from the training data.

Incorrect:

$\mu = \text{mean}(\text{all data})$

$\sigma = \text{standard deviation}(\text{all data})$

Correct:

$\mu = \text{mean}(\text{training data})$

$\sigma = \text{standard deviation}(\text{training data})$

The same training parameters are then applied to validation and test data.

31.8 PCA Leakage

PCA is especially important because the project uses dimensionality reduction before quantum encoding.

Incorrect:

Complete Dataset



PCA



Train/Test Split

Correct:

Dataset



Train/Test Split



Fit PCA on Training Data



Transform Validation Data



Transform Test Data

This ensures that the principal components are derived only from information available during model development.

31.9 Feature Selection Leakage

Feature selection must also be performed only using training information.

Incorrect:

Complete Dataset



Select Top 10 Features



Train/Test Split

Correct:

Training Data



Feature Selection



Selected Features



Apply Same Feature Set



Validation / Test

The final test set must not influence which features are selected.

31.10 Hyperparameter Leakage

Suppose the team tests:

Model A → Test Accuracy = 90%

Model B → Test Accuracy = 92%

Model C → Test Accuracy = 94%

and then chooses Model C.

The test set has effectively become part of model development.

This is leakage.

Correct approach:

Training Data



Validation Experiments



Select Best Model



Lock Configuration



Final Test Evaluation

31.11 QML-Specific Leakage

The same leakage principles apply to quantum machine learning.

Potential leakage can occur during:

- **Quantum feature preparation**
- **Feature scaling**
- **PCA**
- **Feature selection**
- **Quantum model selection**
- **Circuit-depth selection**
- **Qubit-count selection**
- **Hyperparameter optimization**
- **Threshold selection**
- **Backend selection**

For example, repeatedly changing the number of qubits based on final test performance would make the test set part of model development.

Therefore, QML configuration must also be selected using training/validation information.

31.12 Quantum Feature Encoding Leakage

Quantum feature encoding should receive only properly prepared features.

The workflow is:

Training Features



Fit Classical Transformation



Reduced Representation



Quantum Encoding



Quantum Circuit

For validation/test data:

Validation/Test Features



Apply Existing Transformation



Reduced Representation



Quantum Encoding



Quantum Circuit

The encoding procedure itself should remain fixed unless a new experiment is started.

31.13 Patient-Level Leakage

Biomedical datasets may contain multiple records from the same patient.

For example:

Patient A

├── Visit 1
├── Visit 2
└── Visit 3

If:

Visit 1 → Training

Visit 2 → Testing

the model may indirectly recognize the patient rather than learn disease-related patterns.

Where patient identifiers or grouping information are available, the platform should use group-aware splitting.

Example:

Patient A → Training

Patient B → Training

Patient C → Validation

Patient D → Test

rather than randomly splitting individual records.

31.14 Duplicate Detection

Duplicate samples can create another form of leakage.

The platform should check for:

- Exact duplicates

- Duplicate patient records
- Repeated measurements
- Highly similar samples where appropriate

Example:

Training:

Patient X → Record A

Testing:

Patient X → Record A

This can produce unrealistically high performance.

Therefore, duplicate detection should occur during dataset validation.

31.15 Temporal Leakage

Some biomedical datasets contain time-dependent information.

Example:

January:

Patient Symptoms

March:

Confirmed Diagnosis

If the objective is to predict the March diagnosis using information available in January, the model must not receive information recorded after January.

Incorrect:

Past Features + Future Information

↓

Prediction

Correct:

Information Available at Prediction Time



Prediction

This is particularly important when the platform is eventually extended to longitudinal EHR data.

31.16 Target Leakage Detection

The platform should inspect features that may strongly reveal the target.

Potential warning signals include:

- Feature name directly related to the target
- Diagnosis codes
- Post-diagnosis measurements
- Treatment information occurring after diagnosis
- Outcome-derived variables

The platform can flag suspicious features for user review.

Example:

⚠ **Potential Target Leakage**

Feature:

"Final_Diagnosis_Code"

Reason:

May directly encode target information.

Action:

Review / Exclude

This should be treated as an automated warning, not as a substitute for domain expertise.

31.17 Resampling Leakage

If the dataset is imbalanced, techniques such as oversampling should be applied only to the training data.

Incorrect:

Complete Dataset



SMOTE / Oversampling



Train/Test Split

Correct:

Dataset



Train/Test Split



— **Training** → **Resampling**

— **Test** → **Original Distribution**

The test set must remain representative of unseen data.

31.18 Cross-Validation Leakage

When cross-validation is used, preprocessing and feature selection must occur inside each training fold.

Incorrect:

Complete Dataset



PCA



5-Fold Cross Validation

Correct:

Fold 1

Training → Fit PCA → Validation

Fold 2

Training → Fit PCA → Validation

Fold 3

Training → Fit PCA → Validation

...

This ensures each validation fold remains unseen during transformation fitting.

31.19 Pipeline-Based Prevention

The platform should implement preprocessing and model training as a unified pipeline wherever possible.

Conceptually:

Input Data



Validation



Preprocessing



Feature Selection



PCA



Model

This reduces the chance that developers accidentally perform preprocessing outside the controlled training workflow.

31.20 Separate Training and Testing Environments

The software architecture can maintain logical separation between:

Development Data

Used for:

- Experimentation
- Feature engineering
- Model development

Validation Data

Used for:

- Hyperparameter tuning
- Model selection
- Threshold selection

Test Data

Used for:

- Final evaluation

The test dataset should not be exposed to ordinary model-development operations.

31.21 Test Set Locking

Once the test split has been created, it should be treated as locked.

The platform can maintain a state such as:

Test Set Status:

 Locked

The test set becomes available for final evaluation only after the model configuration is finalized.

This creates an additional procedural safeguard.

31.22 Experiment Tracking for Leakage Prevention

Each experiment should record:

- Dataset version
- Data split
- Random seed
- Preprocessing configuration
- Feature selection method
- PCA configuration
- Model configuration
- Hyperparameters
- Validation results
- Test evaluation timestamp

Example:

Experiment: EXP-042

Dataset: v1.0

Split: Stratified 70/15/15

Scaler: StandardScaler

PCA: 8 components

Feature Selection: Mutual Information

Model: VQC

Backend: Noisy Simulator

Test Set:
LOCKED

This makes it possible to audit how the final result was produced.

31.23 Automated Leakage Checks

The platform can implement automated checks before training.

Check 1 — Target Column

Ensure target is not accidentally present in the feature matrix.

Check 2 — Duplicate Samples

Search for duplicates across train/validation/test.

Check 3 — Patient Groups

Ensure patient groups do not cross dataset boundaries where group information exists.

Check 4 — Transformation Fitting

Ensure preprocessing is fitted only on training data.

Check 5 — Feature Selection

Ensure test data is not used for feature ranking.

Check 6 — PCA

Ensure PCA is fitted only on training data.

Check 7 — Resampling

Ensure resampling occurs only inside training folds.

Check 8 — Hyperparameter Optimization

Ensure final test performance is not used during tuning.

31.24 Leakage Detection Report

Before final evaluation, the system can generate a leakage report.

Example:

DATA LEAKAGE AUDIT

Target leakage check	✓ Passed
Duplicate check	✓ Passed
Patient-group check	✓ Passed
Scaler isolation	✓ Passed
PCA isolation	✓ Passed
Feature-selection check	✓ Passed
Resampling check	✓ Passed
Test-set isolation	✓ Passed
Hyperparameter check	✓ Passed

Overall Status:

✓ No detected pipeline leakage

If a problem is found:

⚠ Leakage Risk Detected

PCA was fitted using non-training data.

Final evaluation blocked.

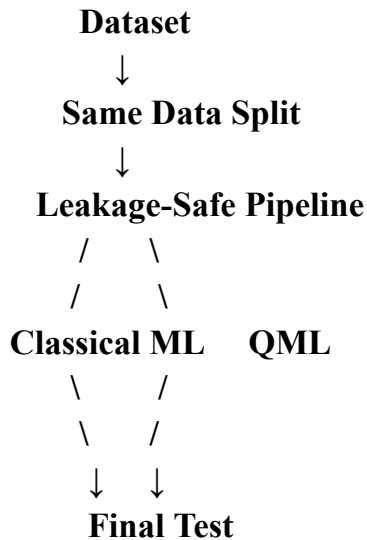
Recommended action:

Re-run preprocessing using training-only fitting.

This makes leakage prevention an actual platform feature rather than only a documentation statement.

31.25 Leakage Prevention in Classical vs QML Benchmarking

Leakage prevention must apply equally to both branches.



Neither model should receive information that the other does not.

This is essential for a fair comparison.

31.26 Leakage Prevention During Explainability

Explainability analysis should also be performed carefully.

For example, feature importance should not be calculated using information from the final test labels to modify the model.

The system can generate explanations after the model is trained without allowing those explanations to influence the final test evaluation.

If explanations lead to a model change, that becomes a new development experiment and requires another final evaluation.

31.27 Leakage Prevention During Threshold Optimization

Suppose the system initially uses:

Threshold = 0.50

and then changes it repeatedly based on test performance.

That is test-set leakage.

Correct:

Training



Validation



Threshold Selection



Lock Threshold



Final Test

The threshold should be selected using validation data.

31.28 Leakage Prevention During Model Comparison

If ten different models are evaluated repeatedly on the test set and the best test score is reported, the test set is effectively being used for model selection.

The platform therefore distinguishes:

Development Results

Used to select models.

Final Test Result

Used once the final configuration is locked.

This provides a cleaner estimate of generalization.

31.29 Leakage Prevention for Future External Validation

A future version of the platform can support an independent external dataset.

Example:

Development Dataset



Training + Validation



Final Model



Independent External Dataset



Generalization Evaluation

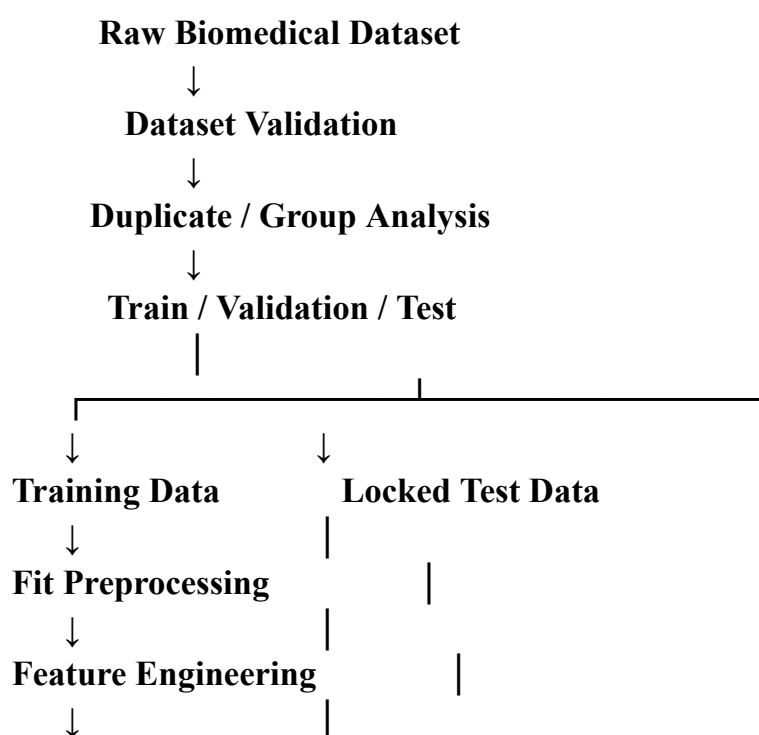
External validation provides stronger evidence of generalization than repeatedly optimizing on a single dataset.

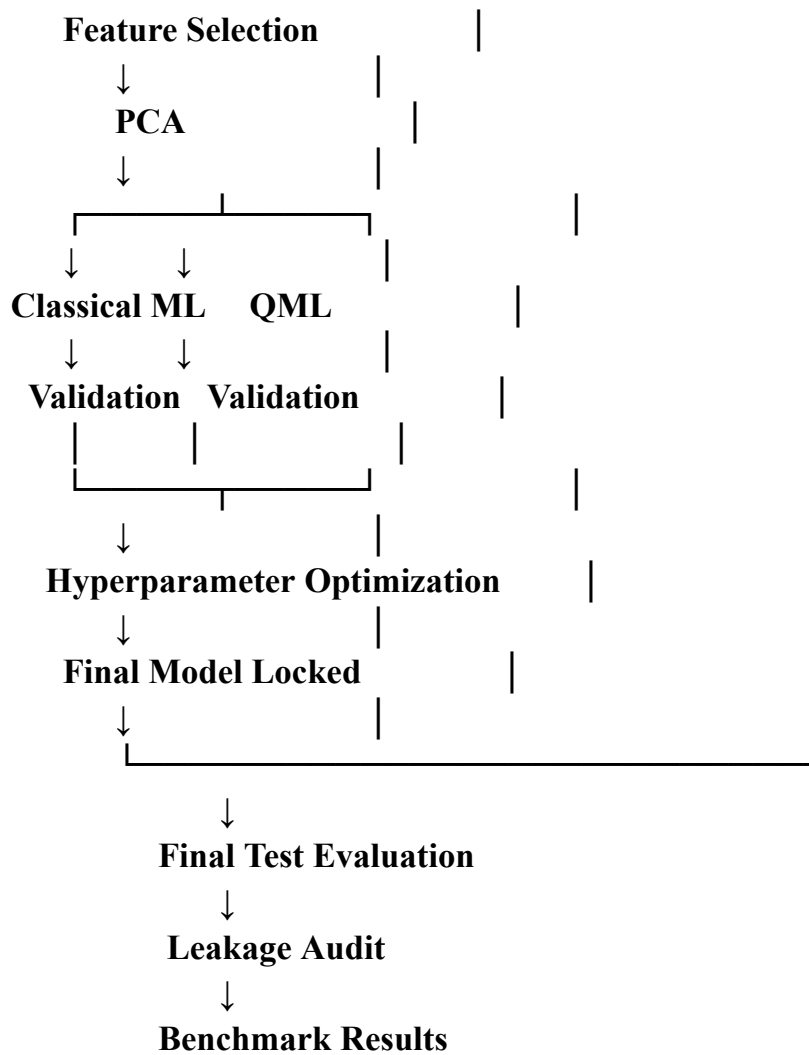
31.30 Data Leakage Risk Matrix

Leakage Type	Risk	Prevention
Target leakage	High	Target-feature audit
Train-test contamination	High	Strict dataset split
Scaling leakage	High	Fit on training only

PCA leakage	High	Training-only PCA
Feature selection leakage	High	Training-only selection
Hyperparameter leakage	High	Validation-based tuning
Duplicate leakage	High	Duplicate detection
Patient leakage	High	Group-aware splitting
Temporal leakage	High	Time-aware validation
Resampling leakage	Medium–High	Training-only resampling
QML configuration leakage	High	Validation-based selection
Threshold leakage	Medium–High	Validation-based threshold

31.31 Leakage-Safe End-to-End Workflow





31.32 Implementation Strategy

The software should enforce leakage prevention at multiple levels.

Data Layer

- Dataset validation
- Duplicate detection
- Group identification
- Target validation

ML Layer

- Pipeline-based preprocessing

- **Training-only transformations**
- **Validation-based optimization**

QML Layer

- **Controlled feature encoding**
- **Validation-based circuit/model selection**
- **Backend configuration tracking**

Evaluation Layer

- **Locked test set**
- **Final evaluation**
- **Metric calculation**

Experiment Layer

- **Configuration tracking**
 - **Dataset versioning**
 - **Random seed tracking**
 - **Audit logs**
-

31.33 Key Design Principle

The platform should not simply tell the user:

“Avoid data leakage.”

It should actively design the workflow so that leakage is difficult to introduce accidentally.

This can be achieved through:

- **Automated pipelines**
- **Locked test sets**
- **Dataset validation**
- **Experiment tracking**
- **Training-only transformations**
- **Leakage audits**
- **Validation-based model selection**

31.34 Final Data Leakage Prevention Strategy

The proposed platform follows a strict leakage-prevention methodology:

Split the data before learning transformations, fit preprocessing and feature-selection operations only on training data, keep patient/group information isolated where applicable, perform hyperparameter optimization using validation data, keep the final test set locked, apply the same leakage-safe principles to classical and QML models, and audit the complete pipeline before reporting final results.

This ensures that reported performance is based on unseen test data rather than accidental information exposure.

Core Principle

“The model must learn from the training data, be selected using validation data, and face the test data only when the experiment is ready for its final evaluation.”

32. Model Reliability

Model reliability defines whether the developed system can provide consistent and dependable outputs when used under different operating conditions. It focuses on the behaviour of the deployed model rather than simply its training score.

32.1 Reliability Monitoring

The platform continuously records important model behaviour during experiments and prediction requests.

It monitors:

- **Prediction success/failure**
- **Invalid input frequency**
- **Model response time**
- **Unexpected output patterns**
- **Backend execution failures**
- **Quantum circuit execution failures**
- **Changes in prediction behaviour**

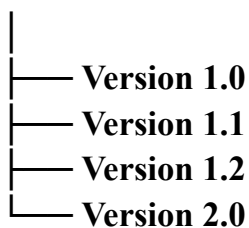
This creates a reliability history for each model version.

32.2 Model Version Control

Every trained model receives a unique version.

For example:

Disease Model



The platform can identify exactly which model generated a particular prediction.

This prevents confusion when several experiments produce different models.

32.3 Model Rollback

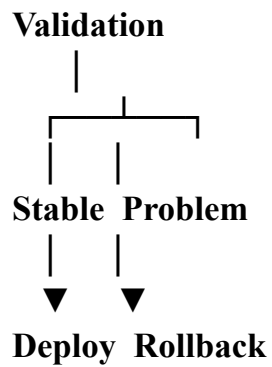
If a newly trained model behaves unexpectedly, the platform can restore a previously validated model version.

Previous Model



New Model





This provides an additional safeguard when experimenting with new QML configurations.

32.4 Input Validation at Prediction Time

Reliability should also begin before the model makes a prediction.

The system checks whether incoming data:

- Contains required fields
- Uses the expected data types
- Follows the trained feature schema
- Contains valid numerical ranges
- Has excessive missing values
- Matches the expected feature ordering

Invalid input is rejected or flagged instead of being silently passed to the model.

32.5 Schema Consistency

A trained model expects a specific feature structure.

For example:

Training:

Age

Blood Pressure

Glucose

Cholesterol

BMI

If prediction data arrives as:

Age

Glucose

BMI

Cholesterol

the system should detect the mismatch rather than assuming that the columns are correctly aligned.

This prevents silent prediction errors.

32.6 Preprocessing Consistency

The prediction environment must use the same transformation configuration used during model development.

The platform therefore stores preprocessing parameters such as:

- **Scaling parameters**
- **Encoding mappings**
- **Selected feature list**
- **Transformation configuration**
- **Dimensionality-reduction parameters**

This ensures that training and prediction operate on compatible representations.

32.7 Model Compatibility Checks

Before deployment or execution, the platform checks whether:

- **Model and dataset schemas match**
- **Required preprocessing artifacts exist**
- **Quantum circuit configuration is valid**
- **Required number of qubits is available**

- Selected backend supports the required operations
- Model artifacts are complete

If a requirement is missing, the system reports the problem before execution.

32.8 Quantum Execution Reliability

Quantum execution can fail for reasons unrelated to the ML algorithm.

The platform can handle:

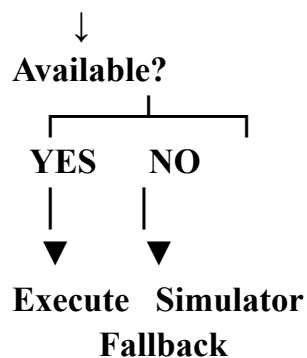
- Backend unavailable
- Circuit compilation failure
- Unsupported gate
- Insufficient qubits
- Execution timeout
- Job failure
- Excessive queue delay

Instead of silently producing an incomplete result, the experiment is marked with an appropriate execution status.

32.9 Automatic Fallback

A practical feature is controlled fallback between execution environments.

Requested Backend



For research experiments, this allows development to continue when a hardware backend is temporarily unavailable.

The system clearly records that a fallback occurred so that results are not confused with hardware results.

32.10 Reproducible Prediction

The platform stores the information required to reproduce an experiment.

A prediction can therefore be associated with:

Model Version

Dataset Version

Feature Configuration

Preprocessing Version

Quantum Configuration

Backend

Execution Settings

Experiment ID

This creates a traceable relationship between input → model configuration → output.

32.11 Audit Trail

Important system events are recorded in an audit log.

Examples:

- **Dataset uploaded**
- **Model trained**
- **Model version created**
- **Model selected**
- **Quantum backend changed**
- **Experiment executed**
- **Prediction generated**
- **Model replaced**
- **Execution failed**

This is useful for debugging and research documentation.

32.12 Failure Classification

Instead of treating every failure as a generic error, the system categorizes it.

Data Failure

Invalid or incomplete input.

Model Failure

Missing or corrupted model artifact.

Computational Failure

Insufficient computational resources.

Quantum Failure

Backend, circuit, or execution problem.

Configuration Failure

Incompatible experiment settings.

This makes troubleshooting significantly easier.

32.13 Reliability Status

Each experiment can receive an operational status:

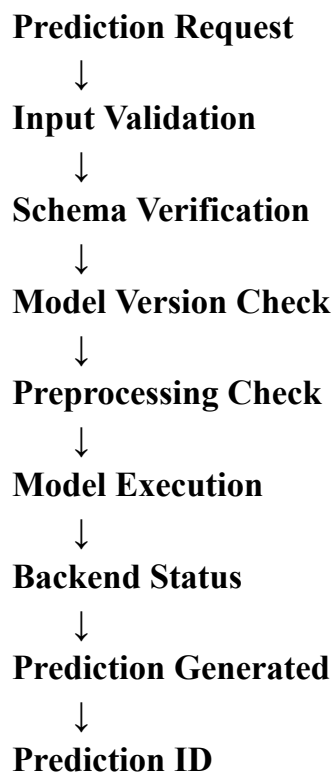
Status	Meaning
Ready	All required components available
Running	Experiment currently executing
Completed	Execution completed successfully
Warning	Completed with a reliability concern

Failed	Execution could not be completed
Review Required	Output requires additional investigation

This is more useful operationally than showing only a model score.

32.14 Reliability Trace

For every prediction, the platform can maintain a trace:



Researchers can use this trace to determine how a particular result was produced.

32.15 Reliability-Centered Dashboard

Instead of displaying only accuracy graphs, the dashboard can show:

- Active model version

- **Backend status**
- **Failed experiments**
- **Warning events**
- **Input validation errors**
- **Model execution history**
- **Fallback events**
- **Prediction trace**
- **System health**

This turns the platform from a simple ML demo into a manageable research system.

32.16 Reliability Principle

The platform follows a simple principle:

“A model should not only produce an output; the system should also be able to verify that the output was generated under valid and traceable conditions.”

Key Innovation

The reliability layer creates a traceable prediction lifecycle:

**Input → Validation → Configuration → Model Version → Execution → Output
→ Audit Trail**

This provides operational confidence without claiming that the system itself is clinically validated.

33. Explainability and Transparency

A biomedical AI system should not behave like a black box. The proposed platform therefore provides an explainability layer that helps researchers understand which input information influenced a prediction and how the model reached its output.

The objective is not to claim that the model discovers medical causality, but to make its computational behaviour inspectable.

33.1 Two Levels of Explanation

The platform provides explanations at two levels:

Global Explanation

Explains the overall behaviour of a trained model.

It identifies:

- **Important features**
- **Frequently influential variables**
- **Feature contribution patterns**
- **Overall model behaviour**

Local Explanation

Explains an individual prediction.

For example:

Prediction: Positive

Important contributing factors:

Feature A → Strong influence

Feature B → Moderate influence

Feature C → Low influence

Feature D → Opposing influence

This allows researchers to inspect individual model decisions.

33.2 Model-Specific Explainability

Different models require different explanation techniques.

Model	Explanation Approach
Logistic Regression	Coefficients
Decision Tree	Decision path
Random Forest	Feature importance / SHAP
XGBoost	SHAP / feature importance
SVM	Suitable post-hoc feature analysis
Quantum Kernel	Feature perturbation / sensitivity
VQC	Input perturbation and circuit analysis
QNN	Sensitivity / attribution approaches

The platform therefore avoids forcing one explanation technique onto every model.

33.3 Feature Contribution Analysis

For a particular prediction, the system can estimate how changes in input features affect the model output.

Conceptually:

Original Input

- **Change Feature A → Prediction changes significantly**
- **Change Feature B → Small change**
- **Change Feature C → No significant change**
- **Change Feature D → Prediction moves in opposite direction**

This identifies features to which the model is sensitive.

33.4 Counterfactual Analysis

The platform can optionally investigate:

“What small change in the input would cause the model's prediction to change?”

For example:

Current prediction



Class A

Controlled feature modification



Model output changes



Class B

Counterfactual analysis is used to understand model sensitivity, not to recommend medical treatment or tell a patient what they should change.

33.5 Quantum Model Transparency

Quantum models can be difficult to interpret because their decision process involves quantum states and parameterized circuits.

The platform can therefore expose important circuit information:

- **Number of qubits**
- **Feature-to-qubit mapping**
- **Circuit depth**
- **Gate structure**
- **Trainable parameters**
- **Measurement configuration**
- **Feature-map configuration**
- **Entanglement configuration**

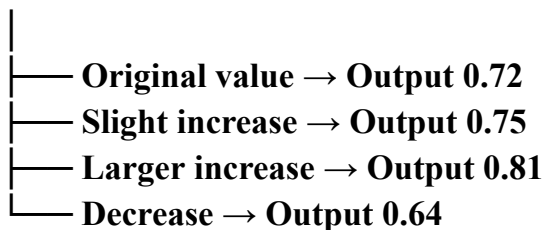
This provides structural transparency even when the internal quantum representation itself is difficult to interpret directly.

33.6 Quantum Sensitivity Analysis

For QML models, the platform can vary one input feature while keeping other conditions fixed.

Example:

Feature X



The resulting response curve provides evidence about how strongly the quantum model reacts to that feature.

33.7 Explanation Consistency

An explanation should not change dramatically for nearly identical inputs without a reason.

The platform can compare explanations for similar samples and identify:

- Stable explanations
- Highly variable explanations
- Features with inconsistent influence

This helps researchers identify potentially unstable model behaviour.

33.8 Explanation vs Clinical Meaning

A critical distinction is maintained:

Model explanation $\neq$ medical explanation

If a model identifies a feature as highly influential, that does not automatically mean:

“This feature causes the disease.”

Instead, the platform reports:

“This feature had a strong influence on the model's prediction under the tested configuration.”

This prevents overinterpretation of AI outputs.

33.9 PCA-Aware Explainability

When dimensionality reduction is used, the model may operate on transformed components instead of the original biomedical features.

For example:

Original Features



PCA



Component 1

Component 2

Component 3



QML Model

Therefore, the platform should distinguish between:

- **Importance of transformed components**
- **Relationship of those components to original features**

This avoids incorrectly presenting a PCA component as if it were a direct clinical variable.

33.10 Explanation Visualization

The dashboard can provide:

- Feature importance charts
- SHAP-style contribution plots
- Local contribution charts
- Sensitivity curves
- Counterfactual comparisons
- Quantum circuit diagrams
- Feature-to-qubit mapping
- Explanation stability views

The objective is to make technical model behaviour understandable without hiding the underlying methodology.

33.11 Model Comparison Through Explanations

The platform can compare explanations from classical and hybrid QML models.

Example:

	Classical	QML
Feature A	High	High
Feature B	Medium	Low
Feature C	Low	High
Feature D	Medium	Medium

This can reveal whether different models are relying on similar or different information.

This is especially useful for investigating why two models with similar predictive performance may behave differently.

33.12 Explanation Audit

Each explanation can be associated with:

- **Model version**
- **Experiment ID**
- **Dataset version**
- **Input sample**
- **Explanation method**
- **Explanation parameters**
- **Timestamp**

This allows researchers to reproduce an explanation later.

33.13 Transparency Report

For every trained model, the platform can generate a compact transparency summary:

MODEL TRANSPARENCY REPORT

Model: Quantum Kernel

Dataset: Version 1.2

Features Used: 12

Qubits: 6

Feature Map: Config-A

Backend: Noisy Simulator

Explanation Method:

Sensitivity Analysis

Model Version:

QK-1.4

Experiment ID:

EXP-028

This gives researchers a clear record of what model configuration produced the reported result.

33.14 Human-Readable Explanation

The system should present technical information in two forms:

Technical View

For researchers and developers:

- Circuit configuration
- Parameters
- Feature contributions
- Backend information
- Experiment metadata

Simplified View

For non-technical users:

- Predicted class
- Main influential features
- Explanation summary
- Important warnings
- Model limitations

This allows the same platform to serve both technical and demonstration purposes.

33.15 Transparency Boundary

The platform explicitly communicates what the explanation does not establish.

It does not prove:

- Causality
- Clinical validity
- Treatment effectiveness
- Biological mechanism
- Individual medical diagnosis

Instead, it provides evidence about the behaviour of the computational model.

33.16 Core Transparency Principle

“Every prediction should be accompanied by enough information to understand which model produced it, what information influenced it, and under which experimental configuration it was generated.”

Key Value

The explainability layer transforms the platform from:

Prediction Only

into:

Prediction → Contribution → Model Behaviour → Technical Evidence → Traceable Explanation

This is particularly valuable for a research-oriented hybrid QML platform because it allows investigators to study not only whether a quantum model works, but also how its behaviour differs from classical models.

34. Security and Privacy

Because the platform processes biomedical information, security and privacy must be considered from the beginning of system design. The proposed platform uses a privacy-aware architecture so that sensitive data is protected during upload, storage, processing, and access.

The goal is to ensure:

“Only authorized users can access the required data, and sensitive biomedical information is never exposed unnecessarily.”

34.1 Privacy-by-Design

Privacy is incorporated into the platform architecture rather than added after development.

The system follows:

- **Minimum required data collection**

- **Controlled data access**
- **Secure storage**
- **Limited data exposure**
- **Traceable user actions**
- **Separation of user and dataset permissions**

For the SIH prototype, public or appropriately de-identified datasets should preferably be used.

34.2 User Authentication

Users must authenticate before accessing protected platform functionality.

Possible authentication methods include:

- **Username/password**
- **Token-based authentication**
- **Institutional authentication**
- **Secure session management**

Unauthenticated users should not be able to access protected datasets, models, or experiments.

34.3 Role-Based Access Control

Different users may require different permissions.

Example:

Role	Access
Administrat or	Users, system configuration, audit logs
Researcher	Datasets, experiments, models, results
Analyst	Results and evaluation

Viewer Read-only dashboard

This prevents unnecessary access to sensitive information.

34.4 Dataset Access Control

Access permissions can be applied at the dataset level.

For example:

Dataset A

- |— Researcher 1 → Read + Experiment**
- |— Researcher 2 → Read**
- |— Viewer → No raw-data access**

Users can therefore work with authorized datasets without automatically receiving access to every dataset stored in the platform.

34.5 Data Encryption

Sensitive information should be protected both:

In Transit

Communication between browser, API, and services should use secure encrypted connections.

At Rest

Stored datasets, credentials, and sensitive metadata should use appropriate storage encryption.

This reduces the risk of unauthorized interception or access.

34.6 Secure API Layer

The backend API is treated as a security boundary.

Security controls include:

- **Authentication checks**
- **Authorization checks**
- **Input validation**
- **Request size limits**
- **Rate limiting where appropriate**
- **Secure error handling**
- **Protection against unauthorized endpoint access**

The API should never assume that a request from the frontend is automatically trustworthy.

34.7 Sensitive Data Minimization

The platform should process only the information required for the experiment.

For example:

Raw Dataset



Identify Required Fields



Remove Unnecessary Information



De-identified Dataset



ML/QML Experiment

This reduces the amount of sensitive information exposed to downstream components.

34.8 De-Identification

When working with sensitive biomedical datasets, directly identifying information should be removed or appropriately transformed before model development whenever possible.

Examples may include:

- **Names**
- **Contact information**
- **Direct identifiers**
- **Unnecessary administrative information**

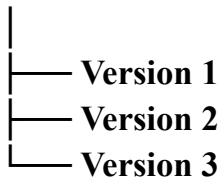
De-identification must be appropriate to the dataset and applicable data-governance requirements.

34.9 Secure Dataset Isolation

Different experiments should not accidentally overwrite or mix datasets.

The system can maintain:

Dataset



Experiment



This provides controlled separation between datasets and experiments.

34.10 Model and Artifact Protection

Security is not limited to raw biomedical data.

The platform should also protect:

- **Trained models**
- **Model parameters**
- **Quantum circuit configurations**
- **Experiment results**
- **Dataset metadata**
- **API credentials**
- **Backend credentials**

Unauthorized modification of these artifacts could affect experimental results.

34.11 Quantum Backend Credential Security

If an external quantum computing service is used, authentication credentials or API keys must never be stored directly in source code.

Instead, sensitive credentials should be managed through:

- **Environment variables**
- **Secure secrets management**
- **Restricted configuration**
- **Access-controlled deployment environments**

The frontend should never receive quantum-provider credentials.

34.12 Audit Logging

Security-relevant activities should be recorded.

Examples:

User Login

Dataset Access

Dataset Upload

Model Creation

Model Download

Experiment Execution

Permission Change

Backend Configuration Change

Audit logs help identify unauthorized or unexpected activity.

34.13 Secure File Upload

Biomedical datasets uploaded to the platform should pass security checks before processing.

The system can validate:

- **File type**
- **File size**
- **File structure**
- **Expected schema**
- **Malformed content**
- **Duplicate uploads**

Files that fail validation should not enter the ML processing environment.

34.14 Database Security

The database should use:

- **Strong authentication**
- **Restricted database permissions**
- **Parameterized queries**
- **Backup mechanisms**
- **Controlled network access**
- **Separation of application and database credentials**

Only required services should have access to specific database resources.

34.15 Backup and Recovery

Important platform data should have controlled backups.

Backup targets may include:

- **Dataset metadata**
- **Experiment configurations**
- **Model artifacts**
- **Evaluation results**
- **Audit records**

Recovery procedures should be tested so that an infrastructure failure does not permanently destroy experimental history.

34.16 Privacy-Aware Quantum Processing

When sensitive biomedical data is involved, the platform should minimize the information sent to external quantum services.

Instead of sending raw biomedical records:

Raw Biomedical Data



Local preprocessing



Feature selection



Dimensionality reduction



Compact representation



Quantum execution

Only the information required for the quantum computation should be passed to the quantum backend, subject to the chosen deployment and privacy requirements.

34.17 Security Incident Handling

The platform should identify and respond to events such as:

- **Unauthorized access**
- **Failed authentication attempts**
- **Invalid API requests**
- **Suspicious file uploads**
- **Unexpected permission changes**
- **Backend credential failures**

The system can log the event, block the relevant operation, and notify the administrator when appropriate.

34.18 Separation of Responsibilities

The architecture separates sensitive responsibilities:

Frontend



API



Authentication / Authorization



Application Services



ML / QML Services



Quantum Backend

The frontend does not directly access the database or quantum backend credentials.

This reduces the attack surface of the platform.

34.19 Security During Demonstration

For the SIH prototype, the demonstration should avoid exposing:

- **Real patient identifiers**
- **Private credentials**

- API keys
- Sensitive clinical records
- Production database information

The demo should use public or appropriately de-identified biomedical data.

34.20 Security and Privacy Objective

The security layer aims to provide:

Confidentiality

Prevent unauthorized access to protected information.

Integrity

Prevent unauthorized modification of datasets, models, and results.

Availability

Keep authorized platform functionality operational.

Traceability

Record important actions and system events.

34.21 Key Security Principle

“Biomedical data should remain protected throughout its lifecycle—from upload and storage to model execution and result generation.”

Key USP

Unlike a simple ML prototype that focuses only on model accuracy, the proposed platform treats data, models, quantum credentials, experiments, and results as protected assets.

This makes the architecture more suitable for future research environments where sensitive biomedical datasets may be involved.

35. Prototype Implementation

The prototype will demonstrate a complete working implementation of the proposed hybrid quantum machine learning platform using a benchmark biomedical dataset.

The objective of the prototype is not to build a clinically deployed diagnostic product. It is to demonstrate that the complete data-to-QML experimentation cycle can be implemented, executed, measured, and compared with classical machine learning.

35.1 Prototype Scope

The SIH prototype will initially focus on:

- One benchmark biomedical dataset
- Binary disease classification
- Classical preprocessing
- Feature selection
- PCA-based dimensionality reduction
- Classical ML baselines
- Quantum Kernel / QSVM
- Variational Quantum Classifier
- Quantum simulator execution
- Explainability
- Performance comparison
- Resource analysis
- Interactive results dashboard

The architecture can later be extended to additional diseases and datasets.

35.2 Prototype Technology

A practical implementation can use:

Component	Technology
-----------	------------

Frontend	React / Next.js
Backend	Python + FastAPI
Data Processing	Pandas + NumPy
Classical ML	Scikit-learn / XGBoost
QML	Qiskit / PennyLane
Quantum Simulator	Qiskit Aer or compatible simulator
Explainability	SHAP + sensitivity analysis
Visualization	Plotly / Matplotlib
Database	PostgreSQL
Containerization	Docker
Testing	Pytest

35.3 Minimum Working Prototype

The first working version should contain the following modules:

Dataset Upload



Data Quality Report



Preprocessing



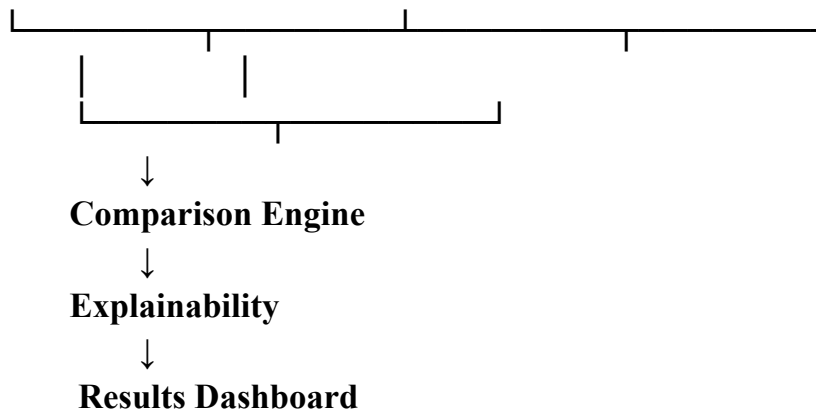
Feature Selection



PCA



Classical ML	QML
SVM	Quantum Kernel
Random Forest	VQC
XGBoost	



This is sufficient to demonstrate the core SIH requirement.

35.4 Dataset Upload Interface

The prototype will provide a dataset upload screen.

The researcher can:

- 1. Upload a dataset.**
- 2. Select the target column.**
- 3. Identify the classification problem.**
- 4. View sample and feature counts.**
- 5. Inspect class distribution.**
- 6. Start the experiment.**

The interface should immediately show whether the dataset is suitable for the selected experiment.

35.5 Automated Dataset Profiling

After upload, the system generates a quick profile containing:

- Number of samples**
- Number of features**
- Numerical features**
- Categorical features**
- Missing values**

- Duplicate records
- Target classes
- Class distribution
- Basic statistical information

This avoids requiring the researcher to manually inspect the dataset before starting an experiment.

35.6 Classical Baseline Implementation

The prototype should implement at least two strong classical models.

Recommended initial models:

- Logistic Regression
- SVM
- Random Forest
- XGBoost

The models use the same prepared dataset used for the QML experiment wherever the benchmarking condition requires a common representation.

This creates a meaningful reference point for QML.

35.7 Quantum Model Implementation

The prototype should initially implement two QML approaches:

Quantum Kernel

The system:

1. Encodes the reduced feature vector.
2. Computes quantum feature-space similarity.
3. Constructs a quantum kernel matrix.
4. Uses a classical SVM for classification.

Variational Quantum Classifier

The system:

1. **Encodes input features.**
2. **Applies a parameterized quantum circuit.**
3. **Measures the circuit.**
4. **Calculates classification loss.**
5. **Uses a classical optimizer.**
6. **Updates circuit parameters.**

Using two different QML approaches makes the prototype more convincing than demonstrating only one quantum model.

35.8 Quantum Simulator Implementation

The first prototype should run primarily on a quantum simulator.

The researcher can configure:

- **Number of qubits**
- **Circuit depth**
- **Number of shots**
- **Feature map**
- **Ansatz**
- **Optimizer**
- **Random seed**
- **Noise model**

The simulator makes the prototype usable even when physical quantum hardware is unavailable.

35.9 Experiment Configuration

Before execution, the researcher can select:

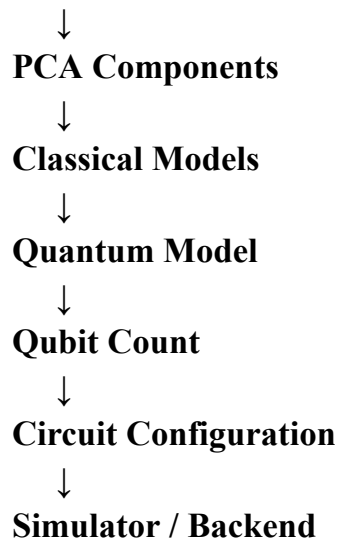
Dataset



Preprocessing Settings



Feature Count



The selected configuration is stored with the experiment.

35.10 Automated Experiment Execution

After the researcher clicks **Run Experiment**, the backend creates an experiment job.

The system records:

- **Experiment ID**
- **Start time**
- **Configuration**
- **Model status**
- **Training status**
- **Completion status**
- **Errors if any**
- **Final results**

This allows long-running QML experiments to execute independently of the dashboard session.

35.11 Results Generation

After training, the platform automatically generates:

- Accuracy
- Precision
- Recall/Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion matrix
- ROC curve
- Prediction results

For QML models, it additionally records quantum-specific information such as:

- Qubit count
- Circuit depth
- Gate information
- Shots
- Backend type

35.12 Classical vs QML Comparison

The prototype should automatically create a comparison table.

Example:

Model	Accuracy	Sensitivity	Specificity	F1	ROC-AUC
Logistic Regression	—	—	—	—	—
SVM	—	—	—	—	—
Random Forest	—	—	—	—	—
Quantum Kernel	—	—	—	—	—
VQC	—	—	—	—	—

The actual values will come from experiments rather than being predetermined.

35.13 Explainability Implementation

The prototype provides explanations for selected predictions.

For classical models:

- Feature importance
- Coefficient-based interpretation
- SHAP where applicable

For QML:

- Feature perturbation
- Sensitivity analysis
- Quantum circuit configuration display

The user can select an individual sample and inspect how the model responds to controlled feature changes.

35.14 Experiment History

The dashboard maintains previous experiments.

Example:

EXP-001

Dataset A

SVM

AUC: ...

EXP-002

Dataset A

Quantum Kernel

AUC: ...

EXP-003

Dataset A

VQC

AUC: ...

Researchers can compare experiments without rerunning completed models.

35.15 Prototype Database

The database can maintain entities such as:

Users

Datasets

Dataset Versions

Experiments

Models

Model Configurations

Metrics

Predictions

Quantum Backends

Audit Events

This supports traceability throughout the prototype.

35.16 API-Based Implementation

The frontend communicates with the backend through APIs.

Example prototype endpoints:

POST /dataset/upload

GET /dataset/{id}/profile

POST /experiment/create

POST /experiment/{id}/run

GET /experiment/{id}/status

GET /experiment/{id}/metrics

GET /experiment/{id}/comparison

POST /prediction

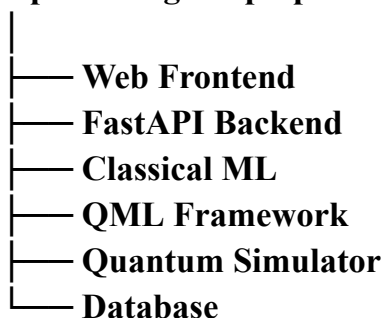
GET /prediction/{id}/explanation

This keeps the UI independent from the computational implementation.

35.17 Prototype Execution Environment

The complete prototype can run on a normal development computer for simulator-based experiments.

Developer / Judge Laptop



Physical quantum hardware is therefore not required to demonstrate the complete core prototype.

35.18 Optional Hardware Demonstration

If access to a compatible quantum backend is available, the same QML experiment can be submitted through the backend abstraction layer.

The demonstration can show:

Simulator Result → Hardware Result

along with:

- Execution status
- Qubit usage
- Circuit information
- Shots
- Noise-related changes
- Performance differences

Hardware execution should be presented as an experimental validation step, not as proof of quantum advantage.

35.19 Prototype Success Criteria

The prototype will be considered successful if it can:

- Accept a biomedical dataset
 - Generate a data-quality profile
 - Prepare the dataset
 - Train classical baselines
 - Train at least one QML model
 - Execute a quantum circuit on a simulator
 - Generate disease-class predictions
 - Calculate evaluation metrics
 - Compare classical and QML results
 - Provide model explanations
 - Record experiment configurations
 - Display results through a dashboard
-

35.20 SIH Demonstration Output

At the end of the demonstration, the judges should be able to see one complete experiment:

Biomedical Dataset



Experiment Created



Classical Baseline



Quantum Model

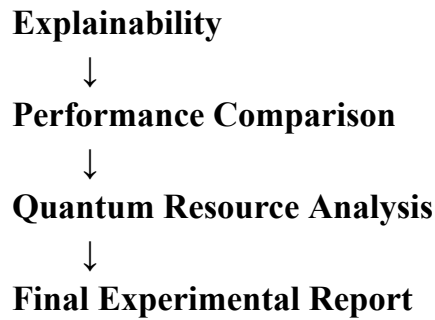


Training Completed



Prediction





The most important demonstration is not simply showing a quantum circuit.

It is showing that the platform can answer:

“For this biomedical dataset and this experimental configuration, how does the hybrid QML approach compare with established classical models?”

35.21 Prototype-to-Production Expansion

The prototype can later be expanded with:

- **Multiple disease datasets**
- **Larger biomedical datasets**
- **Additional QML algorithms**
- **More quantum backends**
- **Real clinical validation studies**
- **Federated/privacy-preserving learning**
- **Advanced uncertainty methods**
- **Automated model selection**
- **Cloud deployment**
- **Multi-institution research support**

These are future extensions and are not required to prove the core SIH concept.

35.22 Prototype Positioning

The SIH prototype should be presented as:

A research and decision-support platform for evaluating hybrid quantum machine learning on biomedical disease-detection problems.

It should not be presented as a clinically approved diagnostic system.

Core Prototype USP

“A working platform where the same biomedical problem can be processed, trained, predicted, explained, and experimentally compared using both classical and quantum-enhanced machine-learning approaches.”

36. Example Prototype Workflow

The prototype can demonstrate the complete platform using one public benchmark biomedical dataset for binary disease classification.

The purpose of this example is to show how a researcher would actually use the platform and obtain an evidence-based comparison between classical ML and hybrid QML.

36.1 Example Experiment

The researcher selects:

- **Biomedical dataset**
- **Binary target variable**
- **Classical baseline models**
- **Quantum Kernel model**
- **VQC model**
- **Number of PCA components**
- **Quantum backend**
- **Evaluation configuration**

The system creates a unique experiment ID.

Experiment ID: EXP-001

Dataset: Biomedical Dataset v1

Task: Binary Classification

36.2 Stage 1 — Dataset Profiling

After upload, the platform automatically displays:

- **Total samples**
- **Total features**
- **Target classes**
- **Missing-value count**
- **Duplicate count**
- **Numerical/categorical feature count**
- **Class distribution**

Example:

Samples : 569

Features : 30

Classes : 2

Missing Values : 0

Duplicate Rows : 0

The actual values depend on the selected dataset.

36.3 Stage 2 — Experiment Configuration

The researcher selects the experimental settings.

Example:

Feature Selection : Enabled

PCA : 8 Components

Classical Models:

✓ **Logistic Regression**

✓ **SVM**

✓ Random Forest

Quantum Models:

✓ Quantum Kernel

✓ VQC

Backend:

✓ Quantum Simulator

The complete configuration is stored under the experiment ID.

36.4 Stage 3 — Dataset Preparation

The selected dataset is converted into the representation required by the experiment.

The platform applies the configured transformations and generates a compact feature representation.

The important point is that the same experiment configuration is recorded and reused consistently for the selected models.

36.5 Stage 4 — Classical Baseline Experiment

The classical models are trained first.

Example:

Logistic Regression



Training



Validation



Test Evaluation

SVM

↓
Training
↓
Validation
↓
Test Evaluation

Random Forest
↓
Training
↓
Validation
↓
Test Evaluation

Their results are stored automatically.

36.6 Stage 5 — Quantum Kernel Experiment

The reduced feature representation is passed to the quantum feature map.

Reduced Features
↓
Quantum Encoding
↓
Quantum Feature Map
↓
Kernel Evaluation
↓
Kernel Matrix
↓
Classical SVM
↓
Prediction

The platform records the quantum configuration used during the experiment.

36.7 Stage 6 — VQC Experiment

The same experimental dataset is used with a variational quantum classifier.

Input Features



Quantum Encoding



Parameterized Circuit



Measurement



Loss Calculation



Classical Optimizer



Parameter Update



Training continues until the configured stopping condition is reached.

36.8 Stage 7 — Prediction

After training, the researcher provides an unseen test sample.

The system validates the input and generates the model output.

Example:

Sample ID: TEST-027

Classical SVM:

Positive

Quantum Kernel:

Positive

VQC:

Negative

When models disagree, the case can be highlighted for further analysis.

36.9 Stage 8 — Explanation

The researcher selects a prediction and opens the explanation panel.

The platform can display:

Prediction: Positive

Most influential features:

Feature A → High influence

Feature B → Moderate influence

Feature C → Moderate influence

Feature D → Low influence

For the QML model, sensitivity analysis can show how controlled changes to input features affect the output.

36.10 Stage 9 — Performance Report

After all models complete execution, the platform generates a unified report.

Example structure:

Model	Accuracy	Sensitivity	Specificity	F1	ROC-AUC
Logistic Regression	Experimental	Experimental	Experimental	Experimental	Experimental
SVM	Experimental	Experimental	Experimental	Experimental	Experimental
Random Forest	Experimental	Experimental	Experimental	Experimental	Experimental

Quantum Kernel	Experimental	Experimental	Experimental	Experimental	Experimental
VQC	Experimental	Experimental	Experimental	Experimental	Experimental

No result is hard-coded. All values are generated from the actual experiment.

36.11 Stage 10 — Quantum Resource Report

For the QML models, the platform additionally reports:

Quantum Resource Report

Qubits Used

:

Experimental

Circuit Depth

:

Experimental

Gate Count

:

Experimental

Shots

:

Configured

Backend

:

Simulator

Execution Time

:

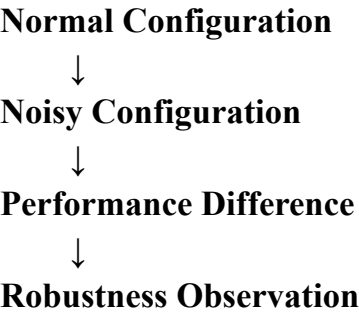
Recorded

This prevents the comparison from focusing only on predictive metrics.

36.12 Stage 11 — Reliability Check

The experiment can then be examined under controlled variations.

For example:



The platform records how the QML result changes when quantum execution conditions are modified.

36.13 Stage 12 — Final Experiment Decision

The platform does not automatically declare quantum superiority.

Instead, it summarizes the evidence.

Possible conclusions include:

Case A — Hybrid QML Better

The tested hybrid model achieves better relevant metrics while remaining within acceptable resource requirements.

Case B — Classical Better

A classical model performs better under the tested conditions.

Case C — Similar Performance

Both approaches provide comparable predictive results.

Case D — Inconclusive

The experiment requires additional datasets, repetitions, or configurations before a meaningful conclusion can be drawn.

This is important because the project is evaluating a research hypothesis rather than assuming the answer beforehand.

36.14 Stage 13 — Experiment Record

The complete experiment is saved as:

EXP-001

|
|— Dataset Version
|— Configuration

- **Classical Results**
- **Quantum Results**
- **Quantum Resources**
- **Explanation**
- **Reliability Information**
- **Final Comparison**

The researcher can reopen the experiment later without losing its configuration.

36.15 What the Judge Sees

A strong SIH demonstration can be completed in the following sequence:

UPLOAD DATASET



SHOW DATA PROFILE



SELECT MODELS



RUN EXPERIMENT



SHOW QUANTUM CIRCUIT



SHOW CLASSICAL RESULTS



SHOW QML RESULTS



COMPARE BOTH



SHOW EXPLANATION



SHOW QUANTUM RESOURCES



SHOW FINAL EVIDENCE

This makes the demonstration concrete and easy to understand.

36.16 Example Final Dashboard

The final screen can combine the experiment into four panels:

1. Prediction

Predicted Class: Positive

2. Model Comparison

Classical vs Quantum-enhanced performance.

3. Explainability

Important features and model sensitivity.

4. Quantum Resources

Qubits, circuit depth, shots, backend and execution information.

This gives the judge a single-screen overview of the experiment.

36.17 Research Insight Generated by the Prototype

The prototype should ultimately answer a measurable research question:

“Under the same biomedical classification problem and defined experimental conditions, does the hybrid quantum model provide a measurable benefit over established classical approaches?”

The result could be positive, negative, or inconclusive.

All three outcomes are scientifically meaningful.

36.18 Prototype Demonstration Principle

The strongest demonstration is not:

“We used quantum computing.”

It is:

“We built a reproducible platform that lets us experimentally determine where hybrid quantum machine learning is useful, where classical ML remains better, and what computational resources are required.”

This makes the prototype a research and benchmarking platform, rather than simply a conventional disease-prediction application with a quantum circuit added to it.

36.19 System Workflow Overview

The end-to-end system workflow consists of the following sequential pipeline stages:

- 1. Biomedical Data Ingestion and Validation**
- 2. Preprocessing and Data Cleaning**
- 3. Feature Engineering and Selection**
- 4. PCA / Dimensionality Reduction**
- 5. Classical Baseline Modeling & Quantum Feature Encoding**
- 6. Hybrid Quantum-Classical Training**
- 7. Disease Prediction Generation**
- 8. Explainability (XAI) Analysis**
- 9. Performance Evaluation and Quantum Resource Tracking**
- 10. Classical vs. QML Benchmarking**
- 11. Dashboard Visualization & Experiment Reporting**

37. Innovation

The innovation of the proposed platform is not simply the use of quantum computing. The main innovation is the complete experimental framework that combines classical machine learning, quantum machine learning, explainability, resource analysis, and fair benchmarking in a single disease-detection platform.

37.1 Hybrid-First Architecture

Instead of replacing classical machine learning with quantum computing, the platform assigns each task to the computing approach that is most practical.

- **Classical computing handles data preparation, feature processing, optimization, and evaluation.**
- **Quantum computing is introduced specifically for the learning components where it can be experimentally investigated.**
- **This makes the system practical with today's quantum limitations.**

37.2 Quantum-Ready Biomedical Data Pipeline

Biomedical datasets are usually too large and high-dimensional to directly map onto current quantum devices.

The platform therefore creates a quantum-ready representation by:

- **identifying useful features**
- **removing redundant information**
- **reducing dimensionality**
- **scaling numerical values**
- **mapping the final representation to available qubits**

This creates a controlled bridge between conventional biomedical data and quantum models.

37.3 Classical vs Quantum Learning Laboratory

The platform acts as an experimental laboratory rather than providing only one trained model.

A researcher can compare:

- **Logistic Regression**
- **SVM**
- **Random Forest**
- **XGBoost**
- **Quantum Kernel/QSVM**
- **Variational Quantum Classifier**
- **Quantum Neural Network**

All models can be tested on comparable experimental conditions.

This allows the platform to answer an important research question:

“When does quantum-enhanced learning actually provide value for biomedical prediction?”

37.4 Resource-Aware Quantum Machine Learning

A major innovation is that model quality is not evaluated using prediction metrics alone.

The platform also records quantum resources such as:

- **number of qubits**
- **circuit depth**
- **gate count**
- **number of circuit executions**
- **shots**
- **backend type**
- **execution time**
- **noise configuration**

Therefore, a model with slightly better accuracy but extremely high quantum cost can be distinguished from a model that provides a better performance-to-resource trade-off.

37.5 Experiment-Based Model Selection

Instead of permanently assuming that one algorithm is the best, the platform treats model selection as an experimental process.

Different combinations can be tested:

Feature Set → Dimensionality → Encoding → Quantum Model → Circuit Configuration → Backend

The system records the results of each experiment and identifies the configuration that provides the best practical trade-off.

37.6 Quantum Hardware Abstraction

The QML layer is separated from the specific quantum backend.

This allows the same experiment to be executed using:

- **ideal quantum simulators**
- **noisy simulators**
- **compatible quantum hardware**

The core application does not need to be redesigned whenever the execution backend changes.

37.7 Noise-Aware Evaluation

Real quantum devices introduce errors that are not present in ideal simulations.

The platform can therefore evaluate the effect of noise on:

- **prediction performance**
- **model stability**
- **circuit behaviour**
- **execution cost**

This makes the research more realistic than evaluating only an ideal quantum circuit.

37.8 Explainable Hybrid Predictions

The platform does not stop at producing a prediction.

It attempts to show:

Input Features → Model Behaviour → Prediction

For classical models, established feature-attribution techniques can be used. For QML models, perturbation, sensitivity, and circuit-level analysis can be used where appropriate.

This makes the experimental results easier for researchers and judges to inspect.

37.9 Reproducible Quantum Experiments

Every experiment can be associated with configuration information such as:

- **dataset version**
- **preprocessing configuration**

- selected features
- dimensionality-reduction settings
- encoding method
- quantum model
- circuit configuration
- optimizer
- random seed
- simulator/backend
- evaluation metrics

This makes experiments repeatable instead of producing isolated results that cannot easily be reconstructed.

37.10 Evidence-Based Quantum Advantage Assessment

The platform does not assume that quantum models will outperform classical models.

Instead, it provides experimental evidence that can lead to three legitimate outcomes:

1. Hybrid QML performs better.
2. Classical ML performs better.
3. Both approaches perform similarly, but have different resource or generalization characteristics.

This is an important scientific innovation because the platform is designed to discover the advantage rather than assume it.

37.11 Multi-Dimensional Evaluation

Model comparison is performed across multiple dimensions:

Predictive Quality + Generalization + Explainability + Computational Cost + Quantum Resource Usage

This provides a more complete assessment than comparing accuracy alone.

37.12 Disease-Independent Design

The platform is not hard-coded for one disease.

The same framework can be adapted to different biomedical datasets involving:

- **cancer**
- **cardiovascular disorders**
- **neurological conditions**
- **genomic disease patterns**
- **structured EHR data**
- **imaging-derived features**

Only the dataset and experimental configuration need to change.

37.13 Research-to-Prototype Bridge

The platform combines research concepts with an executable software system.

Instead of presenting only a theoretical QML model, the prototype demonstrates:

For a detailed step-by-step description of the execution pipeline, see Section 36.19 (System Workflow Overview).

This makes the project suitable for both technical demonstration and experimental research.

37.14 Main Innovation Statement

The core innovation is a resource-aware, explainable, reproducible hybrid quantum-classical experimentation platform that allows biomedical disease-detection models to be trained and objectively compared across classical ML, quantum simulators, and near-term quantum hardware.

Innovation in One Line

“Don’t assume quantum advantage — build a platform that can experimentally prove where quantum machine learning helps, where it does not, and at what computational cost.”

38. Key USP

The Unique Selling Proposition (USP) of the proposed platform is that it does not present Quantum Machine Learning as a standalone algorithm. Instead, it creates a complete, measurable, and reproducible environment for investigating hybrid quantum-classical machine learning for early disease detection.

38.1 Hybrid Intelligence Instead of Quantum-Only Intelligence

The platform combines:

Classical ML + Quantum ML + Explainable AI + Resource Analysis

Rather than forcing the entire problem onto a quantum computer, classical and quantum components work together according to their strengths.

38.2 Same Problem, Multiple Learning Paradigms

A researcher can use the same biomedical problem to evaluate both:

- **Classical machine-learning models**
- **Quantum-enhanced models**
- **Hybrid quantum-classical models**

This creates a direct experimental comparison instead of presenting a quantum model without context.

38.3 Quantum Resource Awareness

Most disease-prediction systems focus mainly on accuracy.

Our platform additionally considers:

- **Qubit requirements**
- **Circuit depth**
- **Gate count**
- **Circuit executions**
- **Shots**
- **Quantum execution time**

- **Noise sensitivity**
- **Backend limitations**

This answers not only “How accurate is the model?” but also “How expensive is the quantum computation required to obtain that result?”

38.4 Hardware-Ready Without Hardware Dependency

The platform can begin development and testing using quantum simulators and later connect to compatible quantum hardware.

This provides two advantages:

- **The prototype remains practical despite limited access to quantum hardware.**
 - **The architecture does not become obsolete when better quantum devices become available.**
-

38.5 Evidence-Based Quantum Advantage

The platform does not hard-code the assumption that QML is superior.

Instead, it allows experimental evidence to determine whether the hybrid approach provides:

- **Better predictive performance**
- **Better sensitivity**
- **Better generalization**
- **Lower or higher computational cost**
- **Different resource requirements**
- **No measurable advantage**

This makes the platform scientifically defensible.

38.6 Explainability Built Into the Experiment

Explainability is treated as part of the platform rather than an afterthought.

The system can connect:

Prediction → Influential Information → Model Behaviour → Experimental Configuration

This helps users understand not only the prediction but also how the model arrived at it.

38.7 Reproducible Experiment Records

Every important experiment can be stored with its configuration.

For example:

Dataset + Features + PCA Settings + Encoding + QML Model + Optimizer + Backend + Seed + Metrics

Another researcher can therefore reproduce the same experiment instead of relying on undocumented settings.

38.8 Resource-Constrained QML Design

Current quantum computers have limited qubits and are affected by noise.

The platform therefore treats quantum resources as a design constraint from the beginning.

Instead of:

Large Dataset → Direct Quantum Model

the approach becomes:

Biomedical Data → Compact Representation → Quantum-Compatible Features → Resource-Aware QML

This makes the concept more realistic for near-term quantum computing.

38.9 Disease-Agnostic Platform

The underlying platform is not restricted to a single disease.

The same architecture can support different biomedical classification problems by changing the dataset and experimental configuration.

This provides a path toward:

One Platform → Multiple Biomedical Applications

38.10 Research and Deployment Thinking in One Platform

The project combines two normally separate perspectives:

Research Perspective

- Compare algorithms
- Study quantum behaviour
- Measure resource requirements
- Investigate noise
- Test generalization

Engineering Perspective

- Data validation
- API services
- Model management
- Experiment tracking
- Security
- Dashboard
- Reproducible execution

This makes the solution more than a theoretical quantum-computing demonstration.

38.11 Judge-Friendly Result Interpretation

The final output is designed to answer the important questions a judge or researcher may ask:

- Which model performed best?
 - Was the quantum model actually better?
 - How much data reduction was required?
 - How many qubits were needed?
 - How deep was the circuit?
 - How did noise affect performance?
 - How explainable is the prediction?
 - Can the experiment be reproduced?
 - Is the improvement worth the quantum cost?
-

38.12 Core USP

“A complete experimental platform that does not merely use Quantum Machine Learning for disease detection, but objectively measures its predictive value, explainability, reproducibility, and quantum resource cost against classical machine learning.”

USP in One Line

“From biomedical data to quantum experimentation — with every prediction measurable, explainable, reproducible, and benchmarked.”

39. Feasibility

The proposed Hybrid Quantum Machine Learning Platform is technically feasible because the system is designed around the capabilities of current classical infrastructure, quantum simulators, and near-term quantum hardware rather than depending on future large-scale quantum computers.

39.1 Technical Feasibility

The required software ecosystem is already available.

The platform can be developed using:

- Python for the ML backend

- **FastAPI for APIs**
- **Pandas and NumPy for biomedical data processing**
- **Scikit-learn and XGBoost for classical ML**
- **Qiskit or PennyLane for QML**
- **Quantum simulators for initial experimentation**
- **React/Next.js for the dashboard**
- **PostgreSQL for experiment and result storage**

Therefore, the project does not require building a quantum computer or developing quantum hardware.

39.2 Simulator-First Development

The prototype can be developed and tested primarily using quantum simulators.

This removes the dependency on:

- **physical quantum hardware availability**
- **hardware queue times**
- **limited qubit access**
- **hardware-specific failures**
- **execution costs**

Once the complete pipeline is stable, selected experiments can be transferred to compatible quantum hardware.

39.3 Manageable Prototype Scope

The SIH prototype does not need to support every disease or every QML algorithm.

A practical first implementation can use:

- **One public biomedical dataset**
- **Binary classification**
- **One or two classical baselines**
- **Quantum Kernel/QSVM**
- **One variational model such as VQC**

- **PCA-based dimensionality reduction**
- **One ideal simulator**
- **One noisy simulation configuration**
- **Complete benchmarking and visualization**

This is sufficient to demonstrate the core concept.

39.4 Computational Feasibility

Biomedical datasets may contain hundreds or thousands of features, while current quantum systems have limited qubit capacity.

The platform addresses this through:

Feature Selection → Dimensionality Reduction → Compact Quantum Representation

For example:

1000+ Raw Features → Selected Features → PCA → 4–12 Quantum-Compatible Features

The exact dimensionality is determined experimentally rather than fixed universally.

This makes the quantum component computationally manageable.

39.5 Algorithmic Feasibility

The selected QML approaches are compatible with near-term quantum computing.

Quantum Kernel / QSVM

Useful for studying quantum-enhanced feature-space representations while keeping the final classification component classical.

Variational Quantum Classifier

Uses a parameterized quantum circuit combined with a classical optimizer.

Quantum Neural Network

Provides an additional experimental architecture for studying trainable quantum representations.

These approaches do not require fault-tolerant quantum computers.

39.6 Development Feasibility

The project can be implemented incrementally.

Phase 1 — Classical Pipeline

Build:

Dataset → Preprocessing → Feature Engineering → Feature Selection → PCA → Classical Models

Phase 2 — Quantum Integration

Add:

Quantum Encoding → Quantum Circuit → QML Training → Prediction

Phase 3 — Hybrid Integration

Connect classical and quantum components into a common experiment pipeline.

Phase 4 — Evaluation

Add:

Benchmarking → Explainability → Resource Analysis → Dashboard

Phase 5 — Hardware Readiness

Connect selected experiments to compatible quantum hardware.

This reduces development risk because the complete project does not depend on the quantum component from day one.

39.7 Data Feasibility

The initial prototype can use publicly available benchmark biomedical datasets.

Suitable datasets can provide:

- patient/sample records
- measurable biomedical features
- disease labels
- sufficient samples for model training
- reproducible experimental conditions

Public benchmark data also makes it easier for judges and researchers to reproduce the experiment.

39.8 Integration Feasibility

The platform can use a modular design.

For example:

Frontend

↓

FastAPI Backend

↓

Experiment Orchestrator

↓

Classical ML / QML Engines

↓

Quantum Backend

Because the quantum backend is abstracted, simulator and hardware execution can be treated as interchangeable execution targets where supported.

39.9 Hardware Feasibility

Near-term quantum hardware can be treated as an optional experimental execution layer, not a mandatory requirement for the core prototype.

This allows the system to operate in three modes:

Mode	Purpose
Ideal Simulator	Validate QML logic
Noisy Simulator	Study realistic quantum errors
Quantum Hardware	Test selected experiments on real devices

This is significantly more practical than designing the entire solution around physical quantum hardware.

39.10 Time Feasibility for SIH

The project can be divided into deliverable-based milestones.

Stage	Major Deliverable
1	Dataset and preprocessing pipeline
2	Classical baseline models
3	Quantum model implementation
4	Hybrid training pipeline
5	Prediction and explainability
6	Benchmarking and resource analysis
7	Dashboard integration
8	Final SIH demonstration

This allows a working prototype to exist even before every advanced feature is completed.

39.11 Team Feasibility

The project can be divided among team members according to technical roles:

- **ML/Data:** preprocessing, feature engineering, classical models
- **Quantum/Research:** encoding, circuits, QML models
- **Backend:** APIs, orchestration, experiment execution
- **Frontend:** dashboard and visualization
- **Testing/Research:** benchmarking, validation, documentation

The modular structure allows parallel development.

39.12 Financial Feasibility

The prototype can primarily use open-source software and public benchmark datasets.

Therefore, the initial development does not require:

- purchasing quantum hardware
- maintaining dedicated quantum infrastructure
- licensing expensive ML frameworks
- collecting large proprietary datasets

Cloud quantum hardware can be used selectively when access is available.

39.13 Scalability Feasibility

The prototype can later be extended without redesigning the entire platform.

Possible scaling directions include:

- additional diseases
- larger datasets
- additional QML algorithms
- additional quantum backends
- multi-class classification

- genomic datasets
- imaging-derived features
- EHR datasets
- distributed experiment execution

The modular architecture supports these extensions.

39.14 Key Feasibility Advantage

The strongest feasibility decision is:

The project does not depend on achieving quantum advantage or owning a quantum computer to demonstrate a working solution.

The complete platform can be developed, tested, benchmarked, and demonstrated using existing classical infrastructure and quantum simulators, while remaining ready for near-term quantum hardware experimentation.

39.15 Feasibility Summary

Data: Public biomedical datasets available

Classical ML: Mature and readily deployable

QML: Available through established frameworks

Quantum Execution: Simulator-first, hardware-ready

Infrastructure: Standard development systems are sufficient

Prototype: Can be developed incrementally

Cost: Low for initial prototype

Scalability: Modular architecture supports future expansion

Final Feasibility Statement

“The proposed system is feasible because it treats quantum computing as an experimentally integrated component rather than a dependency. A complete SIH prototype can run on conventional computing infrastructure with quantum simulation, while the same architecture remains prepared for near-term quantum hardware.”

40. Limitations

Although the proposed platform provides a practical framework for investigating hybrid quantum machine learning in disease detection, several limitations must be acknowledged. These limitations arise from current quantum hardware, biomedical datasets, machine-learning methodology, and the gap between research prototypes and clinical deployment.

40.1 Limited Quantum Hardware

Current quantum computers have a limited number of usable qubits compared with the dimensionality of many biomedical datasets.

This makes it impractical to directly encode large biomedical datasets into quantum circuits.

Mitigation

The platform uses:

Feature Selection → Dimensionality Reduction → Compact Quantum Representation

40.2 Quantum Noise

Near-term quantum processors are affected by noise and operational errors.

Noise can influence:

- measurement results
- circuit behaviour
- training stability
- prediction performance
- reproducibility

Mitigation

The platform supports noisy simulation and records the quantum execution configuration so that the effect of noise can be studied.

40.3 Circuit Depth Constraints

Deep quantum circuits can become difficult to execute reliably on near-term hardware.

Increasing circuit depth may introduce additional errors and execution cost.

Mitigation

The prototype prioritizes:

- **shallow circuits**
 - **resource-aware ansatz design**
 - **limited entanglement where appropriate**
 - **hardware-compatible gates**
 - **experimental circuit-depth comparison**
-

40.4 Quantum Training Can Be Computationally Expensive

Hybrid models may require many repeated quantum-circuit executions during training.

For variational models, each optimization step can involve multiple circuit evaluations.

Therefore, a QML model that achieves good predictive performance may still have a high execution cost.

Mitigation

The platform measures:

Accuracy + Training Time + Circuit Executions + Quantum Resources

rather than considering accuracy alone.

40.5 Quantum Advantage Is Not Guaranteed

The use of a quantum model does not automatically mean that it will outperform classical machine learning.

For some biomedical datasets:

- **classical models may perform better**
- **QML and classical models may perform similarly**
- **quantum execution may have higher computational overhead**

This is an important limitation and is treated as an experimental research question.

Mitigation

The platform performs controlled benchmarking against strong classical baselines.

40.6 Dataset Size and Quality

Biomedical datasets may contain:

- **missing values**
- **noisy measurements**
- **class imbalance**
- **limited samples**
- **inconsistent feature definitions**
- **imperfect labels**

A QML model cannot automatically solve poor-quality data.

Mitigation

The platform includes systematic data validation, preprocessing, quality analysis, and dataset documentation.

40.7 Dataset Generalization

A model that performs well on one benchmark dataset may not perform equally well on another dataset.

Differences in:

- **population**
- **measurement methods**

- feature distributions
- disease prevalence
- data collection procedures

can affect model behaviour.

Mitigation

Future experiments should include external datasets and cross-dataset validation where appropriate.

40.8 Limited Clinical Validation

A software prototype trained on benchmark datasets should not be considered a clinically validated diagnostic system.

Clinical deployment requires substantially more validation, including:

- clinical studies
- independent validation cohorts
- regulatory assessment
- clinical workflow integration
- safety evaluation
- expert review

Therefore, the proposed platform is positioned as a research and decision-support prototype, not an autonomous medical diagnosis system.

40.9 Interpretability of Quantum Models

Explaining a quantum model can be more difficult than explaining many conventional ML models.

Quantum circuits contain:

- parameterized gates
- entanglement
- quantum measurements
- nonlinear optimization behaviour

Therefore, conventional feature-importance methods may not directly explain the complete quantum computation.

Mitigation

The platform uses appropriate approaches such as:

- perturbation analysis
- sensitivity analysis
- ablation experiments
- circuit-level inspection

while clearly distinguishing technical explanations from clinical interpretation.

40.10 Quantum Hardware Accessibility

Real quantum hardware may not always be available during development or demonstration.

Possible constraints include:

- limited hardware access
- queue times
- execution limits
- backend availability
- changing hardware characteristics

Mitigation

The platform remains fully functional using quantum simulators and treats hardware execution as an additional experimental layer.

40.11 Simulator-to-Hardware Gap

A model that performs well on an ideal simulator may behave differently on real quantum hardware.

Differences can result from:

- gate errors
- measurement errors
- connectivity constraints
- noise
- calibration changes

Therefore, simulator results should not automatically be interpreted as hardware performance.

40.12 Hyperparameter Sensitivity

QML models may be sensitive to choices such as:

- learning rate
- optimizer
- circuit depth
- feature map
- initialization
- number of shots
- encoding strategy

Poor configuration can produce unstable or underperforming results.

Mitigation

The platform treats these settings as experimental parameters and records them for reproducibility.

40.13 Classical Preprocessing May Dominate the Pipeline

A large portion of biomedical processing may remain classical.

Feature engineering, dimensionality reduction, data loading, model optimization, and evaluation can require substantially more classical computation than the quantum component.

Therefore, the system should not be described as a purely quantum solution.

This is an inherent characteristic of the hybrid architecture.

40.14 Resource Scaling

As the number of biomedical features, samples, diseases, or experiments increases, computational requirements also increase.

Large-scale experimentation may require:

- parallel processing
- experiment scheduling
- additional storage
- optimized circuit execution
- distributed classical computation

The SIH prototype can therefore focus on a controlled experimental scale before moving toward large-scale deployment.

40.15 Medical Data Privacy Constraints

Real patient data can contain highly sensitive information.

Using such datasets requires appropriate:

- de-identification
- access control
- secure storage
- data governance
- authorization

The prototype can use public or appropriately anonymized benchmark datasets to reduce these constraints during development.

40.16 Threshold and Clinical Decision Limitations

A classification probability or score produced by the model does not automatically represent clinical risk.

Changing the decision threshold can change:

- sensitivity
- specificity
- false positives
- false negatives

Therefore, the optimal threshold depends on the intended application and clinical context.

The platform should present these trade-offs rather than declaring a universally correct threshold.

40.17 Reproducibility Challenges

Quantum hardware characteristics can change over time, and stochastic ML/QML training can produce slightly different results.

Therefore, exact reproduction may be difficult in some hardware experiments.

Mitigation

The platform records:

- random seeds
 - model configuration
 - dataset version
 - preprocessing settings
 - backend
 - circuit parameters
 - shots
 - experiment metadata
-

40.18 Limitation Summary

Area	Limitation	Platform Response
Qubits	Limited quantum resources	Dimensionality reduction
Noise	Hardware errors	Noisy simulation and analysis

Circuit depth	Deep circuits are difficult	Shallow/resource-aware circuits
Training	Repeated quantum execution	Resource tracking
QML performance	Advantage not guaranteed	Fair classical benchmarking
Data	Noise/missingness/imbalance	Data-quality pipeline
Generalization	Dataset dependence	External validation strategy
Explainability	Quantum models are harder to interpret	Sensitivity/perturbation analysis
Hardware	Limited accessibility	Simulator-first architecture
Clinical use	Not clinically validated	Research/decision-support positioning
Privacy	Biomedical data sensitivity	De-identification and access controls
Reproducibility	Stochastic/hardware variation	Experiment metadata and versioning

40.19 Core Limitation Principle

The platform is designed to investigate the practical value of hybrid QML, not to assume that quantum computing will automatically outperform classical machine learning.

Understanding these limitations makes the proposed system more scientifically credible and prevents unrealistic claims about quantum computing or medical diagnosis.

Final Statement

“The current limitations of quantum hardware and biomedical data are treated as experimental constraints within the platform rather than hidden assumptions. This allows the project to produce realistic, reproducible, and scientifically defensible results.”

41. Safety and Medical Responsibility

Because the proposed platform works with biomedical data and disease-related predictions, safety and responsible use are fundamental design requirements. The system is therefore designed as a research and decision-support platform, not as an autonomous medical diagnostic authority.

41.1 Decision-Support, Not Final Diagnosis

The output of the platform should be interpreted as a machine-learning prediction or risk-oriented classification result, not as a confirmed medical diagnosis.

The final medical decision must remain with qualified healthcare professionals.

41.2 Human-in-the-Loop Decision Making

The platform maintains human oversight throughout the prediction process.

The intended decision flow is:

Biomedical Data → Model Prediction → Explanation → Human Review → Medical Decision

The system supports experts rather than replacing them.

41.3 Confidence and Uncertainty Awareness

A prediction should not be presented as absolute certainty.

The interface can distinguish between:

- **prediction score**
- **decision threshold**
- **model confidence where appropriately calibrated**
- **uncertain or ambiguous cases**
- **out-of-distribution warnings where supported**

This prevents users from interpreting every model output as equally reliable.

41.4 False Negative Awareness

In early disease detection, a false negative can be particularly important because a potentially affected patient may be classified as negative.

Therefore, the platform evaluates:

- **sensitivity/recall**
- **false-negative count**
- **confusion matrix**
- **threshold behaviour**

The system should not optimize only for overall accuracy.

41.5 False Positive Awareness

False positives can also create unnecessary concern, testing, or clinical workload.

The platform therefore reports:

- **specificity**
- **false-positive count**
- **precision**
- **precision-recall behaviour**

This makes the trade-off between detecting more potential cases and reducing unnecessary alerts visible.

41.6 Threshold Transparency

The decision threshold should be visible in the experimental configuration.

Changing the threshold can modify:

Sensitivity ↔ Specificity

Therefore, the platform should allow threshold analysis rather than hiding this decision inside the model.

41.7 No Unsupported Medical Claims

The system should never automatically claim:

- that a patient definitely has a disease
- that a patient is definitely disease-free
- that the model replaces a doctor
- that a QML model is clinically superior without evidence
- that a benchmark result represents clinical effectiveness

All claims must remain consistent with the available experimental evidence.

41.8 Dataset-to-Clinical Boundary

A model trained on a benchmark dataset operates within the characteristics of that dataset.

For example, a model may learn patterns associated with:

- a particular population
- a particular hospital
- specific measurement procedures
- specific feature definitions

Those results should not automatically be generalized to every patient population.

41.9 Out-of-Distribution Awareness

A future-ready safety mechanism can identify inputs that differ substantially from the data distribution used during training.

For example:

Training Distribution $\neq$ New Input Distribution

The system can flag such cases for additional review rather than treating the prediction as equally trustworthy.

41.10 Explain Before Trust

Predictions should be accompanied by appropriate technical explanations whenever possible.

The system can provide:

- influential features
- feature contributions
- sensitivity information
- model configuration
- prediction score
- relevant experiment information

This allows users to inspect model behaviour instead of relying blindly on the output.

41.11 Quantum-Specific Responsibility

A quantum model should not receive additional trust simply because it uses quantum computing.

The platform applies the same principle:

Quantum $\neq$ Automatically Better

Quantum predictions must satisfy the same experimental evaluation requirements as classical predictions.

41.12 Human Review of Abnormal Results

Cases with unusual characteristics can be flagged for additional review.

Examples include:

- **incomplete input**
- **unexpected feature values**
- **unfamiliar data distribution**
- **unstable model output**
- **failed quantum execution**
- **inconsistent preprocessing configuration**

The system should avoid silently producing a normal-looking result when the underlying experiment is invalid.

41.13 Safe Failure Behaviour

If a required processing or quantum execution step fails, the platform should report the failure rather than presenting an apparently valid prediction.

Possible states include:

- **Valid Result**
- **Result Requires Review**
- **Execution Failed**
- **Input Invalid**
- **Insufficient Information**

This is safer than hiding technical failures.

41.14 Auditability

Important prediction events should be traceable.

The platform can record:

- **experiment identifier**
- **model version**
- **dataset version**
- **configuration**
- **execution backend**
- **prediction output**

- timestamp
- explanation information
- system status

This supports investigation when an unexpected result occurs.

41.15 Responsible Use of Biomedical Data

The system should follow responsible data-handling principles:

- use appropriately authorized datasets
 - minimize unnecessary personal information
 - use de-identified data where possible
 - restrict access to sensitive datasets
 - protect stored data
 - avoid exposing patient-level information in demonstrations
-

41.16 Bias and Fairness Awareness

Biomedical datasets may contain demographic or population-related biases.

A model can therefore perform differently across different groups.

Where relevant data is available, the platform can evaluate performance across meaningful subgroups using measures such as:

- sensitivity
- specificity
- false-positive rate
- false-negative rate

This helps identify potential disparities in model behaviour.

41.17 Clinical Validation Boundary

Before any real clinical deployment, the system would require substantially more validation.

A future clinical pathway would include:

**Research Prototype → Independent Validation → Clinical Evaluation →
Regulatory/Institutional Review → Controlled Deployment**

The SIH prototype remains within the research/experimental stage.

41.18 Safety-Centered User Interface

The dashboard should clearly distinguish between:

Model Output

What the algorithm predicts.

Experimental Evidence

How well the model performed during evaluation.

Technical Reliability

Whether the computation completed correctly.

Medical Interpretation

What a qualified healthcare professional may conclude.

These categories should not be merged into one simplified “diagnosis” label.

41.19 Responsible QML Communication

The project presentation should use scientifically accurate language.

Instead of saying:

✗ “Quantum AI diagnoses cancer better than doctors.”

Use:

✓ “The platform experimentally evaluates whether hybrid quantum-classical models can improve disease-classification performance compared with classical baselines.”

This distinction is critical for responsible scientific communication.

41.20 Safety Principle

The platform follows four fundamental rules:

1. Prediction is not diagnosis.
2. Quantum is not automatically superior.
3. Uncertainty should not be hidden.
4. Human expertise remains essential.

Safety Architecture

Input Validation



Model Execution



Prediction + Evidence



Explanation + Reliability Status



Human Review



Final Decision

41.21 Final Safety Statement

“The proposed platform is designed to support medical research and decision-making rather than replace clinical expertise. Predictions are presented with appropriate evidence, technical status, and limitations, while final medical interpretation remains under qualified human supervision.”

42. Testing and Validation

Testing and validation ensure that the proposed platform produces technically correct, reproducible, and scientifically meaningful results. Since the system combines classical ML, quantum ML, APIs, data processing, and a user interface, validation is performed at multiple levels.

42.1 Testing Strategy

The platform follows a layered testing strategy:

Component Testing → Integration Testing → Model Validation → Quantum Validation → End-to-End Testing → Experimental Validation

Each layer verifies a different aspect of the system.

42.2 Data Pipeline Testing

The data pipeline is tested to ensure that biomedical datasets are processed correctly.

Tests include:

- file-format validation
- schema validation
- column detection
- data-type detection
- missing-value handling
- duplicate detection
- invalid-value detection
- target-column validation
- class-label validation
- preprocessing output verification

The objective is to ensure that invalid input does not silently enter the ML pipeline.

42.3 Feature Engineering Validation

Feature transformations are tested to verify that:

- **calculations are mathematically correct**
- **expected columns are generated**
- **invalid values are handled**
- **feature dimensions are consistent**
- **transformations are reproducible**

For example, a derived clinical ratio can be independently calculated and compared with the platform's generated value.

42.4 Feature Selection Validation

The feature-selection module is validated by checking:

- **selected feature count**
- **ranking consistency**
- **selection-method correctness**
- **reproducibility across repeated runs**
- **compatibility with downstream models**

The system should also verify that the selected feature set matches the experiment configuration.

42.5 Dimensionality Reduction Validation

For PCA-based processing, testing includes:

- **correct scaling before PCA**
- **expected number of components**
- **transformation dimensions**
- **explained-variance calculation**
- **consistent train/test transformation**

The fitted transformation from training data must be applied consistently to validation and test data.

42.6 Classical Model Testing

Each classical model is independently tested before being used as a benchmark.

Possible models include:

- **Logistic Regression**
- **SVM**
- **Random Forest**
- **XGBoost**
- **Neural Network**

Testing verifies:

- **training completion**
 - **prediction generation**
 - **expected output dimensions**
 - **metric calculation**
 - **model serialization**
 - **model reloading**
 - **inference consistency**
-

42.7 Quantum Circuit Testing

Quantum components require dedicated validation.

The system checks:

- **circuit construction**
- **number of qubits**
- **gate compatibility**
- **parameter handling**
- **measurement configuration**
- **circuit depth**
- **feature-to-qubit mapping**
- **simulator execution**
- **output extraction**

A small controlled circuit can first be tested with known expected behaviour before being integrated into the full QML model.

42.8 Quantum Encoding Validation

The encoding layer is tested independently.

For a given input:

Classical Feature Vector → Encoding Circuit Parameters

The platform verifies that:

- all required features are mapped
 - feature scaling is correct
 - no feature is unintentionally omitted
 - the number of encoded values matches the circuit design
 - repeated input produces the same encoding configuration
-

42.9 Hybrid Training Validation

The hybrid training controller is tested to ensure correct communication between classical and quantum components.

The expected cycle is:

Parameters → Quantum Circuit → Measurement → Loss → Classical Optimizer
→ Updated Parameters

Testing verifies that:

- parameters are updated correctly
 - loss values are returned
 - optimization progresses
 - training terminates correctly
 - invalid quantum executions are handled
 - training history is stored
-

42.10 API Testing

The FastAPI backend is tested independently from the frontend.

Important API tests include:

- **dataset upload**
- **dataset validation**
- **experiment creation**
- **model training**
- **prediction**
- **explanation generation**
- **result retrieval**
- **experiment-history retrieval**

Invalid requests should return meaningful error responses rather than causing silent failures.

42.11 Frontend Testing

The dashboard is tested for:

- **correct page rendering**
- **input validation**
- **experiment configuration**
- **training-status updates**
- **result visualization**
- **error messages**
- **comparison tables**
- **explanation display**
- **responsive behaviour**

The frontend should never display a completed result when the backend reports a failed experiment.

42.12 Integration Testing

Integration testing verifies communication between major modules.

Examples:

Data Integration

Upload → Validation → Preprocessing

ML Integration

Preprocessing → Model Training → Prediction

Quantum Integration

Encoding → Circuit → QML Model → Prediction

Result Integration

Prediction → Evaluation → Dashboard

This confirms that individually working components also work correctly together.

42.13 End-to-End Testing

A complete test experiment is executed from beginning to end:

**Dataset Upload → Validation → Processing → Model Training → Prediction →
Explanation → Evaluation → Result Storage → Dashboard**

The objective is to confirm that the complete platform can execute a real experiment without manual intervention between every stage.

42.14 Reproducibility Testing

The same experiment configuration is executed multiple times.

The system checks whether important outputs remain sufficiently consistent under controlled conditions.

The experiment records:

- **dataset version**
- **preprocessing configuration**
- **selected features**
- **PCA settings**

- **model configuration**
- **random seed**
- **quantum backend**
- **circuit settings**
- **evaluation configuration**

This helps distinguish genuine model behaviour from accidental experimental variation.

42.15 Model Validation

Model validation determines whether the trained models generalize beyond their training samples.

The validation process can include:

- **stratified cross-validation**
- **repeated runs**
- **validation-set evaluation**
- **held-out test-set evaluation**
- **external validation where datasets are available**

The final test set remains isolated until final evaluation.

42.16 Quantum Noise Validation

The quantum component can be evaluated under different execution conditions:

Environment	Purpose
Ideal Simulator	Verify algorithmic behaviour
Noisy Simulator	Study error sensitivity
Real Quantum Hardware	Validate selected experiments under physical conditions

Performance changes across these environments can reveal how sensitive the model is to quantum noise.

42.17 Statistical Validation

A single experimental run should not automatically determine the conclusion.

Where computationally practical, experiments can be repeated using multiple random seeds.

Results can then be reported using:

- mean performance
- standard deviation
- confidence intervals where appropriate
- variation across runs

This provides a more reliable picture of model behaviour.

42.18 Failure Testing

The platform is deliberately tested with invalid and unexpected conditions.

Examples:

- corrupted dataset
- missing target column
- unsupported data type
- empty file
- insufficient features
- invalid model configuration
- unavailable quantum backend
- quantum execution failure
- incomplete experiment

The expected behaviour is a clear failure status rather than an incorrect result.

42.19 Performance Validation

System-level performance can be tested by measuring:

- dataset processing time
- training time
- inference time
- API response time
- memory usage
- quantum execution overhead
- experiment completion time

This evaluates whether the prototype is practical to operate.

42.20 Benchmark Validation

The final benchmark validates whether the comparison between classical and hybrid QML models is fair.

The same experimental framework should be used wherever applicable for:

- data splits
- preprocessing
- evaluation metrics
- validation methodology
- reporting format

The benchmark should report both predictive results and computational/quantum resource requirements.

42.21 Acceptance Criteria

The SIH prototype can define measurable acceptance criteria.

Component	Validation Requirement
Dataset Module	Valid datasets successfully processed
Preprocessing	Correct transformed output
Classical ML	Models train and predict successfully

QML	Quantum circuits execute successfully
Hybrid Training	Optimization completes correctly
Prediction	Valid output generated
XAI	Explanation generated where supported
Evaluation	Metrics calculated correctly
Benchmarking	Classical/QML comparison generated
Dashboard	Results displayed correctly
Experiment Tracking	Configuration and results stored
Error Handling	Invalid experiments fail safely

42.22 Validation Pyramid

The testing strategy can be represented as:

Unit Tests

↓

Module Tests

↓

Integration Tests

↓

Model Validation

↓

Quantum Validation

↓

End-to-End Testing

↓

Experimental Validation

Each higher level depends on the correctness of the lower levels.

42.23 Final Validation Goal

The goal is not simply to demonstrate that the software runs.

The platform must establish three different forms of correctness:

Software Correctness

Does the system perform the requested operations correctly?

Model Correctness

Are training, prediction, and evaluation implemented correctly?

Experimental Validity

Are the resulting classical-vs-QML conclusions supported by a fair and reproducible experiment?

42.24 Core Testing Principle

“A successful demo is not enough; every major component, model, quantum circuit, integration point, and experimental conclusion must be validated.”

Final Statement

“The testing framework validates the platform from individual software components to complete biomedical experiments, ensuring that reported QML results are technically correct, reproducible, and supported by controlled evaluation.”

43. Expected Results

The proposed platform is expected to produce a complete set of technical and experimental results for evaluating hybrid quantum-classical machine learning in early disease detection.

The results will be experimentally measured rather than assumed, particularly for any comparison between classical and quantum-enhanced models.

43.1 Functional Platform

The primary expected result is a working software platform capable of performing the complete experimental process:

The primary expected result is a working software platform capable of performing the complete experimental pipeline as described in Section 36.19 (System Workflow Overview).

The prototype should allow a user to configure and execute experiments through a single interface.

43.2 Successful Biomedical Data Processing

The system is expected to successfully:

- **accept supported biomedical datasets**
- **validate dataset structure**
- **identify feature and target columns**
- **detect missing and invalid values**
- **process numerical and categorical features**
- **generate dataset-quality information**
- **prepare data for ML/QML experiments**

The result will be a standardized dataset representation suitable for controlled experimentation.

43.3 Compact Quantum-Compatible Representation

The platform is expected to transform potentially high-dimensional biomedical data into a compact representation suitable for current quantum experiments.

For example:

High-Dimensional Biomedical Data

↓

Feature Selection

↓

PCA / Dimensionality Reduction

↓

Compact Feature Vector



Quantum Encoding

The exact number of features and components will be determined experimentally.

43.4 Classical Baseline Results

The platform is expected to generate baseline results from established ML algorithms such as:

- **Logistic Regression**
- **SVM**
- **Random Forest**
- **XGBoost**
- **Neural Network**

These models establish a reference against which QML approaches can be evaluated.

43.5 Quantum Machine Learning Results

The QML experiments are expected to produce results from models such as:

- **Quantum Kernel / QSVM**
- **Variational Quantum Classifier**
- **Quantum Neural Network**

For each model, the system will record predictive metrics and relevant quantum configuration information.

43.6 Hybrid Model Performance

The platform will measure the performance of the hybrid quantum-classical models using metrics such as:

- **Accuracy**

- Precision
- Recall / Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion Matrix
- ROC curve
- Precision-Recall curve

The objective is to determine how the hybrid models behave on unseen test data.

43.7 Classical vs QML Comparison

A major expected result is a direct comparison between classical and hybrid QML approaches.

A result table can contain:

Metric	Classical Best Model	Quantum / Hybrid Model
Accuracy	Experimentally measured	Experimentally measured
Sensitivity	Experimentally measured	Experimentally measured
Specificity	Experimentally measured	Experimentally measured
F1-score	Experimentally measured	Experimentally measured
ROC-AUC	Experimentally measured	Experimentally measured
Training Time	Measured	Measured
Inference Time	Measured	Measured

No performance value should be inserted before the experiment is actually conducted.

43.8 Quantum Resource Results

The platform is expected to report the resources required by each quantum experiment.

Important outputs include:

- number of qubits
- circuit depth
- gate count
- number of shots
- circuit executions
- backend
- quantum execution time
- noise configuration

This provides a practical understanding of the cost of the quantum component.

43.9 Simulator vs Noisy Execution Results

The same QML experiment can be evaluated under different conditions.

Ideal Simulation

Measures algorithmic behaviour without simulated hardware noise.

Noisy Simulation

Measures sensitivity to realistic quantum errors.

Hardware Execution

Where available, selected experiments can be executed on compatible quantum hardware.

The comparison can reveal how quantum noise affects model performance.

43.10 Explainability Results

The platform is expected to generate interpretable information about model behaviour.

For suitable classical models, this may include:

- important features
- feature contributions
- global feature importance
- individual prediction explanations

For QML models, appropriate methods such as perturbation or sensitivity analysis can be used.

The output should explain model behaviour, without claiming that a feature is medically causal.

43.11 Generalization Results

The project is expected to evaluate whether model performance remains stable beyond the training data.

Possible outputs include:

- validation performance
- test performance
- cross-validation results
- repeated-run variation
- performance across different random seeds
- external-dataset performance where available

This helps identify whether an apparent improvement is robust or dependent on a particular dataset split.

43.12 Model Stability Results

Repeated experiments can reveal how stable the models are.

The platform can report:

Mean Performance $\pm$ Variation

across multiple controlled runs.

This is particularly useful for QML experiments because optimization and quantum measurement can introduce variability.

43.13 Feature-Dimension Experiment Results

The platform can experimentally investigate how model performance changes as the quantum-compatible feature space changes.

For example:

4 Features $\rightarrow$ 6 Features $\rightarrow$ 8 Features $\rightarrow$ 12 Features

For each configuration, the system can record:

- **predictive performance**
- **circuit complexity**
- **training cost**
- **execution cost**

This can reveal the trade-off between information retained and quantum resource requirements.

43.14 Model Selection Result

The final selected model should not necessarily be the model with the highest accuracy.

The platform can select a model based on a combination of:

Predictive Quality + Generalization + Resource Cost + Stability + Practicality

This produces a more meaningful final experimental configuration.

43.15 Complete Experiment Report

Each experiment is expected to produce a structured report containing:

Dataset

- dataset version
- sample count
- feature count
- class distribution

Processing

- preprocessing configuration
- selected features
- dimensionality-reduction settings

Model

- algorithm
- hyperparameters
- quantum circuit configuration

Execution

- simulator/backend
- qubits
- circuit depth
- shots
- execution information

Results

- predictive metrics
 - confusion matrix
 - generalization results
 - resource measurements
 - explainability information
-

43.16 Possible Experimental Outcomes

The project intentionally allows multiple scientifically valid outcomes.

Outcome A — Hybrid Advantage

The hybrid QML model may outperform selected classical baselines under the tested conditions.

Outcome B — Classical Advantage

Classical ML may provide better performance or efficiency.

This is also a valuable result.

Outcome C — Comparable Performance

Classical and hybrid models may achieve similar predictive performance, while differing in computational or quantum resource requirements.

Outcome D — Quantum Resource Limitation

The QML model may show promising behaviour but become impractical because of circuit depth, execution cost, or noise.

All four outcomes provide useful research information.

43.17 Expected SIH Demonstration Output

During the final demonstration, judges should be able to see:

The demonstration walks through the sequential end-to-end pipeline detailed in Section 36.19 (System Workflow Overview).

43.18 Expected Research Insight

The most important expected result is not simply a high accuracy number.

The platform should help answer:

“Under what biomedical-data and resource conditions can hybrid quantum machine learning provide meaningful value compared with classical machine learning?”

This transforms the project from a simple QML demonstration into an experimental research platform.

43.19 Expected Result Summary

Expected Output	Purpose
Working Platform	Demonstrate complete system
Processed Dataset	Create ML/QML-ready representation
Classical Baselines	Establish reference performance
QML Models	Evaluate quantum-enhanced learning
Predictions	Demonstrate disease classification
XAI Results	Understand model behaviour
Benchmark	Compare approaches fairly
Resource Report	Measure quantum cost
Noise Analysis	Study realistic quantum behaviour
Generalization Analysis	Evaluate robustness
Experiment Report	Preserve reproducibility
Final Model	Select best practical configuration

43.20 Core Expected-Result Principle

“The project will not promise a predetermined quantum advantage; it will produce measurable evidence showing the predictive performance, resource requirements, stability, and practical value of hybrid QML compared with classical machine learning.”

Final Expected Result

A working, reproducible platform that can experimentally determine whether hybrid quantum-classical machine learning provides a meaningful and practical contribution to early disease detection for the selected biomedical dataset.

44. Expected Impact

The proposed Hybrid Quantum Machine Learning Platform has potential impact across AI research, biomedical machine learning, quantum computing, and future healthcare decision-support systems.

The impact is not limited to achieving a particular accuracy value. The platform is intended to create a practical foundation for investigating how emerging quantum technologies can contribute to biomedical AI.

44.1 Impact on Early Disease Detection Research

The platform provides a structured environment for investigating machine-learning approaches for early disease classification.

It can help researchers study:

- **complex biomedical patterns**
- **high-dimensional feature spaces**
- **different ML architectures**
- **quantum-enhanced representations**
- **model generalization**
- **computational trade-offs**

This can support future research into AI-assisted early disease detection.

44.2 Democratizing Quantum Machine Learning for Biomedical Research

Quantum ML experimentation can otherwise require specialized quantum-computing knowledge and infrastructure.

The proposed platform brings multiple stages into one environment:

The proposed platform integrates all pipeline stages (as detailed in Section 36.19 System Workflow Overview) into a single unified environment.

This can make QML experimentation more accessible to:

- **students**
 - **researchers**
 - **universities**
 - **AI developers**
 - **biomedical researchers**
-

44.3 Bridging Classical AI and Quantum Computing

One of the major impacts is creating a practical bridge between two technological areas.

Classical AI

Provides mature tools for:

- **data processing**
- **feature engineering**
- **optimization**
- **prediction**
- **evaluation**

Quantum Computing

Provides an emerging computational paradigm for:

- **quantum feature representations**
- **quantum kernels**
- **variational circuits**
- **quantum-enhanced learning**

The platform allows these technologies to be investigated together rather than as isolated fields.

44.4 Better Understanding of Quantum Advantage

The project can contribute to a more evidence-based understanding of QML.

Instead of asking:

“Can quantum computing replace classical AI?”

the platform encourages a more useful question:

“For which biomedical learning problems, under which conditions, and at what resource cost can quantum methods provide measurable value?”

This can generate useful experimental evidence even when classical ML performs better.

44.5 Resource-Aware Quantum Research

The platform makes quantum-resource requirements visible alongside predictive performance.

This can help researchers understand trade-offs such as:

Performance ↔ Qubits ↔ Circuit Depth ↔ Execution Cost ↔ Noise

Such information becomes increasingly important as quantum hardware evolves.

44.6 Support for Multiple Biomedical Domains

The platform can potentially be adapted to different biomedical applications, including:

- **cancer research**
- **cardiovascular disease**
- **neurological disorders**
- **genomics**
- **EHR-based prediction**

- **imaging-derived feature classification**

The underlying experimental infrastructure remains reusable.

44.7 Improved Research Reproducibility

By storing experiment configurations and results, the platform can make QML experiments easier to reproduce.

Researchers can preserve:

- **dataset versions**
- **preprocessing settings**
- **model configurations**
- **quantum circuit parameters**
- **backend information**
- **random seeds**
- **evaluation results**

This reduces dependence on undocumented experimental settings.

44.8 Educational Impact

The platform can serve as an educational environment for students learning:

- **machine learning**
- **quantum computing**
- **quantum machine learning**
- **biomedical AI**
- **explainable AI**
- **experimental benchmarking**

Students can directly observe the differences between classical and quantum-enhanced learning approaches.

44.9 Research Collaboration

A standardized platform can make it easier for different teams to compare experiments.

For example, researchers could use the same:

- **dataset representation**
- **evaluation metrics**
- **experiment structure**
- **resource measurements**
- **reporting format**

This can support more systematic comparison of QML approaches.

44.10 Future Healthcare Technology Impact

If future research establishes meaningful advantages for specific biomedical applications, the platform could provide an experimental foundation for developing more advanced AI-assisted healthcare systems.

Potential future directions include:

Research Prototype → Validated Model → Clinical Research → Controlled Decision Support

However, clinical impact cannot be claimed until appropriate clinical validation is completed.

44.11 Impact on AI Model Development

The platform encourages a shift from:

“Choose one model and report accuracy.”

toward:

“Compare multiple approaches, measure resources, understand behaviour, and select the most practical model.”

This methodology can improve the quality of experimental AI development beyond the quantum component itself.

44.12 Impact on Responsible AI

The project incorporates responsible principles such as:

- **human oversight**
- **explainability**
- **uncertainty awareness**
- **reproducibility**
- **privacy protection**
- **bias awareness**
- **transparent benchmarking**
- **explicit limitations**

This reduces the risk of presenting an experimental AI model as an unquestionable medical authority.

44.13 Impact on India's Emerging Quantum Ecosystem

The project can contribute to building practical skills at the intersection of:

Quantum Computing + AI + Healthcare

It can help students and researchers gain hands-on experience with:

- **quantum programming**
- **QML algorithms**
- **quantum simulators**
- **hybrid architectures**
- **quantum resource analysis**

This is particularly valuable as quantum computing research and infrastructure continue to develop.

44.14 Impact on Innovation

The platform demonstrates that quantum computing can be explored through practical application-oriented problems rather than only theoretical examples.

The combination of:

Biomedical AI + QML + Explainability + Benchmarking + Resource Analysis
creates a multidisciplinary research environment.

44.15 Long-Term Impact

With further research and validation, the platform could evolve into a broader experimentation environment supporting:

- multiple diseases
- larger biomedical datasets
- additional QML algorithms
- multiple quantum hardware providers
- advanced genomic analysis
- medical imaging pipelines
- federated or privacy-preserving learning
- multi-modal biomedical data

These are future possibilities rather than capabilities claimed for the initial SIH prototype.

44.16 Impact Summary

Impact Area	Expected Contribution
Biomedical AI	Experimental platform for disease classification
QML Research	Practical testing environment
Quantum Computing	Resource-aware near-term experimentation
Classical ML	Strong reference baselines
Explainability	More transparent model analysis

Reproducibility	Structured experiment records
Education	Hands-on AI + quantum learning
Research	Comparable experimental framework
Healthcare	Potential future decision-support foundation
Innovation	Bridge between quantum computing and biomedical AI

44.17 Core Impact Statement

“The primary impact of the project is to create a practical bridge between biomedical AI and emerging quantum computing, enabling researchers to measure—not assume—the real value of hybrid quantum machine learning.”

Final Impact Statement

“Rather than promising immediate clinical transformation, the platform creates a scientifically rigorous foundation from which future quantum-enhanced biomedical applications can be evaluated, improved, and eventually validated.”

45. Future Scope

The proposed SIH prototype establishes the foundation for experimenting with hybrid quantum-classical machine learning for biomedical disease detection. Once the core platform is validated, it can be extended in several directions.

45.1 Multi-Disease Prediction

The initial prototype can focus on a single binary classification problem.

Future versions can support multiple diseases within one platform, such as:

- **Cancer**
- **Cardiovascular disorders**
- **Neurological disorders**
- **Metabolic diseases**

The system could provide separate disease-specific models while maintaining a common experimental infrastructure.

45.2 Multi-Class Disease Classification

The platform can be extended beyond binary classification.

Instead of:

Disease / No Disease

future models could support:

Disease A / Disease B / Disease C / Healthy

This would require appropriate datasets, model design, and multi-class evaluation.

45.3 Multi-Modal Biomedical Data

Future versions could combine multiple types of biomedical information.

For example:

Clinical Data + Genomic Data + Imaging-Derived Features + EHR Data

A multi-modal architecture could investigate whether combining different information sources improves prediction.

45.4 Advanced Genomic Applications

Genomic datasets can contain extremely high-dimensional information.

Future research could investigate quantum-enhanced learning for:

- genomic feature classification
- disease-associated patterns
- biomarker research

- **molecular classification**
- **patient subgroup identification**

Dimensionality reduction and resource-aware encoding would become especially important in this scenario.

45.5 Medical Imaging Integration

The platform could eventually accept imaging-derived representations from modalities such as:

- **retinal images**
- **X-rays**
- **MRI**
- **CT**
- **histopathology images**

Rather than directly placing large images into quantum circuits, classical deep-learning models could first extract compact representations, which could then be investigated using quantum-enhanced models.

45.6 Advanced Quantum Algorithms

The initial implementation can focus on practical QML approaches.

Future versions could investigate:

- **advanced quantum kernels**
- **improved variational circuits**
- **quantum convolutional approaches**
- **quantum generative models**
- **quantum ensemble methods**
- **problem-specific quantum feature maps**

Each new algorithm should still be evaluated against strong classical alternatives.

45.7 Automated Quantum Model Search

Future versions could introduce automated experimentation for selecting:

Feature Representation + Feature Count + Encoding + Circuit + Optimizer + Backend

The platform could search across configurations and identify promising candidates based on multiple objectives.

45.8 Quantum Resource Optimization

Future research could develop optimization methods that explicitly minimize quantum resources.

For example:

Maximize Predictive Performance
while simultaneously
Minimizing Qubits + Circuit Depth + Executions

This could lead to more practical QML models for near-term devices.

45.9 Improved Noise Mitigation

As quantum hardware develops, the platform could incorporate advanced techniques for dealing with noise.

Possible research directions include:

- error mitigation
- noise-aware circuit design
- measurement-error mitigation
- hardware-aware optimization
- robustness analysis

These methods could help determine whether improvements remain meaningful under realistic quantum conditions.

45.10 Broader Quantum Hardware Support

The platform can eventually support multiple quantum execution providers and hardware architectures.

A standardized backend interface could allow experiments to move between:

Simulator A → Simulator B → Quantum Hardware A → Quantum Hardware B
without changing the complete application architecture.

45.11 Federated and Privacy-Preserving Learning

Future healthcare applications may involve data distributed across hospitals or institutions.

A possible extension is:

Hospital A Data + Hospital B Data + Hospital C Data

without directly centralizing sensitive patient records.

Federated or privacy-preserving approaches could then be investigated together with hybrid QML.

This would require substantial additional research and security validation.

45.12 Real-World Clinical Validation

A major long-term direction is moving from benchmark experimentation toward controlled clinical research.

The progression could be:

Public Dataset

↓

Independent Dataset

↓

Multi-Center Validation



Prospective Clinical Evaluation



Regulatory and Institutional Review



Controlled Clinical Deployment

The SIH prototype does not claim to have reached these stages.

45.13 Personalized Risk Modeling

Future systems could investigate patient-specific risk estimation rather than simple disease classification.

Potential inputs could include:

- patient history
- longitudinal measurements
- genomic information
- lifestyle-related variables
- previous clinical observations

Such systems would require careful calibration and clinical validation.

45.14 Continuous Learning

Future versions could investigate updating models as new validated biomedical data becomes available.

A controlled continuous-learning system could:

- monitor new datasets
- evaluate distribution changes
- retrain candidate models
- compare new and previous versions
- require validation before deployment

This would prevent automatic uncontrolled model updates.

45.15 Federated Quantum Machine Learning Research

An advanced research direction could investigate whether quantum-enhanced learning can be incorporated into distributed learning environments.

Potential architecture:

Multiple Data Sources → Local Processing → Hybrid/QML Learning → Aggregation

This could combine research in:

- quantum computing
- distributed ML
- privacy-preserving AI
- healthcare analytics

45.16 Advanced Explainability for QML

Quantum-model explainability remains an active research area.

Future work could investigate more sophisticated methods for understanding:

- quantum feature maps
- circuit parameters
- entanglement effects
- feature interactions
- quantum measurement behaviour

The goal would be to make QML models increasingly transparent without overstating what the explanations mean clinically.

45.17 Automated Experiment Discovery

The platform could eventually become an autonomous research assistant for QML experiments.

A future system could:

- 1. Analyze the dataset.**
- 2. Recommend suitable feature representations.**
- 3. Generate candidate classical baselines.**
- 4. Select quantum-compatible dimensions.**
- 5. Test multiple QML configurations.**
- 6. Compare results.**
- 7. Identify promising experiments.**
- 8. Generate an experimental report.**

Human researchers would still review and validate the conclusions.

45.18 Large-Scale Experiment Management

As the number of experiments grows, the platform could support:

- experiment queues**
- parallel execution**
- distributed training**
- automated result comparison**
- experiment versioning**
- searchable research history**

This would transform the prototype into a larger QML experimentation environment.

45.19 Real-Time Biomedical Decision Support

A distant future direction could involve integrating validated models into healthcare workflows.

Potential applications could include:

- screening assistance**
- clinical prioritization**
- research cohort identification**
- decision-support alerts**

Such applications would require extensive clinical validation, regulatory review, safety testing, and human oversight before real-world use.

45.20 Future Scope Roadmap

Phase 1 — SIH Prototype

One Dataset → Classical + QML → Simulator → Benchmark

Phase 2 — Research Expansion

Multiple Datasets → Multiple QML Models → Noise Analysis → Hardware Experiments

Phase 3 — Advanced Platform

Multi-Disease → Multi-Modal Data → Automated Experiment Search → Multi-Backend Support

Phase 4 — Clinical Research

Independent Validation → Multi-Center Studies → Clinical Evaluation

Phase 5 — Long-Term Vision

Validated AI Decision-Support Platform

45.21 Future Vision

The long-term vision is to evolve the project from a single SIH prototype into a general-purpose experimental infrastructure for quantum-enhanced biomedical AI.

The progression can be represented as:

Biomedical Dataset

→ **Hybrid ML/QML Research**

→ **Resource-Aware Quantum Experimentation**

- **Multi-Dataset Validation**
- **Real Quantum Hardware Evaluation**
- **Clinical Research**
- **Potential Validated Decision Support**

45.22 Core Future-Scope Principle

“Future development should expand the scientific and practical capabilities of the platform only after each new capability is experimentally validated.”

Final Future Vision

“The SIH prototype is the starting point for a broader research platform where emerging quantum technologies can be systematically evaluated across biomedical problems, quantum hardware generations, and increasingly realistic validation environments.”

46. Expected Deliverables

The project will deliver a functional hybrid quantum-classical machine learning platform along with the software modules, experimental results, documentation, and demonstration materials required to validate the proposed solution.

46.1 Complete Hybrid QML Platform

A working web-based platform that integrates the complete experimental pipeline.

Core flow:

A working web-based platform that integrates the complete experimental pipeline (see Section 36.19 System Workflow Overview).

46.2 Biomedical Data Processing Module

The platform will provide:

- **Dataset upload**
 - **Dataset validation**
 - **Schema detection**
 - **Data-quality analysis**
 - **Missing-value analysis**
 - **Duplicate detection**
 - **Feature/target identification**
 - **Class-distribution analysis**
 - **Dataset metadata**
-

46.3 Preprocessing Module

The deliverable will include configurable preprocessing operations such as:

- **Missing-value handling**
 - **Categorical encoding**
 - **Numerical scaling**
 - **Outlier analysis**
 - **Target preparation**
 - **Train/validation/test splitting**
 - **Class-imbalance handling**
-

46.4 Feature Engineering Module

The platform will support meaningful transformations of biomedical features.

Expected capabilities include:

- **Derived features**
 - **Statistical transformations**
 - **Ratio/difference features where appropriate**
 - **Interaction features**
 - **Domain-aware transformations**
 - **Feature-transformation tracking**
-

46.5 Feature Selection Module

The system will provide configurable feature-selection methods such as:

- Correlation-based selection
- Mutual information
- Statistical selection
- Recursive Feature Elimination
- Tree-based feature importance

The selected feature set will be recorded as part of the experiment configuration.

46.6 Dimensionality Reduction Module

The prototype will provide dimensionality reduction, primarily using PCA.

Expected outputs include:

- Number of components
 - Explained variance
 - Reduced feature representation
 - Transformation configuration
 - Quantum-compatible feature representation
-

46.7 Classical ML Benchmark Module

The platform will implement classical reference models such as:

- Logistic Regression
- SVM
- Random Forest
- XGBoost

These models provide the baseline against which hybrid QML models are evaluated.

46.8 Quantum Machine Learning Module

The prototype will include selected QML algorithms, such as:

- **Quantum Kernel / QSVM**
- **Variational Quantum Classifier**
- **Quantum Neural Network where practical**

The final number of implemented models can be adjusted according to prototype development time and computational resources.

46.9 Quantum Feature Encoding Module

The platform will convert compact classical feature vectors into quantum representations.

The implementation can include:

- **Angle encoding**
 - **Feature-to-qubit mapping**
 - **Parameterized feature maps**
 - **Entanglement configuration**
 - **Encoding validation**
-

46.10 Hybrid Training Engine

The platform will provide the classical-quantum training loop:

Classical Features → Quantum Circuit → Measurement → Loss → Classical Optimization → Parameter Update

The engine will support configurable training parameters and experiment tracking.

46.11 Quantum Simulator Integration

The prototype will support quantum simulation for:

- **QML circuit execution**
- **Ideal simulation**
- **Noisy simulation**

- **Circuit inspection**
- **Resource measurement**
- **Reproducible experimentation**

This forms the primary execution environment for the SIH prototype.

46.12 Quantum Hardware Compatibility Layer

The software architecture will include a backend abstraction that can support compatible quantum hardware where access is available.

The deliverable will demonstrate that the QML component is not permanently tied to one simulator implementation.

46.13 Disease Prediction Module

The platform will generate predictions for unseen samples using the selected trained model.

Expected output can include:

- **Predicted class**
- **Prediction score where supported**
- **Decision threshold**
- **Model version**
- **Experiment identifier**
- **Execution status**

The output will be clearly presented as an experimental ML prediction rather than a confirmed medical diagnosis.

46.14 Explainability Module

The platform will provide model-behaviour explanations using suitable methods.

Expected outputs include:

- **Feature importance**
 - **Feature contribution**
 - **Local explanation**
 - **Global explanation**
 - **Perturbation/sensitivity analysis for appropriate QML models**
 - **Explanation visualization**
-

46.15 Performance Evaluation Module

The system will calculate and visualize:

- **Accuracy**
- **Precision**
- **Recall/Sensitivity**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Confusion Matrix**
- **ROC curve**
- **Precision-Recall curve**

Additional computational measurements will also be recorded.

46.16 Quantum Resource Analysis Module

The platform will report relevant quantum-resource information, including:

- **Qubit count**
- **Circuit depth**
- **Gate count**
- **Number of shots**
- **Circuit executions**
- **Backend**
- **Execution time**
- **Noise configuration**

This will allow judges to understand the practical cost of the quantum component.

46.17 Classical vs Hybrid QML Benchmark Report

A dedicated benchmarking output will compare the tested approaches.

Example structure:

Category	Classical ML	Hybrid QML
Predictive Metrics	Measured	Measured
Generalization	Measured	Measured
Training Cost	Measured	Measured
Inference Cost	Measured	Measured
Resource Requirements	Classical	Quantum + Classical
Explainability	Supported methods	Suitable QML analysis
Stability	Measured	Measured

Actual values will be generated from experiments rather than predetermined.

46.18 Experiment Tracking System

Each experiment will receive a unique record containing relevant configuration and results.

The record can include:

- Experiment ID
- Dataset version
- Feature configuration
- PCA settings
- Model
- Hyperparameters
- Encoding

- **Circuit configuration**
 - **Optimizer**
 - **Random seed**
 - **Backend**
 - **Metrics**
 - **Resource measurements**
 - **Final result**
-

46.19 Results Dashboard

A visual dashboard will present the major outputs of the experiment.

The dashboard will include:

Dataset View

Dataset characteristics and quality.

Model View

Classical and QML model configurations.

Prediction View

Model prediction and relevant supporting information.

Benchmark View

Classical vs QML comparison.

Quantum View

Qubits, circuit depth, gates, shots, and execution information.

Explainability View

Model-behaviour analysis.

Experiment View

Historical experiments and configurations.

46.20 Reliability and Error Reporting

The platform will provide clear system-status information.

Possible statuses include:

- **Experiment completed**
- **Prediction generated**
- **Input invalid**
- **Training failed**
- **Quantum backend unavailable**
- **Execution failed**
- **Result requires review**

The system should not display a successful result when a required computation has failed.

46.21 Security and Privacy Implementation

The prototype will include appropriate technical safeguards such as:

- **Authentication**
 - **Role-based access where required**
 - **Secure API communication**
 - **Dataset access control**
 - **Protected experiment records**
 - **Secure file handling**
 - **De-identified/public datasets for demonstration**
-

46.22 Testing Suite

The project will provide tests covering:

- **Data processing**
- **Feature transformations**
- **Model training**

- **Quantum circuits**
- **APIs**
- **Integration**
- **Error handling**
- **End-to-end execution**

This demonstrates that the project is an engineered software system rather than only a research notebook.

46.23 Technical Documentation

Documentation will describe:

- **System architecture**
 - **Installation**
 - **Technology stack**
 - **API structure**
 - **Dataset requirements**
 - **ML models**
 - **QML models**
 - **Quantum encoding**
 - **Experiment configuration**
 - **Evaluation methodology**
 - **Known limitations**
 - **Reproduction instructions**
-

46.24 Experimental Report

The team will prepare a report containing:

- 1. Dataset description**
- 2. Experimental methodology**
- 3. Classical baseline results**
- 4. QML results**
- 5. Quantum resource analysis**
- 6. Noise analysis**
- 7. Explainability results**
- 8. Generalization analysis**

- 9. Classical vs QML comparison
 - 10. Experimental conclusions
 - 11. Limitations
 - 12. Future research directions
-

46.25 SIH Demonstration Package

The final SIH submission/demo package should contain:

- Working prototype
 - Source code
 - README/documentation
 - Dataset or dataset access information
 - Model configurations
 - Experimental results
 - Architecture diagram
 - Workflow diagram
 - Results screenshots
 - Presentation/PPT
 - Demonstration script
 - Testing evidence
 - Experimental report
-

46.26 Core Deliverables Summary

No	Deliverable
.	
1	Hybrid QML Platform
2	Biomedical Data Pipeline
3	Preprocessing Module
4	Feature Engineering Module
5	Feature Selection Module

- 6 Dimensionality Reduction Module**
- 7 Classical ML Models**
- 8 Quantum ML Models**
- 9 Quantum Encoding Module**
- 10 Hybrid Training Engine**
- 11 Quantum Simulator Integration**
- 12 Hardware Compatibility Layer**
- 13 Disease Prediction Module**
- 14 Explainability Module**
- 15 Performance Evaluation**
- 16 Quantum Resource Analysis**
- 17 Benchmarking Module**
- 18 Experiment Tracking**
- 19 Results Dashboard**
- 20 Testing Suite**
- 21 Technical Documentation**
- 22 Experimental Report**
- 23 SIH Demonstration Package**
- 24 Working Prototype**

46.27 Minimum Viable SIH Deliverable

If development time is limited, the minimum working version should prioritize:

- 1. One biomedical dataset**

↓

2. Data preprocessing

↓

3. PCA/dimensionality reduction

↓

4. Classical baseline

↓

5. Quantum Kernel or VQC

↓

6. Quantum simulator

↓

7. Prediction

↓

8. Performance comparison

↓

9. Basic explainability

↓

10. Results dashboard

Advanced hardware execution and additional QML models can remain optional extensions.

46.28 Final Deliverable Statement

“The final deliverable is a functional, reproducible hybrid quantum-classical machine-learning platform that processes biomedical data, trains classical and quantum-enhanced models, generates disease predictions, explains model behaviour, measures quantum resources, and objectively benchmarks the approaches.”

SIH Deliverable in One Line

“Not just a QML model—a complete working platform, experimental pipeline, benchmark, and evidence package.”

47. SIH Demo Flow

47.1 Purpose of the SIH Demo

The SIH demonstration should prove that the proposed system is not only a theoretical concept but a working hybrid quantum machine-learning platform

capable of taking biomedical data, processing it, training classical and quantum-enhanced models, generating predictions, explaining results, and objectively comparing their performance.

The demo should focus on visible functionality and measurable evidence, rather than lengthy explanations of quantum theory or source code.

Main Demo Message

“We are not asking you to believe that quantum machine learning is better. We built a platform that allows us to experimentally measure whether it is better, where it helps, and what it costs.”

47.2 Recommended Demo Duration

A strong SIH demonstration can be completed in approximately 3–5 minutes.

Time	Demonstration Stage	Judge Sees
0:00–0:20	Problem + Solution	One-line project pitch
0:20–0:50	Dashboard	Complete platform
0:50–1:20	Dataset	Biomedical data + quality
1:20–1:45	Experiment Setup	Model configuration
1:45–2:15	Classical Baseline	Classical ML result
2:15–2:50	Quantum Model	QML execution
2:50–3:20	Prediction + XAI	Prediction and explanation

3:20–4:00	Benchmarking	Classical vs QML
0		
4:00–4:30	Quantum Resources	Qubits, depth, gates, shots
0		
4:30–5:00	Final Conclusion	Key experimental finding
0		

47.3 Step-by-Step Live Demonstration

Step 1 — Opening Pitch

Time: 0:00–0:20

Start with the problem rather than opening the code.

Say:

“Early disease detection depends on finding complex patterns in biomedical data. Our project combines classical machine learning with quantum machine learning to investigate whether quantum-enhanced models can provide useful improvements while remaining practical on today's quantum systems.”

Then immediately show the platform.

Key visual

Biomedical Data → Classical Processing → Quantum Learning → Prediction → Explainability → Benchmark

Do not spend the opening minute explaining quantum mechanics.

47.4 Step 2 — Show the Platform Dashboard

Time: 0:20–0:50

Open the main dashboard.

The judge should immediately see:

- **Project name**
- **Dataset status**
- **Selected disease/application**
- **Number of samples**
- **Number of features**
- **Selected model**
- **Quantum backend**
- **Experiment status**
- **Prediction/result area**
- **Benchmarking section**

Recommended dashboard layout

Left: Experiment controls

Center: Main prediction/performance results

Right: Quantum execution/resource information

This makes the platform understandable without navigating through many pages.

Say:

“This is our unified platform. The same environment handles biomedical data preparation, classical baselines, quantum models, prediction, explainability, and experimental benchmarking.”

47.5 Step 3 — Load Biomedical Dataset

Time: 0:50–1:20

Select a prepared benchmark biomedical dataset or upload the dataset.

Show only the important information:

- **Dataset name**
- **Samples**

- **Features**
- **Target variable**
- **Class distribution**
- **Missing values**
- **Data types**
- **Dataset status**

Example:

Dataset: Biomedical Binary Classification Dataset

Samples: 569

Features: 30

Classes: 2

Target: Disease / Non-Disease

If another dataset is used in the actual prototype, display its real values.

Say:

“Before training anything, the platform validates the dataset and generates a quality summary so that the experiment starts from a controlled input.”

Avoid spending time scrolling through hundreds of rows.

47.6 Step 4 — Configure the Experiment

Time: 1:20–1:45

Show a compact experiment configuration panel.

Example:

Experiment ID: EXP-001

Dataset: Biomedical Dataset

Task: Binary Classification

Preprocessing: Enabled

Feature Selection: Enabled

Dimensionality Reduction: PCA

Classical Model: SVM

Quantum Model: Quantum Kernel / VQC

Quantum Backend: Simulator

Shots: Configured

Random Seed: Fixed

Click Run Experiment.

Say:

“We keep the experiment configuration explicit so that the result can be reproduced and compared later.”

47.7 Step 5 — Show Classical Baseline

Time: 1:45–2:15

First show the classical result.

Display:

- **Accuracy**
- **Sensitivity / Recall**
- **Specificity**
- **F1-score**
- **ROC-AUC**
- **Confusion matrix**

Do not claim that the quantum model is better before showing its measured result.

Say:

“We first establish a classical baseline. This gives us a reference point against which the quantum-enhanced model can be fairly evaluated.”

This is an important SIH judge-facing point.

47.8 Step 6 — Run the Quantum Model

Time: 2:15–2:50

Now select the QML model.

Possible demonstration:

Classical Features → Quantum Encoding → Quantum Circuit → Measurement
→ Classical Optimizer

Show the quantum execution status.

Example:

Quantum Model: VQC

Qubits: 8

Circuit Depth: 6

Shots: 1024

Backend: Quantum Simulator

Status:

- ✓ Encoding
- ✓ Circuit Execution
- ✓ Measurement
- ✓ Optimization
- ✓ Model Ready

Say:

“The selected reduced features are encoded into a quantum circuit.
The quantum circuit produces measurements, and a classical
optimizer updates the model parameters.”

Keep this explanation short.

47.9 Step 7 — Demonstrate Disease Prediction

Time: 2:50–3:20

Select a test sample or enter a prepared test case.

Display:

Prediction

Class: Disease Risk

Model: Hybrid QML

Prediction Score: [Measured Value]

Status: Prediction Generated

If probability calibration has not been validated, avoid calling the output a clinical probability.

Instead use terms such as:

- prediction score
- model score
- predicted class
- decision output

Say:

“The trained model can now process an unseen sample and generate a prediction. This is a research decision-support output, not a clinical diagnosis.”

47.10 Step 8 — Show Explainability

Time: 3:20–3:40

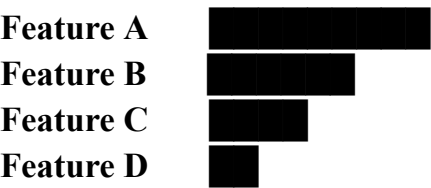
Open the explanation panel.

For classical models, show feature contribution or SHAP where appropriate.

For QML, show the implemented sensitivity/perturbation or ablation-based explanation.

Example:

Prediction Explanation



Explanation Method:
Feature Perturbation / SHAP

Say:

“The platform does not stop at prediction. We also show which input features or perturbations influenced the model output, while clearly separating model explanation from clinical interpretation.”

47.11 Step 9 — Show Classical vs Hybrid Benchmark

Time: 3:40–4:10

This should be one of the most important screens in the entire demo.

Show a direct comparison.

Metric	Classical SVM	Hybrid QML
Accuracy	Measured	Measured

Sensitivity	Measured	Measured
Specificity	Measured	Measured
F1	Measured	Measured
ROC-AUC	Measured	Measured
Inference Time	Measured	Measured
Quantum Resources	—	Measured

Use the actual measured values from your experiment.

Never insert impressive-looking numbers merely for the presentation.

Say:

“This is the key research result. Instead of assuming quantum superiority, we directly compare both approaches using the same experimental framework.”

If classical ML performs better, that is still a valid research result.

47.12 Step 10 — Show Quantum Resource Cost

Time: 4:10–4:30

Open the quantum resource panel.

Show:

- Number of qubits
- Circuit depth
- Gate count
- Shots
- Number of circuit executions
- Backend

- Execution time
- Noise configuration, if applicable

Example:

Quantum Resource Report

Qubits : 8
Circuit Depth : 6
Gate Count : Measured
Shots : 1024
Backend : Simulator
Noise Model : Enabled / Disabled
Execution Time : Measured

Say:

“Performance alone is not enough. We also measure the quantum resources required to obtain that result.”

This makes the demo more scientifically credible.

47.13 Step 11 — Show Final Experiment Decision

Display a compact summary:

EXPERIMENT SUMMARY

Dataset ✓
Preprocessing ✓
Classical ML ✓
Quantum ML ✓
Prediction ✓
Explainability ✓
Benchmark ✓
Resources ✓

Outcome:

Hybrid QML / Classical ML / Comparable

Evidence:

Performance + Generalization + Resource Cost

The final outcome must be based on the actual experiment.

Possible conclusions:

Case A — Hybrid performs better

“Under this experimental configuration, the hybrid model achieved better measured performance.”

Case B — Classical performs better

“The classical model performed better for this dataset, showing that QML does not automatically provide an advantage.”

Case C — Similar performance

“Both approaches produced comparable predictive performance, while the quantum model introduced additional computational resources.”

All three are legitimate outcomes.

47.14 Recommended Backup Demo Strategy

A live quantum execution should never be the single point of failure during SIH.

Quantum hardware or remote simulators can experience:

- **Queue delays**
- **Network failures**
- **Backend unavailability**
- **Execution errors**
- **Timeout**
- **Noise-related instability**

- **API problems**

Therefore maintain two demonstration modes.

Mode 1 — Live Experiment

Live Dataset



Live Training



Live QML Execution



Live Results

Mode 2 — Reproducible Cached Experiment

Stored Dataset



Stored Experiment Configuration



Previously Validated QML Run



Stored Results



Live Dashboard Visualization

If live quantum execution fails, switch immediately to the validated experiment record.

Say:

“The live backend is currently unavailable, so we are switching to our reproducible experiment record. The dashboard, configuration, model outputs, and resource measurements are preserved for the same experiment.”

Do not pretend a failed live execution succeeded.

47.15 What the Team Should NOT Show

Avoid wasting SIH demonstration time on:

- ✗ Long source-code scrolling**
- ✗ Installing libraries**
- ✗ Dataset cleaning line-by-line**
- ✗ Terminal commands**
- ✗ Long mathematical derivations**
- ✗ Quantum theory lectures**
- ✗ Waiting for a slow training process**
- ✗ Hundreds of dataset rows**
- ✗ Unnecessary UI navigation**
- ✗ Unmeasured performance claims**

Instead show:

Input → Experiment → Model → Prediction → Explanation → Benchmark → Evidence

47.16 Judge-Facing Narration Strategy

The team should keep the narration short and consistent.

Problem

“Biomedical datasets contain complex patterns that can be difficult to model efficiently.”

Solution

“We combine classical preprocessing with quantum-enhanced machine learning.”

Differentiation

“We don't assume quantum advantage; we experimentally benchmark it against classical ML.”

Explainability

“Every prediction is accompanied by model-level explanation and experiment information.”

Practicality

“The platform is simulator-first and designed to remain compatible with near-term quantum hardware.”

Final Evidence

“The result is evaluated using predictive metrics, generalization, stability, and quantum resource cost.”

47.17 Likely SIH Judge Questions

Q1. “Why use quantum computing if classical ML already works?”

Answer:

“We are not replacing classical ML. Classical computing handles preprocessing and other tasks where it is currently stronger, while quantum ML is evaluated as an additional learning component.”

Q2. “Are you claiming quantum advantage?”

Answer:

“No. Quantum advantage is an experimental hypothesis in our platform. We measure it rather than assuming it.”

Q3. “Can this run without a quantum computer?”

Answer:

“Yes. The prototype can run on quantum simulators, while the backend abstraction keeps the system compatible with near-term quantum hardware.”

Q4. “What happens if classical ML performs better?”

Answer:

“That is a valid result. Our objective is to determine where hybrid QML provides practical value, not to force a quantum model to win.”

Q5. “Is this already a medical diagnostic system?”

Answer:

“No. It is a research and decision-support platform. Clinical deployment would require extensive clinical validation, regulatory assessment, and real-world testing.”

Q6. “What is innovative here?”

Answer:

“The innovation is the complete experimental platform: biomedical preprocessing, QML, classical benchmarking, explainability, quantum-resource analysis, reproducibility, and hardware readiness in one workflow.”

47.18 Ideal 20-Second Closing

End with a strong statement instead of simply saying “Thank you.”

“Our platform does more than apply quantum machine learning to disease data. It creates a controlled environment where classical and quantum-enhanced models can be trained, explained, benchmarked, and evaluated for real computational cost. We don't assume quantum advantage—we build the evidence needed to determine where it actually exists.”

47.19 SIH Demo Success Criteria

The demo should successfully demonstrate these seven things:

- 1. A real biomedical dataset can be processed.**
- 2. A classical baseline can be trained.**
- 3. A quantum-enhanced model can be executed.**
- 4. An unseen sample can receive a prediction.**
- 5. The prediction can be explained.**

6. Classical and QML performance can be compared.
7. Quantum resource requirements can be measured.

Final Demo Flow

The demonstration follows the end-to-end flow outlined in Section 36.19 (System Workflow Overview).

Core SIH Demonstration USP

“One working platform to experimentally evaluate the predictive value, explainability, reproducibility, and resource cost of hybrid quantum machine learning for early disease detection.”

Is there